Red Hat Enterprise Linux 8

기본 시스템 설정 구성

시스템의 필수 기능을 설정하고 시스템 환경을 사용자 정의

Last Updated: 2024-09-20

Red Hat Enterprise Linux 8 기본 시스템 설정 구성

시스템의 필수 기능을 설정하고 시스템 환경을 사용자 정의

법적 공지

Copyright © 2024 Red Hat, Inc.

The text of and illustrations in this document are licensed by Red Hat under a Creative Commons Attribution–Share Alike 3.0 Unported license ("CC-BY-SA"). An explanation of CC-BY-SA is available at
http://creativecommons.org/licenses/by-sa/3.0/
. In accordance with CC-BY-SA, if you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, Red Hat Enterprise Linux, the Shadowman logo, the Red Hat logo, JBoss, OpenShift, Fedora, the Infinity logo, and RHCE are trademarks of Red Hat, Inc., registered in the United States and other countries.

Linux ® is the registered trademark of Linus Torvalds in the United States and other countries.

Java ® is a registered trademark of Oracle and/or its affiliates.

XFS ® is a trademark of Silicon Graphics International Corp. or its subsidiaries in the United States and/or other countries.

MySQL ® is a registered trademark of MySQL AB in the United States, the European Union and other countries.

Node.js ® is an official trademark of Joyent. Red Hat is not formally related to or endorsed by the official Joyent Node.js open source or commercial project.

The OpenStack ® Word Mark and OpenStack logo are either registered trademarks/service marks or trademarks/service marks of the OpenStack Foundation, in the United States and other countries and are used with the OpenStack Foundation's permission. We are not affiliated with, endorsed or sponsored by the OpenStack Foundation, or the OpenStack community.

All other trademarks are the property of their respective owners.

초록

기본 시스템 관리 작업을 수행하고, 환경 설정을 구성하고, 시스템을 등록하고, 네트워크 액세스 및 시스템 보안을 구성합니다. 사용자, 그룹 및 파일 권한을 관리합니다. 여러 RHEL 시스템에서 시스템 구성 인터페이스를 관리하려면 시스템 역할을 사용합니다. 효율적인 서비스 관리를 위해 systemd를 사용합니다. chrony로 NTP(Network Time Protocol)를 구성합니다. ReaR을 사용하여 시스템을 백업 및 복원합니다. Python 3 및 PHP와 같은 동적 프로그래밍 언어를 설치하고 사용합니다.

차례

RED HAT 문서에 관한 피드백 제공 5

1장. 기본 네트워크 액세스 구성 및 관리 6
1.1. 그래픽 설치 모드에서 네트워크 및 호스트 이름 구성 6
1.2. NMCLI를 사용하여 이더넷 연결 구성 7
1.3. NMTUI를 사용하여 이더넷 연결 구성 9
1.4. RHEL 웹 콘솔에서 네트워킹 관리 12
1.5. RHEL 시스템 역할을 사용하여 네트워킹 관리 13
1.6. 추가 리소스 14

2장. 시스템 등록 및 서브스크립션 관리 15
2.1. 설치 후 시스템 등록 15
2.2. 웹 콘솔에서 인증 정보로 서브스크립션 등록 16
2.3. GNOME에서 RED HAT 계정을 사용하여 시스템 등록 17
2.4. GNOME에서 활성화 키를 사용하여 시스템 등록 18
2.5. 설치 프로그램 GUI를 사용하여 RHEL 8 등록 19

3장. RED HAT 지원에 액세스 21
3.1. RED HAT 고객 포털을 통해 RED HAT 지원 요청 21
3.2. SOSREPORT를 사용하여 문제 해결 21

4장. 기본 환경 설정 변경 23
4.1. 날짜 및 시간 설정 23
4.2. 웹 콘솔을 사용하여 시간 설정 구성 23
4.3. 시스템 로케일 구성 25
4.4. 키보드 레이아웃 구성 26
4.5. 텍스트 콘솔 모드에서 글꼴 크기 변경 26
4.6. 추가 리소스 27

5장. OPENSSH로 두 시스템 간의 보안 통신 사용 28
5.1. SSH 키 쌍 생성 28
5.2. OPENSSH 서버에서 유일한 방법으로 키 기반 인증 설정 29
5.3. SSH-AGENT를 사용하여 SSH 인증 정보 캐싱 30
5.4. 스마트 카드에 저장된 SSH 키로 인증 31
5.5. 추가 리소스 32

6장. 기본 시스템 보안 구성 33
6.1. FIREWALLD 서비스 활성화 33
6.2. RHEL 8 웹 콘솔에서 방화벽 관리 33
6.3. 기본 SELINUX 설정 관리 34
6.4. RHEL 8 웹 콘솔에서 SELINUX 모드 전환 35
6.5. 추가 리소스 35

7장. 소프트웨어 패키지 관리 37
7.1. RHEL 8의 소프트웨어 관리 툴 37
7.2. 애플리케이션 스트림 37
7.3. 소프트웨어 패키지 검색 37
7.4. 소프트웨어 패키지 설치 40
7.5. 소프트웨어 패키지 업데이트 42
7.6. 소프트웨어 패키지 제거 48
7.7. 소프트웨어 패키지 그룹 관리 49
7.8. 패키지 관리 내역 처리 51
7.9. 소프트웨어 리포지토리 관리 53

7.10. YUM 구성 55

8장. RHEL 시스템 역할 소개 57

9장. 로깅 구성 59
9.1. 원격 로깅 솔루션 구성 59
9.2. 로깅 시스템 역할 사용 73

10장. 로그 파일을 사용하여 문제 해결 89
10.1. SYSLOG 메시지를 처리하는 서비스 89
10.2. SYSLOG 메시지를 저장하는 하위 디렉터리 89
10.3. 명령줄을 사용하여 로그 보기 89
10.4. 웹 콘솔에서 로그 검토 91
10.5. 추가 리소스 96

11장. 사용자 및 그룹 관리 97
11.1. 사용자 및 그룹 계정 관리 소개 97
11.2. 사용자 계정 관리 시작하기 98
11.3. 명령줄에서 사용자 관리 99
11.4. 웹 콘솔에서 사용자 계정 관리 104
11.5. 명령줄을 사용하여 사용자 그룹 편집 105
11.6. 루트 암호 변경 및 재설정 109

12장. SUDO 액세스 관리 113
12.1. SUDOERS의 사용자 권한 부여 113
12.2. 사용자에게 SUDO 액세스 권한 부여 114
12.3. 권한이 없는 사용자가 특정 명령 실행 가능 115

13장. 파일 시스템 권한 관리 118
13.1. 파일 권한 관리 118
13.2. 액세스 제어 목록 관리 125
13.3. UMASK 관리 126

14장. SYSTEMD 관리 131
14.1. SYSTEMD 장치 파일 위치 131
14.2. SYSTEMCTL을 사용하여 시스템 서비스 관리 132
14.3. 대상 시스템 상태로 부팅 139
14.4. 시스템 종료, 일시 중지 및 최대 절전 모드 142

15장. 시간 동기화 구성 149
15.1. CHRONY 제품군을 사용하여 NTP 구성 149
15.2. CHRONY 사용 152
15.3. HW 타임스탬프링을 사용하는 CHRONY 161
15.4. CHRONY에서 NTP가 이전에 지원한 일부 설정 수행 165
15.5. CHRONY의 NTP(NETWORK TIME SECURITY) 개요 168

16장. LANGPACK 사용 173
16.1. LANGPACK을 제공하는 언어 확인 173
16.2. RPM에 약한 종속성 기반 LANGPACK 작업 173
16.3. GLIBC-LANGPACK-<LOCALE_CODE>를 사용하여 디스크 공간 절약 175

17장. 나중에 분석을 위해 충돌한 커널 덤프 177
17.1. KDUMP란 무엇입니까? 177
17.2. 웹 콘솔에서 KDUMP 메모리 사용량 및 대상 위치 구성 177
17.3. RHEL 시스템 역할을 사용하는 KDUMP 181
17.4. 추가 리소스 181

18장. 시스템 복구 및 복원 **183**
18.1. REAR 설정 183
18.2. REAR 예약 185
18.3. 64비트 IBM Z 아키텍처에서 REAR RESCUE 이미지 사용 186

19장. 동적 프로그래밍 언어 설치 및 사용 **189**
19.1. PYTHON 소개 189
19.2. PYTHON 설치 및 사용 192
19.3. 버전이 없는 PYTHON 구성 200
19.4. PYTHON 3 RPM 패키지 202
19.5. PYTHON 스크립트에서 인터프리터 지시문 처리 206
19.6. PHP 스크립팅 언어 사용 208
19.7. TCL/TK 시작하기 216

RED HAT 문서에 관한 피드백 제공

문서에 대한 피드백에 감사드립니다. 어떻게 개선할 수 있는지 알려주십시오.

Jira를 통해 피드백 제출 (등록 필요)

1. Jira 웹 사이트에 로그인합니다.
2. 상단 탐색 모음에서 **생성** 을 클릭합니다.
3. **Summary** (요약) 필드에 설명 제목을 입력합니다.
4. **Description** (설명) 필드에 개선을 위한 제안을 입력합니다. 문서의 관련 부분에 대한 링크를 포함합니다.
5. 대화 상자 하단에서 **생성** 을 클릭합니다.

1장. 기본 네트워크 액세스 구성 및 관리

이 섹션에서는 Red Hat Enterprise Linux에서 네트워크 설정을 구성하는 방법에 대한 기본 옵션만 설명합니다.

1.1. 그래픽 설치 모드에서 네트워크 및 호스트 이름 구성

다음 프로세스의 단계에 따라 네트워크 및 호스트 이름을 구성합니다.

절차

1. **Installation Summary** (설치 요약) 창에서 **Network and Host Name(네트워크 및 호스트 이름)을** 클릭합니다.

2. 왼쪽 창의 목록에서 인터페이스를 선택합니다. 세부 정보는 오른쪽 창에 표시됩니다.

3. **ON/OFF(켜기/끄기)** 스위치를 토글하여 선택한 인터페이스를 활성화하거나 비활성화합니다. 인터페이스를 수동으로 추가하거나 제거할 수 없습니다.

4. **+** 를 클릭하여 가상 네트워크 인터페이스를 추가합니다. 이 인터페이스 중 하나는 다음과 같습니다. 팀, 본딩, 브리지 또는 VLAN.

5. 가상 인터페이스를 제거하려면 **-** 를 클릭합니다.

6. **Configure(구성**)를 클릭하여 기존 인터페이스(가상 및 물리적 모두)의 IP 주소, DNS 서버 또는 라우팅 구성과 같은 설정을 변경합니다.

7. **Host Name(호스트 이름) 필드에 시스템의 호스트 이름을 입력합니다.**
   호스트 이름은 ***hostname.domainname*** 형식의 FQDN(정규화된 도메인 이름) 또는 도메인이 없는 짧은 호스트 이름일 수 있습니다. 많은 네트워크에는 도메인 이름을 사용하여 연결된 시스템을 자동으로 제공하는 DHCP(Dynamic Host Configuration Protocol) 서비스가 있습니다. DHCP 서비스에서 이 시스템에 도메인 이름을 할당할 수 있도록 하려면 짧은 호스트 이름만 지정합니다.

   호스트 이름은 영숫자 문자만 포함할 수 있으며 **-** 또는 . **.**. 호스트 이름은 64자 이상이어야 합니다. 호스트 이름은 **-** 및 로 시작하거나 종료할 수 없습니다 **.** DNS를 준수하려면 FQDN의 각 부분이 63자 미만이어야 하며 점을 포함한 FQDN 길이에 255자를 초과해서는 안 됩니다.

   localhost 값은 대상 시스템에 대한 특정 정적 호스트 이름이 구성되어 있지 않으며, 설치된 시스템의 실제 호스트 이름은 DHCP 또는 DNS를 사용하는 NetworkManager를 사용하여 네트워크 구성을 처리하는 동안 구성됩니다.

   고정 IP 및 호스트 이름 구성을 사용하는 경우 계획된 시스템 사용 사례에 따라 짧은 이름 또는 FQDN을 사용할지 여부에 따라 달라집니다. Red Hat Identity Management는 프로비저닝 중에 FQDN을 구성하지만 일부 타사 소프트웨어 제품에는 짧은 이름이 필요할 수 있습니다. 두 경우 모두 모든 상황에서 두 양식의 가용성을 보장하기 위해 **IP FQDN short-alias** 형식으로 **/etc/hosts** 의 호스트에 대한 항목을 추가합니다.

8. **Apply(적용**)를 클릭하여 설치 프로그램 환경에 호스트 이름을 적용합니다.

9. 또는 **Network and Hostname(네트워크 및 호스트 이름)** 창에서 플레이버 옵션을 선택할 수 있습니다. 오른쪽 창에서 **Select network** (네트워크 선택)를 클릭하여 사용 가능한 연결을 선택하고 필요한 경우 암호를 입력하고 **Done(완료)을** 클릭합니다.

추가 리소스

- RHEL 자동 설치
- 네트워크 장치 이름 지정 표준에 대한 자세한 내용은 *네트워킹 구성 및 관리를* 참조하십시오.

1.2. NMCLI를 사용하여 이더넷 연결 구성

이더넷을 통해 호스트를 네트워크에 연결하는 경우 **nmcli** 유틸리티를 사용하여 명령줄에서 연결의 설정을 관리할 수 있습니다.

사전 요구 사항

- 물리적 또는 가상 이더넷 NIC(네트워크 인터페이스 컨트롤러)가 서버 구성에 있습니다.

절차

1. NetworkManager 연결 프로필을 나열합니다.

```
# nmcli connection show
NAME                UUID                                  TYPE      DEVICE
Wired connection 1  a5eb6490-cc20-3668-81f8-0314a27f3f75  ethernet  enp1s0
```

   기본적으로 NetworkManager는 호스트의 각 NIC에 대한 프로필을 생성합니다. 이 NIC를 특정 네트워크에만 연결하려는 경우 자동으로 생성된 프로필을 조정합니다. 이 NIC를 다른 설정으로 네트워크에 연결하려는 경우 각 네트워크에 대한 개별 프로필을 생성합니다.

2. 추가 연결 프로필을 생성하려면 다음을 입력합니다.

```
# nmcli connection add con-name <connection-name> ifname <device-name> type
ethernet
```

   기존 프로필을 수정하려면 이 단계를 건너뜁니다.

3. 선택 사항: 연결 프로필의 이름을 변경합니다.

```
# nmcli connection modify "Wired connection 1" connection.id "Internal-LAN"
```

   프로필이 여러 개인 호스트에서 의미 있는 이름을 사용하면 프로필의 용도를 쉽게 식별할 수 있습니다.

4. 연결 프로필의 현재 설정을 표시합니다.

```
# nmcli connection show Internal-LAN
...
connection.interface-name:     enp1s0
connection.autoconnect:        yes
ipv4.method:                   auto
ipv6.method:                   auto
...
```

5. IPv4 설정을 구성합니다.

   - DHCP를 사용하려면 다음을 입력합니다.

```
# nmcli connection modify Internal-LAN ipv4.method auto
```

ipv4.method 가 이미 **auto** (기본값)로 설정된 경우 이 단계를 건너뜁니다.

- 정적 IPv4 주소, 네트워크 마스크, 기본 게이트웨이, DNS 서버 및 검색 도메인을 설정하려면 다음을 입력합니다.

```
# nmcli connection modify Internal-LAN ipv4.method manual ipv4.addresses
192.0.2.1/24 ipv4.gateway 192.0.2.254 ipv4.dns 192.0.2.200 ipv4.dns-search
example.com
```

6. IPv6 설정을 구성합니다.

- SLAAC(상태 비저장 주소 자동 구성)를 사용하려면 다음을 입력합니다.

```
# nmcli connection modify Internal-LAN ipv6.method auto
```

ipv6.method 가 이미 **auto** (기본값)로 설정된 경우 이 단계를 건너뜁니다.

- 정적 IPv6 주소, 네트워크 마스크, 기본 게이트웨이, DNS 서버 및 검색 도메인을 설정하려면 다음을 입력합니다.

```
# nmcli connection modify Internal-LAN ipv6.method manual ipv6.addresses
2001:db8:1::fffe/64 ipv6.gateway 2001:db8:1::fffe ipv6.dns 2001:db8:1::ffbb
ipv6.dns-search example.com
```

7. 프로필의 다른 설정을 사용자 지정하려면 다음 명령을 사용합니다.

```
# nmcli connection modify <connection-name> <setting> <value>
```

값을 따옴표로 묶거나 spaces로 묶습니다.

8. 프로필을 활성화합니다.

```
# nmcli connection up Internal-LAN
```

검증

1. NIC의 IP 설정을 표시합니다.

```
# ip address show enp1s0
2: enp1s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP
group default qlen 1000
    link/ether 52:54:00:17:b8:b6 brd ff:ff:ff:ff:ff:ff
    inet 192.0.2.1/24 brd 192.0.2.255 scope global noprefixroute enp1s0
       valid_lft forever preferred_lft forever
    inet6 2001:db8:1::fffe/64 scope global noprefixroute
       valid_lft forever preferred_lft forever
```

2. IPv4 기본 게이트웨이를 표시합니다.

```
# ip route show default
default via 192.0.2.254 dev enp1s0 proto static metric 102
```

3. IPv6 기본 게이트웨이를 표시합니다.

```
# ip -6 route show default
default via 2001:db8:1::ffee dev enp1s0 proto static metric 102 pref medium
```

4. DNS 설정을 표시합니다.

```
# cat /etc/resolv.conf
search example.com
nameserver 192.0.2.200
nameserver 2001:db8:1::ffbb
```

여러 연결 프로필이 동시에 활성화된 경우 **이름 서버** 항목의 순서는 이러한 프로필의 DNS 우선 순위 값과 연결 유형에 따라 달라집니다.

5. **ping** 유틸리티를 사용하여 이 호스트가 다른 호스트에 패킷을 보낼 수 있는지 확인합니다.

```
# ping <host-name-or-IP-address>
```

문제 해결

- 네트워크 커넥터가 호스트와 스위치에 연결되어 있는지 확인합니다.
- 링크 실패가 이 호스트에만 있는지 또는 동일한 스위치에 연결된 다른 호스트에 있는지 확인합니다.
- 네트워크 케이블과 네트워크 인터페이스가 예상대로 작동하는지 확인합니다. 하드웨어 진단 단계를 수행하고 결함 케이블 및 네트워크 인터페이스 카드를 교체합니다.
- 디스크의 구성이 장치의 구성과 일치하지 않는 경우 NetworkManager를 시작하거나 다시 시작하면 장치 구성이 반영되는 메모리 내 연결이 생성됩니다. 자세한 내용과 이 문제를 방지하는 방법은 NetworkManager 서비스 솔루션을 다시 시작한 후 NetworkManager 연결 중복을 참조하십시오.

추가 리소스

- 시스템의 **nm-settings(5)** 도움말 페이지

1.3. NMTUI를 사용하여 이더넷 연결 구성

이더넷을 통해 호스트를 네트워크에 연결하는 경우 **nmtui** 애플리케이션을 사용하여 텍스트 기반 사용자 인터페이스에서 연결의 설정을 관리할 수 있습니다. **nmtui** 를 사용하여 새 프로필을 만들고 그래픽 인터페이스 없이 호스트에서 기존 프로필을 업데이트합니다.

참고

nmtui:

- 커서 키를 사용하여 이동합니다.
- 버튼을 선택하고 **Enter** 를 누릅니다.
- **Space** 를 사용하여 확인란을 선택하고 지웁니다.

사전 요구 사항

- 물리적 또는 가상 이더넷 NIC(네트워크 인터페이스 컨트롤러)가 서버 구성에 있습니다.

절차

1. 연결에 사용하려는 네트워크 장치 이름을 모르는 경우 사용 가능한 장치를 표시합니다.

```
# nmcli device status
DEVICE     TYPE      STATE              CONNECTION
enp1s0     ethernet  unavailable        --
...
```

2. nmtui **시작**:

```
# nmtui
```

3. **Edit a connection** 을 선택하고 **Enter** 를 누릅니다.

4. 새 연결 프로필을 추가하거나 기존 프로필을 수정할지 선택합니다.

   - 새 프로필을 생성하려면 다음을 수행합니다.

     i. **추가를** 누릅니다.

     ii. 네트워크 유형 목록에서 **이더넷** 을 선택하고 **Enter** 키를 누릅니다.

   - 기존 프로필을 수정하려면 목록에서 프로필을 선택하고 **Enter** 키를 누릅니다.

5. 선택 사항: 연결 프로필의 이름을 업데이트합니다.
   프로필이 여러 개인 호스트에서 의미 있는 이름을 사용하면 프로필의 용도를 쉽게 식별할 수 있습니다.

6. 새 연결 프로필을 생성하는 경우 장치 필드에 네트워크 **장치** 이름을 입력합니다.

7. 환경에 따라 그에 따라 **IPv4 구성 및 IPv6 구성** 영역에서 IP 주소 설정을 구성합니다. 이를 위해 다음 영역 옆에 있는 버튼을 누른 후 다음을 선택합니다.

   - **비활성화됨** (이 연결에 IP 주소가 필요하지 않은 경우).

   - **자동으로** DHCP 서버가 이 NIC에 IP 주소를 동적으로 할당하는 경우입니다.

   - **수동**: 네트워크에 고정 IP 주소 설정이 필요한 경우입니다. 이 경우 추가 필드를 채워야 합니다.

     i. 추가 필드를 표시하도록 구성할 프로토콜 옆에 **Show** 를 누릅니다.

     ii. **주소** 옆에 있는 **추가** 를 클릭하고 CIDR(Classless Inter-Domain Routing) 형식으로 IP 주소와 서브넷 마스크를 입력합니다.
     서브넷 마스크를 지정하지 않으면 NetworkManager는 IPv4 주소에 대해 **/32** 서브넷 마스크를 설정하고 IPv6 주소에 대해 **/64** 를 설정합니다.

     iii. 기본 게이트웨이의 주소를 입력합니다.

     iv. **DNS 서버** 옆에 있는 **추가** 를 클릭하고 DNS 서버 주소를 입력합니다.

     v. **검색 도메인** 옆에 있는 **추가** 를 클릭하고 DNS 검색 도메인을 입력합니다.

그림 1.1. 고정 IP 주소 설정이 포함된 이더넷 연결 예

```
                          Edit Connection

        Profile name Example-Connection
              Device enp7s0

= ETHERNET                                                    <Show>

= IPv4 CONFIGURATION <Manual>                                 <Hide>
          Addresses 192.0.2.1/24          <Remove>
                    <Add...>
            Gateway 192.0.2.254
        DNS servers 192.0.2.200           <Remove>
                    <Add...>
    Search domains example.com            <Remove>
                    <Add...>

            Routing (No custom routes) <Edit...>
  [ ] Never use this network for default route
  [ ] Ignore automatically obtained routes
  [ ] Ignore automatically obtained DNS parameters

  [ ] Require IPv4 addressing for this connection

= IPv6 CONFIGURATION <Manual>                                 <Hide>
          Addresses 2001:db8:1::1/64        <Remove>
                    <Add...>
            Gateway 2001:db8:1::fffe
        DNS servers 2001:db8:1::ffbb        <Remove>
                    <Add...>
    Search domains example.com            <Remove>
                    <Add...>
            Routing (No custom routes) <Edit...>
  [ ] Never use this network for default route
  [ ] Ignore automatically obtained routes
  [ ] Ignore automatically obtained DNS parameters

  [ ] Require IPv6 addressing for this connection

[X] Automatically connect
[X] Available to all users

                                                   <Cancel> <OK>
```

8. **OK** 를 눌러 새 연결을 만들고 자동으로 활성화합니다.

9. **다시** 키를 눌러 기본 메뉴로 돌아갑니다.

10. **Quit** 를 선택하고 **Enter** 를 눌러 **nmtui** 애플리케이션을 종료합니다.

검증

1. NIC의 IP 설정을 표시합니다.

```
# ip address show enp1s0
2: enp1s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP
group default qlen 1000
    link/ether 52:54:00:17:b8:b6 brd ff:ff:ff:ff:ff:ff
```

```
    inet 192.0.2.1/24 brd 192.0.2.255 scope global noprefixroute enp1s0
      valid_lft forever preferred_lft forever
    inet6 2001:db8:1::fffe/64 scope global noprefixroute
      valid_lft forever preferred_lft forever
```

2. IPv4 기본 게이트웨이를 표시합니다.

```
# ip route show default
default via 192.0.2.254 dev enp1s0 proto static metric 102
```

3. IPv6 기본 게이트웨이를 표시합니다.

```
# ip -6 route show default
default via 2001:db8:1::ffee dev enp1s0 proto static metric 102 pref medium
```

4. DNS 설정을 표시합니다.

```
# cat /etc/resolv.conf
search example.com
nameserver 192.0.2.200
nameserver 2001:db8:1::ffbb
```

여러 연결 프로필이 동시에 활성화된 경우 **이름 서버** 항목의 순서는 이러한 프로필의 DNS 우선 순위 값과 연결 유형에 따라 달라집니다.

5. **ping** 유틸리티를 사용하여 이 호스트가 다른 호스트에 패킷을 보낼 수 있는지 확인합니다.

```
# ping <host-name-or-IP-address>
```

문제 해결

- 네트워크 커넥터가 호스트와 스위치에 연결되어 있는지 확인합니다.
- 링크 실패가 이 호스트에만 있는지 또는 동일한 스위치에 연결된 다른 호스트에 있는지 확인합니다.
- 네트워크 케이블과 네트워크 인터페이스가 예상대로 작동하는지 확인합니다. 하드웨어 진단 단계를 수행하고 결함 케이블 및 네트워크 인터페이스 카드를 교체합니다.
- 디스크의 구성이 장치의 구성과 일치하지 않는 경우 NetworkManager를 시작하거나 다시 시작하면 장치 구성이 반영되는 메모리 내 연결이 생성됩니다. 자세한 내용과 이 문제를 방지하는 방법은 NetworkManager 서비스 솔루션을 다시 시작한 후 NetworkManager 연결 중복을 참조하십시오.

추가 리소스

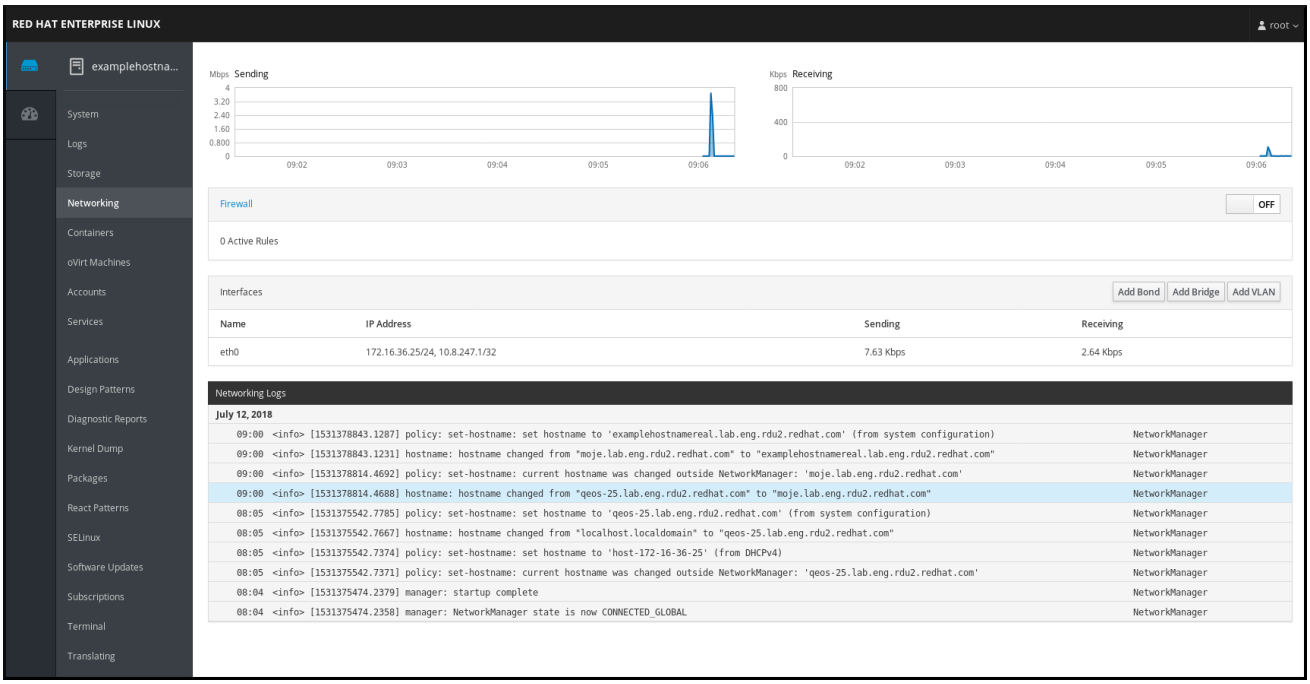
- 기본 게이트웨이를 제공하기 위해 특정 프로필을 사용하지 않도록 NetworkManager 구성
- DNS 서버 순서 구성

1.4. RHEL 웹 콘솔에서 네트워킹 관리

웹 콘솔에서 **Networking** (네트워킹) 메뉴를 사용하면 다음을 수행할 수 있습니다.

- 현재 수신 및 전송된 패킷을 표시하려면
- 사용 가능한 네트워크 인터페이스의 가장 중요한 특성을 표시
- 네트워킹 로그의 콘텐츠를 표시합니다.
- 다양한 유형의 네트워크 인터페이스를 추가하려면 (bond, team, bridge, VLAN)

그림 1.2. RHEL 웹 콘솔에서 네트워킹 관리



1.5. RHEL 시스템 역할을 사용하여 네트워킹 관리

네트워크 역할을 사용하여 여러 대상 머신에서 네트워킹 연결을 구성할 수 있습니다.

네트워크 역할을 사용하면 다음 유형의 인터페이스를 구성할 수 있습니다.

- 이더넷
- 브릿지
- 본딩됨
- VLAN
- MacVLAN
- InfiniBand

각 호스트에 필요한 네트워킹 연결은 **network_connections** 변수 내의 목록으로 제공됩니다.

> **주의**
>
> **네트워크 역할은 network _connections** 변수에 지정된 대로 타겟 시스템의 모든 연결 프로필을 업데이트하거나 생성합니다. 따라서 옵션이 시스템에만 있지만 **network_connections 변수에 없는 경우 네트워크 역할은** 지정된 프로필에서 옵션을 제거합니다.

다음 예제에서는 **네트워크** 역할을 적용하여 필수 매개 변수를 사용한 이더넷 연결이 있는지 확인하는 방법을 보여줍니다.

필수 매개 변수를 사용한 이더넷 연결을 설정하기 위해 네트워크 역할을 적용하는 예제

```
# SPDX-License-Identifier: BSD-3-Clause
---
- hosts: managed-node-01.example.com
  vars:
    network_connections:

      # Create one Ethernet profile and activate it.
      # The profile uses automatic IP addressing
      # and is tied to the interface by MAC address.
      - name: prod1
        state: up
        type: ethernet
        autoconnect: yes
        mac: "00:00:5e:00:53:00"
        mtu: 1450

  roles:
    - rhel-system-roles.network
```

추가 리소스

- RHEL 시스템 역할을 사용하도록 컨트롤 노드 및 관리형 노드 준비

1.6. 추가 리소스

- 네트워킹 구성 및 관리

2장. 시스템 등록 및 서브스크립션 관리

서브스크립션은 운영 체제 자체를 포함하여 Red Hat Enterprise Linux에 설치된 제품을 다룹니다.

Red Hat Content Delivery Network 서브스크립션을 사용하여 다음을 추적할 수 있습니다.

- 등록된 시스템
- 시스템에 제품이 설치되어 있습니다
- 설치된 제품에 첨부된 서브스크립션

2.1. 설치 후 시스템 등록

설치 프로세스 중에 시스템을 등록하지 않은 경우 다음 절차에 따라 시스템을 등록하십시오.

사전 요구 사항

- Red Hat 고객 포털에서 유효한 사용자 계정.
- Red Hat 로그인 생성 페이지를 참조하십시오.
- RHEL 시스템용 활성 서브스크립션.
- 설치 프로세스에 대한 자세한 내용은 설치 미디어에서 RHEL 상호 작용 설치를 참조하십시오.

절차

1. 한 단계로 시스템을 등록하고 자동으로 서브스크립션합니다.

```
# subscription-manager register --username <username> --password <password> --auto-attach
Registering to: subscription.rhsm.redhat.com:443/subscription
The system has been registered with ID: 37to907c-ece6-49ea-9174-20b87ajk9ee7
The registered system name is: client1.idm.example.com
Installed Product Current Status:
Product Name: Red Hat Enterprise Linux for x86_64
Status:       Subscribed
```

명령을 실행하면 Red Hat 고객 포털 사용자 이름과 암호를 입력하라는 메시지가 표시됩니다.

등록 프로세스가 실패하면 시스템을 특정 풀에 등록할 수 있습니다. 수행하는 방법에 대한 지침을 보려면 다음 단계를 따르십시오.

a. 필요한 서브스크립션의 풀 ID를 확인합니다.

```
# subscription-manager list --available
```

이 명령은 Red Hat 계정에 사용 가능한 모든 서브스크립션을 표시합니다. 모든 서브스크립션에 대해 풀 ID를 포함하여 다양한 특성이 표시됩니다.

b. *pool_id* 를 이전 단계에서 확인한 풀 ID로 교체하여 시스템에 적절한 서브스크립션을 연결합니다.

```
# subscription-manager attach --pool=pool_id
```

참고

Red Hat Insights에 시스템을 등록하려면 **rhc connect** 유틸리티를 사용할 수 있습니다. 원격 호스트 구성 설정을 참조하십시오.

2.2. 웹 콘솔에서 인증 정보로 서브스크립션 등록

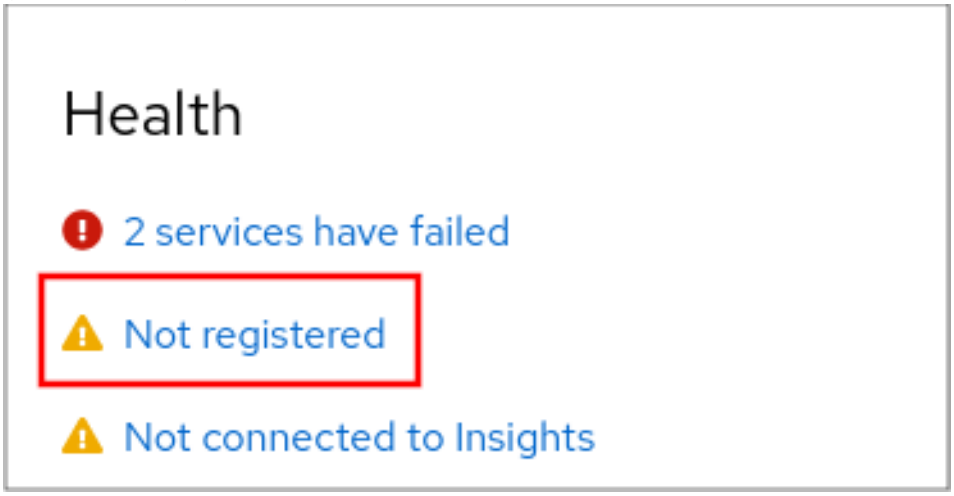
RHEL 웹 콘솔에서 계정 자격 증명으로 새로 설치된 Red Hat Enterprise Linux를 등록할 수 있습니다.

사전 요구 사항

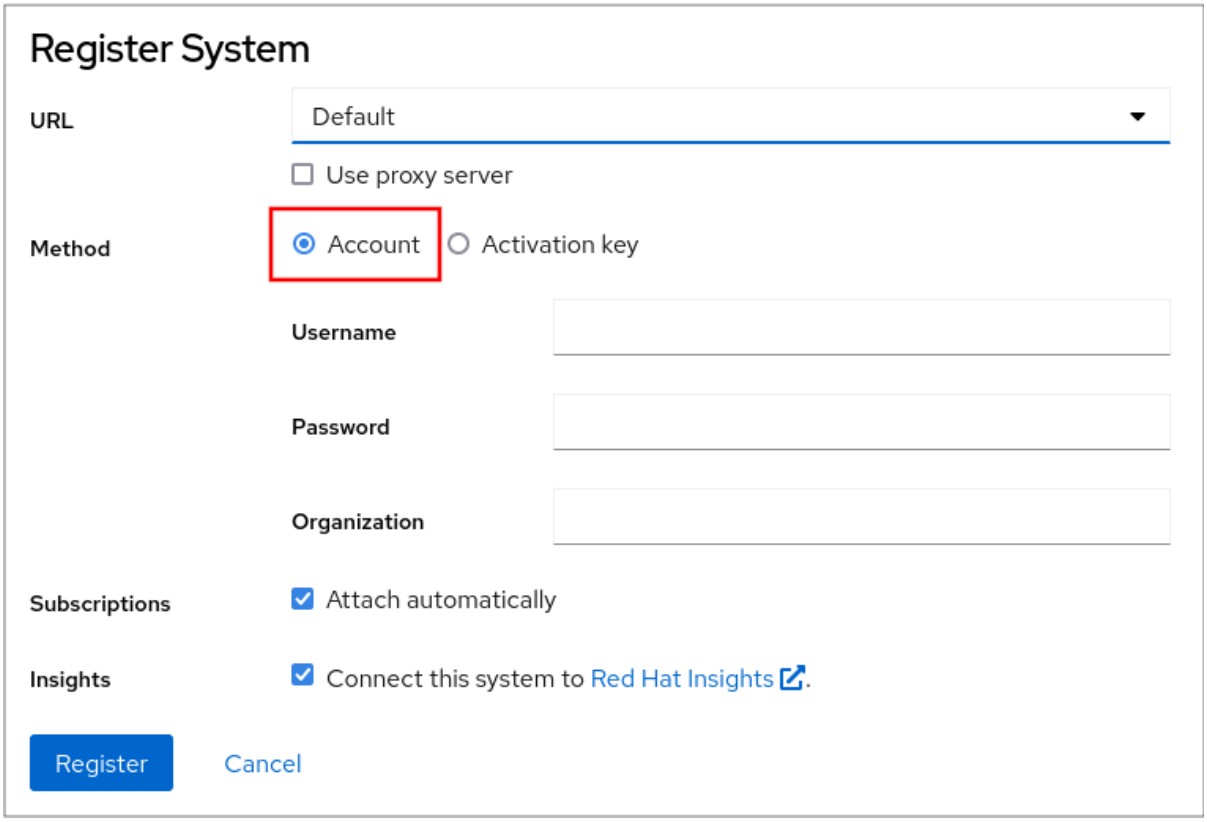
- Red Hat 고객 포털에서 유효한 사용자 계정입니다.
  Red Hat 로그인 생성 페이지를 참조하십시오.
- RHEL 시스템에 대한 활성 서브스크립션입니다.
- RHEL 8 웹 콘솔을 설치했습니다.
  자세한 내용은 웹 콘솔 설치 및 활성화를 참조하십시오.

절차

1. RHEL 8 웹 콘솔에 로그인합니다.
   자세한 내용은 웹 콘솔에 로그인 을 참조하십시오.
2. **개요** 페이지의 **Health** filed에서 **Not registered** 경고를 클릭하거나 메인 메뉴에서 **Subscriptions** (서브스크립션)를 클릭하여 서브스크립션 정보를 페이지로 이동합니다.



3. **개요** 필드에서 **등록을** 클릭합니다.
4. **등록 시스템** 대화 상자에서 **계정** 자격 증명을 사용하여 등록할 계정을 선택합니다.



5. 사용자 이름을 입력합니다.

6. 암호를 입력합니다.

7. 선택 사항: 조직의 이름 또는 ID를 입력합니다.
   계정이 Red Hat 고객 포털에서 두 개 이상의 조직에 속하는 경우 조직 이름 또는 조직 ID를 추가해야 합니다. 조직 ID를 가져오려면 Red Hat 연락처로 이동하십시오.

   - 시스템을 Red Hat Insights에 연결하지 않으려면 **Insights** 확인란을 지웁니다.

8. **Register(등록)**를 클릭합니다.

2.3. GNOME에서 RED HAT 계정을 사용하여 시스템 등록

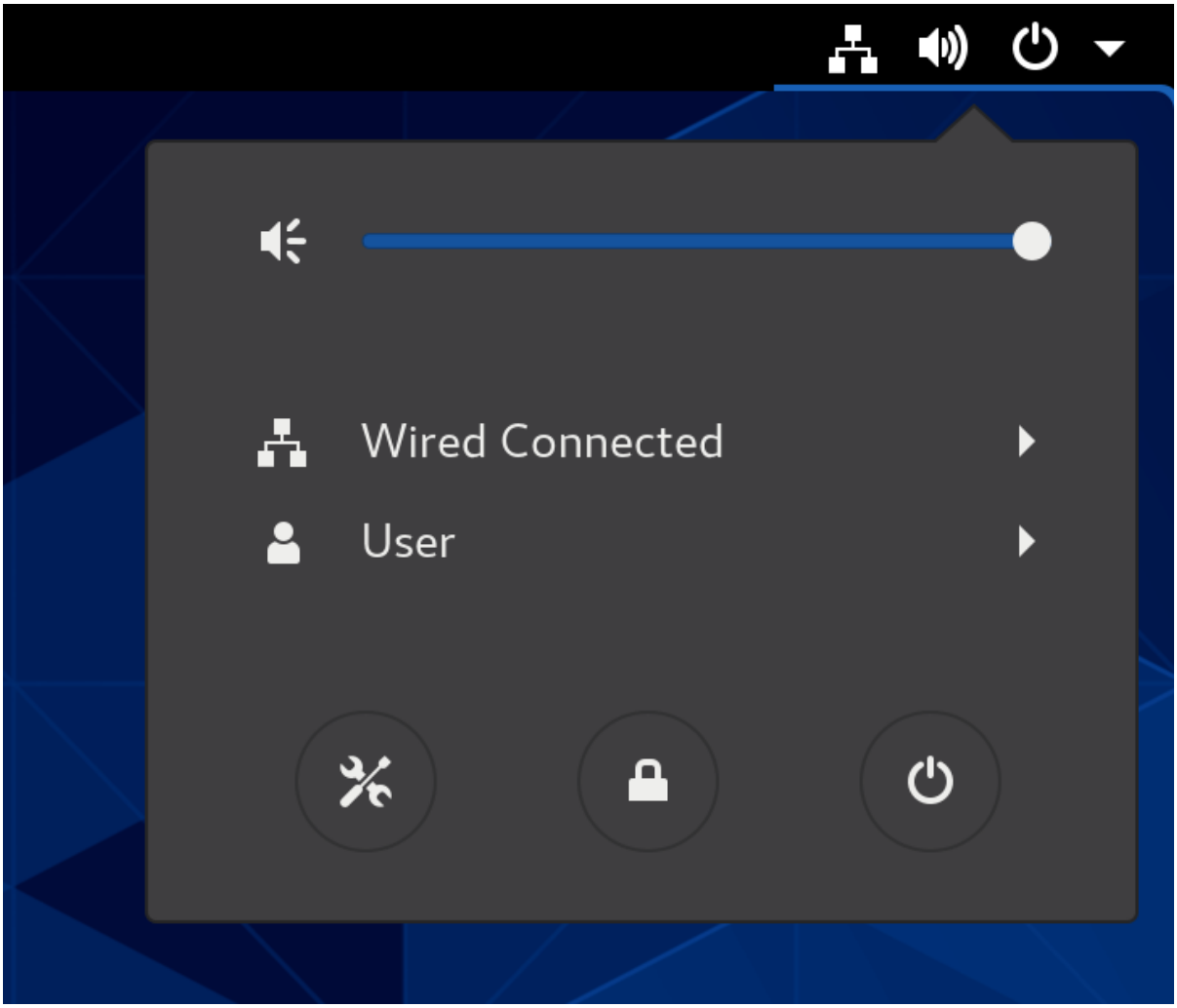
다음 절차의 단계에 따라 시스템을 Red Hat 계정에 등록하십시오.

사전 요구 사항

- Red Hat 고객 포털에서 유효한 계정.
  새 사용자 등록은 Create a Red Hat Login (Red Hat 로그인 만들기) 페이지를 참조하십시오.

절차

1. 오른쪽 상단에 있는 **시스템 메뉴를** 열고 **Settings** 아이콘을 클릭합니다.



2. **Details → About** (세부 정보 정보) 섹션에서 **Register(등록**)를 클릭합니다.
3. **Registration Server** (등록 서버)를 선택합니다.
4. Red Hat 서버를 사용하지 않는 경우 **URL** 필드에 서버 주소를 입력합니다.
5. **등록 유형** 메뉴에서 **Red Hat 계정을** 선택합니다.
6. **등록 정보** 아래 :
   - **로그인** 필드에 Red Hat 계정 사용자 이름을 입력합니다.
   - **암호** 필드에 Red Hat 계정 암호를 입력합니다.
   - **Organization** (조직) 필드에 조직 이름을 입력합니다.
7. **Register(등록**)를 클릭합니다.

2.4. GNOME에서 활성화 키를 사용하여 시스템 등록

이 절차의 단계에 따라 시스템을 활성화 키에 등록합니다. 조직 관리자로부터 활성화 키를 가져올 수 있습니다.

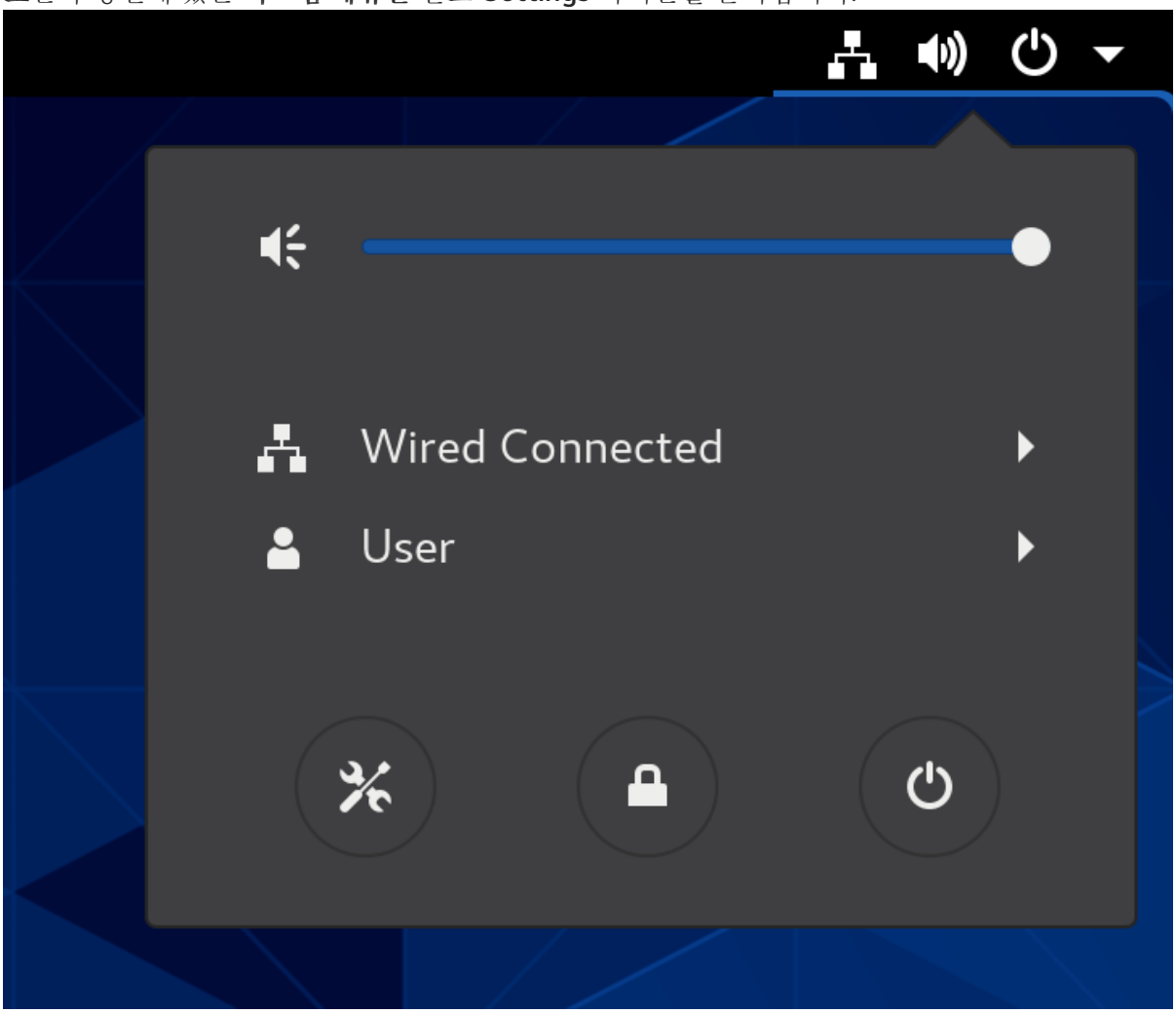
사전 요구 사항

- 활성화 키 또는 키.

새 활성화 키를 생성하려면 활성화 키 페이지를 참조하십시오.

절차

1. 오른쪽 상단에 있는 **시스템 메뉴를** 열고 **Settings** 아이콘을 클릭합니다.



2. **Details → About** (세부 정보 정보) 섹션에서 **Register(등록**)를 클릭합니다.
3. **Registration Server** (등록 서버)를 선택합니다.
4. Red Hat 서버를 사용하지 않는 경우 **URL** 필드에 서버 주소를 입력합니다.
5. **Registration Type(등록 유형**) 메뉴에서 **Activation Keys** (활성화 키)를 선택합니다.
6. **등록 정보** 아래 :
   - 활성화 키 필드에 **활성화 키를** 입력합니다.
     키를 쉼표(,)로 구분합니다.
   - 조직 필드에 조직의 이름 또는 ID를 **입력합니다**.
7. **Register(등록**)를 클릭합니다.

2.5. 설치 프로그램 GUI를 사용하여 RHEL 8 등록

RHEL 설치 프로그램 GUI를 사용하여 Red Hat Enterprise Linux 8을 등록할 수 있습니다.

사전 요구 사항

- Red Hat 고객 포털에 유효한 사용자 계정이 있어야 합니다. Red Hat 로그인 생성 페이지를 참조하십시오.
- 유효한 활성화 키 및 조직 ID가 있습니다.

절차

1. **설치 요약** 화면에서 소프트웨어에서 **Red Hat에 연결을** 클릭합니다.
2. **계정 또는 활성화 키 옵션을 사용하여 Red Hat 계정을 인증합니다.**
3. **선택 사항: Set System Purpose (시스템 용도 설정) 필드에서 드롭다운 메뉴에서 설정할 Role,SLA, Usage 특성을 선택합니다.**
   현재 Red Hat Enterprise Linux 8 시스템이 성공적으로 등록되었습니다.

3장. RED HAT 지원에 액세스

이 섹션에서는 Red Hat 지원 및 **sosreport** 를 사용하여 문제를 효과적으로 해결하는 방법을 설명합니다.

Red Hat으로부터 지원을 받으려면 서브스크립션이 제공되는 모든 항목에 액세스할 수 있는 Red Hat 고객 포털 을 사용하십시오.

3.1. RED HAT 고객 포털을 통해 RED HAT 지원 요청

다음 섹션에서는 Red Hat 고객 포털을 사용하여 도움을 얻는 방법에 대해 설명합니다.

사전 요구 사항

- Red Hat 고객 포털에서 유효한 사용자 계정. Red Hat 로그인 생성 단원을 참조하십시오.
- RHEL 시스템용 활성 서브스크립션.

절차

1. Red Hat 지원에 액세스:
   a. 새 지원 사례 열기.
   b. Red Hat 전문가와 함께 실시간 채팅 시작.
   c. 전화 문의 또는 이메일 전송은 Red Hat 전문가에게 문의하십시오.

3.2. SOSREPORT를 사용하여 문제 해결

sosreport 명령은 Red Hat Enterprise Linux 시스템에서 구성 세부 정보, 시스템 정보 및 진단 정보를 수집합니다.

다음 섹션에서는 **sosreport** 명령을 사용하여 지원 사례에 대한 보고서를 생성하는 방법을 설명합니다.

사전 요구 사항

- Red Hat 고객 포털에서 유효한 사용자 계정. Red Hat 로그인 생성 단원을 참조하십시오.
- RHEL 시스템용 활성 서브스크립션.
- 지원 케이스 번호.

절차

1. **sos** 패키지를 설치합니다.

```
# yum install sos
```

참고

Red Hat Enterprise Linux의 기본 최소 설치에는 **sosreport** 명령을 제공하는 **sos** 패키지가 포함되지 않습니다.

2. 보고서를 생성합니다.

```
# sosreport
```

3. 지원 사례에 보고서를 첨부합니다.
   Red Hat 지원 케이스에 파일을 첨부하려면 어떻게 해야 합니까?자세한 내용은 Red Hat 지식베이스 문서.

   보고서를 첨부할 때 관련 지원 케이스 수를 입력하라는 메시지가 표시됩니다.

추가 리소스

- sosreport는 무엇이며 Red Hat Enterprise Linux에서 하나를 만드는 방법은 무엇입니까?

4장. 기본 환경 설정 변경

기본 환경 설정 구성은 설치 프로세스의 일부입니다. 다음 섹션에서는 나중에 변경할 때 안내합니다. 환경의 기본 구성에는 다음이 포함됩니다.

- 날짜 및 시간
- 시스템 로케일
- 키보드 레이아웃
- 언어

4.1. 날짜 및 시간 설정

정확한 시간 보관은 여러 가지 이유로 중요합니다. Red Hat Enterprise Linux에서는 사용자 공간에서 데몬이 실행되는 **NTP** 프로토콜을 통해 시간 초과를 보장합니다. 사용자 공간 데몬은 커널에서 실행되는 시스템 시계를 업데이트합니다. 시스템 클록은 다양한 시계 소스를 사용하여 시간을 유지할 수 있습니다.

Red Hat Enterprise Linux 8은 **chronyd** 데몬을 사용하여 **NTP** 를 구현합니다.**chronyd** 는 chrony 패키지에서 사용할 수 있습니다. 자세한 내용은 chrony 모음을 사용하여 NTP 구성을 참조하십시오.

4.1.1. 현재 날짜 및 시간 표시

현재 날짜와 시간을 표시하려면 다음 단계 중 하나를 사용합니다.

절차

1. **date** 명령을 입력합니다.

```
$ date
Mon Mar 30 16:02:59 CEST 2020
```

2. 자세한 내용을 보려면 **timedatectl** 명령을 사용합니다.

```
$ timedatectl
Local time: Mon 2020-03-30 16:04:42 CEST
Universal time: Mon 2020-03-30 14:04:42 UTC
  RTC time: Mon 2020-03-30 14:04:41
 Time zone: Europe/Prague (CEST, +0200)
System clock synchronized: yes
NTP service: active
RTC in local TZ: no
```

추가 리소스

- **date(1)** 및 **timedatectl(1)** 매뉴얼 페이지

4.2. 웹 콘솔을 사용하여 시간 설정 구성

시간대를 설정하고 RHEL 웹 콘솔에서 NTP(Network Time Protocol) 서버와 시스템 시간을 동기화할 수 있습니다.

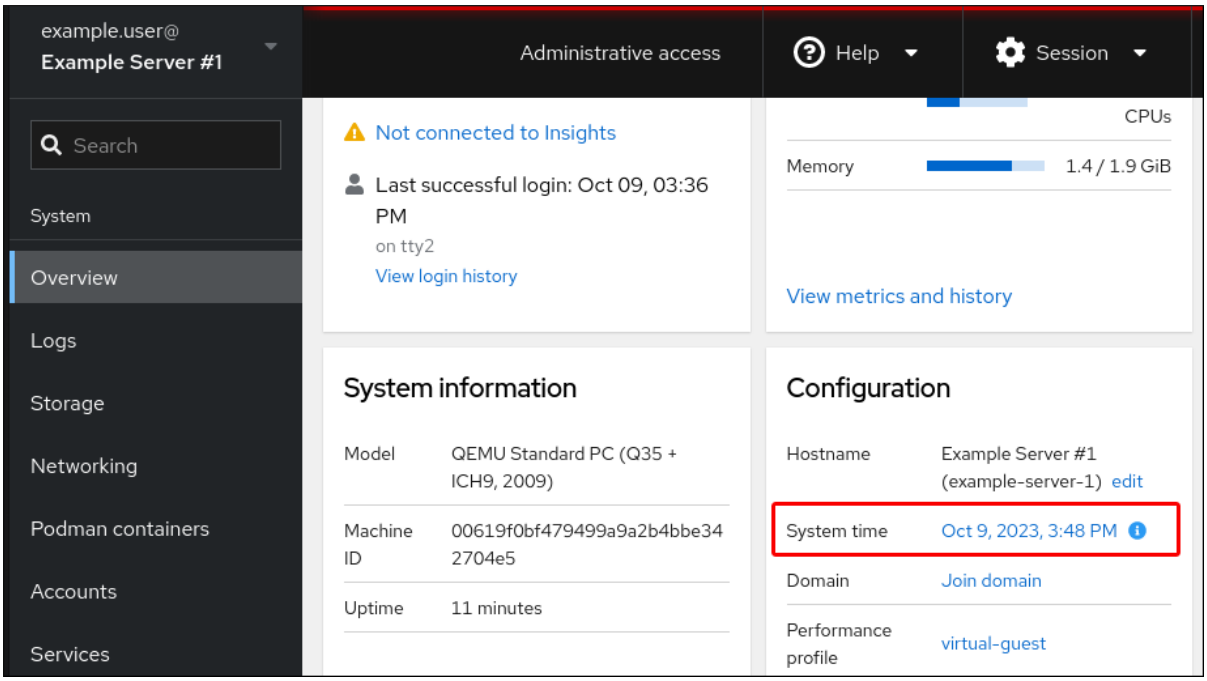
사전 요구 사항

- RHEL 8 웹 콘솔을 설치했습니다.
  자세한 내용은 웹 콘솔 설치 및 활성화를 참조하십시오.

절차

1. RHEL 8 웹 콘솔에 로그인합니다.
   자세한 내용은 웹 콘솔에 로그인 을 참조하십시오.

2. 개요 에서 현재 시스템 시간을 클릭합니다.



3. 시스템 시간을 클릭합니다.

4. 시스템 시간 변경 대화 상자에서 필요한 경우 시간대를 변경합니다.

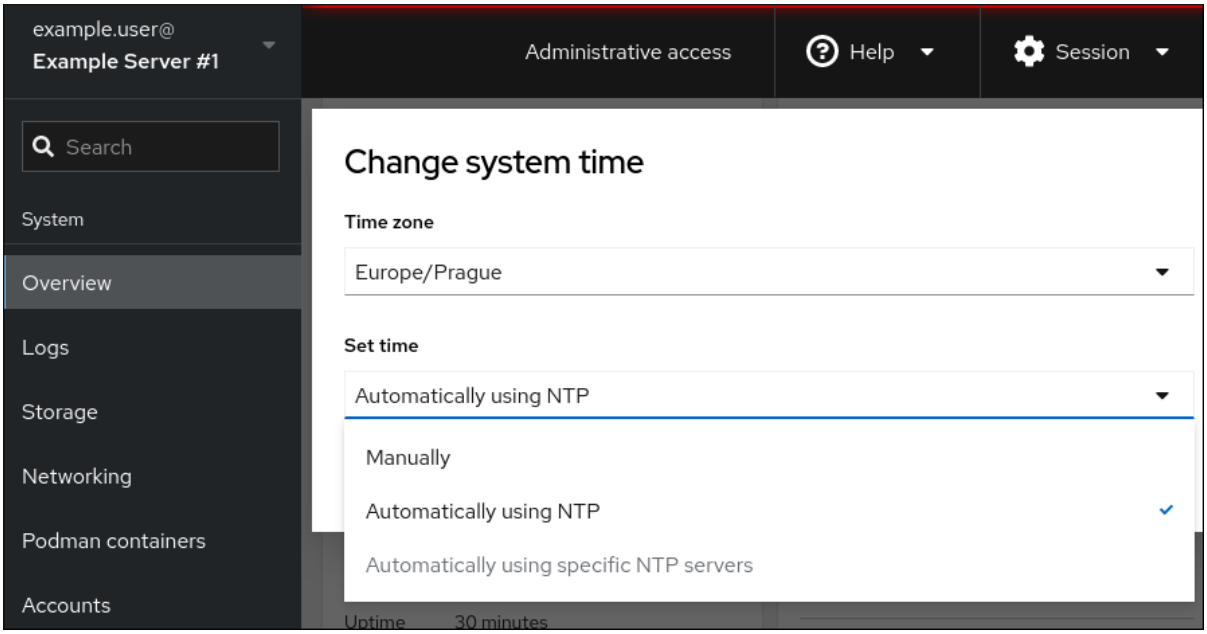
5. 설정 시간 드롭다운 메뉴에서 다음 중 하나를 선택합니다.

   수동
   : NTP 서버 없이 수동으로 시간을 설정해야 하는 경우 이 옵션을 사용합니다.

   NTP 서버 자동 사용
   : 이 옵션은 시간을 사전 설정된 NTP 서버와 자동으로 동기화하는 기본 옵션입니다.

   특정 NTP 서버 자동 사용
   : 시스템을 특정 NTP 서버와 동기화해야 하는 경우에만 이 옵션을 사용합니다. DNS 이름 또는 서버의 IP 주소를 지정합니다.

6. 변경을 클릭합니다.



검증

- 시스템 탭에 표시된 시스템 시간을 확인합니다.

추가 리소스

- Chrony 모음을 사용하여 NTP 구성.

4.3. 시스템 로케일 구성

시스템 전체 로케일 설정은 **systemd** 데몬에서 초기 부팅 시 읽을 수 있는/**etc/locale.conf** 파일에 저장됩니다. 개별 프로그램이나 개별 사용자가 재정의하지 않는 한 모든 서비스 또는 사용자는 /**etc/locale.conf** 에 구성된 로케일 설정을 상속합니다.

절차

- 사용 가능한 시스템 로케일 설정을 나열하려면 다음을 수행합니다.

```
$ localectl list-locales
C.utf8
aa_DJ
aa_DJ.iso88591
aa_DJ.utf8
...
```

- 시스템 로케일 설정의 현재 상태를 표시하려면 다음을 수행합니다.

```
$ localectl status
```

- 기본 시스템 로케일 설정을 설정하거나 변경하려면 **root** 사용자로 **localectl set-locale** 하위 명령을 사용합니다. 예를 들면 다음과 같습니다.

```
# localectl set-locale LANG=en_US
```

참고

GNOME 터미널은 UTF8 이외의 시스템 로케일을 지원하지 않습니다. 자세한 내용은 시스템 로케일을 비UTF8 지식 기반 솔루션으로 설정할 때 gnome-terminal 애플리케이션이 시작되지 않음을 참조하십시오.

추가 리소스

- **man localectl(1), man locale(7) 및 man locale.conf(5)**

4.4. 키보드 레이아웃 구성

키보드 레이아웃 설정은 텍스트 콘솔 및 그래픽 사용자 인터페이스에 사용되는 레이아웃을 제어합니다.

절차

- 사용 가능한 키 맵을 나열하려면 다음을 수행합니다.

```
$ localectl list-keymaps
ANSI-dvorak
al
al-plisi
amiga-de
amiga-us
...
```

- keymaps 설정의 현재 상태를 표시하려면 다음을 수행합니다.

```
$ localectl status
...
VC Keymap: us
...
```

- 기본 시스템 키맵을 설정하거나 변경하려면 다음을 수행합니다. 예를 들면 다음과 같습니다.

```
# localectl set-keymap us
```

추가 리소스

- **man localectl(1), man locale(7) 및 man locale.conf(5)**

4.5. 텍스트 콘솔 모드에서 글꼴 크기 변경

setfont 명령을 사용하여 가상 콘솔에서 글꼴 크기를 변경할 수 있습니다.

- 글꼴 이름과 함께 **setfont** 명령을 입력합니다. 예를 들면 다음과 같습니다.

```
# setfont /usr/lib/kbd/consolefonts/LatArCyrHeb-19.psfu.gz
```

참고

setfont 명령은 기본적으로 여러 하드 코딩된 경로를 검색합니다. 따라서**setfont** 에는 글꼴의 전체 이름과 경로가 필요하지 않습니다.

- 가로 및 수직으로 글꼴 크기를 두 배로 늘리려면 **-d** 매개변수를 사용하여 **setfont** 명령을 입력합니다.

```
# setfont -d LatArCyrHeb-16
```

참고

두 배로 늘릴 수 있는 최대 글꼴 크기는 16x16 component입니다.

- 시스템을 재부팅하는 동안 선택한 글꼴을 유지하려면 **/etc/vconsole.conf** 파일에서 **FONT** 변수를 사용합니다. 예를 들면 다음과 같습니다.

```
# cat /etc/vconsole.conf
KEYMAP="us"
FONT="eurlatgr"
```

- 'kbd' 패키지로 설치된 **kbd-misc** 패키지에서 다양한 글꼴을 찾을 수 있습니다. 예를 들어, 글꼴 **LatArCyrHeb** 에는 다음과 같은 다양한 변형이 있습니다.

```
# rpm -ql kbd-misc | grep LatAr

/usr/lib/kbd/consolefonts/LatArCyrHeb-08.psfu.gz
/usr/lib/kbd/consolefonts/LatArCyrHeb-14.psfu.gz
/usr/lib/kbd/consolefonts/LatArCyrHeb-16+.psfu.gz
/usr/lib/kbd/consolefonts/LatArCyrHeb-16.psfu.gz
/usr/lib/kbd/consolefonts/LatArCyrHeb-19.psfu.gz
```

참고

가상 콘솔에서 지원되는 최대 글꼴 크기는 32픽셀입니다. 콘솔에 대한 작은 해상도를 사용하여 글꼴 가독성 문제를 줄일 수 있습니다.

4.6. 추가 리소스

- 설치 미디어에서 RHEL을 대화형으로 설치

5장. OPENSSH로 두 시스템 간의 보안 통신 사용

SSH(Secure Shell)는 클라이언트-서버 아키텍처를 사용하여 두 시스템 간에 보안 통신을 제공하고 사용자가 서버 호스트 시스템에 원격으로 로그인할 수 있는 프로토콜입니다. FTP 또는 Telnet과 같은 다른 원격 통신 프로토콜과 달리 SSH는 로그인 세션을 암호화하여 침입자가 연결에서 암호화되지 않은 암호를 수집하지 못하도록 합니다.

5.1. SSH 키 쌍 생성

로컬 시스템에서 SSH 키 쌍을 생성하고 생성된 공개 키를 OpenSSH 서버에 복사하여 암호를 입력하지 않고 OpenSSH 서버에 로그인할 수 있습니다. 키를 생성하려는 각 사용자는 이 절차를 실행해야 합니다.

시스템을 다시 설치한 후 이전에 생성된 키 쌍을 유지하려면 새 키를 만들기 전에 ~/**.ssh**/ 디렉터리를 백업하십시오. 다시 설치한 후 홈 디렉터리로 복사합니다. **root** 를 포함하여 시스템의 모든 사용자에 대해 이 작업을 수행할 수 있습니다.

사전 요구 사항

- 키를 사용하여 OpenSSH 서버에 연결하려는 사용자로 로그인했습니다.
- OpenSSH 서버는 키 기반 인증을 허용하도록 구성됩니다.

절차

1. ECDSA 키 쌍을 생성합니다.

```
$ ssh-keygen -t ecdsa
Generating public/private ecdsa key pair.
Enter file in which to save the key (/home/<username>/.ssh/id_ecdsa):
Enter passphrase (empty for no passphrase): <password>
Enter same passphrase again: <password>
Your identification has been saved in /home/<username>/.ssh/id_ecdsa.
Your public key has been saved in /home/<username>/.ssh/id_ecdsa.pub.
The key fingerprint is:
SHA256:Q/x+qms4j7PCQ0qFd09iZEFHA+SqwBKRNaU72oZfaCI
<username>@<localhost.example.com>
The key's randomart image is:
+---[ECDSA 256]---+
|.oo..o=++        |
|.. o .oo .       |
|. .. o. o        |
|....o.+...       |
|o.oo.o +S .      |
|.=.+.   .o       |
|E.*+.  .  . .    |
|.=..+ +..  o     |
|  .  oo*+o.      |
+----[SHA256]-----+
```

   ssh-keygen -t ed25519 명령을 입력하여 매개 변수 또는 **Ed25519** 키 쌍 없이 **ssh-keygen** 명령을 사용하여 RSA 키 쌍을 생성할 수도 있습니다. Ed25519 알고리즘은 FIPS-140과 호환되지 않으며 OpenSSH는 FIPS 모드에서 Ed25519 키와 함께 작동하지 않습니다.

2. 공개 키를 원격 머신에 복사합니다.

```
$ ssh-copy-id <username>@<ssh-server-example.com>
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are
already installed
<username>@<ssh-server-example.com>'s password:
…
Number of key(s) added: 1

Now try logging into the machine, with: "ssh '<username>@<ssh-server-example.com>'" and
check to make sure that only the key(s) you wanted were added.
```

<***username*** > @ <***ssh-server-example.com*** >을 사용자 인증 정보로 바꿉니다.

세션에서 **ssh-agent** 프로그램을 사용하지 않는 경우 이전 명령은 가장 최근에 수정된 **~/.ssh/id*.pub** 공개 키를 아직 설치하지 않은 경우 복사합니다. 다른 공개 키 파일을 지정하거나 **ssh-agent** 로 메모리에 캐시된 키보다 파일의 키 우선 순위를 지정하려면**ssh-copy-id** 명령을 **-i** 옵션과 함께 사용합니다.

검증

1. 키 파일을 사용하여 OpenSSH 서버에 로그인합니다.

```
$ ssh -o PreferredAuthentications=publickey <username>@<ssh-server-example.com>
```

추가 리소스

- 시스템의 **ssh-keygen(1)** 및 **ssh-copy-id(1)** 도움말 페이지

5.2. OPENSSH 서버에서 유일한 방법으로 키 기반 인증 설정

시스템 보안을 강화하려면 OpenSSH 서버에서 암호 인증을 비활성화하여 키 기반 인증을 시행합니다.

사전 요구 사항

- **openssh-server** 패키지가 설치되어 있습니다.
- **sshd** 데몬이 서버에서 실행되고 있습니다.
- 키를 사용하여 OpenSSH 서버에 이미 연결할 수 있습니다.
  자세한 내용은 SSH 키 쌍 생성 섹션을참조하십시오.

절차

1. 텍스트 편집기에서 **/etc/ssh/sshd_config** 구성을 엽니다. 예를 들면 다음과 같습니다.

```
# vi /etc/ssh/sshd_config
```

2. **PasswordAuthentication** 옵션을 **no** 로 변경합니다.

```
PasswordAuthentication no
```

3. 새 기본 설치 이외의 시스템에서 **PubkeyAuthentication** 매개변수가 설정되지 않았거나**yes** 로 설정되어 있는지 확인합니다.

4. **ChallengeResponseAuthentication** 지시문을 **no** 로 설정합니다.
   해당 항목은 구성 파일에서 주석 처리되며 기본값은 **yes** 입니다.

5. NFS로 마운트된 홈 디렉토리에서 키 기반 인증을 사용하려면 **use_nfs_home_dirs** SELinux 부울을 활성화합니다.

```
# setsebool -P use_nfs_home_dirs 1
```

6. 콘솔 또는 대역 외 액세스를 사용하지 않고 원격으로 연결하는 경우 암호 인증을 비활성화하기 전에 키 기반 로그인 프로세스를 테스트합니다.

7. **sshd** 데몬을 다시 로드하여 변경 사항을 적용합니다.

```
# systemctl reload sshd
```

추가 리소스

- **sshd_config(5)** 및 **setsebool(8)** 도움말 페이지

5.3. SSH-AGENT를 사용하여 SSH 인증 정보 캐싱

SSH 연결을 시작할 때마다 암호를 입력하지 않으려면 **ssh-agent** 유틸리티를 사용하여 로그인 세션의 개인 SSH 키를 캐시할 수 있습니다. 에이전트가 실행 중이고 키가 잠금 해제되면 키의 암호를 다시 입력하지 않고도 이러한 키를 사용하여 SSH 서버에 로그인할 수 있습니다. 개인 키와 암호는 계속 안전합니다.

사전 요구 사항

- SSH 데몬이 실행되고 네트워크를 통해 연결할 수 있는 원격 호스트가 있습니다.
- IP 주소 또는 호스트 이름 및 자격 증명을 통해 원격 호스트에 로그인합니다.
- 암호를 사용하여 SSH 키 쌍을 생성하고 공개 키를 원격 시스템으로 전송했습니다.
  자세한 내용은 SSH 키 쌍 생성 섹션을 참조하십시오.

절차

1. 세션에서 **ssh-agent** 를 자동으로 시작하는 명령을 **~/.bashrc** 파일에 추가합니다.

   a. 선택한 텍스트 편집기에서 **~/.bashrc** 를 엽니다. 예를 들면 다음과 같습니다.

```
$ vi ~/.bashrc
```

   b. 파일에 다음 행을 추가합니다.

```
eval $(ssh-agent)
```

   c. 변경 사항을 저장하고 편집기를 종료합니다.

2. **~/.ssh/config** 파일에 다음 행을 추가합니다.

```
AddKeysToAgent yes
```

이 옵션과 **ssh-agent** 가 세션에서 시작되면 에이전트는 호스트에 처음 연결할 때만 암호를 입력하라는 메시지를 표시합니다.

검증

- 에이전트에서 캐시된 개인 키의 해당 공개 키를 사용하는 호스트에 로그인합니다. 예를 들면 다음과 같습니다.

```
$ ssh <example.user>@<ssh-server@example.com>
```

암호를 입력할 필요가 없습니다.

5.4. 스마트 카드에 저장된 SSH 키로 인증

스마트 카드에 ECDSA 및 RSA 키를 생성 및 저장하고 OpenSSH 클라이언트의 스마트 카드로 인증할 수 있습니다. 스마트 카드 인증은 기본 암호 인증을 대체합니다.

사전 요구 사항

- 클라이언트 측에서 **opensc** 패키지가 설치되고 **pcscd** 서비스가 실행 중입니다.

절차

1. PKCS #11 URI를 포함하여 OpenSC PKCS #11 모듈에서 제공하는 모든 키를 나열하고 출력을 **keys.pub** 파일에 저장합니다.

```
$ ssh-keygen -D pkcs11: > keys.pub
```

2. 공개 키를 원격 서버로 전송합니다. 이전 단계에서 만든 **keys.pub** 파일과 함께 **ssh-copy-id** 명령을 사용합니다.

```
$ ssh-copy-id -f -i keys.pub <username@ssh-server-example.com>
```

3. ECDSA 키를 사용하여 < *ssh-server-example.com* >에 연결합니다. 키를 고유하게 참조하는 URI의 하위 집합만 사용할 수 있습니다. 예를 들면 다음과 같습니다.

```
$ ssh -i "pkcs11:id=%01?module-path=/usr/lib64/pkcs11/opensc-pkcs11.so" <ssh-server-example.com>
Enter PIN for 'SSH key':
[ssh-server-example.com] $
```

OpenSSH는 **p11-kit-proxy** 래퍼를 사용하고 OpenSC PKCS #11 모듈이 **p11-kit** 툴에 등록되므로 이전 명령을 단순화할 수 있습니다.

```
$ ssh -i "pkcs11:id=%01" <ssh-server-example.com>
Enter PIN for 'SSH key':
[ssh-server-example.com] $
```

PKCS #11 URI의 **id=** 부분을 건너뛰면 OpenSSH는 proxy 모듈에서 사용할 수 있는 모든 키를 로드합니다. 이렇게 하면 필요한 입력 횟수가 줄어듭니다.

```
$ ssh -i pkcs11: <ssh-server-example.com>
Enter PIN for 'SSH key':
[ssh-server-example.com] $
```

4. **선택 사항: ~/.ssh/config 파일에서 동일한 URI 문자열을 사용하여 구성을 영구적으로 만들 수 있습니다.**

```
$ cat ~/.ssh/config
IdentityFile "pkcs11:id=%01?module-path=/usr/lib64/pkcs11/opensc-pkcs11.so"
$ ssh <ssh-server-example.com>
Enter PIN for 'SSH key':
[ssh-server-example.com] $
```

이제 ssh 클라이언트 유틸리티에서 이 URI와 스마트 카드의 키를 자동으로 사용합니다.

추가 리소스

- **p11-kit(8), opensc.conf(5), pcscd(8), ssh(1), 및 ssh-keygen(1) 도움말 페이지**

5.5. 추가 리소스

- **sshd(8), ssh(1), scp(1), sftp(1), ssh-keygen(1), ssh-copy-id(1), ssh_config(5), sshd_config(5), update-crypto-policies(8) 및 crypto-policies(7) 도움말 페이지.**
- 비표준 구성을 사용하여 애플리케이션 및 서비스에 대한 SELinux 구성
- firewalld를 사용하여 네트워크 트래픽 제어

6장. 기본 시스템 보안 구성

컴퓨터 보안은 컴퓨터 시스템과 하드웨어, 소프트웨어, 정보 및 서비스의 도난, 손상, 중단 및 오작동으로부터 보호하는 것입니다. 특히 중요한 데이터를 처리하고 비즈니스 트랜잭션을 처리하는 기업에서 컴퓨터 보안이 필수적인 작업인지 확인합니다.

이 섹션에서는 운영 체제를 설치한 후 구성할 수 있는 기본 보안 기능만 설명합니다.

6.1. FIREWALLD 서비스 활성화

방화벽은 구성된 보안 규칙에 따라 들어오고 나가는 네트워크 트래픽을 모니터링하고 제어하는 네트워크 보안 시스템입니다. 방화벽은 일반적으로 신뢰할 수 있는 안전한 내부 네트워크와 다른 외부 네트워크 간의 장벽을 설정합니다.

Red Hat Enterprise Linux에서 방화벽을 제공하는 **firewalld** 서비스는 설치 중에 자동으로 활성화됩니다.

firewalld 서비스를 활성화하려면 다음 절차를 따르십시오.

절차

- **firewalld** 의 현재 상태를 표시합니다 :

```
$ systemctl status firewalld
● firewalld.service - firewalld - dynamic firewall daemon
   Loaded: loaded (/usr/lib/systemd/system/firewalld.service; disabled; vendor preset:
enabled)
   Active: inactive (dead)
...
```

- **firewalld** 가 활성화되어 실행되고 있지 않으면**root** 사용자로 전환하고**firewalld** 서비스를 시작하고 시스템이 다시 시작되면 자동으로 시작합니다.

```
# systemctl enable --now firewalld
```

검증

- **firewalld** 가 실행 중이고 활성화되어 있는지 확인합니다.

```
$ systemctl status firewalld
● firewalld.service - firewalld - dynamic firewall daemon
   Loaded: loaded (/usr/lib/systemd/system/firewalld.service; enabled; vendor preset:
enabled)
   Active: active (running)
...
```

추가 리소스

- firewalld 사용 및 구성
- **man firewalld(1)**

6.2. RHEL 8 웹 콘솔에서 방화벽 관리

웹 콘솔에서 **firewalld** 서비스를 구성하려면 Networking → Firewall (방화벽) 로 이동합니다.

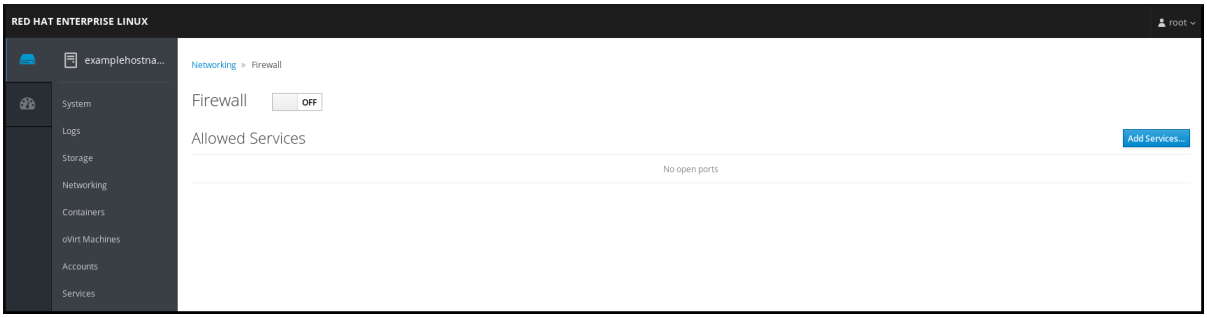
기본적으로 **firewalld** 서비스는 활성화됩니다.

사전 요구 사항

- RHEL 8 웹 콘솔을 설치했습니다.
  자세한 내용은 웹 콘솔 설치 및 활성화를 참조하십시오.

절차

1. RHEL 8 웹 콘솔에 로그인합니다.
   자세한 내용은 웹 콘솔에 로그인 을 참조하십시오.
2. 웹 콘솔에서 **firewalld** 를 활성화하거나 비활성화하려면 **Firewall** 토글 버튼을 전환합니다.



참고

또한 **Add services...**를 사용하여 방화벽을 통해 서비스에 대한 보다 세밀한 액세스를 정의할 수 있습니다. 버튼.

6.3. 기본 SELINUX 설정 관리

SELinux(Security-Enhanced Linux)는 어떤 프로세스가 어떤 파일, 디렉터리 및 포트에 액세스할 수 있는지를 결정하는 추가 시스템 보안 계층입니다. 이러한 권한은 SELinux 정책에 정의되어 있습니다. 정책은 SELinux 보안 엔진을 안내하는 규칙 집합입니다.

SELinux의 가능한 상태는 다음과 같습니다.

- 비활성화됨
- 활성화됨

SELinux가 활성화되면 다음 모드 중 하나로 실행됩니다.

- 활성화됨
  - 강제
  - 허용

강제 모드에서 SELinux는 로드된 정책을 적용합니다. SELinux는 SELinux 정책 규칙을 기반으로 액세스를 거부하고 명시적으로 허용되는 상호 작용만 활성화합니다. 강제 모드는 가장 안전한 SELinux 모드이며 설치 후 기본 모드입니다.

허용 모드에서 SELinux는 로드된 정책을 시행하지 않습니다. SELinux는 액세스를 거부하지 않지만 규칙을 **/var/log/audit/audit.log** 로그로 손상시키는 작업을 보고합니다. 허용 모드는 설치 중에 기본 모드입니다. 허용 모드는 특정 사례(예: 문제를 해결할 때 유용함)에서도 유용합니다.

추가 리소스

- SELinux 사용

6.4. RHEL 8 웹 콘솔에서 SELINUX 모드 전환

SELinux 메뉴 항목의 RHEL 8 웹 콘솔을 통해 SELinux 모드를 설정할 수 있습니다.

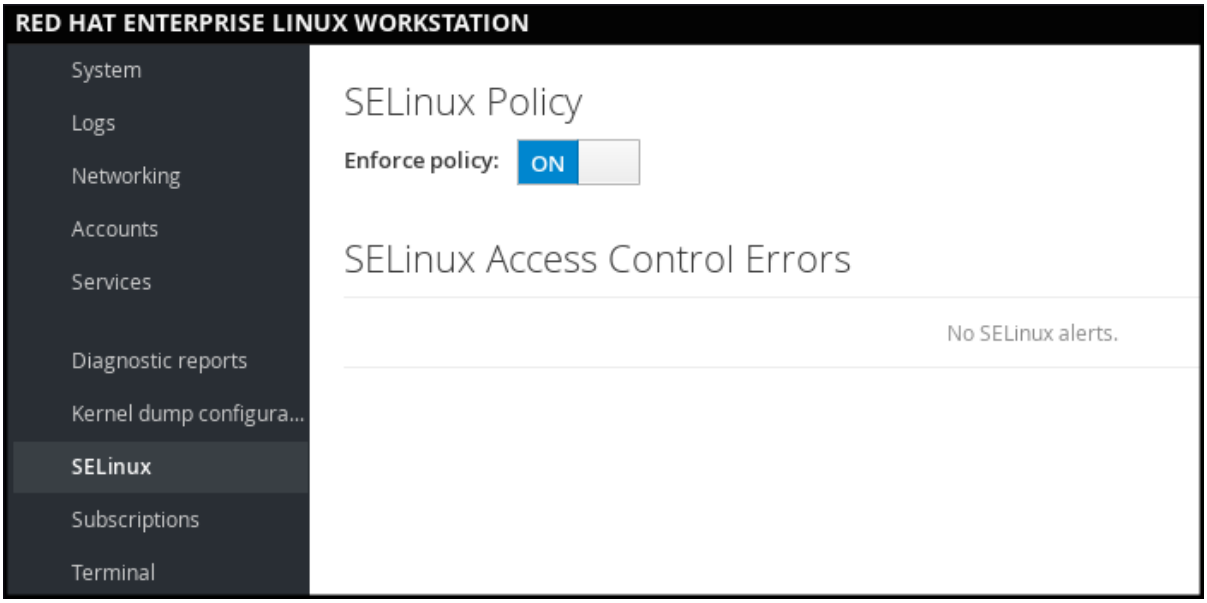
기본적으로 웹 콘솔의 SELinux 강제 정책은 이며 SELinux는 강제 모드로 작동합니다. 이를 끄면 SELinux를 허용 모드로 전환합니다. 다음 부팅 시 **/etc/sysconfig/selinux** 파일에 정의된 구성으로 이 선택 사항이 자동으로 복원됩니다.

사전 요구 사항

- RHEL 8 웹 콘솔을 설치했습니다.
  자세한 내용은 웹 콘솔 설치 및 활성화를 참조하십시오.

절차

1. RHEL 8 웹 콘솔에 로그인합니다.
   자세한 내용은 웹 콘솔에 로그인 을 참조하십시오.
2. 웹 콘솔에서 SELinux 메뉴 항목에서 **Enforce policy** toggle(강제 정책 토글) 버튼을 사용하여 SELinux 강제 정책 켜거나 끕니다.



6.5. 추가 리소스

- SSH 키 쌍 생성
- 키 기반 인증을 위한 OpenSSH 서버 설정
- 보안 강화
- SELinux 사용

- 네트워크 보안
- 여러 시스템에 동일한 SELinux 구성 배포

7장. 소프트웨어 패키지 관리

7.1. RHEL 8의 소프트웨어 관리 툴

RHEL 8에서는 DNF 기술을 기반으로 하는 새로운 버전의YUM 툴(YUM v4)을 통해 소프트웨어 설치가 활성화됩니다.

참고

업스트림 설명서는 기술을 DNF 로 식별하고 도구는 업스트림에서DNF 라고 합니다. 결과적으로 RHEL 8의 새로운 YUM 툴에서 반환한 일부 출력에는DNF 가 표시됩니다.

RHEL 8에서 사용되는 YUM v4 는 DNF 를 기반으로 하지만 RHEL 7에서 사용되는YUM v3 과 호환됩니다. 소프트웨어 설치의 경우 **yum** 명령과 대부분의 옵션은 RHEL 7에서처럼 RHEL 8에서와 동일한 방식으로 작동합니다.

선택한 yum 플러그인과 유틸리티는 새 DNF 백엔드로 포팅되었으며 RHEL 7과 동일한 이름으로 설치할 수 있습니다. 패키지는 호환성 심볼릭 링크를 제공하므로 바이너리, 구성 파일 및 디렉터리를 일반적인 위치에서 찾을 수 있습니다.

YUM v3 에서 제공하는 레거시 Python API는 더 이상 사용할 수 없습니다. 플러그인 및 스크립트를 안정적이고 완벽하게 지원되는 YUM v4 (DNF Python API)에서 제공하는 새 API로 마이그레이션할 수 있습니다. 자세한 내용은 DNF API 참조를 참조하십시오.

7.2. 애플리케이션 스트림

RHEL 8에는 Application Streams의 개념이 도입되었습니다. 이제 여러 버전의 사용자 공간 구성 요소가 핵심 운영 체제 패키지보다 더 자주 제공되고 업데이트됩니다. 이는 플랫폼 또는 특정 배포의 기본 안정성에 영향을 주지 않고 Red Hat Enterprise Linux를 사용자 지정할 수 있는 유연성을 향상시킵니다.

Application Streams로 제공되는 구성 요소는 모듈 또는 RPM 패키지로 패키징할 수 있으며 RHEL 8의 AppStream 리포지토리를 통해 제공됩니다. 각 Application Stream에는 RHEL 8과 동일하거나 짧은 라이프사이클이 있으며 특정 애플리케이션에 더 적합합니다. 라이프사이클이 짧은 Application Streams는 Red Hat Enterprise Linux 8 Application Streams 라이프 사이클 페이지에 나열되어 있습니다.

모듈은 논리 단위, 애플리케이션, 언어 스택, 데이터베이스 또는 툴 세트를 나타내는 패키지 컬렉션입니다. 이러한 패키지는 함께 빌드, 테스트, 릴리스됩니다.

모듈 스트림은 Application Stream 구성 요소의 버전을 나타냅니다. 예를 들어 postgresql 모듈에서 두 개의 스트림(버전)의 PostgreSQL 데이터베이스 서버를 사용할 수 있습니다. PostgreSQL 10(기본 스트림) 및 PostgreSQL 9.6. 시스템에는 하나의 모듈 스트림만 설치할 수 있습니다. 개별 컨테이너에서 서로 다른 버전을 사용할 수 있습니다.

자세한 모듈 명령은 사용자 공간 구성 요소 설치, 관리 및 제거문서에서 설명합니다. AppStream에서 사용 가능한 모듈 목록은 패키지 매니페스트를 참조하십시오.

7.3. 소프트웨어 패키지 검색

yum 을 사용하면 소프트웨어 패키지를 사용하여 전체 작업 세트를 수행할 수 있습니다.

다음 섹션에서는 yum 을 사용하여 다음을 수행하는 방법을 설명합니다.

- 패키지를 검색합니다.

- 패키지 나열.
- 리포지토리를 나열합니다.
- 패키지에 대한 정보를 표시합니다.
- 패키지 그룹 나열.
- yum 입력에서 전역 표현식을 지정합니다.

7.3.1. YUM을 사용하여 패키지 검색

다음 절차에 따라 특정 애플리케이션 또는 기타 콘텐츠를 제공하는 패키지를 찾으십시오.

절차

- 패키지를 검색하려면 다음을 사용합니다.

```
# yum search term
```

  *용어*를 패키지와 관련된 용어로 바꿉니다.

  yum search 명령은 패키지 이름 및 요약 내에서 일치하는 용어를 반환합니다. 그러면 검색이 더 빨라지고 이름을 모르는 패키지를 검색할 수 있지만 관련 용어를 알고 있습니다.

- 패키지 설명 내에서 일치하는 용어를 포함하려면 다음을 사용하십시오.

```
# yum search --all term
```

  *용어*를 패키지 이름, 요약 또는 설명에서 검색하려는 용어로 바꿉니다.

  yum search --all 을 사용하면 더 포괄적이지만 느린 검색을 사용할 수 있습니다.

7.3.2. YUM을 사용하여 패키지 나열

다음 절차에 따라 설치 및 사용 가능한 패키지를 나열하십시오.

절차

- 설치된 모든 패키지 및 사용 가능한 패키지에 대한 정보를 나열하려면 다음을 사용합니다.

```
# yum list --all
```

- 시스템에 설치된 모든 패키지를 나열하려면 다음을 사용합니다.

```
# yum list --installed
```

- 설치할 수 있는 모든 활성화된 리포지토리의 모든 패키지를 나열하려면 다음을 사용합니다.

```
# yum list --available
```

글로벌 표현식을 인수로 추가하여 결과를 필터링할 수 있습니다. 자세한 내용은 yum 입력에서 글로벌 표현식 지정을 참조하십시오.

7.3.3. YUM을 사용하여 리포지토리 나열

다음 절차에 따라 활성화된 리포지토리를 나열하고 비활성화합니다.

절차

- 시스템에서 활성화된 모든 리포지토리를 나열하려면 다음을 사용합니다.

```
# yum repolist
```

- 시스템에서 비활성화된 리포지토리를 모두 나열하려면 다음을 사용합니다.

```
# yum repolist --disabled
```

- 활성화된 리포지토리와 비활성화된 리포지토리를 모두 나열하려면 다음을 사용합니다.

```
# yum repolist --all
```

- 리포지토리에 대한 추가 정보를 나열하려면 다음을 사용합니다.

```
# yum repoinfo
```

리포지토리의 ID 또는 이름을 인수로 전달하거나 전역 표현식을 추가하여 결과를 필터링할 수 있습니다. 자세한 내용은 yum 입력에서 글로벌 표현식 지정을 참조하십시오.

7.3.4. YUM을 사용하여 패키지 정보 표시

YUM(버전, 릴리스, 크기, 로드된 플러그인 등)을 사용하여 패키지에 대한 다양한 유형의 정보를 표시할 수 있습니다.

절차

- 하나 이상의 패키지에 대한 정보를 표시하려면 다음을 사용합니다.

```
# yum info package-name
```

  package-name 을 패키지 이름으로 바꿉니다.

글로벌 표현식을 인수로 추가하여 결과를 필터링할 수 있습니다. 자세한 내용은 yum 입력에서 글로벌 표현식 지정을 참조하십시오.

7.3.5. YUM을 사용하여 패키지 그룹 나열

yum 을 사용하여 설치된 패키지 그룹을 보고 목록 결과를 필터링합니다.

절차

- 설치되어 사용 가능한 그룹 수를 보려면 다음을 사용합니다.

```
# yum group summary
```

- 설치되어 사용 가능한 그룹을 모두 나열하려면 다음을 사용합니다.

```
# yum group list
```

yum group list 명령(**--hidden, -- available**)에 대한 명령줄 옵션을 추가하여 결과를 필터링할 수 있습니다. 사용 가능한 옵션은 도움말 페이지를 참조하십시오.

- 특정 그룹에 포함된 필수 및 선택적 패키지를 나열하려면 다음을 사용합니다.

```
# yum group info group-name
```

group-name 을 그룹 이름으로 바꿉니다.

글로벌 표현식을 인수로 추가하여 결과를 필터링할 수 있습니다. 자세한 내용은 yum 입력에서 글로벌 표현식 지정을 참조하십시오.

7.3.6. YUM 입력에서 글로벌 표현식 지정

yum 명령을 사용하면 하나 이상의 *glob 표현식*을 인수로 추가하여 결과를 필터링할 수 있습니다. 전역 표현식을 **yum** 명령에 인수로 전달할 때 이스케이프해야 합니다.

절차

- 전역 표현식이 의도한 대로 **yum** 에 전달되도록 하려면 다음 방법 중 하나를 사용합니다.
  - 전체 글로벌 표현식을 큰따옴표 또는 단일 따옴표로 묶습니다.

```
# yum provides "*/file-name"
```

file-name 을 파일 이름으로 바꿉니다.

  - 와일드카드 문자를 백슬래시(\)문자로이스케이프합니다.

```
# yum provides \*/file-name
```

file-name 을 파일 이름으로 바꿉니다.

7.4. 소프트웨어 패키지 설치

다음 섹션에서는 yum 을 사용하여 다음을 수행하는 방법을 설명합니다.

- 패키지 설치.
- 패키지 그룹을 설치합니다.
- yum 입력으로 패키지 이름을 지정합니다.

7.4.1. YUM을 사용하여 패키지 설치

- 패키지 및 모든 패키지 종속성을 설치하려면 다음을 사용합니다.

```
# yum install package-name
```

package-name 을 패키지 이름으로 바꿉니다.

- 여러 패키지와 해당 종속 패키지를 동시에 설치하려면 다음을 사용합니다.

```
# yum install package-name-1 package-name-2
```

package-name-1 및 *package-name-2* 를 패키지 이름으로 바꿉니다.

- *멀티 라이브러리 시스템* (AMD64, Intel 64 시스템)에 패키지를 설치할 때 패키지 이름에 패키지를 추가하여 패키지의 아키텍처를 지정할 수 있습니다.

```
# yum install package-name.arch
```

package-name.arch 를 패키지의 이름 및 아키텍처로 바꿉니다.

- 설치하려는 바이너리의 이름을 알고 있지만 패키지 이름은 아닌 경우 바이너리의 경로를 인수로 사용할 수 있습니다.

```
# yum install /usr/sbin/binary-file
```

/usr/sbin/binary-file 을 바이너리 파일의 경로로 바꿉니다.

yum 은 패키지 목록을 검색하고 ***/usr/sbin/binary-file*** 을 제공하는 패키지를 찾은 다음 설치 여부를 묻는 메시지를 표시합니다.

- 로컬 디렉터리에서 이전에 다운로드된 패키지를 설치하려면 다음을 사용합니다.

```
# yum install /path/
```

/path/ 를 패키지 경로로 바꿉니다.

인수를 구문 분석하는 방법을 명시적으로 정의하여 패키지 검색을 최적화할 수 있습니다. 자세한 내용은 7.4.3절. "YUM 입력에서 패키지 이름 지정"를 참조하십시오.

7.4.2. YUM을 사용하여 패키지 그룹 설치

다음 절차에서는 **yum** 을 사용하여 그룹 이름 또는 groupID로 패키지 그룹을 설치하는 방법을 설명합니다.

절차

- 그룹 이름으로 패키지 그룹을 설치하려면 다음을 사용합니다.

```
# yum group install group-name
```

또는

```
# yum install @group-name
```

group-name 을 그룹 또는 환경 그룹의 전체 이름으로 바꿉니다.

- groupID로 패키지 그룹을 설치하려면 다음을 사용합니다.

```
# yum group install groupID
```

groupID 를 그룹의 ID로 바꿉니다.

7.4.3. YUM 입력에서 패키지 이름 지정

설치 및 제거 프로세스를 최적화하기 위해 **yum install** 및 **yum remove** 명령에 **-n**,**-na** 또는 **-nevra** 접미사를 추가하여 인수를 구문 분석하는 방법을 명시적으로 정의할 수 있습니다.

- 정확한 이름을 사용하여 패키지를 설치하려면 다음을 사용합니다.

  ```
  # yum install-n name
  ```

  name 을 정확한 패키지 이름으로 바꿉니다.

- 정확한 이름 및 아키텍처를 사용하여 패키지를 설치하려면 다음을 사용합니다.

  ```
  # yum install-na name.architecture
  ```

  이름 및 *아키텍처*를 패키지의 정확한 이름 및 아키텍처로 바꿉니다.

- 정확한 이름, epoch, 버전, 릴리스 및 아키텍처를 사용하여 패키지를 설치하려면 다음을 사용합니다.

  ```
  # yum install-nevra name-epoch:version-release.architecture
  ```

  name,*epoch*,*version*,*release* 및 *architecture* 을 정확한 이름, epoch, 버전, 릴리스 및 아키텍처로 바꿉니다.

7.5. 소프트웨어 패키지 업데이트

yum 을 사용하면 시스템에 보류 중인 업데이트가 있는지 확인할 수 있습니다. 업데이트해야 하는 패키지를 나열하고 단일 패키지, 여러 패키지 또는 모든 패키지를 한 번에 업데이트하도록 선택할 수 있습니다. 업데이트하도록 선택한 패키지에 종속성이 있는 경우 해당 패키지도 업데이트됩니다.

다음 섹션에서는 yum 을 사용하여 다음을 수행하는 방법을 설명합니다.

- 업데이트를 확인합니다.
- 단일 패키지를 업데이트합니다.
- 패키지 그룹을 업데이트합니다.
- 모든 패키지 및 해당 종속성을 업데이트합니다.
- 보안 업데이트 적용.
- 소프트웨어 업데이트 자동화.

7.5.1. YUM을 사용하여 업데이트 확인

다음 절차에서는 **yum** 을 사용하여 시스템에 설치된 패키지에 대해 사용 가능한 업데이트를 확인하는 방법을 설명합니다.

절차

- 시스템에 설치된 패키지를 확인할 수 있는 업데이트를 보려면 다음을 사용합니다.

  ```
  # yum check-update
  ```

  출력에는 사용 가능한 업데이트가 있는 패키지 목록과 해당 종속 항목이 반환됩니다.

7.5.2. YUM을 사용하여 단일 패키지 업데이트

yum 을 사용하여 단일 패키지 및 해당 종속 항목을 업데이트하려면 다음 절차를 사용하십시오.

중요

업데이트를 커널에 적용하는 경우 **yum update** 또는 **yum install** 명령을 사용하는지에 관계없이 **yum** 은 항상 새 커널을 설치합니다.

- 패키지를 업데이트하려면 다음을 사용합니다.

```
# yum update package-name
```

package-name 을 패키지 이름으로 바꿉니다.

중요

BIOS 또는 IBM Power 시스템에서 GRUB 부트 로더 패키지를 업그레이드한 경우 GRUB을 다시 설치하십시오. GRUB 재설치를 참조하십시오.

7.5.3. YUM을 사용하여 패키지 그룹 업데이트

다음 절차에 따라 **yum** 을 사용하여 패키지 그룹과 해당 종속 항목을 업데이트합니다.

절차

- 패키지 그룹을 업데이트하려면 다음을 사용합니다.

```
# yum group update group-name
```

group-name 을 패키지 그룹의 이름으로 바꿉니다.

중요

BIOS 또는 IBM Power 시스템에서 GRUB 부트 로더 패키지를 업그레이드한 경우 GRUB을 다시 설치하십시오. GRUB 재설치를 참조하십시오.

7.5.4. YUM을 사용하여 모든 패키지 및 종속 항목 업데이트

다음 절차에 따라 **yum** 을 사용하여 모든 패키지 및 해당 종속 항목을 업데이트합니다.

절차

- 모든 패키지 및 해당 종속성을 업데이트하려면 다음을 사용합니다.

```
# yum update
```

중요

BIOS 또는 IBM Power 시스템에서 GRUB 부트 로더 패키지를 업그레이드한 경우 GRUB을 다시 설치하십시오. GRUB 재설치를 참조하십시오.

7.5.5. YUM을 사용하여 보안 관련 패키지 업데이트

다음 절차에 따라 **yum** 을 사용하여 보안 에라타가 있는 사용 가능한 패키지를 업데이트합니다.

절차

- 보안 에라타가 있는 최신 패키지로 업그레이드하려면 다음을 사용합니다.

```
# yum update --security
```

- 마지막 보안 에라타 패키지로 업그레이드하려면 다음을 사용합니다.

```
# yum update-minimal --security
```

중요

BIOS 또는 IBM Power 시스템에서 GRUB 부트 로더 패키지를 업그레이드한 경우 GRUB을 다시 설치하십시오. GRUB 재설치를 참조하십시오.

7.5.6. 소프트웨어 업데이트 자동화

패키지 업데이트를 자동으로 및 정기적으로 확인하고 다운로드하려면 **dnf-automatic** 패키지에서 제공하는 DNF 자동 도구를 사용할 수 있습니다.

DNF Automatic 은 systemd 타이머, cron 작업 및 기타해당 도구를 사용하여 자동 및 일반 실행에 적합한 대체 명령줄 인터페이스입니다.

DNF Automatic 은 필요에 따라 패키지 메타데이터를 동기화한 다음 사용 가능한 업데이트를 확인합니다. 그런 다음 도구는 구성 방법에 따라 다음 작업 중 하나를 수행할 수 있습니다.

- 종료
- 업데이트된 패키지 다운로드
- 업데이트 다운로드 및 적용

그런 다음 표준 출력 또는 이메일과 같이 선택한 메커니즘에서 작업 결과를 보고합니다.

7.5.6.1. DNF 자동 설치

다음 절차에서는 DNF 자동 툴을 설치하는 방법을 설명합니다.

절차

- **dnf-automatic** 패키지를 설치하려면 다음을 사용합니다.

```
# yum install dnf-automatic
```

검증

- 설치에 성공했는지 확인하려면 다음 명령을 실행하여 **dnf-automatic** 패키지가 있는지 확인합니다.

```
# rpm -qi dnf-automatic
```

7.5.6.2. DNF 자동 구성 파일

기본적으로 DNF 자동에서는 **/etc/dnf/automatic.conf** 를 구성 파일로 사용하여 동작을 정의합니다.

구성 파일은 다음 주제 섹션으로 구분됩니다.

- **[명령]** 섹션
  DNF 자동의 작동 모드를 설정합니다.
- **[emitters]** 섹션
  DNF 자동의 결과를 보고하는 방법을 정의합니다.
- **[command_email]** 섹션
  이메일을 보내는 데 사용되는 외부 명령의 이메일 발송자 구성을 제공합니다.
- **[이메일]** 섹션
  전자 메일 발송자 구성을 제공합니다.
- **[base]** 섹션
  yum의 기본 구성 파일의 설정을 재정의합니다.

/etc/dnf/automatic.conf 파일의 기본 설정을 사용하여DNF 자동 검사에서 사용 가능한 업데이트를 확인하고 다운로드한 다음 결과를 표준 출력으로 보고합니다.

> 주의
>
> **[commands]** 섹션에서 작업 모드 설정은**dnf-automatic.timer** 을 제외한 모든 타이머 장치에 대해 systemd 타이머 장치에서 사용하는 설정으로 재정의됩니다.

추가 리소스

- 특정 섹션에 대한 자세한 내용은 DNF 자동 설명서를 참조하십시오.
- systemd 타이머 장치에 대한 자세한 내용은 **man dnf-automatic** 도움말 페이지를 참조하십시오.
- **dnf-automatic** 패키지에 포함된 systemd 타이머 단위의 개요는 dnf-automatic 패키지에 포함된systemd 타이머 단위의 섹션 개요를참조하십시오.

7.5.6.3. DNF 자동 활성화

DNF 자동을 실행하려면 항상 특정 systemd 타이머 장치를 활성화하고 시작해야 합니다.**dnf-automatic** 패키지에 제공된 타이머 장치 중 하나를 사용하거나 필요에 따라 고유한 타이머 장치를 작성할 수 있습니다.

다음 섹션에서는 DNF 자동 활성화 방법을 설명합니다.

사전 요구 사항

- **/etc/dnf/automatic.conf** 구성 파일을 수정하여 DNF 자동 동작을 지정했습니다.

DNF 자동 구성 파일에 대한 자세한 내용은 섹션 2.5.6.2, "DNF 자동 구성 파일"을 참조하십시오.

절차

- 필요에 맞는 systemd 타이머 장치를 선택, 활성화 및 시작합니다.

```
# systemctl enable --now <unit>
```

여기서 **<unit>** 은 다음 타이머 중 하나입니다.

- **dnf-automatic-download.timer**
- **dnf-automatic-install.timer**
- **dnf-automatic-notifyonly.timer**
- **dnf-automatic.timer**
  - 사용 가능한 업데이트를 다운로드하려면 다음을 사용하십시오.

```
# systemctl enable dnf-automatic-download.timer
# systemctl start dnf-automatic-download.timer
```

  - 사용 가능한 업데이트를 다운로드하고 설치하려면 다음을 사용합니다.

```
# systemctl enable dnf-automatic-install.timer
# systemctl start dnf-automatic-install.timer
```

  - 사용 가능한 업데이트에 대해 보고 하려면 다음을 사용합니다.

```
# systemctl enable dnf-automatic-notifyonly.timer
# systemctl start dnf-automatic-notifyonly.timer
```

  - 선택적으로 다음을 사용할 수 있습니다.

```
# systemctl enable dnf-automatic.timer
# systemctl start dnf-automatic.timer
```

업데이트를 다운로드하고 적용하는 측면에서 본 타이머 장치는 **/etc/dnf/automatic.conf** 구성 파일의 설정에 따라 작동합니다. 기본 동작은 **dnf-automatic-download.timer**: 업데이트된 패키지를 다운로드하지만 설치하지는 않습니다.

참고

또는 명령줄 또는 사용자 지정 스크립트에서 직접 **/usr/bin/dnf-automatic** 파일을 실행하여 DNF 자동 을 실행할 수도 있습니다.

검증

- 타이머가 활성화되었는지 확인하려면 다음 명령을 실행합니다.

```
# systemctl status <systemd timer unit>
```

추가 리소스

- dnf-automatic 타이머에 대한 자세한 내용은 **man dnf-automatic** 매뉴얼 페이지를 참조하십시오.

- dnf-automatic 패키지에 포함된 systemd 타이머 장치의 개요는 **dnf-automatic** 패키지에 포함된 systemd 타이머 단위의 섹션 개요를참조하십시오.

7.5.6.4. dnf-automatic 패키지에 포함된 systemd 타이머 단위 개요

systemd 타이머 단위는 업데이트를 다운로드하고 적용하는 데 관한 **/etc/dnf/automatic.conf** 구성 파일의 설정을 우선하고 재정의합니다.

예를 들어 **/etc/dnf/automatic.conf** 구성 파일에서 다음 옵션을 설정했지만**dnf-automatic-notifyonly.timer** 장치를 활성화한 경우 패키지는 다운로드되지 않습니다.

```
download_updates = yes
```

dnf-automatic 패키지에는 다음과 같은 systemd 타이머 장치가 포함되어 있습니다.

타이머 단위	함수	/etc/dnf/automatic.conf 파일의 설정을 재정의합니까?
dnf-automatic-download.timer	캐시할 패키지를 다운로드하고 업데이트할 수 있도록 합니다. 참고: 이 타이머 장치는 업데이트된 패키지를 설치하지 않습니다. 설치를 수행하려면 **dnf update** 명령을 실행해야 합니다.	있음
dnf-automatic-install.timer	업데이트된 패키지를 다운로드하고 설치합니다.	있음
dnf-automatic-notifyonly.timer	리포지토리 데이터만 다운로드하여 리포지토리 캐시를 최신 상태로 유지하고 사용 가능한 업데이트에 대해 알립니다. 참고: 이 타이머 장치는 업데이트된 패키지를 다운로드하거나 설치하지 않습니다	있음
dnf-automatic.timer	업데이트를 다운로드하고 적용하기 위한 이 타이머의 동작은 **/etc/dnf/automatic.conf** 구성 파일의 설정에 의해 지정됩니다. 기본 동작은 **dnf-automatic-download.timer** 단위와 동일합니다. 패키지만 다운로드하지만 설치하지는 않습니다.	없음

추가 리소스

- **dnf-automatic** 타이머에 대한 자세한 내용은**man dnf-automatic** 매뉴얼 페이지를 참조하십시오.

- **/etc/dnf/automatic.conf** 구성 파일에 대한 자세한 내용은 섹션DNF 자동 구성 파일을참조하십시오.

7.6. 소프트웨어 패키지 제거

다음 섹션에서는 yum 을 사용하여 다음을 수행하는 방법을 설명합니다.

- 패키지 제거.
- 패키지 그룹을 제거합니다.
- yum 입력으로 패키지 이름을 지정합니다.

7.6.1. YUM을 사용하여 패키지 제거

다음 절차에 따라 그룹 이름 또는 groupID로 패키지를 제거합니다.

절차

- 특정 패키지 및 모든 종속 패키지를 제거하려면 다음을 사용합니다.

  ```
  # yum remove package-name
  ```

  package-name 을 패키지 이름으로 바꿉니다.

- 여러 패키지와 해당 종속 항목을 동시에 제거하려면 다음을 사용합니다.

  ```
  # yum remove package-name-1 package-name-2
  ```

  package-name-1 및 *package-name-2* 를 패키지 이름으로 바꿉니다.

참고

yum 은 패키지를 제거하지 않고 패키지를 제거할 수 없습니다.

인수를 구문 분석하는 방법을 명시적으로 정의하여 패키지 검색을 최적화할 수 있습니다. 자세한 내용은 yum input에서 패키지 이름 지정을 참조하십시오.

7.6.2. YUM을 사용하여 패키지 그룹 제거

다음 절차에 따라 그룹 이름 또는 groupID로 패키지를 제거합니다.

절차

- 그룹 이름으로 패키지 그룹을 제거하려면 다음을 사용합니다.

  ```
  # yum group remove group-name
  ```

  또는

  ```
  # yum remove @group-name
  ```

  group-name 을 그룹의 전체 이름으로 바꿉니다.

- groupID로 패키지 그룹을 제거하려면 다음을 사용합니다.

  ```
  # yum group remove groupID
  ```

  groupID 를 그룹의 ID로 바꿉니다.

7.6.3. YUM 입력에서 패키지 이름 지정

설치 및 제거 프로세스를 최적화하기 위해 **yum install** 및 **yum remove** 명령에 **-n**,**-na** 또는 **-nevra** 접미사를 추가하여 인수를 구문 분석하는 방법을 명시적으로 정의할 수 있습니다.

- 정확한 이름을 사용하여 패키지를 설치하려면 다음을 사용합니다.

  ```
  # yum install-n name
  ```

  name 을 정확한 패키지 이름으로 바꿉니다.

- 정확한 이름 및 아키텍처를 사용하여 패키지를 설치하려면 다음을 사용합니다.

  ```
  # yum install-na name.architecture
  ```

  이름 및 *아키텍처*를 패키지의 정확한 이름 및 아키텍처로 바꿉니다.

- 정확한 이름, epoch, 버전, 릴리스 및 아키텍처를 사용하여 패키지를 설치하려면 다음을 사용합니다.

  ```
  # yum install-nevra name-epoch:version-release.architecture
  ```

  name,*epoch*,*version*,*release* 및 *architecture* 을 정확한 이름, epoch, 버전, 릴리스 및 아키텍처로 바꿉니다.

7.7. 소프트웨어 패키지 그룹 관리

패키지 그룹은 공통 목적을 제공하는 패키지 컬렉션입니다(시스템 도구, 사운드 및 비디오). 패키지 그룹을 설치하면 종속 패키지 집합이 가져와 시간을 크게 절약할 수 있습니다.

다음 섹션에서는 yum 을 사용하여 다음을 수행하는 방법을 설명합니다.

- 패키지 그룹 나열.
- 패키지 그룹을 설치합니다.
- 패키지 그룹을 제거합니다.
- yum 입력에서 전역 표현식을 지정합니다.

7.7.1. YUM을 사용하여 패키지 그룹 나열

yum 을 사용하여 설치된 패키지 그룹을 보고 목록 결과를 필터링합니다.

절차

- 설치되어 사용 가능한 그룹 수를 보려면 다음을 사용합니다.

```
# yum group summary
```

- 설치되어 사용 가능한 그룹을 모두 나열하려면 다음을 사용합니다.

```
# yum group list
```

yum group list 명령(**--hidden, -- available**)에 대한 명령줄 옵션을 추가하여 결과를 필터링할 수 있습니다. 사용 가능한 옵션은 도움말 페이지를 참조하십시오.

- 특정 그룹에 포함된 필수 및 선택적 패키지를 나열하려면 다음을 사용합니다.

```
# yum group info group-name
```

group-name 을 그룹 이름으로 바꿉니다.

글로벌 표현식을 인수로 추가하여 결과를 필터링할 수 있습니다. 자세한 내용은 yum 입력에서 글로벌 표현식 지정을 참조하십시오.

7.7.2. YUM을 사용하여 패키지 그룹 설치

다음 절차에서는 **yum** 을 사용하여 그룹 이름 또는 groupID로 패키지 그룹을 설치하는 방법을 설명합니다.

절차

- 그룹 이름으로 패키지 그룹을 설치하려면 다음을 사용합니다.

```
# yum group install group-name
```

또는

```
# yum install @group-name
```

group-name 을 그룹 또는 환경 그룹의 전체 이름으로 바꿉니다.

- groupID로 패키지 그룹을 설치하려면 다음을 사용합니다.

```
# yum group install groupID
```

groupID 를 그룹의 ID로 바꿉니다.

7.7.3. YUM을 사용하여 패키지 그룹 제거

다음 절차에 따라 그룹 이름 또는 groupID로 패키지를 제거합니다.

절차

- 그룹 이름으로 패키지 그룹을 제거하려면 다음을 사용합니다.

```
# yum group remove group-name
```

또는

```
# yum remove @group-name
```

group-name 을 그룹의 전체 이름으로 바꿉니다.

- groupID로 패키지 그룹을 제거하려면 다음을 사용합니다.

```
# yum group remove groupID
```

groupID 를 그룹의 ID로 바꿉니다.

7.7.4. YUM 입력에서 글로벌 표현식 지정

yum 명령을 사용하면 하나 이상의 *glob 표현식*을 인수로 추가하여 결과를 필터링할 수 있습니다. 전역 표현식을 **yum** 명령에 인수로 전달할 때 이스케이프해야 합니다.

절차

- 전역 표현식이 의도한 대로 **yum** 에 전달되도록 하려면 다음 방법 중 하나를 사용합니다.
  - 전체 글로벌 표현식을 큰따옴표 또는 단일 따옴표로 묶습니다.

    ```
    # yum provides "*/file-name"
    ```

    file-name 을 파일 이름으로 바꿉니다.
  - 와일드카드 문자를 백슬래시(\)문자로이스케이프합니다.

    ```
    # yum provides \*/file-name
    ```

    file-name 을 파일 이름으로 바꿉니다.

7.8. 패키지 관리 내역 처리

yum history 명령을 사용하면 yum 트랜잭션의 타임라인, 발생한 날짜 및 시간, 영향을 받는 패키지 수, 이러한 트랜잭션의 성공 또는 중단 여부, 트랜잭션 간에 RPM 데이터베이스 변경에 대한 정보를 검토할 수 있습니다. 또한 **yum history** 명령을 사용하여 트랜잭션을 실행 취소하거나 다시 실행할 수 있습니다.

다음 섹션에서는 yum 을 사용하여 다음을 수행하는 방법을 설명합니다.

- 트랜잭션 나열.
- 트랜잭션 되돌리기.
- 트랜잭션 반복.
- yum 입력에서 전역 표현식을 지정합니다.

7.8.1. YUM을 사용하여 트랜잭션 나열

다음 절차에 따라 최신 트랜잭션, 선택한 패키지에 대한 최신 작업 및 특정 트랜잭션의 세부 정보를 나열합니다.

절차

- 최신 yum 트랜잭션 목록을 표시하려면 다음을 사용합니다.

```
# yum history
```

- 선택한 패키지에 대한 모든 최신 작업 목록을 표시하려면 다음을 사용합니다.

```
# yum history list package-name
```

package-name 을 패키지 이름으로 바꿉니다. 전역 표현식을 추가하여 명령 출력을 필터링할 수 있습니다. 자세한 내용은 yum 입력에서 전역 표현식 지정을 참조하십시오.

- 특정 트랜잭션을 검사하려면 다음을 사용합니다.

```
# yum history info transactionID
```

transactionID 를 트랜잭션의 ID로 바꿉니다.

7.8.2. YUM을 사용하여 트랜잭션 복원

다음 절차에서는 **yum** 을 사용하여 선택한 트랜잭션 또는 마지막 트랜잭션을 되돌리는 방법을 설명합니다.

절차

- 특정 트랜잭션을 되돌리려면 다음을 사용합니다.

```
# yum history undo transactionID
```

transactionID 를 트랜잭션의 ID로 바꿉니다.

- 마지막 트랜잭션을 되돌리려면 다음을 사용합니다.

```
# yum history undo last
```

yum history undo 명령은 트랜잭션 중에 수행된 단계만 되돌립니다. 트랜잭션이 새 패키지를 설치한 경우 **yum history undo** 명령은 이를 제거합니다. 트랜잭션에서 패키지를 제거한 경우**yum history undo** 명령으로 다시 설치합니다. **yum history**는 이전 패키지를 사용할 수 있는 경우 업데이트된 모든 패키지를 이전 버전으로 다운그레이드하려고 시도합니다.

7.8.3. YUM을 사용하여 트랜잭션 반복

다음 절차에 따라 **yum** 을 사용하여 선택한 트랜잭션 또는 마지막 트랜잭션을 반복합니다.

절차

- 특정 트랜잭션을 반복하려면 다음을 사용합니다.

```
# yum history redo transactionID
```

transactionID 를 트랜잭션의 ID로 바꿉니다.

- 마지막 트랜잭션을 반복하려면 다음을 사용합니다.

```
# yum history redo last
```

yum history redo 명령은 트랜잭션 중에 수행된 단계만 반복합니다.

7.8.4. YUM 입력에서 글로벌 표현식 지정

yum 명령을 사용하면 하나 이상의 *glob 표현식*을 인수로 추가하여 결과를 필터링할 수 있습니다. 전역 표현식을 **yum** 명령에 인수로 전달할 때 이스케이프해야 합니다.

절차

- 전역 표현식이 의도한 대로 **yum** 에 전달되도록 하려면 다음 방법 중 하나를 사용합니다.
  - 전체 글로벌 표현식을 큰따옴표 또는 단일 따옴표로 묶습니다.

    ```
    # yum provides "*/file-name"
    ```

    file-name 을 파일 이름으로 바꿉니다.
  - 와일드카드 문자를 백슬래시(\)문자로이스케이프합니다.

    ```
    # yum provides \*/file-name
    ```

    file-name 을 파일 이름으로 바꿉니다.

7.9. 소프트웨어 리포지토리 관리

yum 및 관련 유틸리티의 구성 정보는/etc/yum.conf 파일에 저장됩니다. 이 파일에는 리포지토리별 옵션을 설정할 수 있는 하나 이상의 [*repository*] 섹션이 포함되어 있습니다.

/etc/yum.repos .d/ 디렉터리에 새 또는 기존.repo 파일에 개별 리포지토리를 정의하는 것이 좋습니다.

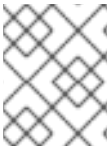
/etc/yum.conf 파일의 개별 [*repository*] 섹션에 정의된 값은[main] 섹션에 설정된 값을 재정의합니다.

다음 섹션에서는 다음을 수행하는 방법을 설명합니다.

- [*repository*] 옵션을 설정합니다.
- yum 리포지토리를 추가합니다.
- yum 리포지토리를 활성화합니다.
- yum 리포지토리를 비활성화합니다.

7.9.1. YUM 리포지토리 옵션 설정

/etc/yum.conf 구성 파일에는[*repository*] 섹션이 포함되어 있습니다. 여기서 *리포지토리* 는 고유한 리포지토리 ID입니다. [*repository*] 섹션을 통해 개별yum 리포지토리를 정의할 수 있습니다.



참고

충돌을 피하기 위해 Red Hat 리포지토리에서 사용하는 사용자 정의 리포지토리 이름을 지정하지 마십시오.

사용 가능한 [*repository*] 옵션의 전체 목록은yum.conf(5) 도움말 페이지의 [*repository*] **OPTIONS** 섹션을 참조하십시오.

7.9.2. YUM 리포지토리 추가

절차

새 리포지토리를 정의하려면 다음을 수행할 수 있습니다.

- **[*repository*]** 섹션을 **/etc/yum.conf** 파일에 추가합니다.
- **[*repository*]** 섹션을 **/etc/yum .repos.d/** 디렉터리의. **repo** 파일에 추가합니다.
  Yum 리포지토리는 일반적으로 고유한.**repo** 파일을 제공합니다.

참고

/etc/yum.conf 대신 **.repo** 파일에 있는 리포지토리를 이 디렉토리의. **repo** 파일 확장자가 있는 모든 파일은 yum 으로 정의하는 것이 좋습니다.

- 시스템에 리포지터리를 추가하고 활성화하려면 다음을 사용합니다.

```
# yum-config-manager --add-repo repository_URL
```

repository_url 을 리포지토리를 가리키는 URL로 바꿉니다.

주의

Red Hat Certificate-basedCDN(**Content Delivery Network**) 이외의 검증 또는 신뢰할 수 없는 소스에서 소프트웨어 패키지를 구입하고 설치하면 잠재적인 보안 위험이 되며 보안, 안정성, 호환성 및 유지 관리 문제로 이어질 수 있습니다.

7.9.3. YUM 리포지토리 활성화

yum 리포지토리를 시스템에 추가한 후 이를 활성화하여 설치 및 업데이트를 확인합니다.

절차

- 리포지터리를 활성화하려면 다음을 사용합니다.

```
# yum-config-manager --enable repositoryID
```

*repositoryID*를 고유한 리포지토리 ID로 바꿉니다.

사용 가능한 리포지토리 ID를 나열하려면 yum을 사용하여 패키지 나열 을 참조하십시오.

7.9.4. YUM 리포지토리 비활성화

특정 YUM 리포지토리를 비활성화하여 특정 패키지가 설치 또는 업데이트되지 않도록 합니다.

절차

- yum 리포지토리를 비활성화하려면 다음을 사용합니다.

```
# yum-config-manager --disable repositoryID
```

*repositoryID*를 고유한 리포지토리 ID로 바꿉니다.

사용 가능한 리포지토리 ID를 나열하려면 yum을 사용하여 패키지 나열 을 참조하십시오.

7.10. YUM 구성

yum 및 관련 유틸리티의 구성 정보는/etc/yum.conf 파일에 저장됩니다. 이 파일에는 필수[main] 섹션 하나가 포함되어 있어 글로벌 효과가 있는 yum 옵션을 설정할 수 있습니다.

다음 섹션에서는 다음을 수행하는 방법을 설명합니다.

- 현재 yum 구성을 봅니다.
- yum [main] 옵션을 설정합니다.
- yum 플러그인 사용.

7.10.1. 현재 YUM 구성 보기

현재 yum 구성을 보려면 다음 절차를 사용하십시오.

절차

- /etc/ yum.conf 파일의 [main] 섹션에 지정된 글로벌 yum 옵션의 현재 값을 표시하려면 다음을 사용합니다.

```
# yum config-manager --dump
```

7.10.2. YUM 기본 옵션 설정

/etc/yum.conf 구성 파일에는[main] 섹션 하나가 포함되어 있습니다. 아래에 나열된 키-값 쌍은yum 이 리포지토리를 작동하고 처리하는 방식에 영향을 미칩니다.

/etc/yum.conf 의 [main] 섹션 제목 아래에 추가 옵션을 추가할 수 있습니다.

사용 가능한 [main] 옵션의 전체 목록은yum.conf(5) 도움말 페이지의 [main] OPTIONS 섹션을 참조하십시오.

7.10.3. YUM 플러그인 사용

yum 은 운영을 확장하고 개선하는 플러그인을 제공합니다. 특정 플러그인은 기본적으로 설치됩니다.

다음 섹션에서는 yum 플러그인을 활성화, 구성 및 비활성화하는 방법을 설명합니다.

7.10.3.1. YUM 플러그인 관리

절차

플러그인 구성 파일에는 항상 yum 명령을 실행할 때 플러그인이활성화되는 지 여부를 제어하는[main] 섹션이 있습니다. 이 옵션이 없는 경우 파일에 수동으로 추가할 수 있습니다.

설치된 모든 플러그인에는 **/etc/dnf/plugins/** 디렉터리에 자체 구성 파일이 있습니다. 이러한 파일에서 플러그인별 옵션을 활성화하거나 비활성화할 수 있습니다.

7.10.3.2. YUM 플러그인 활성화

다음 절차에서는 모든 YUM 플러그인을 비활성화하거나 활성화하거나 특정 명령의 모든 플러그인을 비활성화하거나 단일 명령의 특정 YUM 플러그인을 비활성화하는 방법을 설명합니다.

절차

- 모든 yum 플러그인을 활성화하려면 다음을 수행합니다.
  1. **plugins=** 로 시작하는 행이 **/etc/yum.conf** 파일의 **[main]** 섹션에 있는지 확인합니다.
  2. **plugins=** 의 값을 **1** 로 설정합니다.

     ```
     plugins=1
     ```

7.10.3.3. YUM 플러그인 비활성화

- 모든 yum 플러그인을 비활성화하려면 다음을 수행합니다.
  1. **plugins=** 로 시작하는 행이 **/etc/yum.conf** 파일의 **[main]** 섹션에 있는지 확인합니다.
  2. **plugins=** 의 값을 **0** 으로 설정합니다.

     ```
     plugins=0
     ```

     중요

     모든 플러그인을 비활성화하는 것은 바람직하지 않습니다. 특정 플러그인은 중요한 yum 서비스를 제공합니다. 특히 product-id 및 subscription-manager 플러그인은 인증서 기반CDN(**Content Delivery Network**)에 대한 지원을 제공합니다. 플러그인을 전역적으로 비활성화하는 것은 편의성 옵션으로 제공되며 yum 에서 잠재적인 문제를 진단할 때만 권장됩니다.

- 특정 명령에 대해 모든 yum 플러그인을 비활성화하려면 명령에 **--noplugins** 옵션을 추가합니다.

  ```
  # yum --noplugins update
  ```

- 단일 명령에 특정 yum 플러그인을 비활성화하려면 명령에 **--disableplugin=*plugin-name*** 옵션을 추가합니다.

  ```
  # yum update --disableplugin=plugin-name
  ```

  plugin-name 을 플러그인 이름으로 바꿉니다.

8장. RHEL 시스템 역할 소개

RHEL 시스템 역할을 사용하면 RHEL의 주요 버전에서 여러 RHEL 시스템의 시스템 구성을 원격으로 관리할 수 있습니다.

중요한 용어 및 개념

다음은 Ansible 환경의 중요한 용어 및 개념을 설명합니다.

제어 노드

제어 노드는 Ansible 명령 및 플레이북을 실행하는 시스템입니다. 제어 노드는 Ansible Automation Platform, Red Hat Satellite 또는 RHEL 9, 8 또는 7 호스트일 수 있습니다. 자세한 내용은 RHEL 8에서 제어 노드 준비를 참조하십시오.

관리형 노드

관리형 노드는 Ansible로 관리하는 서버 및 네트워크 장치입니다. 관리 노드를 호스트라고도 합니다. Ansible을 관리형 노드에 설치할 필요가 없습니다. 자세한 내용은 관리 노드 준비를 참조하십시오.

Ansible 플레이북

플레이북에서는 관리형 노드에서 수행할 구성을 정의하거나 수행할 관리 노드의 시스템의 단계 집합을 정의합니다. 플레이북은 Ansible의 구성, 배포 및 오케스트레이션 언어입니다.

인벤토리

인벤토리 파일에서 관리 노드를 나열하고 각 관리 노드의 IP 주소와 같은 정보를 지정합니다. 인벤토리에서는 더 쉽게 스케일링할 수 있도록 그룹을 생성하고 중첩하여 관리 노드를 구성할 수도 있습니다. 인벤토리 파일을 호스트 파일이라고도 합니다.

Red Hat Enterprise Linux 8 제어 노드에서 사용 가능한 역할

Red Hat Enterprise Linux 8 제어 노드에서 **rhel-system-roles** 패키지는 다음 역할을 제공합니다.

역할 이름	역할 설명	장 제목
certificate	인증서 발행 및 갱신	RHEL 시스템 역할을 사용하여 인증서 요청
cockpit	웹 콘솔	cockpit RHEL 시스템 역할을 사용하여 웹 콘솔 설치 및 구성
crypto_policies	시스템 전체 암호화 정책	시스템 전체에서 사용자 정의 암호화 정책 설정
방화벽	firewalld	시스템 역할을 사용하여 firewalld 구성
ha_cluster	HA 클러스터	시스템 역할을 사용하여 고가용성 클러스터 구성
kdump	커널 덤프	RHEL 시스템 역할을 사용하여 kdump 구성
kernel_settings	커널 설정	Ansible 역할을 사용하여 커널 매개변수를 영구적으로 구성
logging	로깅	로깅 시스템 역할 사용
메트릭	메트릭 (PCP)	RHEL 시스템 역할을 사용하여 성능 모니터링

역할 이름	역할 설명	장 제목
microsoft.sql.server	Microsoft SQL Server	Cryostat.sql.server Ansible 역할을 사용하여 Microsoft SQL Server 구성
network	네트워킹	네트워크 RHEL 시스템 역할을 사용하여 InfiniBand 연결 관리
nbde_client	네트워크 Bound 디스크 암호화 클라이언트	nbde_client 및 nbde_server 시스템 역할 사용
nbde_server	네트워크 Bound 디스크 암호화 서버	nbde_client 및 nbde_server 시스템 역할 사용
postfix	Postfix	시스템 역할에서 postfix 역할의 변수
postgresql	PostgreSQL	postgresql RHEL 시스템 역할을 사용하여 PostgreSQL 설치 및 구성
selinux	SELinux	시스템 역할을 사용하여 SELinux 구성
ssh	SSH 클라이언트	ssh 시스템 역할과 보안 통신 구성
sshd	SSH 서버	ssh 시스템 역할과 보안 통신 구성
storage	스토리지	RHEL 시스템 역할을 사용하여 로컬 스토리지 관리
tlog	터미널 세션 레코딩	tlog RHEL 시스템 역할을 사용하여 세션 레코딩 시스템 구성
timesync	시간 동기화	RHEL 시스템 역할을 사용하여 시간 동기화 구성
vpn	VPN	vpn RHEL 시스템 역할을 사용하여 IPsec으로 VPN 연결 구성

추가 리소스

- RHEL 시스템 역할을 사용하여 시스템 관리 자동화
- RHEL(Red Hat Enterprise Linux) 시스템 역할
- **/usr/share/ansible/roles/rhel-system-roles. <*role_name*> /README.md** 파일
- **/usr/share/doc/rhel-system-roles/<*role_name*>/** directory

9장. 로깅 구성

Red Hat Enterprise Linux의 대부분의 서비스는 상태 메시지, 경고 및 오류를 기록합니다. **rsyslogd** 서비스를 사용하여 이러한 항목을 로컬 파일 또는 원격 로깅 서버에 기록할 수 있습니다.

9.1. 원격 로깅 솔루션 구성

환경에서 다양한 시스템의 로그가 로깅 서버에 중앙에 기록되도록 하기 위해 클라이언트 시스템에서 서버에 대한 특정 기준에 맞는 로그를 기록하도록 Rsyslog 애플리케이션을 구성할 수 있습니다.

9.1.1. Rsyslog 로깅 서비스

systemd-journald 서비스와 함께 Rsyslog 애플리케이션은 Red Hat Enterprise Linux에서 로컬 및 원격 로깅 지원을 제공합니다. **rsyslogd** 데몬은 journal에서 **systemd-journald** 서비스에서 수신한 **syslog** 메시지를 지속적으로 읽습니다. **rsyslogd** 는 이러한 **syslog** 이벤트를 필터링 및 처리하고 **rsyslog** 로그 파일에 기록하거나 구성에 따라 다른 서비스에 전달합니다.

rsyslogd 데몬은 또한 확장된 필터링, 메시지의 암호화 보호 릴레이, 입력 및 출력 모듈, TCP 및 UDP 프로토콜을 사용하여 운송을 지원합니다.

rsyslog의 기본 구성 파일인 **/etc/rsyslog.conf** 에서 메시지를 처리하는 **rsyslog d** 에 따라 규칙을 지정할 수 있습니다. 일반적으로 소스 및 주제(유효성) 및 긴급성(우선 순위)에 따라 메시지를 분류한 다음, 메시지가 이러한 기준에 맞는 경우 수행해야 하는 작업을 할당할 수 있습니다.

/etc/rsyslog.conf 에서 **rsyslogd** 가 유지 관리하는 로그 파일 목록을 볼 수도 있습니다. 대부분의 로그 파일은 **/var/log/** 디렉토리에 있습니다. **httpd** 및 **samba** 와 같은 일부 애플리케이션은 **/var/log/** 내의 하위 디렉터리에 로그 파일을 저장합니다.

추가 리소스

- **rsyslogd(8)** 및 **rsyslog.conf(5)** 도움말 페이지.
- **/usr/share/doc/rsyslog/html/index.html** 파일의 **rsyslog-doc** 패키지와 함께 설치된 설명서입니다.

9.1.2. Rsyslog 문서 설치

Rsyslog 애플리케이션에는 https://www.rsyslog.com/doc/ 에서 제공되는 광범위한 온라인 문서가 있지만, **rsyslog-doc** 설명서 패키지를 로컬로 설치할 수도 있습니다.

사전 요구 사항

- 시스템에서 **AppStream** 리포지토리를 활성화했습니다.
- **sudo** 를 사용하여 새 패키지를 설치할 권한이 있습니다.

절차

- **rsyslog-doc** 패키지를 설치합니다.

```
# yum install rsyslog-doc
```

검증

- 선택한 브라우저에서 **/usr/share/doc/rsyslog/html/index.html** 파일을 엽니다. 예를 들면 다음과 같습니다.

```
$ firefox /usr/share/doc/rsyslog/html/index.html &
```

9.1.3. TCP를 통한 원격 로깅을 위한 서버 구성

Rsyslog 애플리케이션을 사용하면 로깅 서버를 실행하고 로그 파일을 로깅 서버로 보내도록 개별 시스템을 구성할 수 있습니다. TCP를 통해 원격 로깅을 사용하려면 서버와 클라이언트를 둘 다 구성합니다. 서버는 하나 이상의 클라이언트 시스템에서 전송한 로그를 수집하고 분석합니다.

Rsyslog 애플리케이션을 사용하면 로그 메시지가 네트워크를 통해 서버로 전달되는 중앙 집중식 로깅 시스템을 유지 관리할 수 있습니다. 서버를 사용할 수 없는 경우 메시지 손실을 방지하기 위해 전달 작업에 대한 작업 대기열을 구성할 수 있습니다. 이렇게 하면 전송에 실패한 메시지는 서버에 다시 연결할 때까지 로컬에 저장됩니다. 이러한 큐는 UDP 프로토콜을 사용하여 연결에 대해 구성할 수 없습니다.

omfwd 플러그인은 UDP 또는 TCP를 통해 전달을 제공합니다. 기본 프로토콜은 UDP입니다. 플러그인이 빌드되었으므로 로드할 필요가 없습니다.

기본적으로 **rsyslog** 는 포트 **514** 에서 TCP를 사용합니다.

사전 요구 사항

- rsyslog가 서버 시스템에 설치되어 있습니다.
- 서버에서 **root** 로 로그인했습니다.
- **semanage** 명령을 사용하여 선택적 단계를 위해 **policycoreutils-python-utils** 패키지가 설치됩니다.
- **firewalld** 서비스가 실행 중입니다.

절차

1. 선택 사항: **rsyslog** 트래픽에 다른 포트를 사용하려면 **syslogd_port_t** SELinux 유형을 포트에 추가합니다. 예를 들어 포트 **30514** 를 활성화합니다.

```
# semanage port -a -t syslogd_port_t -p tcp 30514
```

2. 선택 사항: **rsyslog** 트래픽에 다른 포트를 사용하려면 해당 포트에서 들어오는 **rsyslog** 트래픽을 허용하도록 **firewalld** 를 구성합니다. 예를 들어 포트 **30514** 에서 TCP 트래픽을 허용합니다.

```
# firewall-cmd --zone=<zone-name> --permanent --add-port=30514/tcp
success
# firewall-cmd --reload
```

3. **/etc/rsyslog.d/** 디렉토리(예: **remotelog.conf**)에 새 파일을 생성하고 다음 내용을 삽입합니다.

```
# Define templates before the rules that use them
# Per-Host templates for remote systems
template(name="TmplAuthpriv" type="list") {
    constant(value="/var/log/remote/auth/")
    property(name="hostname")
    constant(value="/")
```

```
    property(name="programname" SecurePath="replace")
    constant(value=".log")
    }

template(name="TmplMsg" type="list") {
    constant(value="/var/log/remote/msg/")
    property(name="hostname")
    constant(value="/")
    property(name="programname" SecurePath="replace")
    constant(value=".log")
    }

# Provides TCP syslog reception
module(load="imtcp")

# Adding this ruleset to process remote messages
ruleset(name="remote1"){
     authpriv.*   action(type="omfile" DynaFile="TmplAuthpriv")
      *.info;mail.none;authpriv.none;cron.none
action(type="omfile" DynaFile="TmplMsg")
}

input(type="imtcp" port="30514" ruleset="remote1")
```

4. **/etc/rsyslog.d/remotelog.conf** 파일에 변경 사항을 저장합니다.

5. **/etc/rsyslog.conf** 파일의 구문을 테스트합니다.

```
# rsyslogd -N 1
rsyslogd: version 8.1911.0-2.el8, config validation run...
rsyslogd: End of config validation run. Bye.
```

6. 로깅 서버에서 **rsyslog** 서비스가 실행 중이고 활성화되어 있는지 확인합니다.

```
# systemctl status rsyslog
```

7. **rsyslog** 서비스를 다시 시작합니다.

```
# systemctl restart rsyslog
```

8. 선택 사항: **rsyslog** 가 활성화되지 않은 경우 재부팅 후**rsyslog** 서비스가 자동으로 시작되는지 확인하십시오.

```
# systemctl enable rsyslog
```

이제 로그 서버가 환경의 다른 시스템에서 로그 파일을 수신하고 저장하도록 구성되어 있습니다.

추가 리소스

- **rsyslogd(8), rsyslog.conf(5), semanage(8)** 및 **firewall-cmd(1)** 도움말 페이지.
- **/usr/share/doc/rsyslog/html/index.html** 파일의 **rsyslog-doc** 패키지와 함께 설치된 설명서입니다.

9.1.4. TCP를 통해 서버에 원격 로깅 구성

TCP 프로토콜을 통해 서버로 로그 메시지를 전달하도록 시스템을 구성할 수 있습니다. **omfwd** 플러그인은 UDP 또는 TCP를 통해 전달을 제공합니다. 기본 프로토콜은 UDP입니다. 플러그인이 빌드되었으므로 로드할 필요가 없습니다.

사전 요구 사항

- **rsyslog** 패키지는 서버에 보고해야 하는 클라이언트 시스템에 설치됩니다.
- 원격 로깅을 위해 서버를 구성했습니다.
- 지정된 포트는 SELinux에서 허용되며 방화벽에서 열립니다.
- 시스템에는 SELinux 구성에 비표준 포트를 추가하기 위한 **semanage** 명령을 제공하는 **policycoreutils-python-utils** 패키지가 포함되어 있습니다.

절차

1. **/etc/rsyslog.d/** 디렉토리에 라는 새 파일을 만듭니다(예:**10-remotelog.conf**).

```
*.* action(type="omfwd"
     queue.type="linkedlist"
     queue.filename="example_fwd"
     action.resumeRetryCount="-1"
     queue.saveOnShutdown="on"
     target="example.com" port="30514" protocol="tcp"
    )
```

다음과 같습니다.

- **queue.type="linkedlist"** 설정은 LinkedList in-memory 큐를 활성화합니다.
- **queue.filename** 설정은 디스크 스토리지를 정의합니다. 백업 파일은 이전 global **workDirectory** 지시문에서 지정한 작업 디렉터리에**example_fwd** 접두사를 사용하여 생성됩니다.
- **action.resumeRetryCount -1** 설정은 서버가 응답하지 않는 경우 연결을 다시 시도할 때 **rsyslog** 가 메시지를 삭제하지 않도록 합니다.
- **rsyslog** 가 종료되면 **queue.saveOnShutdown="on"** 설정은 메모리 내 데이터를 저장합니다.
- 마지막 줄은 수신된 모든 메시지를 로깅 서버로 전달합니다. 포트 사양은 선택 사항입니다. 이 구성을 사용하면 **rsyslog** 에서 서버로 메시지를 전송하지만 원격 서버에 연결할 수 없는 경우 메시지를 메모리에 유지합니다. 디스크의 파일은 **rsyslog** 가 구성된 메모리 대기열 공간에서 부족하거나 시스템 성능을 향상시킬 수 있는 를 종료해야 하는 경우에만 생성됩니다.

참고

rsyslog는 사전 순서로 구성 파일 **/etc/rsyslog.d/** 를 처리합니다.

2. **rsyslog** 서비스를 다시 시작합니다.

```
# systemctl restart rsyslog
```

검증

클라이언트 시스템이 서버에 메시지를 전송하는지 확인하려면 다음 단계를 따르십시오.

1. 클라이언트 시스템에서 테스트 메시지를 전송합니다.

```
# logger test
```

2. 서버 시스템에서 **/var/log/messages** 로그를 확인합니다. 예를 들면 다음과 같습니다.

```
# cat /var/log/remote/msg/hostname/root.log
Feb 25 03:53:17 hostname root[6064]: test
```

여기서 *hostname* 은 클라이언트 시스템의 호스트 이름입니다. 로그에는로거 명령을 입력한 사용자의 사용자 이름이 포함됩니다(이 경우 **root**).

추가 리소스

- **rsyslogd(8)** 및 **rsyslog.conf(5)** 도움말 페이지.
- **/usr/share/doc/rsyslog/html/index.html** 파일의 **rsyslog-doc** 패키지와 함께 설치된 설명서입니다.

9.1.5. TLS 암호화 원격 로깅 구성

기본적으로 Rsyslog는 일반 텍스트 형식으로 원격 블로그 통신을 보냅니다. 시나리오에 이 통신 채널을 보호해야 하는 경우 TLS를 사용하여 암호화할 수 있습니다.

TLS를 통해 암호화된 전송을 사용하려면 서버와 클라이언트 모두를 구성합니다. 서버는 하나 이상의 클라이언트 시스템에서 전송한 로그를 수집하고 분석합니다.

ossl 네트워크 스트림 드라이버(OpenSSL) 또는**gtls** 스트림 드라이버(GnuTLS)를 사용할 수 있습니다.

참고

더 높은 보안이 있는 별도의 시스템이 있는 경우(예: 네트워크에 연결되지 않거나 엄격한 권한이 있는 시스템)를 인증 기관(CA)으로 별도의 시스템을 사용하십시오.

글로벌 ,모듈 및 입력 수준의 서버 측과 글로벌 및 작업 수준의 클라이언트 측의 스트림 드라이버를 사용하여 연결 설정을 사용자 지정할 수 있습니다. 보다 구체적인 구성은 보다 일반적인 구성을 덮어씁니다. 예를 들어 대부분의 연결에서 **ossl** 을 사용하고 특정 연결에 대해서만**gtls** 를 사용할 수 있습니다.

사전 요구 사항

- 클라이언트와 서버 시스템에 모두 **root** 액세스 권한이 있습니다.
- 다음 패키지는 서버 및 클라이언트 시스템에 설치됩니다.
  - **rsyslog** 패키지입니다.
  - **ossl** 네트워크 스트림 드라이버의 경우**rsyslog-openssl** 패키지입니다.
  - **gtls** 네트워크 스트림 드라이버의 경우**rsyslog-gnutls** 패키지입니다.
  - **certtool** 명령을 사용하여 인증서를 생성하는 경우**gnutls-utils** 패키지를 사용합니다.

- 로깅 서버에서 다음 인증서는 **/etc/pki/ca-trust/source/anchors/** 디렉터리에 있으며 **update-ca-trust** 명령을 사용하여 시스템 구성을 업데이트합니다.
  - **ca-cert.pem** - 로깅 서버 및 클라이언트의 키와 인증서를 확인할 수 있는 CA 인증서입니다.
  - **server-cert.pem** - 로깅 서버의 공개 키입니다.
  - **server-key.pem** - 로깅 서버의 개인 키입니다.
- 로깅 클라이언트에서 다음 인증서는 **/etc/pki/ca-trust/source/anchors/** 디렉터리에 있으며, **update-ca-trust** 를 사용하여 시스템 설정을 업데이트합니다.
  - **ca-cert.pem** - 로깅 서버 및 클라이언트의 키와 인증서를 확인할 수 있는 CA 인증서입니다.
  - **client-cert.pem** - 클라이언트의 공개 키입니다.
  - **client-key.pem** - 클라이언트의 개인 키입니다.

절차

1. 클라이언트 시스템에서 암호화된 로그를 수신하도록 서버를 구성합니다.
   a. **/etc/journal.d/** 디렉토리에 라는 새 파일을 만듭니다(예:**securelogser.conf**).
   b. 통신을 암호화하려면 구성 파일에 서버의 인증서 파일 경로, 선택한 인증 방법 및 TLS 암호화를 지원하는 스트림 드라이버가 포함되어야 합니다. **/etc/octets.d/securelogser.conf** 파일에 다음 행을 추가합니다.

```
# Set certificate files
global(
  DefaultNetstreamDriverCAFile="/etc/pki/ca-trust/source/anchors/ca-cert.pem"
  DefaultNetstreamDriverCertFile="/etc/pki/ca-trust/source/anchors/server-cert.pem"
  DefaultNetstreamDriverKeyFile="/etc/pki/ca-trust/source/anchors/server-key.pem"
)

# TCP listener
module(
  load="imtcp"
  PermittedPeer=["client1.example.com", "client2.example.com"]
  StreamDriver.AuthMode="x509/name"
  StreamDriver.Mode="1"
  StreamDriver.Name="ossl"
)

# Start up listener at port 514
input(
  type="imtcp"
  port="514"
)
```

참고

GnuTLS 드라이버를 선호하는 경우 **StreamDriver.Name="gtls"** 구성 옵션을 사용합니다. **x509/name** 보다 엄격한 인증 모드에 대한 자세한 내용은**rsyslog-doc** 패키지로 설치된 설명서를 참조하십시오.

c. **/etc/octets.d/securelogser.conf** 파일에 변경 사항을 저장합니다.

d. **/etc/octets.conf** 파일 구문 및 **/etc/octets.d/** 디렉토리에 있는 모든 파일을 확인합니다.

```
# rsyslogd -N 1
rsyslogd: version 8.1911.0-2.el8, config validation run (level 1)...
rsyslogd: End of config validation run. Bye.
```

e. 로깅 서버에서 **rsyslog** 서비스가 실행 중이고 활성화되어 있는지 확인합니다.

```
# systemctl status rsyslog
```

f. **rsyslog** 서비스를 다시 시작하십시오.

```
# systemctl restart rsyslog
```

g. 선택 사항: Rsyslog가 활성화되지 않은 경우 재부팅 후 **rsyslog** 서비스가 자동으로 시작됩니다.

```
# systemctl enable rsyslog
```

2. 암호화된 로그를 서버에 전송하도록 클라이언트를 구성합니다.

a. 클라이언트 시스템에서 **/etc/journal.d/** 디렉토리에 라는 새 파일을 만듭니다(예: **securelogcli.conf**).

b. **/etc/octets.d/securelogcli.conf** 파일에 다음 행을 추가합니다.

```
# Set certificate files
global(
  DefaultNetstreamDriverCAFile="/etc/pki/ca-trust/source/anchors/ca-cert.pem"
  DefaultNetstreamDriverCertFile="/etc/pki/ca-trust/source/anchors/client-cert.pem"
  DefaultNetstreamDriverKeyFile="/etc/pki/ca-trust/source/anchors/client-key.pem"
)


# Set up the action for all messages
*.* action(
  type="omfwd"
  StreamDriver="ossl"
  StreamDriverMode="1"
  StreamDriverPermittedPeers="server.example.com"
  StreamDriverAuthMode="x509/name"
  target="server.example.com" port="514" protocol="tcp"
)
```

참고

GnuTLS 드라이버를 선호하는 경우 **StreamDriver.Name="gtls"** 구성 옵션을 사용합니다.

c. **/etc/rsyslog.d/securelogcli.conf** 파일에 변경 사항을 저장합니다.

d. **/etc/rsyslog.conf** 파일 및 기타 파일의 구문을 **/etc/rsyslog.d/** 디렉터리에 확인합니다.

```
# rsyslogd -N 1
rsyslogd: version 8.1911.0-2.el8, config validation run (level 1)...
rsyslogd: End of config validation run. Bye.
```

e. 로깅 서버에서 **rsyslog** 서비스가 실행 중이고 활성화되어 있는지 확인합니다.

```
# systemctl status rsyslog
```

f. **rsyslog** 서비스를 다시 시작하십시오.

```
# systemctl restart rsyslog
```

g. 선택 사항: Rsyslog가 활성화되지 않은 경우 재부팅 후 **rsyslog** 서비스가 자동으로 시작됩니다.

```
# systemctl enable rsyslog
```

검증

클라이언트 시스템이 서버에 메시지를 전송하는지 확인하려면 다음 단계를 따르십시오.

1. 클라이언트 시스템에서 테스트 메시지를 전송합니다.

```
# logger test
```

2. 서버 시스템에서 **/var/log/messages** 로그를 확인합니다. 예를 들면 다음과 같습니다.

```
# cat /var/log/remote/msg/<hostname>/root.log
Feb 25 03:53:17 <hostname> root[6064]: test
```

여기서 ***<hostname>***은 클라이언트 시스템의 호스트 이름입니다. 로그에는 로거 명령을 입력한 사용자의 사용자 이름이 포함됩니다(이 경우 **root**).

추가 리소스

- **certtool(1), openssl(1), update-ca-trust(8), rsyslogd(8), rsyslog.conf(5)** 매뉴얼 페이지.
- **/usr/share/doc/rsyslog/html/index.html** 의 **rsyslog-doc** 패키지와 함께 설치된 설명서입니다.
- TLS에서 로깅 시스템 역할 사용.

9.1.6. UDP를 통해 원격 로깅 정보를 받기 위한 서버 설정

Rsyslog 애플리케이션을 사용하면 원격 시스템에서 로깅 정보를 수신하도록 시스템을 구성할 수 있습니다. UDP를 통해 원격 로깅을 사용하려면 서버와 클라이언트를 둘 다 구성합니다. 수신 서버는 하나 이상의 클라이언트 시스템에서 전송한 로그를 수집하고 분석합니다. 기본적으로 **rsyslog** 는 포트 **514** 에서 UDP를 사용하여 원격 시스템에서 로그 정보를 받습니다.

다음 절차에 따라 UDP 프로토콜을 통해 하나 이상의 클라이언트 시스템에서 전송한 로그를 수집하고 분석하기 위해 서버를 구성합니다.

사전 요구 사항

- rsyslog가 서버 시스템에 설치되어 있습니다.

- 서버에서 **root** 로 로그인했습니다.
- **semanage** 명령을 사용하여 선택적 단계를 위해 **policycoreutils-python-utils** 패키지가 설치됩니다.
- **firewalld** 서비스가 실행 중입니다.

절차

1. 선택 사항: 기본 포트 **514** 가 아닌 **rsyslog** 트래픽에 다른 포트를 사용하려면 다음을 수행합니다.
   a. **syslogd_port_t** SELinux 유형을 SELinux 정책 구성에 추가하고 ***portno*** 를 **rsyslog** 에서 사용할 포트 번호로 바꿉니다.

      ```
      # semanage port -a -t syslogd_port_t -p udp portno
      ```

   b. 들어오는 **rsyslog** 트래픽을 허용하고 ***portno*** 를 포트 번호 및 영역으로 교체하여 **rsyslog** 에서 사용할 영역으로 교체하도록 **firewalld** 를 구성합니다.

      ```
      # firewall-cmd --zone=zone --permanent --add-port=portno/udp
      success
      # firewall-cmd --reload
      ```

   c. 방화벽 규칙을 다시 로드합니다.

      ```
      # firewall-cmd --reload
      ```

2. **/etc/rsyslog .d/** 디렉토리(예: **remotelogserv. conf**)에 새 **.conf** 파일을 생성하고 다음 내용을 삽입합니다.

   ```
   # Define templates before the rules that use them
   # Per-Host templates for remote systems
   template(name="TmplAuthpriv" type="list") {
      constant(value="/var/log/remote/auth/")
      property(name="hostname")
      constant(value="/")
      property(name="programname" SecurePath="replace")
      constant(value=".log")
      }

   template(name="TmplMsg" type="list") {
      constant(value="/var/log/remote/msg/")
      property(name="hostname")
      constant(value="/")
      property(name="programname" SecurePath="replace")
      constant(value=".log")
      }

   # Provides UDP syslog reception
   module(load="imudp")

   # This ruleset processes remote messages
   ruleset(name="remote1"){
       authpriv.*   action(type="omfile" DynaFile="TmplAuthpriv")
        *.info;mail.none;authpriv.none;cron.none
   ```

```
action(type="omfile" DynaFile="TmplMsg")
}

input(type="imudp" port="514" ruleset="remote1")
```

여기서 **514** 는 기본적으로 사용하는 포트 번호**rsyslog** 입니다. 대신 다른 포트를 지정할 수 있습니다.

3. **/etc/rsyslog. d/** 디렉터리에 있는 **/etc/rsyslog .conf** 파일의 구문과 **all.** conf 파일을 확인합니다.

```
# rsyslogd -N 1
rsyslogd: version 8.1911.0-2.el8, config validation run...
```

4. **rsyslog** 서비스를 다시 시작합니다.

```
# systemctl restart rsyslog
```

5. 선택 사항: **rsyslog** 가 활성화되지 않은 경우 재부팅 후**rsyslog** 서비스가 자동으로 시작되는지 확인하십시오.

```
# systemctl enable rsyslog
```

추가 리소스

- **rsyslogd(8) , rsyslog.conf(5), semanage(8)** 및 **firewall-cmd(1)** 도움말 페이지.
- **/usr/share/doc/rsyslog/html/index.html** 파일의 **rsyslog-doc** 패키지와 함께 설치된 설명서입니다.

9.1.7. UDP를 통해 서버에 원격 로깅 구성

UDP 프로토콜을 통해 서버로 로그 메시지를 전달하도록 시스템을 구성할 수 있습니다. **omfwd** 플러그인은 UDP 또는 TCP를 통해 전달을 제공합니다. 기본 프로토콜은 UDP입니다. 플러그인이 빌드되었으므로 로드할 필요가 없습니다.

사전 요구 사항

- **rsyslog** 패키지는 서버에 보고해야 하는 클라이언트 시스템에 설치됩니다.
- UDP를 통해 원격 로깅 정보를 받기 위한 서버 구성에 설명된 대로 원격 로깅을 위해 서버를 구성했습니다.

절차

1. **/etc/rsyslog.d/** 디렉토리에 새 **.conf** 파일을 만듭니다(예:**10-remotelogcli.conf**).

```
*.* action(type="omfwd"
     queue.type="linkedlist"
     queue.filename="example_fwd"
     action.resumeRetryCount="-1"
     queue.saveOnShutdown="on"
     target="example.com" port="portno" protocol="udp"
    )
```

다음과 같습니다.

- **queue.type="linkedlist"** 설정은 LinkedList in-memory 큐를 활성화합니다.
- **queue.filename** 설정은 디스크 스토리지를 정의합니다. 백업 파일은 이전 글로벌 **workDirectory** 지시문으로 지정된 작업 디렉터리에 ***example_fwd*** 접두사를 사용하여 생성됩니다.
- **action.resumeRetryCount -1** 설정은 서버가 응답하지 않는 경우 연결을 재시도할 때**rsyslog** 가 메시지를 삭제하지 않도록 합니다.
- **rsyslog** 가 종료되면 활성화된 **queue.saveOnShutdown="on"** 설정은 메모리 내 데이터를 저장합니다.
- ***portno*** 값은 **rsyslog** 에서 사용할 포트 번호입니다. 기본값은**514** 입니다.
- 마지막 행은 수신된 모든 메시지를 로깅 서버로 전달하고 포트 사양은 선택 사항입니다. 이 구성을 사용하면 **rsyslog** 에서 서버로 메시지를 전송하지만 원격 서버에 연결할 수 없는 경우 메시지를 메모리에 유지합니다. 디스크의 파일은 **rsyslog** 가 구성된 메모리 대기열 공간에서 부족하거나 시스템 성능을 향상시킬 수 있는 를 종료해야 하는 경우에만 생성됩니다.

참고

rsyslog는 사전 순서로 구성 파일 **/etc/rsyslog.d/** 를 처리합니다.

2. **rsyslog** 서비스를 다시 시작합니다.

```
# systemctl restart rsyslog
```

3. 선택 사항: **rsyslog** 가 활성화되지 않은 경우 재부팅 후**rsyslog** 서비스가 자동으로 시작되는지 확인하십시오.

```
# systemctl enable rsyslog
```

검증

클라이언트 시스템이 서버에 메시지를 전송하는지 확인하려면 다음 단계를 따르십시오.

1. 클라이언트 시스템에서 테스트 메시지를 전송합니다.

```
# logger test
```

2. 서버 시스템에서 **/var/log/remote/msg/*hostname*/root.log** 로그를 확인합니다. 예를 들면 다음과 같습니다.

```
# cat /var/log/remote/msg/hostname/root.log
Feb 25 03:53:17 hostname root[6064]: test
```

여기서 **hostname** 은 클라이언트 시스템의 호스트 이름입니다. 로그에는 로거 명령을 입력한 사용자의 사용자 이름이 포함됩니다(이 경우 **root**).

추가 리소스

- **rsyslogd(8)** 및 **rsyslog.conf(5)** 도움말 페이지.

- **/usr/share/doc/rsyslog/html/index.html** 의 **rsyslog-doc** 패키지와 함께 설치된 설명서입니다.

9.1.8. Rsyslog의 로드 밸런싱 도우미

RebindInterval 설정은 현재 연결이 끊어지고 다시 설정된 간격을 지정합니다. 이 설정은 TCP, UDP 및 RELP 트래픽에 적용됩니다. 로드 밸런서는 새 연결로 인식하여 메시지를 다른 물리적 대상 시스템으로 전달합니다.

RebindInterval 설정은 대상 시스템이 IP 주소를 변경한 시나리오에서 유용합니다. Rsyslog 애플리케이션은 연결이 설정될 때 IP 주소를 캐시하므로 메시지가 동일한 서버로 전송됩니다. IP 주소가 변경되면 Rsyslog 서비스가 다시 시작될 때까지 UDP 패킷이 손실됩니다. 연결을 다시 설정하면 DNS에서 IP를 다시 확인합니다.

```
action(type="omfwd" protocol="tcp" RebindInterval="250" target="example.com" port="514" …)

action(type="omfwd" protocol="udp" RebindInterval="250" target="example.com" port="514" …)

action(type="omrelp" RebindInterval="250" target="example.com" port="6514" …)
```

9.1.9. 신뢰할 수 있는 원격 로깅 구성

RELP(Reliable Event Logging Protocol)를 사용하면 메시지 손실 위험이 훨씬 감소하여 TCP를 통해 **syslog** 메시지를 보내고 받을 수 있습니다. RELP는 메시지 손실이 허용되지 않는 환경에서 유용하게 이벤트 메시지를 안정적으로 전송합니다. RELP를 사용하려면 서버에서 실행되고 로그를 수신하는 **imrelp** 입력 모듈을 구성하고 클라이언트에서 실행되고 로깅 서버에 로그를 전송하는 **omrelp** 출력 모듈을 구성합니다.

사전 요구 사항

- 서버 및 클라이언트 시스템에 **rsyslog**,**librelp** 및 **rsyslog-relp** 패키지를 설치했습니다.
- 지정된 포트는 SELinux에서 허용되며 방화벽에서 열립니다.

절차

1. 안정적인 원격 로깅을 위해 클라이언트 시스템을 구성합니다.
   a. 클라이언트 시스템에서 **/etc/journal.d/** 디렉토리에 새 **. conf** 파일을 만듭니다(예: **relpclient.conf**).

      ```
      module(load="omrelp")
      *.* action(type="omrelp" target="_target_IP_" port="_target_port_")
      ```

      다음과 같습니다.

      - ***target_IP*** 는 로깅 서버의 IP 주소입니다.
      - ***target_port*** 는 로깅 서버의 포트입니다.

   b. **/etc/octets.d/relpclient.conf** 파일에 변경 사항을 저장합니다.
   c. **rsyslog** 서비스를 다시 시작합니다.

      ```
      # systemctl restart rsyslog
      ```

d. 선택 사항: **rsyslog** 가 활성화되지 않은 경우 재부팅 후**rsyslog** 서비스가 자동으로 시작되는지 확인하십시오.

```
# systemctl enable rsyslog
```

2. 안정적인 원격 로깅을 위해 서버 시스템을 구성합니다.

a. 서버 시스템에서 **/etc/rsyslog .d/** 디렉터리에 새**. conf** 파일을 생성합니다(예:**relpserv.conf**).

```
ruleset(name="relp"){
*.* action(type="omfile" file="_log_path_")
}


module(load="imrelp")
input(type="imrelp" port="_target_port_" ruleset="relp")
```

다음과 같습니다.

- ***log_path*** 는 메시지를 저장하는 경로를 지정합니다.
- ***target_port*** 는 로깅 서버의 포트입니다. 클라이언트 구성 파일과 동일한 값을 사용합니다.

b. **/etc/rsyslog.d/relpserv.conf** 파일에 변경 사항을 저장합니다.

c. **rsyslog** 서비스를 다시 시작합니다.

```
# systemctl restart rsyslog
```

d. 선택 사항: **rsyslog** 가 활성화되지 않은 경우 재부팅 후**rsyslog** 서비스가 자동으로 시작되는지 확인하십시오.

```
# systemctl enable rsyslog
```

검증

클라이언트 시스템이 서버에 메시지를 전송하는지 확인하려면 다음 단계를 따르십시오.

1. 클라이언트 시스템에서 테스트 메시지를 전송합니다.

```
# logger test
```

2. 서버 시스템에서 지정된 ***log_path*** 의 로그를 확인합니다. 예를 들면 다음과 같습니다.

```
# cat /var/log/remote/msg/hostname/root.log
Feb 25 03:53:17 hostname root[6064]: test
```

여기서 **hostname** 은 클라이언트 시스템의 호스트 이름입니다. 로그에는 로거 명령을 입력한 사용자의 사용자 이름이 포함됩니다(이 경우 **root**).

추가 리소스

- **rsyslogd(8)** 및 **rsyslog.conf(5)** 도움말 페이지.

- **/usr/share/doc/rsyslog/html/index.html** 파일의 **rsyslog-doc** 패키지와 함께 설치된 설명서입니다.

9.1.10. 지원되는 Rsyslog 모듈

Rsyslog 애플리케이션의 기능을 확장하려면 특정 모듈을 사용할 수 있습니다. 모듈은 추가 입력(Input Modules), 출력(Output Modules) 및 기타 기능을 제공합니다. 모듈은 모듈을 로드한 후 사용할 수 있는 추가 구성 지시문도 제공할 수 있습니다.

다음 명령을 입력하여 시스템에 설치된 입력 및 출력 모듈을 나열할 수 있습니다.

```
# ls /usr/lib64/rsyslog/{i,o}m*
```

rsyslog -doc 패키지를 설치한 후 **/usr/share/doc/rsyslog/html/configuration/modules/idx_output.html** 파일에서 사용 가능한 모든 rsyslog 모듈 목록을 볼 수 있습니다.

9.1.11. 커널 메시지를 원격 호스트에 기록하도록 netconsole 서비스 구성

디스크에 로깅하거나 직렬 콘솔을 사용할 수 없는 경우 **netconsole** 커널 모듈과 동일한 이름의 서비스를 사용하여 네트워크를 통해 커널 메시지를 원격 **rsyslog** 서비스에 기록할 수 있습니다.

사전 요구 사항

- **rsyslog** 와 같은 시스템 로그 서비스가 원격 호스트에 설치되어 있습니다.
- 원격 시스템 로그 서비스는 이 호스트에서 들어오는 로그 항목을 수신하도록 구성됩니다.

절차

1. **netconsole-service** 패키지를 설치합니다.

```
# yum install netconsole-service
```

2. **/etc/sysconfig/netconsole** 파일을 편집하고 **SYSLOGADDR** 매개변수를 원격 호스트의 IP 주소로 설정합니다.

```
# SYSLOGADDR=192.0.2.1
```

3. **netconsole** 서비스를 활성화하고 시작합니다.

```
# systemctl enable --now netconsole
```

검증

- 원격 시스템 로그 서버에 **/var/log/** ECDHE 파일을 표시합니다.

추가 리소스

원격 로깅 솔루션 구성

9.1.12. 추가 리소스

- **/usr/share/doc/rsyslog/html/index.html** 파일의 **rsyslog-doc** 패키지로 설치된 문서
- **rsyslog.conf(5)** 및 **rsyslogd(8)** 도움말 페이지
- journald 없이 또는 minimized journald 사용 Knowledgebase 문서를 사용하여 시스템 로깅 구성
- 성능 및 완화 기술 자료에서 RHEL 기본 로깅 설정의 부정적인 영향
- 로깅 시스템 역할 사용 장

9.2. 로깅 시스템 역할 사용

시스템 관리자는 로깅 시스템 역할을 사용하여 Red Hat Enterprise Linux 호스트를 로깅 서버로 구성하여 많은 클라이언트 시스템에서 로그를 수집할 수 있습니다.

9.2.1. 로깅 RHEL 시스템 역할

로깅 RHEL 시스템 역할을 사용하면 로컬 및 원격 호스트에 로깅 구성을 배포할 수 있습니다.

로깅 솔루션은 여러 가지 방법으로 로그를 읽고 다양한 로깅 출력을 제공합니다.

예를 들어 로깅 시스템은 다음 입력을 수신할 수 있습니다.

- 로컬 파일
- **systemd/journal**
- 네트워크를 통한 다른 로깅 시스템

또한 로깅 시스템에는 다음과 같은 출력이 있을 수 있습니다.

- **/var/log** 디렉터리의 로컬 파일에 저장된 로그
- Elasticsearch로 전송되는 로그
- 다른 로깅 시스템으로 전달된 로그

로깅 RHEL 시스템 역할을 사용하면 입력 및 출력을 결합하여 시나리오에 적합할 수 있습니다. 예를 들어 로컬 파일에 저널 의 입력을 저장하는 로깅 솔루션을 구성할 수 있지만 파일에서 읽은 입력은 다른 로깅 시스템으로 전달되고 로컬 로그 파일에 저장됩니다.

추가 리소스

- **/usr/share/ansible/roles/rhel-system-roles.logging/README.md** 파일
- **/usr/share/doc/rhel-system-roles/logging/** 디렉터리
- RHEL 시스템 역할

9.2.2. 로컬 로깅 RHEL 시스템 역할 적용

Ansible 플레이북을 준비하고 적용하여 별도의 시스템 집합에서 로깅 솔루션을 구성합니다. 각 머신 레코드는 로컬로 로그됩니다.

사전 요구 사항

- 컨트롤 노드 및 관리형 노드를 준비했습니다.
- 관리 노드에서 플레이북을 실행할 수 있는 사용자로 제어 노드에 로그인되어 있습니다.
- 관리 노드에 연결하는 데 사용하는 계정에는 **sudo** 권한이 있습니다.

참고

배포 시 RHEL 시스템 역할이 **rsyslog** 를 설치하므로 **rsyslog** 패키지가 설치되지 않아도 됩니다.

절차

1. 다음 콘텐츠를 사용하여 플레이북 파일(예: ~/**playbook.yml**)을 생성합니다.

```
---
- name: Deploying basics input and implicit files output
  hosts: managed-node-01.example.com
  roles:
    - rhel-system-roles.logging
  vars:
    logging_inputs:
      - name: system_input
        type: basics
    logging_outputs:
      - name: files_output
        type: files
    logging_flows:
      - name: flow1
        inputs: [system_input]
        outputs: [files_output]
```

2. 플레이북 구문을 확인합니다.

```
$ ansible-playbook --syntax-check ~/playbook.yml
```

이 명령은 구문만 검증하고 잘못되었지만 유효한 구성으로부터 보호하지 않습니다.

3. 플레이북을 실행합니다.

```
$ ansible-playbook ~/playbook.yml
```

검증

1. **/etc/rsyslog.conf** 파일의 구문을 테스트합니다.

```
# rsyslogd -N 1
rsyslogd: version 8.1911.0-6.el8, config validation run...
rsyslogd: End of config validation run. Bye.
```

2. 시스템이 로그에 메시지를 전송하는지 확인합니다.

   a. 테스트 메시지를 전송합니다.

```
# logger test
```

b. **/var/log/messages** 로그를 확인합니다. 예를 들면 다음과 같습니다.

```
# cat /var/log/messages
Aug  5 13:48:31 <hostname> root[6778]: test
```

여기서 *<hostname>*은 클라이언트 시스템의 호스트 이름입니다. 로그에는 로거 명령을 입력한 사용자의 사용자 이름이 포함됩니다(이 경우 **root**).

추가 리소스

- **/usr/share/ansible/roles/rhel-system-roles.logging/README.md** 파일
- **/usr/share/doc/rhel-system-roles/logging/** 디렉터리

9.2.3. 로컬 로깅 RHEL 시스템 역할에서 로그 필터링

rsyslog 속성 기반 필터를 기반으로 로그를 필터링하는 로깅 솔루션을 배포할 수 있습니다.

사전 요구 사항

- 컨트롤 노드 및 관리형 노드를 준비했습니다.
- 관리 노드에서 플레이북을 실행할 수 있는 사용자로 제어 노드에 로그인되어 있습니다.
- 관리 노드에 연결하는 데 사용하는 계정에는 **sudo** 권한이 있습니다.

참고

배포 시 RHEL 시스템 역할이 **rsyslog** 를 설치하므로 **rsyslog** 패키지가 설치되지 않아도 됩니다.

절차

1. 다음 콘텐츠를 사용하여 플레이북 파일(예: **~/playbook.yml**)을 생성합니다.

```
---
- name: Deploying files input and configured files output
  hosts: managed-node-01.example.com
  roles:
    - rhel-system-roles.logging
  vars:
    logging_inputs:
      - name: files_input
        type: basics
    logging_outputs:
      - name: files_output0
        type: files
        property: msg
        property_op: contains
        property_value: error
        path: /var/log/errors.log
      - name: files_output1
```

```
      type: files
      property: msg
      property_op: "!contains"
      property_value: error
      path: /var/log/others.log
    logging_flows:
      - name: flow0
        inputs: [files_input]
        outputs: [files_output0, files_output1]
```

이 구성을 사용하면 오류 문자열이 포함된 모든 메시지가/var/log/errors.log 에 기록되고 다른 모든 메시지는 /var/log/others.log 에 기록됩니다.

오류 속성 값을 필터링할 문자열로 교체할 수 있습니다.

환경 설정에 따라 변수를 수정할 수 있습니다.

2. 플레이북 구문을 확인합니다.

```
$ ansible-playbook --syntax-check ~/playbook.yml
```

이 명령은 구문만 검증하고 잘못되었지만 유효한 구성으로부터 보호하지 않습니다.

3. 플레이북을 실행합니다.

```
$ ansible-playbook ~/playbook.yml
```

검증

1. /etc/rsyslog.conf 파일의 구문을 테스트합니다.

```
# rsyslogd -N 1
rsyslogd: version 8.1911.0-6.el8, config validation run...
rsyslogd: End of config validation run. Bye.
```

2. 시스템에서 오류 문자열이 포함된 메시지를 로그에 전송하는지 확인합니다.

   a. 테스트 메시지를 전송합니다.

```
# logger error
```

   b. /var/log/errors.log 로그를 확인합니다. 예를 들면 다음과 같습니다.

```
# cat /var/log/errors.log
Aug  5 13:48:31 hostname root[6778]: error
```

   여기서 *hostname* 은 클라이언트 시스템의 호스트 이름입니다. 로그에는 로거 명령을 입력한 사용자의 사용자 이름이 포함됩니다(이 경우 root).

추가 리소스

- /usr/share/ansible/roles/rhel-system-roles.logging/README.md 파일
- /usr/share/doc/rhel-system-roles/logging/ 디렉터리

9.2.4. 로깅 RHEL 시스템 역할을 사용하여 원격 로깅 솔루션 적용

다음 단계에 따라 Red Hat Ansible Core 플레이북을 준비 및 적용하여 원격 로깅 솔루션을 구성합니다. 이 플레이북에서 하나 이상의 클라이언트는 **systemd-journal** 에서 로그를 가져와 원격 서버로 전달합니다. 서버는 remote **_rsyslog** 및 **remote_ files** 에서 원격 입력을 수신하고 원격 호스트 이름으로 이름이 지정된 디렉터리의 로컬 파일에 로그를 출력합니다.

사전 요구 사항

- 컨트롤 노드 및 관리형 노드를 준비했습니다.
- 관리 노드에서 플레이북을 실행할 수 있는 사용자로 제어 노드에 로그인되어 있습니다.
- 관리 노드에 연결하는 데 사용하는 계정에는 **sudo** 권한이 있습니다.

참고

배포 시 RHEL 시스템 역할이 **rsyslog** 를 설치하므로 **rsyslog** 패키지가 설치되지 않아도 됩니다.

절차

1. 다음 콘텐츠를 사용하여 플레이북 파일(예: ~/**playbook.yml**)을 생성합니다.

```
---
- name: Deploying remote input and remote_files output
  hosts: managed-node-01.example.com
  roles:
    - rhel-system-roles.logging
  vars:
    logging_inputs:
      - name: remote_udp_input
        type: remote
        udp_ports: [ 601 ]
      - name: remote_tcp_input
        type: remote
        tcp_ports: [ 601 ]
    logging_outputs:
      - name: remote_files_output
        type: remote_files
    logging_flows:
      - name: flow_0
        inputs: [remote_udp_input, remote_tcp_input]
        outputs: [remote_files_output]

- name: Deploying basics input and forwards output
  hosts: managed-node-02.example.com
  roles:
    - rhel-system-roles.logging
  vars:
    logging_inputs:
      - name: basic_input
        type: basics
    logging_outputs:
      - name: forward_output0
```

```
        type: forwards
        severity: info
        target: <host1.example.com>
        udp_port: 601
      - name: forward_output1
        type: forwards
        facility: mail
        target: <host1.example.com>
        tcp_port: 601
    logging_flows:
      - name: flows0
        inputs: [basic_input]
        outputs: [forward_output0, forward_output1]

  [basic_input]
  [forward_output0, forward_output1]
```

여기서 & *lt;host1.example.com>*은 로깅 서버입니다.

참고

요구 사항에 맞게 플레이북의 매개 변수를 수정할 수 있습니다.

주의

로깅 솔루션은 서버 또는 클라이언트 시스템의 SELinux 정책에 정의된 포트에서만 작동하며 방화벽에서 열립니다. 기본 SELinux 정책에는 포트 601, 514, 6514, 10514, 20514가 포함됩니다. 다른 포트를 사용하려면 클라이언트 및 서버 시스템에서 SELinux 정책을 수정합니다.

2. 플레이북 구문을 확인합니다.

```
$ ansible-playbook --syntax-check ~/playbook.yml
```

이 명령은 구문만 검증하고 잘못되었지만 유효한 구성으로부터 보호하지 않습니다.

3. 플레이북을 실행합니다.

```
$ ansible-playbook ~/playbook.yml
```

검증

1. 클라이언트 및 서버 시스템 모두에서 **/etc/rsyslog.conf** 파일의 구문을 테스트합니다.

```
# rsyslogd -N 1
rsyslogd: version 8.1911.0-6.el8, config validation run (level 1), master config
/etc/rsyslog.conf
rsyslogd: End of config validation run. Bye.
```

2. 클라이언트 시스템이 서버에 메시지를 전송하는지 확인합니다.

   a. 클라이언트 시스템에서 테스트 메시지를 전송합니다.

   ```
   # logger test
   ```

   b. 서버 시스템에서 **/var/log/ <*host2.example.com*> /messages** 로그를 확인합니다. 예를 들면 다음과 같습니다.

   ```
   # cat /var/log/<host2.example.com>/messages
   Aug  5 13:48:31 <host2.example.com> root[6778]: test
   ```

   여기서 ***<host2.example.com*** >은 클라이언트 시스템의 호스트 이름입니다. 로그에는 로거 명령을 입력한 사용자의 사용자 이름이 포함됩니다(이 경우 **root**).

추가 리소스

- **/usr/share/ansible/roles/rhel-system-roles.logging/README.md** 파일
- **/usr/share/doc/rhel-system-roles/logging/** 디렉터리

9.2.5. TLS에서 로깅 RHEL 시스템 역할 사용

TLS(Transport Layer Security)는 컴퓨터 네트워크를 통해 보안 통신을 허용하도록 설계된 암호화 프로토콜입니다.

관리자는 로깅 RHEL 시스템 역할을 사용하여 Red Hat Ansible Automation Platform을 사용하여 보안 로그 전송을 구성할 수 있습니다.

9.2.5.1. TLS를 사용하여 클라이언트 로깅 구성

로깅 RHEL 시스템 역할과 함께 Ansible 플레이북을 사용하여 RHEL 클라이언트에 대한 로깅을 구성하고 TLS 암호화를 사용하여 원격 로깅 시스템으로 로그를 전송할 수 있습니다.

이 절차에서는 개인 키 및 인증서를 생성하고 Ansible 인벤토리의 clients 그룹에 있는 모든 호스트에 TLS를 구성합니다. TLS 프로토콜은 네트워크를 통해 로그를 안전하게 전송할 수 있도록 메시지 전송을 암호화합니다.

참고

인증서를 생성하기 위해 플레이북에서 인증서 RHEL 시스템 역할을 호출할 필요가 없습니다. 로깅 RHEL 시스템 역할은 자동으로 호출합니다.

CA에서 생성된 인증서에 서명하려면 관리형 노드를 IdM 도메인에 등록해야 합니다.

사전 요구 사항

- 컨트롤 노드 및 관리형 노드를 준비했습니다.
- 관리 노드에서 플레이북을 실행할 수 있는 사용자로 제어 노드에 로그인되어 있습니다.
- 관리 노드에 연결하는 데 사용하는 계정에는 **sudo** 권한이 있습니다.
- 관리형 노드는 IdM 도메인에 등록됩니다.

절차

1. 다음 콘텐츠를 사용하여 플레이북 파일(예: ~/playbook.yml)을 생성합니다.

```
---
- name: Deploying files input and forwards output with certs
  hosts: managed-node-01.example.com
  roles:
    - rhel-system-roles.logging
  vars:
    logging_certificates:
      - name: logging_cert
        dns: ['localhost', 'www.example.com']
        ca: ipa
    logging_pki_files:
      - ca_cert: /local/path/to/ca_cert.pem
        cert: /local/path/to/logging_cert.pem
        private_key: /local/path/to/logging_cert.pem
    logging_inputs:
      - name: input_name
        type: files
        input_log_path: /var/log/containers/*.log
    logging_outputs:
      - name: output_name
        type: forwards
        target: your_target_host
        tcp_port: 514
        tls: true
        pki_authmode: x509/name
        permitted_server: 'server.example.com'
    logging_flows:
      - name: flow_name
        inputs: [input_name]
        outputs: [output_name]
```

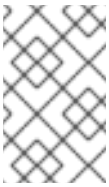
Playbook은 다음 매개 변수를 사용합니다.

logging_certificates

이 매개변수의 값은 인증서 RHEL 시스템 역할의 **certificate _requests** 에 전달되며 개인 키와 인증서를 생성하는 데 사용됩니다.

logging_pki_files

이 매개변수를 사용하여 로깅에서 사용하는 경로 및 기타 설정을 구성하여 로깅에서 사용하는 CA, 인증서 및 TLS에 사용되는 키 파일을 찾습니다.**ca_cert_src, ca_cert** _src ,**cert,cert_src,** private_key ,**private_key _src, tls.**



참고

logging_certificates 를 사용하여 대상 노드에서 파일을 생성하는 경우 **logging_certificates** 에서 생성되지 않은 파일을 복사하는 데 사용되는 **ca_cert_src,cert_src** 및 **private_key_src** 를 사용하지 마십시오.

ca_cert

대상 노드의 CA 인증서 파일의 경로를 나타냅니다. 기본 경로는 **/etc/pki/tls/certs/ca.pem** 이며 파일 이름은 사용자가 설정합니다.

인증서

대상 노드의 인증서 파일의 경로를 나타냅니다. 기본 경로는 **/etc/pki/tls/certs/server-cert.pem** 이며 파일 이름은 사용자가 설정합니다.

private_key

대상 노드의 개인 키 파일의 경로를 나타냅니다. 기본 경로는 **/etc/pki/tls/private/server-key.pem** 이며 파일 이름은 사용자가 설정합니다.

ca_cert_src

ca_cert 에서 지정한 위치로 대상 호스트에 복사되는 제어 노드의 CA 인증서 파일의 경로를 나타냅니다. **logging_certificates** 를 사용하는 경우 이 사용하지 마십시오.

cert_src

인증서에서 지정한 위치로 대상 호스트에 복사되는 제어 노드의 인증서 파일의 경로를 나타냅니다. **logging_certificates** 를 사용하는 경우 이 사용하지 마십시오.

private_key_src

private_key 에서 지정한 위치로 대상 호스트에 복사되는 제어 노드에서 개인 키 파일의 경로를 나타냅니다. **logging_certificates** 를 사용하는 경우 이 사용하지 마십시오.

tls

이 매개변수를 **true** 로 설정하면 네트워크를 통한 로그 전송이 안전합니다. 보안 래퍼가 필요하지 않은 경우 **tls: false** 를 설정할 수 있습니다.

2. 플레이북 구문을 확인합니다.

```
$ ansible-playbook --syntax-check ~/playbook.yml
```

이 명령은 구문만 검증하고 잘못되었지만 유효한 구성으로부터 보호하지 않습니다.

3. 플레이북을 실행합니다.

```
$ ansible-playbook ~/playbook.yml
```

추가 리소스

- **/usr/share/ansible/roles/rhel-system-roles.logging/README.md** 파일
- **/usr/share/doc/rhel-system-roles/logging/** 디렉터리
- RHEL 시스템 역할을 사용하여 인증서 요청

9.2.5.2. TLS를 사용하여 서버 로깅 구성

로깅 RHEL 시스템 역할과 함께 Ansible 플레이북을 사용하여 RHEL 서버에서 로깅을 구성하고 TLS 암호화를 사용하여 원격 로깅 시스템에서 로그를 수신하도록 설정할 수 있습니다.

이 절차에서는 개인 키 및 인증서를 생성하고 Ansible 인벤토리의 서버 그룹에 있는 모든 호스트에 TLS를 구성합니다.

참고

인증서를 생성하기 위해 플레이북에서 인증서 RHEL 시스템 역할을 호출할 필요가 없습니다. 로깅 RHEL 시스템 역할은 자동으로 호출합니다.

CA에서 생성된 인증서에 서명하려면 관리형 노드를 IdM 도메인에 등록해야 합니다.

사전 요구 사항

- 컨트롤 노드 및 관리형 노드를 준비했습니다.
- 관리 노드에서 플레이북을 실행할 수 있는 사용자로 제어 노드에 로그인되어 있습니다.
- 관리 노드에 연결하는 데 사용하는 계정에는 **sudo** 권한이 있습니다.
- 관리형 노드는 IdM 도메인에 등록됩니다.

절차

1. 다음 콘텐츠를 사용하여 플레이북 파일(예: ~/**playbook.yml**)을 생성합니다.

```
---
- name: Deploying remote input and remote_files output with certs
  hosts: managed-node-01.example.com
  roles:
    - rhel-system-roles.logging
  vars:
    logging_certificates:
      - name: logging_cert
        dns: ['localhost', 'www.example.com']
        ca: ipa
    logging_pki_files:
      - ca_cert: /local/path/to/ca_cert.pem
        cert: /local/path/to/logging_cert.pem
        private_key: /local/path/to/logging_cert.pem
    logging_inputs:
      - name: input_name
        type: remote
        tcp_ports: 514
        tls: true
        permitted_clients: ['clients.example.com']
    logging_outputs:
      - name: output_name
        type: remote_files
        remote_log_path: /var/log/remote/%FROMHOST%/%PROGRAMNAME:::secpath-
replace%.log
        async_writing: true
        client_count: 20
        io_buffer_size: 8192
    logging_flows:
      - name: flow_name
        inputs: [input_name]
        outputs: [output_name]
```

Playbook은 다음 매개 변수를 사용합니다.

logging_certificates

이 매개변수의 값은 인증서 RHEL 시스템 역할의 **certificate _requests** 에 전달되며 개인 키와 인증서를 생성하는 데 사용됩니다.

logging_pki_files

이 매개변수를 사용하여 로깅에서 사용하는 경로 및 기타 설정을 구성하여 로깅에서 사용하는 CA, 인증서 및 TLS에 사용되는 키 파일을 찾습니다.**ca_cert_src**, **ca_cert** _src ,**cert**,**cert_src**, private_key ,**private_key** _**src**, **tls**.

> 참고
>
> **logging_certificates** 를 사용하여 대상 노드에서 파일을 생성하는 경우 **logging_certificates** 에서 생성되지 않은 파일을 복사하는 데 사용되는 **ca_cert_src**,**cert_src** 및 **private_key_src** 를 사용하지 마십시오.

ca_cert

대상 노드의 CA 인증서 파일의 경로를 나타냅니다. 기본 경로는 **/etc/pki/tls/certs/ca.pem** 이며 파일 이름은 사용자가 설정합니다.

인증서

대상 노드의 인증서 파일의 경로를 나타냅니다. 기본 경로는 **/etc/pki/tls/certs/server-cert.pem** 이며 파일 이름은 사용자가 설정합니다.

private_key

대상 노드의 개인 키 파일의 경로를 나타냅니다. 기본 경로는 **/etc/pki/tls/private/server-key.pem** 이며 파일 이름은 사용자가 설정합니다.

ca_cert_src

ca_cert 에서 지정한 위치로 대상 호스트에 복사되는 제어 노드의 CA 인증서 파일의 경로를 나타냅니다. **logging_certificates** 를 사용하는 경우 이 사용하지 마십시오.

cert_src

인증서에서 지정한 위치로 대상 호스트에 복사되는 제어 노드의 인증서 파일의 경로를 나타냅니다. **logging_certificates** 를 사용하는 경우 이 사용하지 마십시오.

private_key_src

private_key 에서 지정한 위치로 대상 호스트에 복사되는 제어 노드에서 개인 키 파일의 경로를 나타냅니다. **logging_certificates** 를 사용하는 경우 이 사용하지 마십시오.

tls

이 매개변수를 **true** 로 설정하면 네트워크를 통한 로그 전송이 안전합니다. 보안 래퍼가 필요하지 않은 경우 **tls: false** 를 설정할 수 있습니다.

2. 플레이북 구문을 확인합니다.

```
$ ansible-playbook --syntax-check ~/playbook.yml
```

이 명령은 구문만 검증하고 잘못되었지만 유효한 구성으로부터 보호하지 않습니다.

3. 플레이북을 실행합니다.

```
$ ansible-playbook ~/playbook.yml
```

추가 리소스

- **/usr/share/ansible/roles/rhel-system-roles.logging/README.md** 파일
- **/usr/share/doc/rhel-system-roles/logging/** 디렉터리
- RHEL 시스템 역할을 사용하여 인증서 요청

9.2.6. RELP에서 로깅 RHEL 시스템 역할 사용

RELP(Reliable Event Logging Protocol)는 TCP 네트워크를 통한 데이터 및 메시지 로깅을 위한 네트워킹 프로토콜입니다. 이 솔루션은 이벤트 메시지를 안정적으로 전송하며 메시지 손실을 허용하지 않는 환경에서 사용할 수 있습니다.

RELP 발신자는 로그 항목을 명령 형태로 전송하고 수신자는 처리되면 이를 승인합니다. 일관성을 보장하기 위해 RELP는 모든 종류의 메시지 복구를 위해 전송된 각 명령에 트랜잭션 번호를 저장합니다.

RELP Client와 RELP 서버 간에 원격 로깅 시스템을 고려할 수 있습니다. RELP Client는 로그를 원격 로깅 시스템으로 전송하고 RELP 서버는 원격 로깅 시스템에서 보낸 모든 로그를 수신합니다.

관리자는 로깅 RHEL 시스템 역할을 사용하여 로그 항목을 안정적으로 전송하고 수신하도록 로깅 시스템을 구성할 수 있습니다.

9.2.6.1. RELP로 클라이언트 로깅 구성

로깅 RHEL 시스템 역할을 사용하여 로컬 머신에 로깅되는 RHEL 시스템에 로그인을 구성하고 Ansible 플레이북을 실행하여 RELP를 사용하여 원격 로깅 시스템으로 로그를 전송할 수 있습니다.

이 절차에서는 Ansible 인벤토리의 **clients** 그룹에 있는 모든 호스트에서 RELP를 구성합니다. RELP 구성은 TLS(전송 계층 보안)를 사용하여 네트워크를 통해 로그를 안전하게 전송할 수 있도록 메시지 전송을 암호화합니다.

사전 요구 사항

- 컨트롤 노드 및 관리형 노드를 준비했습니다.
- 관리 노드에서 플레이북을 실행할 수 있는 사용자로 제어 노드에 로그인되어 있습니다.
- 관리 노드에 연결하는 데 사용하는 계정에는 **sudo** 권한이 있습니다.

절차

1. 다음 콘텐츠를 사용하여 플레이북 파일(예: ~/**playbook.yml**)을 생성합니다.

```
---
- name: Deploying basic input and relp output
  hosts: managed-node-01.example.com
  roles:
    - rhel-system-roles.logging
  vars:
    logging_inputs:
      - name: basic_input
        type: basics
    logging_outputs:
      - name: relp_client
        type: relp
        target: logging.server.com
        port: 20514
        tls: true
        ca_cert: /etc/pki/tls/certs/ca.pem
        cert: /etc/pki/tls/certs/client-cert.pem
        private_key: /etc/pki/tls/private/client-key.pem
        pki_authmode: name
        permitted_servers:
```

```
        - '*.server.example.com'
    logging_flows:
      - name: example_flow
        inputs: [basic_input]
        outputs: [relp_client]
```

플레이북은 다음 설정을 사용합니다.

대상

이는 원격 로깅 시스템이 실행 중인 호스트 이름을 지정하는 필수 매개 변수입니다.

port

원격 로깅 시스템이 수신 대기 중인 포트 번호입니다.

tls

네트워크를 통한 로그를 안전하게 전송합니다. 보안 래퍼가 필요하지 않은 경우 **tls** 변수를 **false**로 설정할 수 있습니다. 기본적으로 **tls** 매개변수는 RELP로 작업하는 동안 true로 설정되며 키/인증서 및 트립릿 **{ca_cert,cert,private_key}** 및/또는 **{ca_cert_src,cert_src,private_key_src}**가 필요합니다.

- **{ca_cert_src,cert_src,private_key_src}** triplet가 설정되면 기본 위치 **/etc/pki/tls/certs** 및 **/etc/pki/tls/private** 이 제어 노드에서 파일을 전송하는 대상으로 사용됩니다. 이 경우 파일 이름은 Triplet에서 원래 이름과 동일합니다.
- **{ca_cert,cert,private_key}** triplet가 설정되면 로깅 구성 전에 파일이 기본 경로에 있어야 합니다.
- 두 트래블릿이 설정되어 있으면 파일이 로컬 경로에서 제어 노드에서 관리 노드의 특정 경로로 전송됩니다.

ca_cert

CA 인증서 경로를 나타냅니다. 기본 경로는 **/etc/pki/tls/certs/ca.pem** 이며 파일 이름은 사용자가 설정합니다.

인증서

인증서 경로를 나타냅니다. 기본 경로는 **/etc/pki/tls/certs/server-cert.pem** 이며 파일 이름은 사용자가 설정합니다.

private_key

개인 키의 경로를 나타냅니다. 기본 경로는 **/etc/pki/tls/private/server-key.pem** 이며 파일 이름은 사용자가 설정합니다.

ca_cert_src

대상 호스트에 복사되는 로컬 CA 인증서 파일 경로를 나타냅니다. **ca_cert** 를 지정하면 위치에 복사됩니다.

cert_src

대상 호스트에 복사되는 로컬 인증서 파일 경로를 나타냅니다. **cert** 가 지정되면 해당 인증서가 위치에 복사됩니다.

private_key_src

대상 호스트에 복사되는 로컬 키 파일 경로를 나타냅니다. **private_key** 가 지정되면 해당 키가 위치에 복사됩니다.

pki_authmode

인증 모드를 이름 또는지문으로 허용합니다.

permitted_servers

로깅 클라이언트에서 TLS를 통해 연결하고 로그를 전송하도록 허용하는 서버 목록입니다.

입력

로깅 입력 사전 목록.

출력

로깅 출력 사전 목록.

2. 플레이북 구문을 확인합니다.

```
$ ansible-playbook --syntax-check ~/playbook.yml
```

이 명령은 구문만 검증하고 잘못되었지만 유효한 구성으로부터 보호하지 않습니다.

3. 플레이북을 실행합니다.

```
$ ansible-playbook ~/playbook.yml
```

추가 리소스

- **/usr/share/ansible/roles/rhel-system-roles.logging/README.md** 파일
- **/usr/share/doc/rhel-system-roles/logging/** 디렉터리

9.2.6.2. RELP로 서버 로깅 구성

로깅 RHEL 시스템 역할을 사용하여 RHEL 시스템에서 서버로 로깅을 구성하고 Ansible 플레이북을 실행하여 RELP를 사용하여 원격 로깅 시스템에서 로그를 수신할 수 있습니다.

이 절차에서는 Ansible 인벤토리의 서버 그룹에 있는 모든 호스트에서 RELP를 구성합니다. RELP 구성은 TLS를 사용하여 네트워크를 통해 로그를 안전하게 전송할 수 있도록 메시지 전송을 암호화합니다.

사전 요구 사항

- 컨트롤 노드 및 관리형 노드를 준비했습니다.
- 관리 노드에서 플레이북을 실행할 수 있는 사용자로 제어 노드에 로그인되어 있습니다.
- 관리 노드에 연결하는 데 사용하는 계정에는 **sudo** 권한이 있습니다.

절차

1. 다음 콘텐츠를 사용하여 플레이북 파일(예: **~/playbook.yml**)을 생성합니다.

```
---
- name: Deploying remote input and remote_files output
  hosts: managed-node-01.example.com
  roles:
    - rhel-system-roles.logging
  vars:
    logging_inputs:
      - name: relp_server
        type: relp
        port: 20514
        tls: true
```

```
      ca_cert: /etc/pki/tls/certs/ca.pem
      cert: /etc/pki/tls/certs/server-cert.pem
      private_key: /etc/pki/tls/private/server-key.pem
      pki_authmode: name
      permitted_clients:
        - '*example.client.com'
    logging_outputs:
      - name: remote_files_output
        type: remote_files
    logging_flows:
      - name: example_flow
        inputs: relp_server
        outputs: remote_files_output
```

플레이북은 다음 설정을 사용합니다.

port

원격 로깅 시스템이 수신 대기 중인 포트 번호입니다.

tls

네트워크를 통한 로그를 안전하게 전송합니다. 보안 래퍼가 필요하지 않은 경우 **tls** 변수를 **false**로 설정할 수 있습니다. 기본적으로 **tls** 매개변수는 RELP로 작업하는 동안 true로 설정되며 키/인증서 및 트립릿 {**ca_cert,cert,private_key**} 및/또는 {**ca_cert_src,cert_src,private_key_src**}가 필요합니다.

- {**ca_cert_src,cert_src,private_key_src**} triplet가 설정되면 기본 위치 **/etc/pki/tls/certs** 및 **/etc/pki/tls/private** 이 제어 노드에서 파일을 전송하는 대상으로 사용됩니다. 이 경우 파일 이름은 Triplet에서 원래 이름과 동일합니다.
- {**ca_cert,cert,private_key**} triplet가 설정되면 로깅 구성 전에 파일이 기본 경로에 있어야 합니다.
- 두 트래블릿이 설정되어 있으면 파일이 로컬 경로에서 제어 노드에서 관리 노드의 특정 경로로 전송됩니다.

ca_cert

CA 인증서 경로를 나타냅니다. 기본 경로는 **/etc/pki/tls/certs/ca.pem** 이며 파일 이름은 사용자가 설정합니다.

인증서

인증서의 경로를 나타냅니다. 기본 경로는 **/etc/pki/tls/certs/server-cert.pem** 이며 파일 이름은 사용자가 설정합니다.

private_key

개인 키의 경로를 나타냅니다. 기본 경로는 **/etc/pki/tls/private/server-key.pem** 이며 파일 이름은 사용자가 설정합니다.

ca_cert_src

대상 호스트에 복사되는 로컬 CA 인증서 파일 경로를 나타냅니다. **ca_cert** 를 지정하면 위치에 복사됩니다.

cert_src

대상 호스트에 복사되는 로컬 인증서 파일 경로를 나타냅니다. **cert** 가 지정되면 해당 인증서가 위치에 복사됩니다.

private_key_src

대상 호스트에 복사되는 로컬 키 파일 경로를 나타냅니다. **private_key** 가 지정되면 해당 키가 위치에 복사됩니다.

pki_authmode

인증 모드를 이름 또는지문으로 허용합니다.

permitted_clients

로깅 서버가 TLS를 통해 연결하고 로그를 전송하도록 허용하는 클라이언트 목록입니다.

입력

로깅 입력 사전 목록.

출력

로깅 출력 사전 목록.

2. 플레이북 구문을 확인합니다.

```
$ ansible-playbook --syntax-check ~/playbook.yml
```

이 명령은 구문만 검증하고 잘못되었지만 유효한 구성으로부터 보호하지 않습니다.

3. 플레이북을 실행합니다.

```
$ ansible-playbook ~/playbook.yml
```

추가 리소스

- **/usr/share/ansible/roles/rhel-system-roles.logging/README.md** 파일
- **/usr/share/doc/rhel-system-roles/logging/** 디렉터리

9.2.7. 추가 리소스

- RHEL 시스템 역할을 사용하도록 컨트롤 노드 및 관리형 노드 준비
- **/usr/share/ansible/ roles/rhel-system-roles.logging/README.html**에 **rhel -system-roles** 패키지와 함께 설치된 설명서.
- RHEL 시스템 역할
- 시스템의 **ansible-playbook(1)** 도움말 페이지.

10장. 로그 파일을 사용하여 문제 해결

로그 파일에는 커널, 서비스 및 실행 중인 애플리케이션을 포함하여 시스템에 대한 메시지가 포함되어 있습니다. 여기에는 문제를 해결하거나 시스템 기능을 모니터링하는 데 도움이 되는 정보가 포함됩니다. Red Hat Enterprise Linux의 로깅 시스템은 기본 제공된 **syslog** 프로토콜을 기반으로 합니다. 특정 프로그램은 이 시스템을 사용하여 이벤트를 기록하고 이를 로그 파일로 구성하여 운영 체제를 감사하고 다양한 문제를 해결할 때 유용합니다.

10.1. SYSLOG 메시지를 처리하는 서비스

다음 두 서비스는 **syslog** 메시지를 처리합니다.

- **systemd-journald** 데몬
- **Rsyslog** 서비스

systemd-journald 데몬은 다양한 소스에서 메시지를 수집하고 추가 처리를 위해**Rsyslog** 에 전달합니다. **systemd-journald** 데몬은 다음 소스에서 메시지를 수집합니다.

- 커널
- 부팅 프로세스 초기 단계
- 데몬을 시작하고 실행할 때의 표준 및 오류 출력
- **syslog**

Rsyslog 서비스는 **syslog** 메시지를 유형 및 우선 순위에 따라 정렬하여**/var/log** 디렉터리의 파일에 기록합니다. **/var/log** 디렉토리는 로그 메시지를 영구적으로 저장합니다.

10.2. SYSLOG 메시지를 저장하는 하위 디렉터리

/var/log 디렉토리 아래에 있는 다음 하위 디렉터리는**syslog** 메시지를 저장합니다.

- **/var/log/messages** - 다음을 제외한 모든**syslog** 메시지
- **/var/log/secure** - 보안 및 인증 관련 메시지 및 오류
- **/var/log/maillog** - 메일 서버 관련 메시지 및 오류
- **/var/log/cron** - 정기적으로 실행된 작업과 관련된 로그 파일
- **/var/log/boot.log** - 시스템 시작과 관련된 로그 파일

10.3. 명령줄을 사용하여 로그 보기

저널은 로그 파일을 보고 관리하는 데 도움이 되는 systemd의 구성 요소입니다. 기존 로깅과 연결된 문제를 해결하고 나머지 시스템과 긴밀하게 통합되며 로그 파일의 다양한 로깅 기술과 액세스 관리를 지원합니다.

journalctl 명령을 사용하여 명령줄을 사용하여 시스템 저널의 메시지를 볼 수 있습니다. 예를 들면 다음과 같습니다.

```
$ journalctl -b | grep kvm
May 15 11:31:41 localhost.localdomain kernel: kvm-clock: Using msrs 4b564d01 and 4b564d00
May 15 11:31:41 localhost.localdomain kernel: kvm-clock: cpu 0, msr 76401001, primary cpu clock
```

```
...
```

표 10.1. 시스템 정보 보기

명령	설명
journalctl	는 수집된 모든 저널 항목을 표시합니다.
journalctl FILEPATH	는 특정 파일과 관련된 로그를 표시합니다. 예를 들어 **journalctl /dev/sda** 명령은 **/dev/sda** 파일 시스템과 관련된 로그를 표시합니다.
journalctl -b	현재 부팅에 대한 로그를 표시합니다.
journalctl -k -b -1	현재 부팅에 대한 커널 로그를 표시합니다.

표 10.2. 특정 서비스에 대한 정보 보기

명령	설명
journalctl -b _SYSTEMD_UNIT=*<name.service>*	로그를 필터링하여 **systemd** 서비스와 일치하는 항목을 표시합니다.
journalctl -b _SYSTEMD_UNIT=*<name.service>* _PID=*<number>*	일치 결합. 예를 들어 이 명령은 < ***name.service*** > 및 PID < ***number*** >와 일치하는 **systemd-units** 에 대한 로그를 표시합니다.
journalctl -b _SYSTEMD_UNIT= *<name.service* > _PID= *<number* > + _SYSTEMD_UNIT= *<name2.service>*	더하기 기호(+) 구분 기호는 논리 OR로 두 표현식을 결합합니다. 예를 들어 이 명령은 < ***name.service*** > 서비스 프로세스의 모든 메시지와 < ***name2.service*** > 서비스의 모든 메시지(프로세스 중 하나)를 표시합니다.
journalctl -b _SYSTEMD_UNIT=*<name.service>* _SYSTEMD_UNIT=*<name2.service>*	이 명령은 동일한 필드를 참조하여 두 표현식과 일치하는 모든 항목을 표시합니다. 여기에서 이 명령은 systemd-unit < ***name.service*** > 또는 systemd-unit < ***name2.service*** >와 일치하는 로그를 표시합니다.

표 10.3. 특정 부팅과 관련된 로그 보기

명령	설명
journalctl --list-boots	에는 부팅 번호, ID 및 부팅과 관련된 첫 번째 및 마지막 메시지의 타임스탬프 목록이 표시됩니다. 다음 명령의 ID를 사용하여 자세한 정보를 볼 수 있습니다.

명령	설명
journalctl --boot=ID _SYSTEMD_UNIT=*<name.service>*	지정된 부팅 ID에 대한 정보를 표시합니다.

10.4. 웹 콘솔에서 로그 검토

RHEL 웹 콘솔에서 로그에 액세스, 검토 및 필터링하는 방법을 알아봅니다.

10.4.1. 웹 콘솔에서 로그 검토

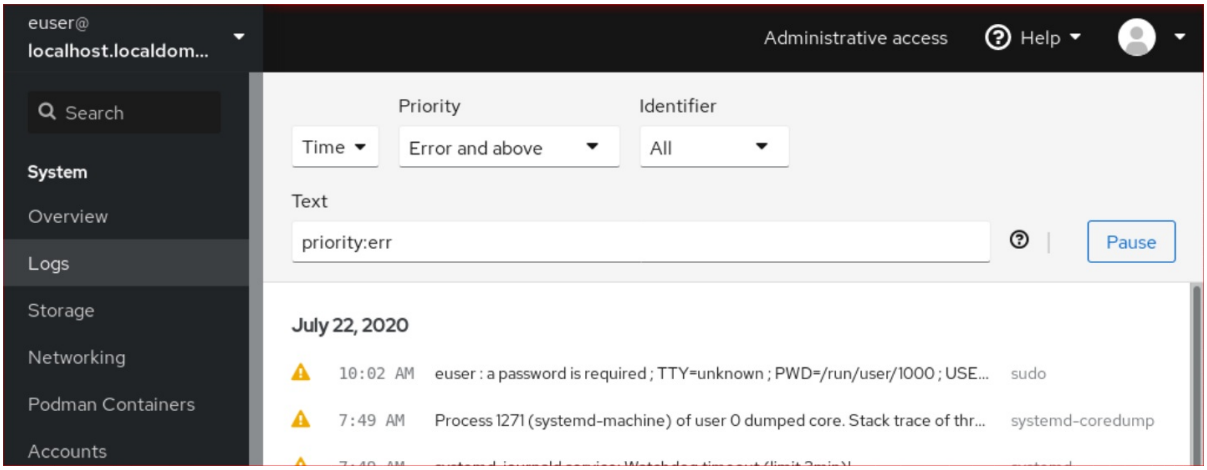
RHEL 8 웹 콘솔 로그 섹션은 **journalctl** 유틸리티의 UI입니다. 웹 콘솔 인터페이스에서 시스템 로그에 액세스할 수 있습니다.

사전 요구 사항

- RHEL 8 웹 콘솔을 설치했습니다.
  자세한 내용은 웹 콘솔 설치 및 활성화를 참조하십시오.

절차

1. RHEL 8 웹 콘솔에 로그인합니다.
   자세한 내용은 웹 콘솔에 로그인 을 참조하십시오.
2. 로그를 클릭합니다.



3. 목록에서 선택한 로그 항목을 클릭하여 로그 항목 세부 정보를 엽니다.

> 참고
>
> 일시 중지 버튼을 사용하여 새 로그 항목이 표시되지 않도록 일시 중지할 수 있습니다. 새 로그 항목을 다시 시작하면 웹 콘솔은 일시 정지 버튼을 사용한 후 보고된 모든 로그 항목을 로드합니다.

로그는 시간, 우선 순위 또는 식별자별로 필터링할 수 있습니다. 자세한 내용은 웹 콘솔에서 로그 필터링을 참조하십시오.

10.4.2. 웹 콘솔에서 로그 필터링

웹 콘솔에서 로그 항목을 필터링할 수 있습니다.

사전 요구 사항

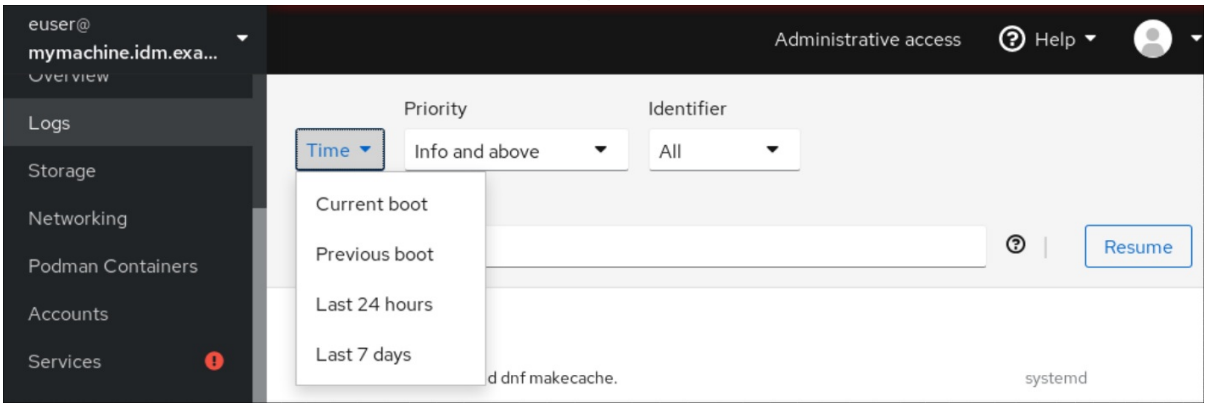
- RHEL 8 웹 콘솔을 설치했습니다.
  자세한 내용은 웹 콘솔 설치 및 활성화를 참조하십시오.

절차

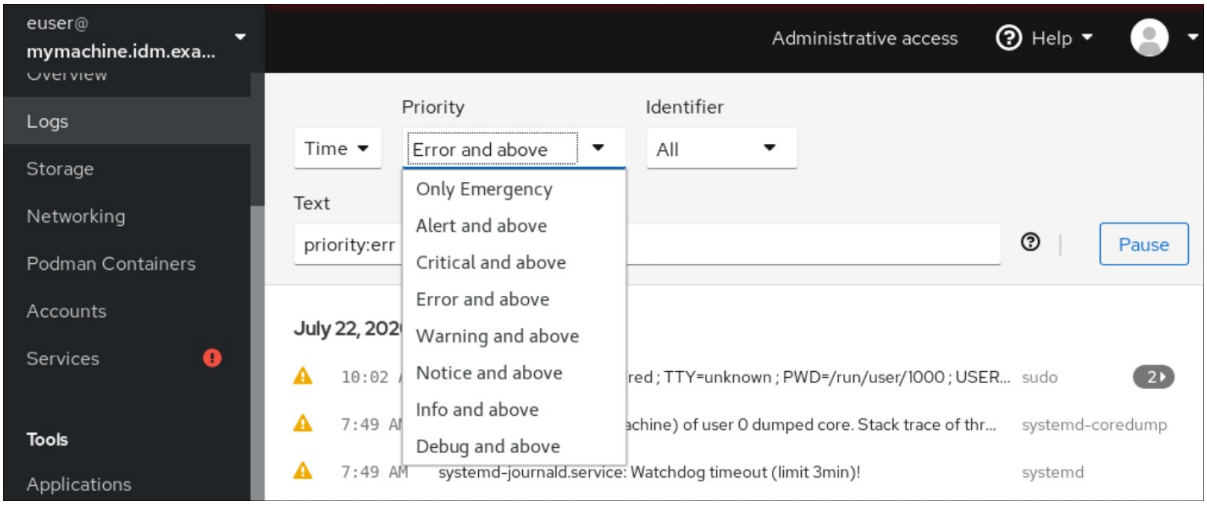
1. RHEL 8 웹 콘솔에 로그인합니다.
   자세한 내용은 웹 콘솔에 로그인 을 참조하십시오.

2. 로그를 클릭합니다.

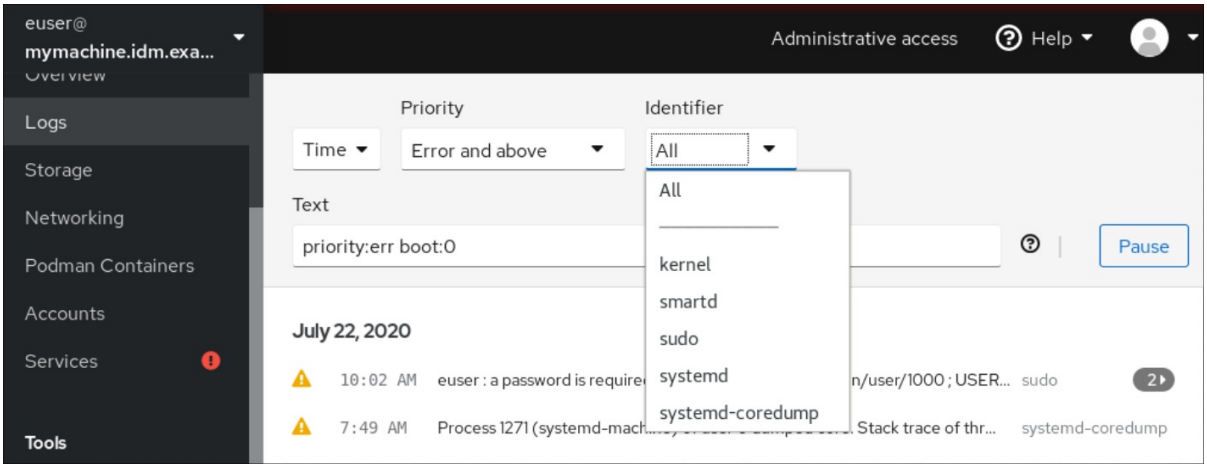
3. 기본적으로 웹 콘솔은 최신 로그 항목을 표시합니다. 특정 시간 범위별로 필터링하려면 시간 드롭다운 메뉴를 클릭하고 선호하는 옵션을 선택합니다.



4. 오류 및 심각도 로그 목록이 기본적으로 표시됩니다. 다른 우선 순위로 필터링하려면Error and above 드롭다운 메뉴를 클릭하고 선호하는 우선 순위를 선택합니다.



5. 기본적으로 웹 콘솔은 모든 식별자에 대한 로그를 표시합니다. 특정 식별자에 대한 로그를 필터링하려면 모두 드롭다운 메뉴를 클릭하고 식별자를 선택합니다.



6. 로그 항목을 열려면 선택한 로그를 클릭합니다.

10.4.3. 웹 콘솔에서 로그 필터링을 위한 텍스트 검색 옵션

텍스트 검색 옵션 기능은 로그 필터링을 위한 다양한 옵션을 제공합니다. 텍스트 검색을 사용하여 로그를 필터링하려면 세 드롭다운 메뉴에 정의된 사전 정의된 옵션을 사용하거나 전체 검색을 직접 입력할 수 있습니다.

드롭다운 메뉴

검색의 기본 매개변수를 지정하는 데 사용할 수 있는 세 가지 드롭다운 메뉴가 있습니다.

- 시간: 이 드롭다운 메뉴에는 다양한 검색 시간 범위에 대한 사전 정의된 검색이 포함되어 있습니다.
- priority: 이 드롭다운 메뉴에서는 다양한 우선 순위 수준에 대한 옵션을 제공합니다.**journalctl --priority** 옵션에 해당합니다. 기본 우선순위 값은Error 이상입니다. 다른 우선 순위를 지정하지 않을 때마다 설정됩니다.
- 식별자: 이 드롭다운 메뉴에서 필터링할 식별자를 선택할 수 있습니다.**journalctl --identifier** 옵션에 해당합니다.

Cryostatifiers

검색을 지정하는 데 사용할 수 있는 6가지 한정자가 있습니다. 로그 필터링 테이블에는 옵션이 포함되어 있습니다.

로그 필드

특정 로그 필드를 검색하려면 해당 콘텐츠와 함께 필드를 지정할 수 있습니다.

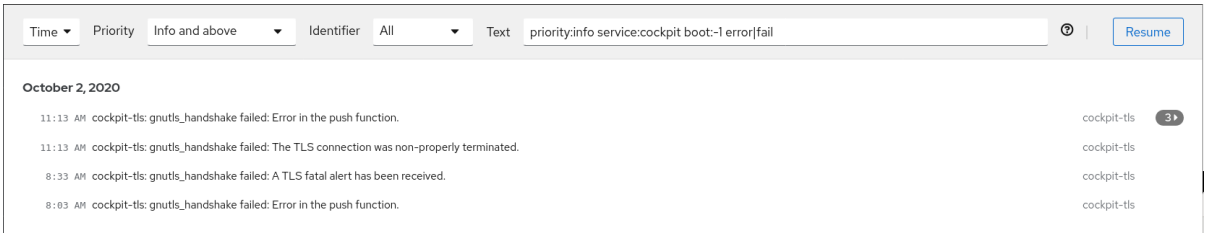
로그 메시지에서 자유형 텍스트 검색

로그 메시지에서 선택한 텍스트 문자열을 필터링할 수 있습니다. 문자열은 정규식 형식일 수도 있습니다.

고급 로그 필터링 I

2020년 10월 22일 자정 이후 발생한 'systemd'로 식별된 모든 로그 메시지를 필터링하고 'JOB_TYPE' 필드는 'start' 또는 'restart'입니다.

1. type **identifier:systemd since:2020-10-22 JOB_TYPE=start,restart** to search field.
2. 결과를 확인하십시오.

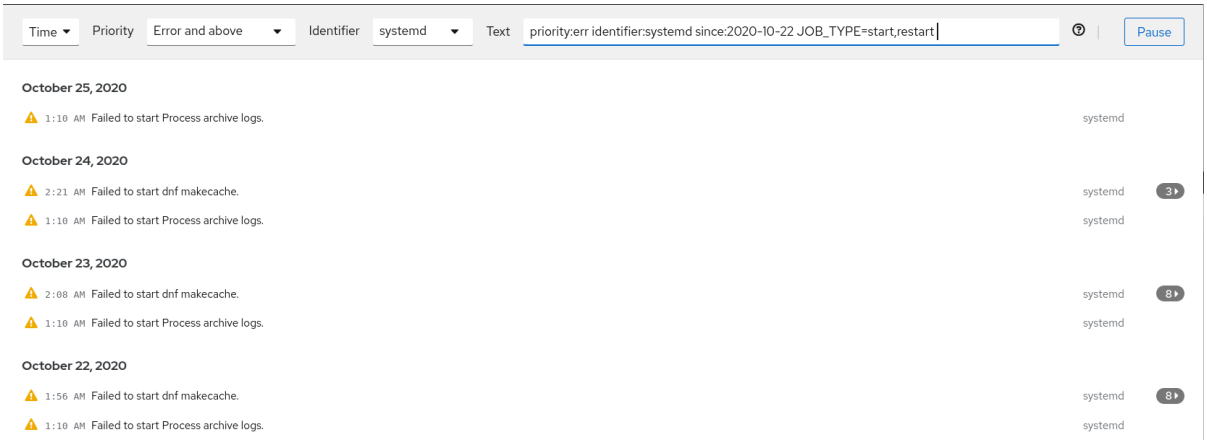


고급 로그 필터링 II

마지막 전에 부팅에 발생한 'cockpit.service' systemd 장치에서 제공되는 모든 로그 메시지를 필터링하고 메시지 본문에는 "error" 또는 "fail"가 포함됩니다.

1. **service:cockpit boot:-1 error|fail** to the search field를 입력합니다.

2. 결과를 확인하십시오.



10.4.4. 텍스트 검색 상자를 사용하여 웹 콘솔에서 로그를 필터링

웹 콘솔의 텍스트 검색 상자를 사용하여 다양한 매개변수에 따라 로그를 필터링할 수 있습니다. 검색에서는 필터링 드롭다운 메뉴, 정량자, 로그 필드 및 자유 형식 문자열 검색의 사용을 결합합니다.

사전 요구 사항

- RHEL 8 웹 콘솔을 설치했습니다.
  자세한 내용은 웹 콘솔 설치 및 활성화를 참조하십시오.

절차

1. RHEL 웹 콘솔에 로그인합니다.
   자세한 내용은 웹 콘솔에 로그인을 참조하십시오.

2. 로그를 클릭합니다.

3. 드롭다운 메뉴를 사용하여 필터링하려는 세 가지 기본 한정자(시간 범위, 우선 순위 및 식별자)를 지정합니다.
   우선 순위 한정자는 항상 값이 있어야 합니다. 이를 지정하지 않으면Error 및 above 우선 순위를 자동으로 필터링합니다. 설정한 옵션은 텍스트 검색 상자에 반영됩니다.

4. 필터링할 로그 필드를 지정합니다.
   여러 로그 필드를 추가할 수 있습니다.

5. 자유 형식 문자열을 사용하여 다른 항목을 검색할 수 있습니다. 검색 상자에는 정규식도 사용할 수 있습니다.

10.4.5. 로그 필터링 옵션

웹 콘솔에서 로그를 필터링하는 데 사용할 수 있는 여러 journalctl 옵션이 있습니다. 이 옵션은 유용할 수 있습니다. 이 중 일부는 웹 콘솔 인터페이스에서 드롭다운 메뉴의 일부로 이미 포함되어 있습니다.

표 10.4. 표

옵션 이름	사용법	참고
priority	메시지 우선 순위에 따라 출력을 필터링합니다. 단일 숫자 또는 텍스트 로그 수준을 사용합니다. 로그 수준은 일반적인 syslog 로그 수준입니다. 단일 로그 수준이 지정되면 이 로그 수준이 있는 모든 메시지 또는 하위(가장 중요함) 로그 수준이 표시됩니다.	**우선 순위** 드롭다운 메뉴에서 다룹니다.
identifier	지정된 syslog 식별자 SYSLOG_IDENTIFIER에 대한 메시지를 표시합니다. 여러 번 지정할 수 있습니다.	**ID** 드롭다운 메뉴에서 다룹니다.
팔로우	가장 최근의 저널 항목만 표시하고 저널에 추가되는 대로 새 항목을 지속적으로 출력합니다.	드롭다운에는 적용되지 않습니다.
서비스	지정된 **systemd** 장치에 대한 메시지를 표시합니다. 여러 번 지정할 수 있습니다.	드롭다운에 포함되지 않습니다. **journalctl --unit** 매개변수에 해당합니다.
부팅	특정 부팅의 메시지를 표시합니다. 양의 정수는 저널의 시작부터 부팅을 조회하고, 동일하거나 무의미한 0 정수는 저널이 끝날 때부터 부팅을 조회합니다. 따라서 1은 저널에 있는 첫 번째 부팅을 시간순으로, 2 번 이상 두 번째 부팅을 의미합니다. -0은 마지막 부팅이고, 마지막 부팅은 -1이 됩니다.	**현재 부팅** 또는 **시간** 드롭다운 메뉴에서 **이전 부팅** 으로만 적용됩니다. 다른 옵션은 수동으로 작성해야 합니다.

옵션 이름	사용법	참고
이후	지정된 날짜보다 크거나 지정된 날짜보다 항목을 각각 표시하거나 그 이전 날짜를 표시합니다. 날짜 사양은 "2012-10-30 18:17:16" 형식이어야 합니다. 시간 부분이 생략되면 "00:00:00"이 사용됩니다. 초 구성 요소만 생략하면 ":00"이 사용됩니다. 날짜 구성 요소를 생략하면 현재 날짜로 가정합니다. 또는 문자열 "yesterday", "today", "tomorrow"가 이해되며, 이는 현재 날 전날, 현재 날짜 또는 하루 전의 00:00:00을 나타냅니다. "현재"는 현재 시간을 나타냅니다. 마지막으로, 현재 시간 전이나 후에 각각 "-" 또는 "+" 접두사를 지정하여 상대 시간을 지정할 수 있습니다.	드롭다운에는 적용되지 않습니다.

10.5. 추가 리소스

- **journalctl(1) 시스템의 도움말 페이지**
- 원격 로깅 솔루션 구성

11장. 사용자 및 그룹 관리

파일 및 프로세스에 대한 무단 액세스를 방지하려면 정확한 사용자 및 그룹 관리가 필요합니다. 계정을 중앙에서 관리하지 않거나 특정 시스템에서만 사용자 계정 또는 그룹이 필요한 경우 이 호스트에서 로컬로 생성할 수 있습니다.

11.1. 사용자 및 그룹 계정 관리 소개

사용자 및 그룹의 제어는 RHEL(Red Hat Enterprise Linux) 시스템 관리의 핵심 요소입니다. 각 RHEL 사용자에게는 고유한 로그인 자격 증명이 있으며 다양한 그룹에 할당하여 시스템 권한을 사용자 지정할 수 있습니다.

11.1.1. 사용자 및 그룹 소개

파일을 생성하는 사용자는 해당 파일의 소유자와 해당 *파일의* 그룹 소유자입니다. 파일에는 소유자, 그룹 및 해당 그룹 외부에 대한 별도의 읽기, 쓰기, 실행 권한이 할당됩니다. 파일 소유자는 **root** 사용자만 변경할 수 있습니다. 파일에 대한 액세스 권한은 **root** 사용자와 파일 소유자가 모두 변경할 수 있습니다. 일반 사용자는 소유한 파일의 그룹 소유권을 멤버인 그룹으로 변경할 수 있습니다.

각 사용자는UID(사용자 *ID*)라는 고유한 숫자 식별 번호와 연결됩니다. 각 그룹은GID(*그룹 ID*)와 연결됩니다. 그룹 내의 사용자는 동일한 권한을 공유하여 해당 그룹이 소유한 파일을 읽고, 쓰고, 실행할 수 있습니다.

11.1.2. 예약된 사용자 및 그룹 ID 구성

RHEL은 1000 미만의 사용자 및 그룹 ID를 시스템 사용자 및 그룹을 위해 예약합니다. 예약된 사용자 및 그룹 ID는 **setup** 패키지에서 찾을 수 있습니다. 예약된 사용자 및 그룹 ID를 보려면 다음을 사용합니다.

```
cat /usr/share/doc/setup*/uidgid
```

향후 예약 범위가 증가할 수 있으므로 5000부터 시작하는 새 사용자 및 그룹에 ID를 할당하는 것이 좋습니다.

새 사용자에게 할당된 ID가 기본적으로 5000에서 시작되도록 하려면 **/etc/login.defs** 파일에서 **UID _MIN** 및 **GID_MIN** 매개 변수를 수정합니다.

절차

새 사용자에게 할당된 ID를 수정하려면 기본적으로 5000에서 시작합니다.

1. 선택한 편집기에서 **/etc/login.defs** 파일을 엽니다.
2. 자동 **UID** 선택에 대한 최소 값을 정의하는 행을 찾습니다.

```
# Min/max values for automatic uid selection in useradd
#
UID_MIN                  1000
```

3. 5000에서 시작되도록 **UID_MIN** 값을 수정합니다.

```
# Min/max values for automatic uid selection in useradd
#
UID_MIN                  5000
```

4. 자동 **GID** 선택에 대한 최소 값을 정의하는 행을 찾습니다.

```
# Min/max values for automatic gid selection in groupadd
#
GID_MIN                 1000
```

5. **GID_MIN** 값을 수정하여 5000에서 시작합니다.

```
# Min/max values for automatic gid selection in groupadd
#
GID_MIN                 5000
```

일반 사용자에 대해 동적으로 할당된 UID 및 GID는 이제 5000에서 시작합니다.

참고

UID_MIN 및 GID_MIN 값을 변경하기 전에 생성된 UID 및 GID의 사용자 및 그룹은 변경되지 않습니다.

이렇게 하면 새 사용자 그룹이 UID 및 GID와 5000+ ID를 가질 수 있습니다.

주의

1000 제한을 유지하는 시스템과의 충돌을 피하도록 **SYS_UID_MAX** 를 변경하여 1000 이상의 시스템에서 예약한 ID를 늘리지 마십시오.

11.1.3. 사용자 개인 그룹

RHEL은 *사용자 개인 그룹* (UPG) 시스템 구성을 사용하므로 UNIX 그룹을 더 쉽게 관리할 수 있습니다. 새 사용자가 시스템에 추가될 때마다 사용자 개인 그룹이 생성됩니다. 사용자 개인 그룹에는 생성된 사용자와 이름이 동일하며 사용자는 사용자 개인 그룹의 유일한 멤버입니다.

UPG는 여러 사용자 간의 프로젝트에서의 협업을 간소화합니다. 또한 UPG 시스템 구성을 사용하면 새로 생성된 파일 또는 디렉토리에 대한 기본 권한을 안전하게 설정할 수 있으며, 이 사용자는 파일 또는 디렉터리를 수정할 수 있습니다.

모든 그룹 목록은 **/etc/group** 구성 파일에 저장됩니다.

11.2. 사용자 계정 관리 시작하기

Red Hat Enterprise Linux는 여러 컴퓨터의 여러 사용자가 하나의 시스템에 설치된 단일 시스템에 액세스할 수 있도록 하는 다중 사용자 운영 체제입니다. 모든 사용자는 자체 계정으로 작동하므로 사용자 계정을 관리하므로 Red Hat Enterprise Linux 시스템 관리의 핵심 요소가 됩니다.

다음은 다양한 유형의 사용자 계정입니다.

- 일반 사용자 계정:
  특정 시스템의 사용자에 대한 일반 계정이 생성됩니다. 이러한 계정은 일반 시스템 관리 중에 추가, 제거 및 수정할 수 있습니다.

- 시스템 사용자 계정:

시스템 사용자 계정은 시스템의 특정 애플리케이션 식별자를 나타냅니다. 이러한 계정은 일반적으로 소프트웨어 설치 시에만 추가 또는 조작되며 나중에 수정되지 않습니다.

> **주의**
>
> 시스템 계정은 시스템에서 로컬로 사용할 수 있다고 가정합니다. LDAP 구성 인스턴스와 같이 이러한 계정을 원격으로 구성하고 제공되는 경우 시스템 중단 및 서비스 시작 오류가 발생할 수 있습니다.

시스템 계정의 경우 1000 미만의 사용자 ID가 예약됩니다. 일반 계정의 경우 1000부터 시작하는 ID를 사용할 수 있습니다. 그러나 5000에서 시작하는 ID를 할당하는 것이 좋습니다. ID를 할당하려면 **/etc/login.defs** 파일을 참조하십시오.

- 그룹:
  그룹은 특정 파일에 대한 액세스 권한 부여 등 공통 목적으로 여러 사용자 계정을 연결하는 엔터티입니다.

11.2.1. 명령줄 도구를 사용하여 계정 및 그룹 관리

다음 기본 명령줄 툴을 사용하여 사용자 계정 및 그룹을 관리합니다.

- 사용자 및 그룹 ID를 표시하려면 다음을 수행합니다.

```
$ id
uid=1000(example.user) gid=1000(example.user)
groups=1000(example.user),10(wheel)
context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
```

- 새 사용자 계정을 생성하려면 다음을 수행합니다.

```
# useradd example.user
```

- *example.user에 속하는 사용자 계정에 새 암호를 할당하려면 다음을 수행합니다.*

```
# passwd example.user
```

- 그룹에 사용자를 추가하려면 다음을 수행합니다.

```
# usermod -a -G example.group example.user
```

추가 리소스

- **man useradd(8), man passwd(1)** 및 **man usermod(8)**

11.3. 명령줄에서 사용자 관리

CLI(명령줄 인터페이스)를 사용하여 사용자와 그룹을 관리할 수 있습니다. 이를 통해 Red Hat Enterprise Linux 환경에서 사용자와 사용자 그룹을 추가, 제거 및 수정할 수 있습니다.

11.3.1. 명령줄에서 새 사용자 추가

useradd 유틸리티를 사용하여 새 사용자를 추가할 수 있습니다.

사전 요구 사항

- 루트 액세스

절차

- 새 사용자를 추가하려면 다음을 사용합니다.

```
# useradd options username
```

옵션을 **useradd** 명령의 명령줄 옵션으로 바꾸고 *username* 을 사용자 이름으로 바꿉니다.

예 11.1. 새 사용자 추가

사용자 ID **5000** 으로 사용자 **sarah** 를 추가하려면 다음을 사용합니다.

```
# useradd -u 5000 sarah
```

검증

- 새 사용자가 추가되었는지 확인하려면 **id** 유틸리티를 사용합니다.

```
# id sarah
```

출력은 다음을 반환합니다.

```
uid=5000(sarah) gid=5000(sarah) groups=5000(sarah)
```

추가 리소스

- 시스템에 **useradd** 도움말 페이지

11.3.2. 명령줄에서 새 그룹 추가

groupadd 유틸리티를 사용하여 새 그룹을 추가할 수 있습니다.

사전 요구 사항

- 루트 액세스

절차

- 새 그룹을 추가하려면 다음을 사용합니다.

```
# groupadd options group-name
```

옵션을 **groupadd** 명령의 명령줄 옵션으로 바꾸고 *group-name* 을 그룹 이름으로 바꿉니다.

> 예 11.2. 새 그룹 추가
>
> 그룹 ID가 **5000** 인 **sysadmins** 그룹을 추가하려면 다음을 사용합니다.
>
> ```
> # groupadd -g 5000 sysadmins
> ```

검증

- 새 그룹이 추가되었는지 확인하려면 **tail** 유틸리티를 사용합니다.

  ```
  # tail /etc/group
  ```

  출력은 다음을 반환합니다.

  ```
  sysadmins:x:5000:
  ```

추가 리소스

- 시스템의 **groupadd** 도움말 페이지

11.3.3. 명령줄에서 사용자를 보조 그룹에 추가

보조 그룹에 사용자를 추가하여 권한을 관리하거나 특정 파일 또는 장치에 대한 액세스를 활성화할 수 있습니다.

사전 요구 사항

- 루트 액세스

절차

- 사용자의 보조 그룹에 그룹을 추가하려면 다음을 사용합니다.

  ```
  # usermod --append -G group-name username
  ```

  group-name 을 그룹의 이름으로 바꾸고*username* 을 사용자 이름으로 바꿉니다.

> 예 11.3. 보조 그룹에 사용자 추가
>
> 사용자 **sysadmin** 을 **system-admins** 그룹에 추가하려면 다음을 사용합니다.
>
> ```
> # usermod --append -G system-administrators sysadmin
> ```

검증

- 새 그룹이 사용자 **sysadmin** 의 보조 그룹에 추가되었는지 확인하려면 다음을 사용합니다.

  ```
  # groups sysadmin
  ```

출력이 표시됩니다.

```
sysadmin : sysadmin system-administrators
```

11.3.4. 그룹 디렉토리 생성

UPG 시스템 구성에서 *set-group 식별 권한*(setgid bit)을 디렉터리에 적용할 수 있습니다.**setgid** 비트를 사용하면 디렉터리를 공유하는 그룹 프로젝트를 더 간단하게 관리할 수 있습니다. 디렉터리에 **setgid** 비트를 적용하면 해당 디렉터리 내에 생성된 파일이 디렉토리를 소유한 그룹에 자동으로 할당됩니다. 이 그룹 내에서 작성하고 실행할 수 있는 권한이 있는 사용자는 이제 디렉터리에 파일을 생성, 수정 및 삭제할 수 있습니다.

다음 섹션에서는 그룹 디렉터리를 만드는 방법을 설명합니다.

사전 요구 사항

- 루트 액세스

절차

1. 디렉터리를 생성합니다.

   ```
   # mkdir directory-name
   ```

   directory-name 을 디렉터리 이름으로 바꿉니다.

2. 그룹을 생성합니다.

   ```
   # groupadd group-name
   ```

   group-name 을 그룹 이름으로 바꿉니다.

3. 그룹에 사용자를 추가합니다.

   ```
   # usermod --append -G group-name username
   ```

   group-name 을 그룹의 이름으로 바꾸고*username* 을 사용자 이름으로 바꿉니다.

4. 디렉터리의 사용자 및 그룹 소유권을 *group-name* 그룹과 연결합니다.

   ```
   # chgrp group-name directory-name
   ```

   group-name 을 그룹의 이름으로 바꾸고,*directory-name* 을 디렉터리 이름으로 바꿉니다.

5. 사용자가 파일 및 디렉터리를 생성 및 수정하고 **setgid** 비트를 설정하여 *directory-name* 디렉토리 내에서 이 권한을 적용하도록 허용하도록 쓰기 권한을 설정합니다.

   ```
   # chmod g+rwxs directory-name
   ```

   directory-name 을 디렉터리 이름으로 바꿉니다.

   이제 ***group-name*** 그룹의 모든 멤버가 ***directory-name*** 디렉터리에 파일을 만들고 편집할 수 있습니다. 새로 생성된 파일은 ***group-name*** 그룹의 그룹 소유권을 유지합니다.

검증

- 권한 설정의 정확성을 확인하려면 다음을 사용합니다.

```
# ls -ld directory-name
```

directory-name 을 디렉터리 이름으로 바꿉니다.

출력은 다음을 반환합니다.

```
drwxrwsr-x. 2 root group-name 6 Nov 25 08:45 directory-name
```

11.3.5. 명령줄에서 사용자 제거

명령줄을 사용하여 사용자 계정을 제거할 수 있습니다. 사용자 계정을 제거하는 것 외에도 선택적으로 홈 디렉터리 및 구성 파일과 같은 사용자 데이터 및 메타데이터를 제거할 수 있습니다.

사전 요구 사항

- **root** 액세스 권한이 있습니다.
- 사용자가 현재 존재합니다.
- 사용자가 로그아웃되었는지 확인합니다.

```
# loginctl terminate-user user-name
```

절차

- 사용자 데이터가 아닌 사용자 계정만 제거하려면 다음을 수행합니다.

```
# userdel user-name
```

- 사용자, 데이터 및 메타데이터를 제거하려면 다음을 수행합니다.

  a. 사용자, 해당 홈 디렉터리, 메일 스풀 및 해당 SELinux 사용자 매핑을 제거합니다.

  ```
  # userdel --remove --selinux-user user-name
  ```

  b. 추가 사용자 메타데이터 제거:

  ```
  # rm -rf /var/lib/AccountsService/users/user-name
  ```

  이 디렉터리는 홈 디렉터리를 사용할 수 있기 전에 시스템에 필요한 정보를 사용자 에 저장합니다. 시스템 구성에 따라 사용자가 로그인 화면에서 인증할 때까지 홈 디렉터리를 사용할 수 없을 수 있습니다.

  중요

  이 디렉터리를 제거하지 않고 나중에 동일한 사용자를 다시 생성하는 경우 다시 생성한 사용자는 삭제된 사용자에게 상속된 특정 설정을 계속 사용합니다.

추가 리소스

- 시스템의 **userdel(8)** 도움말 페이지.

11.4. 웹 콘솔에서 사용자 계정 관리

RHEL 웹 콘솔은 시스템 사용자 계정을 추가, 편집 및 제거하기 위한 그래픽 인터페이스를 제공합니다.

웹 콘솔에서 암호 만료를 설정하고 사용자 세션을 종료할 수도 있습니다.

11.4.1. 웹 콘솔을 사용하여 새 계정 추가

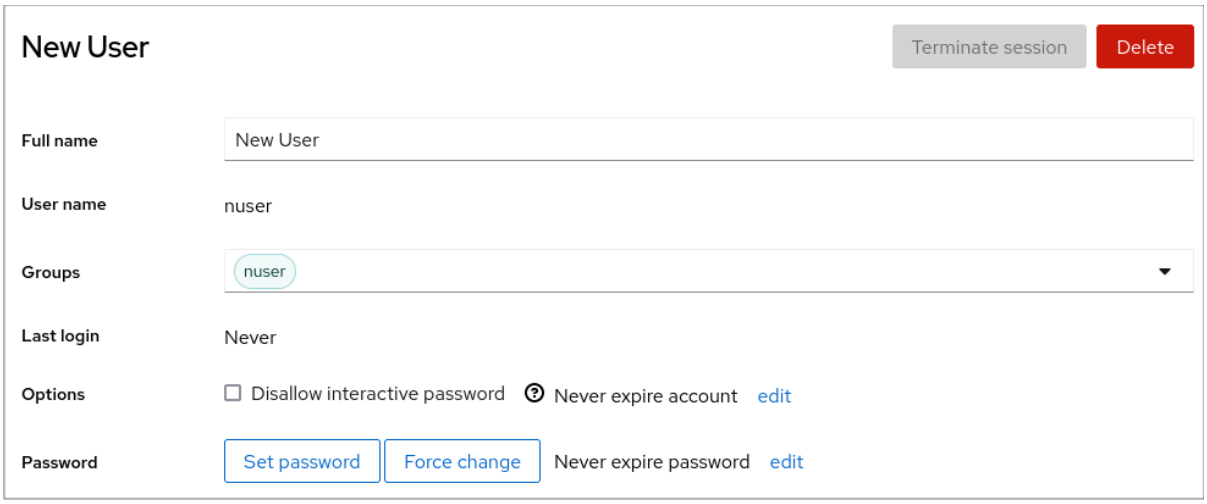
RHEL 웹 콘솔을 통해 사용자 계정을 시스템에 추가하고 계정에 대한 관리 권한을 설정할 수 있습니다.

사전 요구 사항

- RHEL 8 웹 콘솔을 설치했습니다.
  자세한 내용은 웹 콘솔 설치 및 활성화를 참조하십시오.

절차

1. RHEL 8 웹 콘솔에 로그인합니다.
   자세한 내용은 웹 콘솔에 로그인 을 참조하십시오.
2. **Accounts** (계정)를 클릭합니다.
3. **Create New Account** (새 계정 만들기)를 클릭합니다.
4. Full Name (전체 이름) 필드에 사용자의 전체 이름을 입력합니다.
   RHEL 웹 콘솔은 전체 이름에서 사용자 이름을 자동으로 제안하고 User Name(사용자 이름) 필드에 채웁니다. 이름 및 전체 성의 첫 번째 문자로 구성된 원래 명명 규칙을 사용하지 않으려면 제안 사항을 업데이트합니다.
5. Password/Confirm (암호/확인) 필드에 암호를 입력하고 암호가 올바른지 확인하기 위해 암호를 다시 입력합니다.
   필드 아래의 색상 표시줄은 입력한 암호의 보안 수준을 보여주므로 암호가 약한 사용자를 만들 수 없습니다.
6. **Create**(만들기)를 클릭하여 설정을 저장하고 대화 상자를 닫습니다.
7. 새로 생성된 계정을 선택합니다.
8. 그룹 드롭다운 메뉴에서 새 계정에 추가할 그룹을 선택합니다.



이제 계정 설정에서 새 계정을 볼 수 있으며 해당 자격 증명을 사용하여 시스템에 연결할 수 있습니다.

11.4.2. 웹 콘솔에서 암호 만료 강제 적용

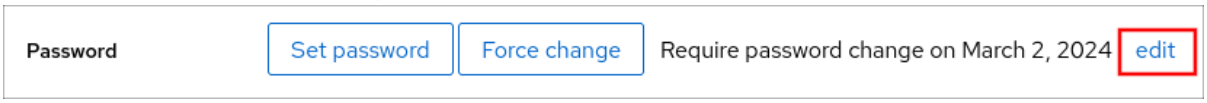
기본적으로 사용자 계정은 암호가 만료되지 않도록 설정되었습니다. 정의된 기간(일) 후에 만료되도록 시스템 암호를 설정할 수 있습니다. 암호가 만료되면 다음 로그인 시도에서 암호 변경을 요청하는 메시지를 표시합니다.

사전 요구 사항

- RHEL 8 웹 콘솔을 설치했습니다.
  자세한 내용은 웹 콘솔 설치 및 활성화를 참조하십시오.

절차

1. RHEL 8 웹 콘솔에 로그인합니다.
2. **Accounts** (계정)를 클릭합니다.
3. 암호 만료를 적용할 사용자 계정을 선택합니다.
4. 암호 행에서 **edit** 를 클릭합니다.



5. 암호 만료 대화 상자에서 암호 변경 시 모든 ...을 선택합니다. days을(를) 입력하고 암호가 만료된 날 수를 나타내는 양의 정수 숫자를 입력합니다.
6. **Change**(변경)를 클릭합니다.
   웹 콘솔은 암호 줄에 향후 암호 변경 요청 날짜를 즉시 표시합니다.

11.5. 명령줄을 사용하여 사용자 그룹 편집

사용자는 파일 및 폴더에 대한 유사한 액세스 권한을 가진 사용자의 논리적 컬렉션을 허용하는 특정 그룹 세트에 속합니다. 명령줄에서 기본 및 보조 사용자 그룹을 편집하여 사용자의 권한을 변경할 수 있습니다.

11.5.1. 기본 및 보조 사용자 그룹

그룹은 특정 파일에 대한 액세스 권한 부여 등 공통 목적으로 여러 사용자 계정을 연결하는 엔터티입니다.

Linux에서 사용자 그룹은 기본 또는 보조 역할을 할 수 있습니다. 기본 및 보조 그룹에는 다음 속성이 있습니다.

기본 그룹

- 모든 사용자에게는 항상 하나의 기본 그룹만 있습니다.
- 사용자의 기본 그룹을 변경할 수 있습니다.

보조 그룹

- 기존 사용자를 기존 보조 그룹에 추가하여 그룹 내에서 동일한 보안 및 액세스 권한으로 사용자를 관리할 수 있습니다.

- 사용자는 0개 또는 여러 개의 보조 그룹의 구성원이 될 수 있습니다.

11.5.2. 사용자의 기본 및 보조 그룹 나열

사용자 그룹을 나열하여 자신이 속한 기본 및 보조 그룹을 확인할 수 있습니다.

절차

- 사용자의 기본 그룹과 보조 그룹의 이름을 표시합니다.

```
$ groups user-name
```

user-name 을 사용자 이름으로 변경합니다. 사용자 이름을 제공하지 않으면 명령은 현재 사용자의 그룹 멤버십을 표시합니다. 첫 번째 그룹은 기본 그룹 다음에 선택적 보조 그룹이 옵니다.

예 11.4. 사용자 sarah의 그룹 목록:

```
$ groups sarah
```

출력이 표시됩니다.

```
sarah : sarah wheel developer
```

User **sarah** 에는 기본 group **sarah** 가 있으며, 보조 그룹 **wheel** 및 **developer** 의 구성원입니다.

예 11.5. 사용자 marc의 그룹 목록:

```
$ groups marc
```

출력이 표시됩니다.

```
marc : marc
```

사용자 **marc** 에는 기본 그룹 **marc** 및 보조 그룹이 없습니다.

11.5.3. 사용자의 기본 그룹 변경

기존 사용자의 기본 그룹을 새 그룹으로 변경할 수 있습니다.

사전 요구 사항

1. 루트 액세스
2. 새 그룹이 존재해야 합니다.

절차

- 사용자의 기본 그룹을 변경합니다.

```
# usermod -g group-name user-name
```

group-name 을 새 기본 그룹의 이름으로 바꾸고*user-name* 을 사용자 이름으로 교체합니다.

참고

사용자의 기본 그룹을 변경하면 명령은 사용자 홈 디렉터리에 있는 모든 파일의 그룹 소유권을 새 기본 그룹으로 자동 변경합니다. 사용자의 홈 디렉터리 외부에서 파일의 그룹 소유권을 수동으로 수정해야 합니다.

예 11.6. 사용자의 기본 그룹을 변경하는 예:

user **sarah**가 기본 group **sarah1** 에 속하고 사용자의 기본 그룹을**sarah2** 로 변경하려면 다음을 사용합니다.

```
# usermod -g sarah2 sarah
```

검증

- 사용자의 기본 그룹을 변경했는지 확인합니다.

```
$ groups sarah
```

출력이 표시됩니다.

```
sarah : sarah2
```

11.5.4. 명령줄에서 사용자를 보조 그룹에 추가

보조 그룹에 사용자를 추가하여 권한을 관리하거나 특정 파일 또는 장치에 대한 액세스를 활성화할 수 있습니다.

사전 요구 사항

- 루트 액세스

절차

- 사용자의 보조 그룹에 그룹을 추가하려면 다음을 사용합니다.

```
# usermod --append -G group-name username
```

group-name 을 그룹의 이름으로 바꾸고*username* 을 사용자 이름으로 바꿉니다.

예 11.7. 보조 그룹에 사용자 추가

사용자 **sysadmin** 을 **system-admins** 그룹에 추가하려면 다음을 사용합니다.

```
# usermod --append -G system-administrators sysadmin
```

검증

- 새 그룹이 사용자 **sysadmin** 의 보조 그룹에 추가되었는지 확인하려면 다음을 사용합니다.

```
# groups sysadmin
```

출력이 표시됩니다.

```
sysadmin : sysadmin system-administrators
```

11.5.5. 보조 그룹에서 사용자 제거

보조 그룹에서 기존 사용자를 제거하여 파일 및 장치에 대한 권한을 제한할 수 있습니다.

사전 요구 사항

- 루트 액세스

절차

- 보조 그룹에서 사용자 제거:

```
# gpasswd -d user-name group-name
```

user-name 을 사용자 이름으로 바꾸고*group-name* 을 보조 그룹 이름으로 교체합니다.

예 11.8. 보조 그룹에서 사용자 제거

사용자 sarah에 기본 group **sarah2** 가 있고 보조 그룹**wheel** 및 **developers** 에 속하고 그룹 개발자에서 해당 사용자를 제거하려면 다음을 사용합니다.

```
# gpasswd -d sarah developers
```

검증

- 보조 그룹 developers에서 사용자 sarah가 제거되었는지 확인합니다.

```
$ groups sarah
```

출력이 표시됩니다.

```
sarah : sarah2 wheel
```

11.5.6. 사용자의 모든 보조 그룹 변경

사용자가 멤버로 유지하려는 보조 그룹 목록을 덮어쓸 수 있습니다.

사전 요구 사항

- 루트 액세스

- 보조 그룹이 있어야 합니다.

절차

- 사용자의 보조 그룹 목록을 덮어씁니다.

```
# usermod -G group-names username
```

group-names 를 하나 이상의 보조 그룹의 이름으로 바꿉니다. 사용자를 한 번에 여러 보조 그룹에 추가하려면 그룹 이름을 쉼표와 중간 공백을 사용하여 구분합니다. 예: **wheel,developer**.

user-name 을 사용자 이름으로 변경합니다.

중요

사용자가 현재 지정하지 않는 그룹의 멤버인 경우 명령은 그룹에서 사용자를 제거합니다.

예 11.9. 사용자의 보조 그룹 목록 변경

user **sarah** 에 기본 group **sarah2** 가 있고 보조 그룹 **wheel** 에 속하고 사용자가 3개의 보조 그룹 **developer**,**sysadmin** 및 **security** 에 속하게 하려면 다음을 사용합니다.

```
# usermod -G wheel,developer,sysadmin,security sarah
```

검증

- 보조 그룹 목록을 올바르게 설정했는지 확인합니다.

```
# groups sarah
```

출력이 표시됩니다.

```
sarah : sarah2 wheel developer sysadmin security
```

11.6. 루트 암호 변경 및 재설정

기존 root 암호가 더 이상 만족스럽지 않거나 잊혀진 경우 **root** 사용자 및 비 루트 사용자로 변경하거나 재설정할 수 있습니다.

11.6.1. root 사용자로 root 암호 변경

passwd 명령을 사용하여 **root** 사용자로 **root** 암호를 변경할 수 있습니다.

사전 요구 사항

- 루트 액세스

절차

- 루트 암호를 변경하려면 다음을 사용합니다.

```
# passwd
```

변경하기 전에 현재 암호를 입력하라는 메시지가 표시됩니다.

11.6.2. root가 아닌 사용자로 잊어버리는 루트 암호 변경 또는 재설정

passwd 명령을 사용하여 루트가 아닌 사용자로 알 수 있는**root** 암호를 변경하거나 재설정할 수 있습니다.

사전 요구 사항

- 루트가 아닌 사용자로 로그인할 수 있습니다.
- 관리 **wheel** 그룹의 구성원입니다.

절차

- 루트가 아닌 사용자로 루트가 아닌 사용자로 **root** 암호를 변경하거나 재설정하려면 다음을 사용합니다.

```
$ sudo passwd root
```

루트가 아닌 암호를 변경하기 전에 현재 루트가 아닌 암호를 입력하라는 메시지가 표시됩니다 **.**

11.6.3. 부팅시 root 암호 재설정

루트가 아닌 사용자로 로그인할 수 없거나 관리 **wheel** 그룹에 속하지 않는 경우 특수**chroot jail** 환경으로 전환하여 부팅 시 루트 암호를 재설정할 수 있습니다.

절차

1. 시스템을 재부팅하고 GRUB 2 부팅 화면에서 **e** 키를 눌러 부팅 과정을 중단합니다.
   커널 부팅 매개 변수가 표시됩니다.

```
load_video
set gfx_payload=keep
insmod gzio
linux ($root)/vmlinuz-4.18.0-80.e18.x86_64 root=/dev/mapper/rhel-root ro crash\
kernel=auto resume=/dev/mapper/rhel-swap rd.lvm.lv/swap rhgb quiet
initrd ($root)/initramfs-4.18.0-80.e18.x86_64.img $tuned_initrd
```

2. **linux** 로 시작하는 행의 끝으로 이동합니다.

```
linux ($root)/vmlinuz-4.18.0-80.e18.x86_64 root=/dev/mapper/rhel-root ro crash\
kernel=auto resume=/dev/mapper/rhel-swap rd.lvm.lv/swap rhgb quiet
```

   Ctrl+e 를 눌러 행 끝으로 건너뜁니다.

3. **linux** 로 시작하는 행의 끝에 Add**rd.break.**

```
linux ($root)/vmlinuz-4.18.0-80.e18.x86_64 root=/dev/mapper/rhel-root ro crash\
kernel=auto resume=/dev/mapper/rhel-swap rd.lvm.lv/swap rhgb quiet rd.break
```

4. **Ctrl+x** 를 눌러 변경된 매개 변수로 시스템을 시작합니다.
   switch_root 프롬프트가 나타납니다.

5. 파일 시스템을 쓰기 가능으로 다시 마운트합니다.

```
mount -o remount,rw /sysroot
```

   파일 시스템은 **/sysroot** 디렉토리에 읽기 전용으로 마운트됩니다. 파일 시스템을 쓰기 가능으로 다시 마운트하면 암호를 변경할 수 있습니다.

6. **chroot** 환경을 입력합니다.

```
chroot /sysroot
```

   sh-4.4# 프롬프트가 표시됩니다.

7. 루트 암호를 재설정합니다.

```
passwd
```

   명령줄에 표시된 지침에 따라 루트 암호 변경을 완료합니다.

8. 다음 시스템 부팅 시 SELinux 레이블 변경 프로세스를 활성화합니다.

```
touch /.autorelabel
```

9. **chroot** 환경을 종료합니다.

```
exit
```

10. **switch_root** 프롬프트를 종료합니다.

```
exit
```

11. SELinux 레이블 변경 프로세스가 완료될 때까지 기다립니다. 큰 디스크의 레이블을 다시 지정하는 데 시간이 오래 걸릴 수 있습니다. 프로세스가 완료되면 시스템이 자동으로 재부팅됩니다.

검증

1. 루트 암호가 성공적으로 변경되었는지 확인하려면 일반 사용자로 로그인하여 터미널을 엽니다.

2. root로 대화형 쉘을 실행합니다.

```
$ su
```

3. 새 루트 암호를 입력합니다.

4. 현재 유효한 사용자 ID와 관련된 사용자 이름을 인쇄합니다.

```
# whoami
```

   출력은 다음을 반환합니다.

root

12장. SUDO 액세스 관리

시스템 관리자는 루트가 아닌 사용자가 일반적으로 root 사용자로 예약된 관리 명령을 실행할 수 있도록 **sudo** 액세스 권한을 부여할 수 있습니다. 결과적으로 루트가 아닌 사용자는 root 사용자 계정에 로그인하지 않고 이러한 명령을 실행할 수 있습니다.

12.1. SUDOERS의 사용자 권한 부여

/etc/sudoers 파일은 **sudo** 명령을 사용하여 다른 명령을 실행할 수 있는 사용자를 지정합니다. 규칙은 개별 사용자 및 사용자 그룹에 적용할 수 있습니다. 별칭을 사용하여 호스트 그룹, 명령 및 사용자도 더 쉽게 정의할 수도 있습니다. 기본 별칭은 **/etc/sudoers** 파일의 첫 번째 부분에서 정의됩니다.

사용자가 권한이 없는 **sudo** 를 사용하여 명령을 입력하면 시스템은 **<*username*> :** 사용자가 저널 로그에 대한 **sudoers**에 없는 메시지를 기록합니다.

기본 **/etc/sudoers** 파일은 권한 부여의 정보와 예를 제공합니다. 해당 행의 주석을 제거하여 특정 예제 규칙을 활성화할 수 있습니다. 사용자 권한 부여 섹션은 다음 도입으로 표시됩니다.

```
## Next comes the main part: which users can run what software on
## which machines  (the sudoers file can be shared between multiple
## systems).
```

다음 형식을 사용하여 새 **sudoers** 권한을 생성하고 기존 권한 부여를 수정할 수 있습니다.

```
<username> <hostname.example.com>=(<run_as_user>:<run_as_group>) <path/to/command>
```

다음과 같습니다.

- **<*username*>**은 명령을 입력하는 사용자입니다(예:**user1**). 값이 **%** 로 시작되면 그룹을 정의합니다(예: **%group1**).
- **<*hostname.example.com*>**은 규칙이 적용되는 호스트의 이름입니다.
- 섹션 (**<*run_as_user*> : <*run_as_group*>**) 은 명령이 실행되는 사용자 또는 그룹을 정의합니다. 이 섹션을 생략하면 **< *username*>** 에서 root로 명령을 실행할 수 있습니다.
- **<*path/to/command*>**는 명령의 전체 절대 경로입니다. 명령 경로 뒤에 해당 옵션을 추가하여 특정 옵션 및 인수가 있는 명령만 수행하도록 사용자를 제한할 수도 있습니다. 옵션을 지정하지 않으면 사용자는 모든 옵션과 함께 명령을 사용할 수 있습니다.

이러한 변수를 **ALL** 로 교체하여 모든 사용자, 호스트 또는 명령에 규칙을 적용할 수 있습니다.

> **주의**
>
> **ALL =(ALL) ALL** 과 같은 과도하게 허용 가능한 규칙을 사용하면 모든 사용자가 모든 호스트에서 모든 사용자로 모든 명령을 실행할 수 있습니다. 이는 심각한 보안 위험을 초래합니다.

! 연산자를 사용하여 인수를 부정적인 방식으로 지정할 수 있습니다. 예를 들어!root 는 root를 제외한 모든 사용자를 지정합니다. 특정 사용자, 그룹 및 명령을 허용하는 것은 특정 사용자, 그룹 및 명령을 허용하지 않는 것보다 안전합니다. 이는 규칙이 권한이 없는 새 사용자 또는 그룹도 차단하기 때문입니다.

> **주의**
>
> 사용자가 **alias** 명령으로 명령 이름을 변경하여 이러한 규칙을 극복할 수 있으므로 명령에 음수 규칙을 사용하지 마십시오.

시스템은 **/etc/sudoers** 파일을 처음부터 끝까지 읽습니다. 따라서 파일에 사용자의 여러 항목이 포함된 경우 항목이 순서대로 적용됩니다. 값이 충돌하는 경우 시스템은 가장 구체적인 일치 항목이 아니어도 마지막 일치 항목을 사용합니다.

시스템 업데이트 중 규칙을 유지하고 오류를 보다 쉽게 수정하려면 **/etc/sudoers.d/** 디렉터리에 새 파일을 **/etc/sudoers** 파일에 직접 입력하는 대신 새 규칙을 입력합니다./**etc/sudoers.d** 디렉터리에서 /etc/sudoers 파일의 다음 행에 도달하면 시스템이 **/etc/sudoers**.d 디렉토리의 파일을 읽습니다.

```
#includedir /etc/sudoers.d
```

이 줄의 시작 부분에 있는 숫자 기호(#)는 구문의 일부이며 행이 주석임을 의미하지는 않습니다. 해당 디렉터리의 파일 이름은 마침표를 포함하지 않아야 하며 틸드(~)로 끝나지 않아야 합니다.

추가 리소스

- **sudo(8)** 및 **sudoers(5)** 도움말 페이지

12.2. 사용자에게 SUDO 액세스 권한 부여

시스템 관리자는 루트가 아닌 사용자가 **sudo** 액세스 권한을 부여하여 관리 명령을 실행할 수 있도록 허용할 수 있습니다. **sudo** 명령은 root 사용자의 암호를 사용하지 않고 사용자에게 관리 액세스 권한을 제공합니다.

사용자가 관리 명령을 수행해야 하는 경우 **sudo** 를 사용하여 해당 명령 앞에 를 사용할 수 있습니다. 사용자에게 명령에 대한 권한 부여가 있는 경우 root인 것처럼 명령이 실행됩니다.

다음 제한 사항에 유의하십시오.

- **/etc/sudoers** 구성 파일에 나열된 사용자만**sudo** 명령을 사용할 수 있습니다.
- 명령은 root 쉘이 아닌 사용자의 쉘에서 실행됩니다. 그러나 모든 사용자에게 전체 **sudo** 권한이 부여된 경우와 같은 몇 가지 예외가 있습니다. 이러한 경우 사용자는 root 쉘에서 로 전환하고 명령을 실행할 수 있습니다. 예를 들면 다음과 같습니다.
- **sudo -i**
- **sudo su -**

사전 요구 사항

- 시스템에 대한 root 액세스 권한이 있습니다.

절차

1. 루트로 **/etc/sudoers** 파일을 엽니다.

```
# visudo
```

/etc/sudoers 파일은 **sudo** 명령에서 적용하는 정책을 정의합니다.

2. **/etc/sudoers** 파일에서 관리 **wheel** 그룹의 사용자에게 **sudo** 액세스 권한을 부여하는 행을 찾습니다.

```
## Allows people in group wheel to run all commands
%wheel        ALL=(ALL)       ALL
```

3. **%wheel** 으로 시작하는 행이 숫자 기호(**#**)를 사용하여 주석 처리되지 않았는지 확인합니다.

4. 변경 사항을 저장하고 편집기를 종료합니다.

5. 에 대한 **sudo** 액세스 권한을 관리 **wheel** 그룹에 부여하려는 사용자를 추가합니다.

```
# usermod --append -G wheel <username>
```

& ***lt;username*** >을 사용자 이름으로 바꿉니다.

검증

- 사용자가 관리 **wheel** 그룹에 있는지 확인합니다.

```
# id <username>
uid=5000(<username>) gid=5000(<username>) groups=5000(<username>),10(wheel)
```

추가 리소스

- **sudo(8)**, **visudo(8)**, **sudoers(5)** 매뉴얼 페이지

12.3. 권한이 없는 사용자가 특정 명령 실행 가능

관리자는 **/etc/sudoers.d/** 디렉터리에 정책을 구성하여 권한이 없는 사용자가 특정 워크스테이션에서 특정 명령을 실행하도록 허용할 수 있습니다. 이는 사용자에게 전체 **sudo** 액세스 권한을 부여하거나 다음과 같은 이유로 사용자에게 root 암호를 제공하는 것보다 더 안전합니다.

- 권한 있는 작업을 보다 세밀하게 제어합니다. 사용자가 전체 관리 액세스 권한을 부여하는 대신 특정 호스트에서 특정 작업을 수행하도록 허용할 수 있습니다.

- 더 나은 로깅. 사용자가 **sudo** 를 통해 작업을 수행하면 root뿐만 아니라 사용자 이름으로 작업이 기록됩니다.

- 투명한 제어. 사용자가 **sudo** 권한을 사용하려고 할 때마다 이메일 알림을 설정할 수 있습니다.

사전 요구 사항

- 시스템에 대한 root 액세스 권한이 있습니다.

절차

1. **root 로서 /etc/: 아래에 새 sudoers.d 디렉토리를 만듭니다.**

```
# mkdir -p /etc/sudoers.d/
```

2. **/etc/sudoers.d 디렉토리에 새 파일을 만듭니다.**

```
# visudo -f /etc/sudoers.d/<filename>
```

파일이 자동으로 열립니다.

3. **/etc/sudoers.d/ <*filename* > 파일에 다음 행을 추가합니다.**

```
<username> <hostname.example.com> = (<run_as_user>:<run_as_group>)
<path/to/command>
```

- & ***lt;username*** >을 사용자 이름으로 바꿉니다.
- & ***lt;hostname.example.com&*** gt;을 호스트의 URL로 바꿉니다.
- **(<*run_as_user* > : <*run_as_group*>)** 명령을 실행할 수 있는 사용자 또는 그룹으로 바꿉니다. 이 섹션을 생략하면 < ***username*>** 에서 root로 명령을 실행할 수 있습니다.
- & ***lt;path/to/command&*** gt;를 명령의 전체 절대 경로로 바꿉니다. 명령 경로 뒤에 해당 옵션을 추가하여 특정 옵션 및 인수가 있는 명령만 수행하도록 사용자를 제한할 수도 있습니다. 옵션을 지정하지 않으면 사용자는 모든 옵션과 함께 명령을 사용할 수 있습니다.
- 한 줄에 동일한 호스트에서 두 개 이상의 명령을 허용하려면 쉼표로 구분한 뒤에 공백으로 구분하여 나열할 수 있습니다.
  예를 들어 **user1** 이 **dnf** 를 실행하고 **host1.example.com** 에서 **reboot** 명령을 실행할 수 있도록 하려면 **user1 host1.example.com = /bin/dnf, /sbin/reboot** 를 입력합니다.

4. **선택 사항: 사용자가 sudo 권한을 사용하려고 할 때마다 이메일 알림을 받으려면 파일에 다음 행을 추가합니다.**

```
Defaults    mail_always
Defaults    mailto="<email@example.com>"
```

5. **변경 사항을 저장하고 편집기를 종료합니다.**

검증

1. **사용자가 sudo 권한으로 명령을 실행할 수 있는지 확인하려면 계정을 전환합니다.**

```
# su <username> -
```

2. **사용자로 sudo 명령을 사용하여 명령을 입력합니다.**

```
$ sudo <command>
[sudo] password for <username>:
```

사용자의 **sudo** 암호를 입력합니다.

3. **권한이 올바르게 구성된 경우 시스템은 명령 및 옵션 목록을 표시합니다. 예를 들어 dnf 명령을 사용하면 다음 출력이 표시됩니다.**

```
...
usage: dnf [options] COMMAND
...
```

시스템이 오류 메시지 <*username>*이 sudoers 파일에 없는 경우. 이 문제는 /etc/sudoers.d/의 < *username* >에 대한 파일인 것으로 보고됩니다.

시스템이 < *username* > 오류 메시지를 반환하는 경우 < *host.example.com*>에서 sudo를 실행할 수 없는 경우 구성이 올바르게 완료되지 않았습니다. root로 로그인하고 구성이 올바르게 수행되었는지 확인합니다.

시스템이 오류 메시지 Sorry를 반환하는 경우 user < *username* >은 <path/to/command> ' ' <*path/to/command*>'를 < *host.example.com* >에서 root로 실행할 수 없습니다.. 에서는 사용자의 규칙에 명령이 올바르게 정의되지 않습니다.

추가 리소스

- **sudo(8), visudo(8), sudoers(5) 매뉴얼 페이지**

13장. 파일 시스템 권한 관리

파일 시스템 권한은 사용자 및 그룹 계정에서 파일의 내용을 읽고 수정 및 실행하고 디렉토리를 입력할 수 있는 기능을 제어합니다. 무단 액세스로부터 데이터를 보호하려면 권한을 신중하게 설정합니다.

13.1. 파일 권한 관리

모든 파일 또는 디렉터리에는 세 가지 수준의 소유권이 있습니다.

- 사용자 소유자(u).
- 그룹 소유자(g).
- 기타(o).

각 소유권 수준은 다음 권한을 할당할 수 있습니다.

- 읽기(r).
- 쓰기(w).
- 실행(x).

파일에 대한 실행 권한을 사용하여 해당 파일을 실행할 수 있습니다. 디렉터리에 대한 실행 권한을 사용하면 디렉터리 내용에 액세스할 수 있지만 실행할 수는 없습니다.

새 파일 또는 디렉터리가 생성되면 기본 권한 집합이 자동으로 할당됩니다. 파일 또는 디렉토리에 대한 기본 권한은 다음 두 가지 요소를 기반으로 합니다.

- 기본 권한.
- *사용자 파일 생성 모드 마스크*(umask).

13.1.1. 기본 파일 권한

새 파일 또는 디렉터리가 생성될 때마다 기본 권한이 자동으로 할당됩니다. 파일 또는 디렉터리에 대한 기본 권한을 *심볼릭* 또는 *8진수* 값으로 표시할 수 있습니다.

권한	심볼릭 값	8진수 값
권한 없음	---	0
실행	--x	1
쓰기	-w-	2
쓰기 및 실행	-wx	3
읽기	r--	4
읽기 및 실행	r-x	5

읽기 및 쓰기	rw-	6
읽기, 쓰기, 실행	rwx	7

디렉터리의 기본 권한은 **777 (drwxrwxrwx)**으로, 모든 사용자에게 읽기, 쓰기, 실행 권한을 부여합니다. 즉, 디렉터리 소유자, 그룹 및 기타 사용자가 디렉터리의 콘텐츠를 나열하고 디렉터리 내에서 항목을 만들고 삭제하고 편집할 수 있습니다.

디렉터리 내의 개별 파일에는 무제한 디렉터리에 대한 액세스 권한이 있더라도 편집을 방해할 수 있는 자체 권한이 있을 수 있습니다.

파일의 기본 권한은 **666 (-rw-rw-rw-)**으로, 모든 사용자에게 읽기 및 쓰기 권한을 부여합니다. 즉, 파일 소유자, 그룹 및 기타 사용자가 파일을 읽고 편집할 수 있습니다.

예 13.1. 파일에 대한 권한

파일에 다음 권한이 있는 경우:

```
$ ls -l
-rwxrw----. 1 sysadmins sysadmins 2 Mar 2 08:43 file
```

- **-** 파일이 파일임을 나타냅니다.
- **rwx** 는 파일 소유자가 파일을 읽고 쓰고 실행할 수 있는 권한이 있음을 나타냅니다.
- **rw-** 는 그룹에 읽기 및 쓰기 권한이 있지만 파일을 실행하지 않음을 나타냅니다.
- **---** 다른 사용자에게 파일을 읽고, 쓰거나, 실행할 수 있는 권한이 없음을 나타냅니다.
- **.** 은 SELinux 보안 컨텍스트가 파일에 대해 설정되었음을 나타냅니다.

예 13.2. 디렉토리에 대한 권한

디렉터리에 다음과 같은 권한이 있는 경우:

```
$ ls -dl directory
drwxr-----. 1 sysadmins sysadmins 2 Mar 2 08:43 directory
```

- **d** 는 디렉토리임을 나타냅니다.
- **rwx** 는 디렉터리 소유자가 디렉터리의 내용을 읽고 쓰고 액세스할 수 있는 권한이 있음을 나타냅니다.
  디렉터리 소유자로서 디렉터리 내의 항목(파일, 하위 디렉터리)을 나열하고 해당 항목의 내용에 액세스하여 수정할 수 있습니다.
- **R -x** 는 그룹에 디렉터리의 내용을 읽을 수 있는 권한이 있지만 쓸 수는 없음을 나타냅니다. 새 항목을 생성하거나 파일을 삭제할 수 없습니다. **x** 권한은 **cd** 명령을 사용하여 디렉터리에 액세스할 수도 있음을 의미합니다.
- **---** 다른 사용자에게 디렉터리 내용을 읽고, 쓰거나, 액세스할 수 있는 권한이 없음을 나타냅니다. 사용자 소유자가 아니거나 디렉토리의 그룹 소유자로서는 디렉터리 내의 항목을 나열하거나, 해당 항목에 대한 정보에 액세스하거나, 수정할 수 없습니다.

- . 은 SELinux 보안 컨텍스트가 디렉터리에 대해 설정되었음을 나타냅니다.

참고

파일 또는 디렉터리에 자동으로 할당되는 기본 권한은 파일 또는 디렉터리가 로 끝나는 기본 권한이 아닙니다. 파일 또는 디렉토리를 만들면 기본 권한이*umask* 에 의해 변경됩니다. 기본 권한과 *umask* 의 조합은 파일 및 디렉터리에 대한 기본 권한을 생성합니다.

13.1.2. 사용자 파일 생성 모드 마스크

user file-creation mode mask(*umask*)는 새로 생성된 파일 및 디렉터리에 대해 파일 권한이 설정되는 방법을 제어하는 변수입니다. CloudEvent 는 Linux 시스템의 전체 보안을 높이기 위해 기본 권한 값에서 권한을 자동으로 제거합니다. *umask* 는 *심볼릭* 또는 *8진수* 값으로 나타낼 수 있습니다.

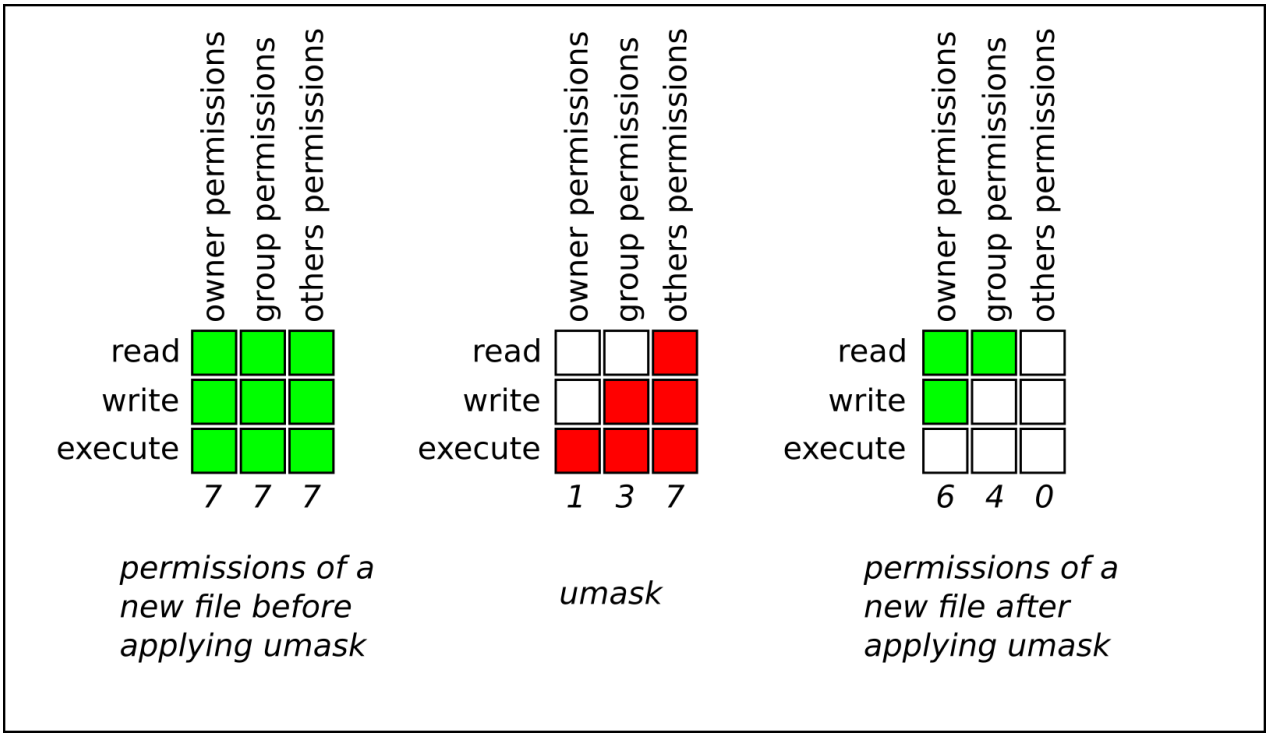
권한	심볼릭 값	8진수 값
읽기, 쓰기 및 실행	rwx	0
읽기 및 쓰기	rw-	1
읽기 및 실행	r-x	2
읽기	r--	3
쓰기 및 실행	-wx	4
쓰기	-w-	5
실행	--x	6
권한 없음	---	7

표준 사용자의 기본 *umask* 는 **0002** 입니다. **root** 사용자의 기본 *umask* 는 **0022** 입니다.

umask 의 첫 번째 숫자는 특별 권한(고정 비트,)을 나타냅니다.*umask* 의 마지막 세 자리 숫자는 사용자 소유자(u), 그룹 소유자(g) 및 기타(o)에서 각각 제거된 권한을 나타냅니다.

예 13.3. 파일을 만들 때 umask 적용

다음 예제에서는 기본 권한이 **640** 인 파일을 생성하기 위해 8진수 값이**0137** 인 *umask* 가 **777** 기본 권한이 있는 파일에 적용되는 방법을 보여줍니다.



13.1.3. 기본 파일 권한

기본 권한은 새로 생성된 모든 파일 및 디렉터리에 대해 자동으로 설정됩니다. 기본 권한의 값은 기본 권한에 *umask* 를 적용하여 결정됩니다.

예 13.4. 표준 사용자가 생성한 디렉터리에 대한 기본 권한

표준 사용자가 새 디렉토리 를 만들면 *umask* 가 **002 (rwxrwxr-x)**로 설정되고 디렉터리의 기본 권한이 **777 (rwxrwxrwx)**으로 설정됩니다. 그러면 **775 (drwxrwxr-x)**에 기본 권한이 부여됩니다.

	심볼릭 값	8진수 값
기본 권한	rwxrwxrwx	777
umask	rwxrwxr-x	002
기본 권한	rwxrwxr-x	775

즉, 디렉터리 소유자와 그룹은 디렉터리의 콘텐츠를 나열하고 디렉터리 내에서 항목을 만들고 삭제하고 편집할 수 있습니다. 다른 사용자는 디렉터리의 내용만 나열할 수 있으며 디렉토리의 하위 항목만 나열할 수 있습니다.

예 13.5. 표준 사용자가 생성한 파일에 대한 기본 권한

표준 사용자가 새 파일을 만들면 *umask* 가 **002 (rwxrwxr-x)**로 설정되고 파일의 기본 권한이 **666 (rw-rw-rw-)**으로 설정됩니다. 이렇게 하면 기본 권한은 **664 (-rw-rw-r--)**입니다.

	심볼릭 값	8진수 값

기본 권한	rw-rw-rw-	666
umask	rwxrwxr-x	002
기본 권한	rw-rw-r--	664

즉, 파일 소유자와 그룹은 파일을 읽고 편집할 수 있지만 다른 사용자는 파일을 읽을 수만 있습니다.

예 13.6. root 사용자가 만든 디렉토리에 대한 기본 권한

루트 사용자가 새 디렉토리 를 만들면 *umask* 가 **022 (rwxr-xr-x)**로 설정되고 디렉터리의 기본 권한이 **777 (rwxrwxrwx)**으로 설정됩니다. 이렇게 하면 기본 권한은 **755 (rwxr-xr-x)**입니다.

	심볼릭 값	8진수 값
기본 권한	rwxrwxrwx	777
umask	rwxr-xr-x	022
기본 권한	rwxr-xr-x	755

즉, 디렉터리 소유자가 디렉터리의 내용을 나열하고 디렉터리 내에서 항목을 만들고 삭제하고 편집할 수 있습니다. 그룹 및 기타 그룹은 디렉터리의 내용만 나열하고 해당 디렉토리로 이동할 수 있습니다.

예 13.7. root 사용자가 생성한 파일에 대한 기본 권한

루트 사용자가 새 파일을 만들면 *umask* 가 **022 (rwxr-xr-x)**로 설정되고 파일의 기본 권한이 **666 (rw-rw-rw-)**으로 설정됩니다. 이렇게 하면 기본 권한이 **644 (-rw-r-r--)**.

	심볼릭 값	8진수 값
기본 권한	rw-rw-rw-	666
umask	rwxr-xr-x	022
기본 권한	rw-r—r--	644

즉, file 소유자는 파일을 읽고 편집할 수 있지만 그룹 및 기타 사용자는 파일을 읽을 수 있습니다.

참고

보안상의 이유로 *umask* 가 **000 (rwxrwxrwx)**으로 설정된 경우에도 일반 파일은 기본적으로 실행 권한을 가질 수 없습니다. 그러나 실행 권한으로 디렉터리를 생성할 수 있습니다.

13.1.4. 심볼릭 값을 사용하여 파일 권한 변경

chmod 유틸리티를 심볼릭 값(복합 문자 및 기호)과 함께 사용하여 파일 또는 디렉토리의 파일 권한을 변경할 수 있습니다.

다음 *권한을* 할당할 수 있습니다.

- 읽기 (r)
- 쓰기 (w)
- 실행 (x)

권한을 다음 *수준의 소유권*에 할당할 수 있습니다.

- 사용자 소유자 (u)
- 그룹 소유자(g)
- 기타 (o)
- 모두 (a)

권한을 추가하거나 제거하려면 다음 *기호를* 사용할 수 있습니다:

- **+**: 기존 권한 위에 권한을 추가합니다.
- **-** 기존 권한에서 권한을 제거하려면 **-**
- **=** - 기존 권한을 제거하고 명시적으로 새 권한을 정의합니다.

절차

- 파일 또는 디렉토리에 대한 권한을 변경하려면 다음을 사용합니다.

  ```
  $ chmod <level><operation><permission> file-name
  ```

  <level> 을 권한을 설정할 소유권 수준으로 바꿉니다. ***<operation>*** 를 기호 중 하나로 바꿉니다. ***<permission>*** 을 할당하려는 권한 으로 바꿉니다. *file-name* 을 파일 또는 디렉터리의 이름으로 바꿉니다. 예를 들어 모든 사용자에게 읽기, 쓰기, 실행(**rwx**) **my-script.sh** 에 대한 권한을 부여하려면 **chmod a=rwx my-script.sh** 명령을 사용합니다.

  자세한 내용은 기본 파일 권한을 참조하십시오.

검증

- 특정 파일의 권한을 보려면 다음을 사용합니다.

  ```
  $ ls -l file-name
  ```

  file-name 을 파일 이름으로 바꿉니다.

- 특정 디렉터리에 대한 권한을 보려면 다음을 사용합니다.

  ```
  $ ls -dl directory-name
  ```

directory-name 을 디렉터리 이름으로 바꿉니다.

- 특정 디렉토리 내의 모든 파일에 대한 권한을 보려면 다음을 사용합니다.

```
$ ls -l directory-name
```

directory-name 을 디렉터리 이름으로 바꿉니다.

예 13.8. 파일 및 디렉토리에 대한 권한 변경

- **my-file.txt** 의 파일 권한을 **-rw-rw-r--** 에서 **-rw------** 로 변경하려면 다음을 사용합니다.

  1. **my-file.txt** 의 현재 권한을 표시합니다 :

```
$ ls -l my-file.txt
-rw-rw-r--. 1 username username 0 Feb 24 17:56 my-file.txt
```

  2. 그룹 소유자(**g**) 및 기타(**o**)에서 파일을 읽기, 쓰기, 실행(**rwx**)할 수 있는 권한을 제거합니다.

```
$ chmod go= my-file.txt
```

  등호(**=**) 후에 지정되지 않는 모든 권한은 자동으로 금지됩니다.

  3. **my-file.txt** 에 대한 권한이 올바르게 설정되었는지 확인합니다.

```
$ ls -l my-file.txt
-rw-------. 1 username username 0 Feb 24 17:56 my-file.txt
```

- **my-directory** 의 파일 권한을 **drwxrwx---**에서 **drwx rwxr-x** 로 변경하려면 다음을 사용합니다.

  1. **my-directory** 에 대한 현재 권한을 표시합니다.

```
$ ls -dl my-directory
drwxrwx---. 2 username username 4096 Feb 24 18:12 my-directory
```

  2. 모든 사용자(**a**)에 대해 읽기 및 실행(**r-x**) 액세스를 추가합니다.

```
$ chmod o+rx my-directory
```

  3. **my-directory** 및 해당 콘텐츠에 대한 권한이 올바르게 설정되었는지 확인합니다.

```
$ ls -dl my-directory
drwxrwxr-x. 2 username username 4096 Feb 24 18:12 my-directory
```

13.1.5. 8진수 값을 사용하여 파일 권한 변경

chmod 유틸리티를 8진수 값(숫자)과 함께 사용하여 파일 또는 디렉터리의 파일 권한을 변경할 수 있습니다.

절차

- 기존 파일 또는 디렉터리에 대한 파일 권한을 변경하려면 다음을 사용합니다.

```
$ chmod octal_value file-name
```

file-name 을 파일 또는 디렉터리의 이름으로 바꿉니다.*octal_value* 를 8진수 값으로 교체합니다. 자세한 내용은 기본 파일 권한을 참조하십시오.

13.2. 액세스 제어 목록 관리

각 파일과 디렉터리에는 한 번에 하나의 사용자 소유자와 하나의 그룹 소유자만 있을 수 있습니다. 다른 파일 및 디렉터리를 비공개로 유지하면서 다른 사용자 또는 그룹에 속하는 특정 파일 또는 디렉토리에 액세스할 수 있는 권한을 사용자에게 부여하려면 ACL(Linux Access Control Lists)을 활용할 수 있습니다.

13.2.1. 현재 액세스 제어 목록 표시

getfacl 유틸리티를 사용하여 현재 ACL을 표시할 수 있습니다.

절차

- 특정 파일 또는 디렉토리에 대한 현재 ACL을 표시하려면 다음을 사용합니다.

```
$ getfacl file-name
```

file-name 을 파일 또는 디렉터리의 이름으로 바꿉니다.

13.2.2. 액세스 제어 목록 설정

setfacl 유틸리티를 사용하여 파일 또는 디렉터리에 대한 ACL을 설정할 수 있습니다.

사전 요구 사항

- 루트 액세스.

절차

- 파일 또는 디렉터리에 대한 ACL을 설정하려면 다음을 사용합니다.

```
# setfacl -m u:username:symbolic_value file-name
```

username 을 사용자 이름으로,*symbolic_value* 를 심볼릭 값으로,*file-name* 을 파일 또는 디렉토리 이름으로 바꿉니다. 자세한 내용은 시스템의 **setfacl** 도움말 페이지를 참조하십시오.

예 13.9. 그룹 프로젝트에 대한 권한 수정

다음 예제에서는 이 파일이 다음과 같이 root 그룹에 속하는 **root** 사용자가 소유한 **group -project** 파일에 대한 권한을 수정하는 방법을 설명합니다.

- 누구나 실행할 수 없습니다.
- 사용자 **andrew** 에는 **rw-** 권한이 있습니다.
- 사용자 **susan** 은 **---** 권한을 갖습니다.

- 기타 사용자에게는 **r--** 권한이 있습니다.

절차

```
# setfacl -m u:andrew:rw- group-project
# setfacl -m u:susan:--- group-project
```

검증

- 사용자 **andrew** 에 **rw-** 권한이 있는지 확인하려면 **susan** 사용자에게 **---** 권한이 있으며 기타 사용자에게 **r--** 권한이 있는지 확인합니다.

```
$ getfacl group-project
```

출력은 다음을 반환합니다.

```
# file: group-project
# owner: root
# group: root
user:andrew:rw-
user:susan:---
group::r--
mask::rw-
other::r--
```

13.3. UMASK 관리

umask 유틸리티를 사용하여 **umask** 의 현재 또는 기본값을 표시, 설정 또는 변경할 수 있습니다.

13.3.1. umask의 현재 값 표시

umask 유틸리티를 사용하여 현재 **umask** 값을 *심볼릭* 또는 8진수 모드로 표시할 수 있습니다.

절차

- 현재 *umask* 값을 심볼릭 모드로 표시하려면 다음을 사용합니다.

```
$ umask -S
```

- 8진수 모드에서 현재 *umask* 값을 표시하려면 다음을 사용합니다.

```
$ umask
```

참고

8진수 모드로 *umask* 를 표시할 때 4자리 숫자(**0002** 또는 **0022**)로 표시될 수 있습니다. *umask* 의 첫 번째 숫자는 특수 비트(고정 비트, SGID 비트 또는 SUID 비트)를 나타냅니다. 첫 번째 숫자가 **0** 으로 설정되면 특수 비트가 설정되지 않습니다.

13.3.2. 기본 bash umask 표시

사용할 수 있는 쉘에는 **bash**,**ksh**,**zsh** 및 **tcsh** 와 같이 사용할 수 있습니다. 이러한 쉘은 로그인 또는 비로그인 쉘로 작동할 수 있습니다. 네이티브 또는 GUI 터미널을 열어 로그인 쉘을 호출할 수 있습니다.

로그인 또는 비로그인 쉘에서 명령을 실행 중인지 확인하려면 **echo alpha** 명령을 사용합니다.

예 13.10. 로그인 또는 비로그인 bash 쉘에서 작업 중인지 확인

- **echo Zero** 명령 의 출력이 **bash** 를 반환하면 비로그인 쉘에서 명령을 실행합니다.

  ```
  $ echo $0
  bash
  ```

  로그인이 아닌 쉘의 기본 *umask* 는 **/etc/bashrc** 구성 파일에 설정됩니다.

- **echo Zero** 명령 의 출력이 **-bash** 를 반환하면 로그인 쉘에서 명령을 실행합니다.

  ```
  # echo $0
  -bash
  ```

  로그인 쉘의 기본 *umask* 는 **/etc/profile** 구성 파일에 설정되어 있습니다.

절차

- 로그인이 아닌 쉘의 기본 **bash** *umask* 를 표시하려면 다음을 사용합니다.

  ```
  $ grep umask /etc/bashrc
  ```

  출력은 다음을 반환합니다.

  ```
  # By default, we want umask to get set. This sets it for non-login shell.
        umask 002
        umask 022
  ```

- 로그인 쉘의 기본 **bash** *umask* 를 표시하려면 다음을 사용합니다.

  ```
  $ grep umask /etc/profile
  ```

  출력은 다음을 반환합니다.

  ```
  # By default, we want umask to get set. This sets it for login shell
        umask 002
        umask 022
  ```

13.3.3. 심볼릭 값을 사용하여 umask 설정

umask 유틸리티를 심볼릭 값(결합 문자 및 기호)과 함께 사용하여 현재 쉘 세션의*umask* 를 설정할 수 있습니다.

다음 *권한을* 할당할 수 있습니다.

- 읽기 (r)

- 쓰기 (w)
- 실행 (x)

권한을 다음 *수준의 소유권*에 할당할 수 있습니다.

- 사용자 소유자 (u)
- 그룹 소유자(g)
- 기타 (o)
- 모두 (a)

권한을 추가하거나 제거하려면 다음 *기호*를 사용할 수 있습니다:

- **+**: 기존 권한 위에 권한을 추가합니다.
- **-** 기존 권한에서 권한을 제거하려면 **-**
- **=** - 기존 권한을 제거하고 명시적으로 새 권한을 정의합니다.

참고

등호(**=**) 이후에 지정되지 않은 모든 권한은 자동으로 금지됩니다.

절차

- 현재 쉘 세션의 *umask* 를 설정하려면 다음을 사용합니다.

```
$ umask -S <level><operation><permission>
```

<level> 을 umask를 설정하려는소유권 수준으로 바꿉니다. ***<operation>*** 를 기호 중 하나로 바꿉니다. ***<permission>*** 을 할당하려는 권한 으로 바꿉니다. 예를 들어 umask를*umask* 를 **u=rwx,g=rwx,o=rwx** 로 설정하려면 **umask -S a=rwx** 를 사용합니다.

자세한 내용은 사용자 파일 생성 모드를 참조하십시오.

참고

umask 는 현재 쉘 세션에만 유효합니다.

13.3.4. 8진수 값을 사용하여 umask 설정

8진수 값(숫자)과 함께 **umask** 유틸리티를 사용하여 현재 쉘 세션의*umask* 를 설정할 수 있습니다.

절차

- 현재 쉘 세션의 *umask* 를 설정하려면 다음을 사용합니다.

```
$ umask octal_value
```

octal_value 를 8진수 값으로 교체합니다. 자세한 내용은User file-creation mode mask를 참조하십시오.

참고

umask 는 현재 쉘 세션에만 유효합니다.

13.3.5. 로그인이 아닌 쉘의 기본 umask 변경

/etc/bashrc 파일을 수정하여 표준 사용자의 기본**bash** *umask* 를 변경할 수 있습니다.

사전 요구 사항

- 루트 액세스

절차

1. 루트 로서 편집기에서 **/etc/bashrc** 파일을 엽니다.
2. 다음 섹션을 수정하여 새로운 기본 **bash** *umask* 를 설정합니다.

```
if [ $UID -gt 199 ] && [ “id -gn” = “id -un” ]; then
  umask 002
else
  umask 022
fi
```

   umask (002)의 기본 8진수 값을다른8진수 값으로 바꿉니다. 자세한 내용은User file-creation mode mask 를 참조하십시오.

3. 변경 사항을 저장하고 편집기를 종료합니다.

13.3.6. 로그인 쉘의 기본 umask 변경

/etc/profile 파일을 수정하여 **root** 사용자의 기본 **bash** *umask* 를 변경할 수 있습니다.

사전 요구 사항

- 루트 액세스

절차

1. 루트 권한으로 편집기에서 **/etc/profile** 파일을 엽니다.
2. 다음 섹션을 수정하여 새로운 기본 **bash** *umask* 를 설정합니다.

```
if [ $UID -gt 199 ] && [ “/usr/bin/id -gn” = “/usr/bin/id -un” ]; then
   umask 002
else
   umask 022
fi
```

   umask **(022)**의 기본 8진수 값을 다른 8진수 값으로 바꿉니다. 자세한 내용은User file-creation mode mask 를 참조하십시오.

3. 변경 사항을 저장하고 편집기를 종료합니다.

13.3.7. 특정 사용자의 기본 umask 변경

해당 사용자의. **bashrc** 를 수정하여 특정 사용자의 기본*umask* 를 변경할 수 있습니다.

절차

- *umask* 의 8진수 값을 특정 사용자의**.bashrc** 파일에 지정하는 행을 추가합니다.

```
$ echo 'umask octal_value' >> /home/username/.bashrc
```

octal_value 를 8진수 값으로 바꾸고*username* 을 사용자 이름으로 바꿉니다. 자세한 내용은User file-creation mode mask 를 참조하십시오.

13.3.8. 새로 생성된 홈 디렉터리에 대한 기본 권한 설정

/etc/login.defs 파일을 수정하여 새로 생성된 사용자의 홈 디렉토리에 대한 권한 모드를 변경할 수 있습니다.

절차

1. 루트 권한으로 편집기에서 **/etc/login.defs** 파일을 엽니다.
2. 다음 섹션을 수정하여 새로운 기본 *HOME_MODE* 를 설정합니다.

```
# HOME_MODE is used by useradd(8) and newusers(8) to set the mode for new
# home directories.
# If HOME_MODE is not set, the value of UMASK is used to create the mode.
HOME_MODE       0700
```

기본 8진수 값**(0700)**을 다른 8진수 값으로 교체합니다. 선택한 모드는 홈 디렉토리에 대한 권한을 생성하는 데 사용됩니다.

3. *HOME_MODE* 가 설정되어 있으면 변경 사항을 저장하고 편집기를 종료합니다.
4. *HOME_MODE* 가 설정되지 않은 경우*UMASK* 를 수정하여 새로 생성된 홈 디렉터리의 모드를 설정합니다.

```
# Default initial "umask" value used by login(1) on non-PAM enabled systems.
# Default "umask" value for pam_umask(8) on PAM enabled systems.
# UMASK is also used by useradd(8) and newusers(8) to set the mode for new
# home directories if HOME_MODE is not set.
# 022 is the default value, but 027, or even 077, could be considered
# for increased privacy. There is no One True Answer here: each sysadmin
# must make up their mind.

UMASK           022
```

기본 8진수 값**(022)**을 다른 8진수 값으로 바꿉니다. 자세한 내용은User file-creation mode mask 를 참조하십시오.

5. 변경 사항을 저장하고 편집기를 종료합니다.

14장. SYSTEMD 관리

시스템 관리자는 **systemd** 를 사용하여 시스템의 중요한 측면을 관리할 수 있습니다. Linux 운영 체제의 시스템 및 서비스 관리자 역할을 하는 **systemd** 소프트웨어 제품군은 제어, 보고 및 시스템 초기화를 위한 툴과 서비스를 제공합니다. **systemd** 의 주요 기능은 다음과 같습니다.

- 부팅 중 시스템 서비스의 병렬 시작
- 데몬의 온 디맨드 활성화
- 종속성 기반 서비스 제어 논리

systemd 가 관리하는 기본 오브젝트는 시스템 리소스 및 서비스를 나타내는*systemd* 장치입니다. **systemd** 장치는 특정 작업을 정의하고 관리하는 이름, 유형 및 구성 파일로 구성됩니다. 단위 파일을 사용하여 시스템 동작을 구성할 수 있습니다. 다양한 systemd 장치 유형의 다음 예제를 참조하십시오.

Service
: 개별 시스템 서비스를 제어 및 관리합니다.

대상
: 시스템 상태를 정의하는 단위 그룹을 나타냅니다.

장치
: 하드웨어 장치 및 가용성을 관리합니다.

Mount
: 파일 시스템 마운트를 처리합니다.

타이머
: 특정 간격으로 실행되도록 작업을 예약합니다.

참고

사용 가능한 유닛 유형을 모두 표시하려면 다음을 수행합니다.

```
# systemctl -t help
```

14.1. SYSTEMD 장치 파일 위치

다음 디렉토리 중 하나에서 단위 구성 파일을 찾을 수 있습니다.

표 14.1. Systemd 유닛 파일 위치

디렉터리	설명
/usr/lib/systemd/system/	설치된 RPM 패키지와 함께 배포된 **systemd** 장치 파일.
/run/systemd/system/	런타임에 생성된 **systemd** 장치 파일입니다. 이 디렉토리는 서비스 유닛 파일이 설치된 디렉토리보다 우선합니다.

디렉터리	설명
/etc/systemd/system/	**systemctl enable** 명령과 서비스 확장을 위해 추가된 유닛 파일을 사용하여 생성된 **systemd** 장치 파일입니다. 이 디렉토리는 런타임 유닛 파일이 있는 디렉터리보다 우선합니다.

systemd 의 기본 구성은 컴파일 중에 정의되며/**etc/systemd/system.conf** 파일에서 구성을 찾을 수 있습니다. 이 파일을 편집하여 전역적으로 **systemd** 단위의 값을 재정의하여 기본 구성을 수정할 수 있습니다.

예를 들어 90초로 설정된 시간 제한 제한의 기본값을 재정의하려면 **DefaultTimeoutStartSec** 매개 변수를 사용하여 필요한 값을 초 단위로 입력합니다.

```
DefaultTimeoutStartSec=required value
```

14.2. SYSTEMCTL을 사용하여 시스템 서비스 관리

시스템 관리자는 **systemctl** 유틸리티를 사용하여 시스템 서비스를 관리할 수 있습니다. 실행 중인 서비스 시작, 중지, 다시 시작, 서비스 활성화 및 비활성화, 사용 가능한 서비스 나열, 시스템 서비스 상태 표시 등 다양한 작업을 수행할 수 있습니다.

14.2.1. 시스템 서비스 나열

현재 로드된 모든 서비스 단위를 나열하고 사용 가능한 모든 서비스 단위의 상태를 표시할 수 있습니다.

절차

systemctl 명령을 사용하여 다음 작업을 수행합니다.

- 현재 로드된 모든 서비스 단위를 나열합니다.

```
$ systemctl list-units --type service
UNIT                    LOAD   ACTIVE SUB     DESCRIPTION
abrt-ccpp.service       loaded active exited  Install ABRT coredump hook
abrt-oops.service       loaded active running ABRT kernel log watcher
abrtd.service           loaded active running ABRT Automated Bug Reporting Tool
...
systemd-vconsole-setup.service loaded active exited  Setup Virtual Console
tog-pegasus.service            loaded active running OpenPegasus CIM Server

LOAD   = Reflects whether the unit definition was properly loaded.
ACTIVE = The high-level unit activation state, or a generalization of SUB.
SUB    = The low-level unit activation state, values depend on unit type.

46 loaded units listed. Pass --all to see loaded but inactive units, too.
To show all installed unit files use 'systemctl list-unit-files'
```

기본적으로 **systemctl list-units** 명령은 활성 유닛만 표시합니다. 각 서비스 단위 파일에 대해 명령은 다음 매개변수에 대한 개요를 제공합니다.

단위

서비스 유닛의 전체 이름

LOAD

구성 파일의 로드 상태

ACTIVE 또는 **SUB**

현재 고급 및 낮은 수준의 장치 파일 활성화 상태

DESCRIPTION

유닛의 목적과 기능에 대한 간략한 설명

- all 또는 **-a** 명령줄 옵션과 함께 다음 명령을 사용하여 상태에 관계없이 로드된모든 유닛 을 나열합니다.

```
$ systemctl list-units --type service --all
```

- 사용 가능한 모든 서비스 단위의 상태(활성화 또는 비활성화)를 나열합니다.

```
$ systemctl list-unit-files --type service
UNIT FILE                          STATE
abrt-ccpp.service                    enabled
abrt-oops.service                    enabled
abrtd.service                      enabled
...
wpa_supplicant.service               disabled
ypbind.service                     disabled

208 unit files listed.
```

서비스 유닛마다 이 명령이 표시됩니다.

단위 파일

서비스 유닛의 전체 이름

상태

부팅 중에 서비스 단위가 활성화되거나 비활성화되었는지 여부에 대한 정보

추가 리소스

- 시스템 서비스 상태 표시

14.2.2. 시스템 서비스 상태 표시

서비스 단위를 검사하여 자세한 정보를 가져오고 부팅 중에 시작되도록 활성화되었는지 또는 현재 실행 중인 서비스 상태인지 확인할 수 있습니다. 특정 서비스 유닛 이후 또는 그 전에 시작되도록 주문된 서비스를 볼 수도 있습니다.

절차

systemctl 명령을 사용하여 다음 작업을 수행합니다.

- 시스템 서비스에 해당하는 서비스 유닛에 대한 자세한 정보를 표시합니다.

```
$ systemctl status <name>.service
```

& *lt;name* >을 검사하려는 서비스 단위의 이름으로 바꿉니다(예:gdm).

이 명령은 다음 정보를 표시합니다.

- 선택한 서비스 단위의 이름 뒤에 간단한 설명
- 사용 가능한 서비스 단위 정보에설명된 하나 이상의 필드
- 서비스 유닛의 실행: 루트 사용자가 유닛을 실행하는 경우
- 최신 로그 항목

표 14.2. 사용 가능한 서비스 유닛 정보

필드	설명
로드됨	서비스 유닛이 로드되었는지 여부, 단위 파일의 절대 경로, 부팅 중에 장치를 시작할 수 있는지 여부를 확인합니다.
활성	서비스 유닛의 실행 여부와 타임스탬프에 대한 정보입니다.
Main PID	프로세스 ID 및 해당 시스템 서비스의 이름입니다.
상태	해당 시스템 서비스에 대한 추가 정보입니다.
프로세스	관련 프로세스에 대한 추가 정보.
cgroup	관련 제어 그룹(**cgroup**)에 대한 추가 정보.

예 14.1. 서비스 상태 표시

GNOME Display Manager의 서비스 유닛은 **gdm.service** 라고 합니다. 이 서비스 유닛의 현재 상태를 확인하려면 쉘 프롬프트에서 다음을 입력합니다.

```
# systemctl status gdm.service
gdm.service - GNOME Display Manager
   Loaded: loaded (/usr/lib/systemd/system/gdm.service; enabled)
   Active: active (running) since Thu 2013-10-17 17:31:23 CEST; 5min ago
 Main PID: 1029 (gdm)
   CGroup: /system.slice/gdm.service
           ├─1029 /usr/sbin/gdm
           └─1047 /usr/bin/Xorg :0 -background none -verbose -auth /r...

Oct 17 17:31:23 localhost systemd[1]: Started GNOME Display Manager.
```

- 특정 서비스 장치가 실행 중인지 확인합니다.

```
$ systemctl is-active <name>.service
```

- 부팅 중에 특정 서비스 단위가 시작되도록 활성화되어 있는지 확인합니다.

```
$ systemctl is-enabled <name>.service
```

> **참고**
>
> 지정된 서비스 장치가 실행 중이거나 활성화된 경우 **systemctl is-active** 및 **systemctl is-enabled** 명령은 종료 상태 **0** 을 반환합니다.

- 지정된 서비스 단위 전에 **systemd** 주문을 시작할 서비스 확인

```
# systemctl list-dependencies --after <name>.service
```

예를 들어 **gdm** 이전에 시작되도록 순서가 지정된 서비스 목록을 보려면 다음을 입력합니다.

```
# systemctl list-dependencies --after gdm.service
gdm.service
├─dbus.socket
├─getty@tty1.service
├─livesys.service
├─plymouth-quit.service
├─system.slice
├─systemd-journald.socket
├─systemd-user-sessions.service
└─basic.target
[output truncated]
```

- 지정된 서비스 단위 후에 시작할 서비스 **systemd** 주문을 확인합니다.

```
# systemctl list-dependencies --before <name>.service
```

예를 들어 **gdm** 후에 시작할 서비스**systemd** 주문 목록을 보려면 다음을 입력합니다.

```
# systemctl list-dependencies --before gdm.service
gdm.service
├─dracut-shutdown.service
├─graphical.target
│ ├─systemd-readahead-done.service
│ ├─systemd-readahead-done.timer
│ └─systemd-update-utmp-runlevel.service
└─shutdown.target
  ├─systemd-reboot.service
  └─final.target
    └─systemd-reboot.service
```

추가 리소스

- 시스템 서비스 나열

14.2.3. 시스템 서비스 시작

start 명령을 사용하여 현재 세션에서 시스템 서비스를 시작할 수 있습니다.

사전 요구 사항

- 루트 액세스

절차

- 현재 세션에서 시스템 서비스를 시작합니다.

```
# systemctl start <name>.service
```

& *lt;name* >을 시작하려는 서비스 단위의 이름으로 바꿉니다(예:**httpd.service**).

참고

systemd 에서는 서비스 간에 양수 및 음수 종속성이 있습니다. 특정 서비스를 시작하려면 하나 이상의 다른 서비스(공속성)를 시작하거나 하나 이상의 서비스(거의종속성)를 중지해야 할수 있습니다.

새 서비스를 시작하려고 하면 **systemd** 는 사용자에게 명시적 통지 없이 모든 종속 항목을 자동으로 해결합니다. 즉, 이미 서비스를 실행 중이고 음수 종속성을 사용하여 다른 서비스를 시작하려고 하면 첫 번째 서비스가 자동으로 중지됩니다.

예를 들어 **postfix** 서비스를 실행 중이고 **sendmail** 서비스를 시작하려고 하는 경우 **systemd** 는 먼저 **postfix** 를 자동으로 중지합니다. 이 두 서비스는 충돌하고 동일한 포트에서 실행할 수 없기 때문입니다.

추가 리소스

- 시스템의 **systemctl(1)** 도움말 페이지
- 부팅 시 시스템 서비스가 시작되도록 활성화
- 시스템 서비스 상태 표시

14.2.4. 시스템 서비스 중지

현재 세션에서 시스템 서비스를 중지하려면 **stop** 명령을 사용합니다.

사전 요구 사항

- 루트 액세스

절차

- 시스템 서비스를 중지합니다.

```
# systemctl stop <name>.service
```

& *lt;name* >을 중지하려는 서비스 단위의 이름으로 바꿉니다(예:**bluetooth**).

추가 리소스

- 시스템의 **systemctl(1)** 도움말 페이지

- 부팅할 때 시작할 시스템 서비스 비활성화
- 시스템 서비스 상태 표시

14.2.5. 시스템 서비스 다시 시작

restart 명령을 사용하여 현재 세션에서 시스템 서비스를 다시 시작하여 다음 작업을 수행할 수 있습니다.

- 현재 세션에서 선택한 서비스 장치를 중지하고 즉시 다시 시작합니다.
- 해당 서비스가 이미 실행 중인 경우에만 서비스 장치를 다시 시작합니다.
- 실행을 중단하지 않고 시스템 서비스의 구성을 다시 로드합니다.

사전 요구 사항

- 루트 액세스

절차

- 시스템 서비스를 다시 시작하십시오.

```
# systemctl restart <name>.service
```

& *lt;name* >을 재시작할 서비스 단위의 이름으로 바꿉니다(예:**httpd**).

> 참고
>
> 선택한 서비스 유닛이 실행 중이 아니면 이 명령도 시작합니다.

- 선택 사항: 해당 서비스가 이미 실행 중인 경우에만 서비스 장치를 다시 시작하십시오.

```
# systemctl try-restart <name>.service
```

- 선택 사항: 서비스 실행을 중단하지 않고 구성을 다시 로드합니다.

```
# systemctl reload <name>.service
```

> 참고
>
> 이 기능을 지원하지 않는 시스템 서비스, 이 명령을 무시합니다. 이러한 서비스를 다시 시작하려면 대신 **reload-or-restart** 및 **reload-or-try-restart** 명령을 사용합니다.

추가 리소스

- 시스템의 **systemctl** man page
- 시스템 서비스 상태 표시

14.2.6. 부팅 시 시스템 서비스가 시작되도록 활성화

부팅 시 서비스가 자동으로 시작되도록 활성화할 수 있습니다. 이러한 변경 사항은 다음 재부팅 시 적용됩니다.

사전 요구 사항

- 루트 액세스
- 사용하려는 서비스는 마스킹할 수 없습니다. 마스킹된 서비스가 있는 경우 먼저 마스킹을 해제합니다.

```
# systemctl unmask <name>.service
```

절차

- 부팅 시 시작할 서비스를 활성화합니다.

```
# systemctl enable <name>.service
```

& *lt;name* >을 활성화하려는 서비스 단위의 이름으로 바꿉니다(예:**httpd**).

- 선택 사항: 단일 명령을 사용하여 서비스를 활성화하고 시작할 수도 있습니다.

```
# systemctl enable --now <name>.service
```

추가 리소스

- 시스템의 **systemctl(1)** 도움말 페이지
- 시스템 서비스 상태 표시
- 시스템 서비스 시작

14.2.7. 부팅할 때 시작할 시스템 서비스 비활성화

부팅 시 서비스 유닛이 자동으로 시작되지 않도록 할 수 있습니다. 서비스를 비활성화하면 부팅 시 시작되지 않지만 수동으로 시작할 수 있습니다. 수동으로 시작할 수 없도록 서비스를 마스킹할 수도 있습니다. 마스킹은 서비스를 다시 마스크 해제할 때까지 영구적으로 사용할 수 없게 하는 서비스를 비활성화하는 방법입니다.

사전 요구 사항

- 루트 액세스

절차

- 부팅 시 시작할 서비스를 비활성화합니다.

```
# systemctl disable <name>.service
```

& *lt;name* >을 비활성화하려는 서비스 단위의 이름으로 바꿉니다(예:**bluetooth**).

- 선택 사항: 서비스를 영구적으로 사용할 수 없게 하려면 서비스를 마스크하십시오.

```
# systemctl mask <name>.service
```

이 명령은 **/etc/systemd/system/***name***.service** 파일을 **/dev/null** 에 대한 심볼릭 링크로 교체하여 **systemd** 에 액세스할 수 없는 실제 유닛 파일을 렌더링합니다.

추가 리소스

- 시스템의 **systemctl(1)** 도움말 페이지
- 시스템 서비스 상태 표시
- 시스템 서비스 중지

14.3. 대상 시스템 상태로 부팅

시스템 관리자는 시스템의 부팅 프로세스를 제어하고 시스템을 부팅할 상태를 정의할 수 있습니다. 이를 **systemd** 대상이라고 하며 시스템이 특정 수준의 기능에 도달하기 시작하는**systemd** 장치 세트입니다. systemd 대상으로 작업하는 동안 기본 대상을 보고, 런타임에 대상을 선택하고, 기본 부팅 대상을 변경하고, 긴급 또는 복구 대상으로 부팅할 수 있습니다.

14.3.1. 대상 단위 파일

systemd 의 대상은 시스템을 시작하는 동안 동기화 지점 역할을 하는 관련 장치의 그룹입니다.**target** 파일 확장자로 끝나는 대상 장치 파일은 **systemd** 대상을 나타냅니다. 대상 장치의 목적은 종속성 체인을 통해 다양한 **systemd** 장치를 그룹화하는 것입니다.

다음 예제를 고려하십시오.

- 그래픽 세션을 시작하기 위한 **graphical.target** 단위는 GNOME Display Manager(**gdm.service**) 또는 계정 서비스(**accounts-daemon.service**)와 같은 시스템 서비스를 시작하고,**multi-user.target** 단위 도 활성화합니다.
- 마찬가지로, **multi-user.target** 단위는 NetworkManager(**NetworkManager.service**) 또는 D-Bus(**dbus.service**)와 같은 다른 필수 시스템 서비스를 시작하고**basic.target** 이라는 다른 대상 장치를 활성화합니다.

다음 **systemd** 대상을 기본값 또는 현재 대상으로 설정할 수 있습니다.

표 14.3. 일반적인 **systemd** 대상

rescue	기본 시스템에서 가져오고 복구 쉘을 생성하는 단위 대상
multi-user	다중 사용자 시스템을 설정하기 위한 단위 대상
graphical	그래픽 로그인 화면을 설정하는 단위 대상
emergency	기본 콘솔에서 긴급 쉘을 시작하는 단위 대상

추가 리소스

- 시스템의 **systemd.special(7)** 및 **systemd.target(5)** 도움말 페이지

14.3.2. 부팅할 기본 대상 변경

시스템이 시작되면 **systemd** 는 실제 대상 장치를 가리키는**default.target** 심볼릭 링크를 활성화합니다. **/etc/systemd/system/default.target** 파일에서 현재 선택한 기본 대상 장치를 찾을 수 있습니다. 각 대상은 특정 수준의 기능을 나타내며 다른 단위를 그룹화하는 데 사용됩니다. 또한 대상 단위는 부팅 중에 동기화 지

점 역할을 합니다. 시스템이 부팅되는 기본 대상을 변경할 수 있습니다. 기본 대상 장치를 설정하면 다음에 다시 부팅할 때까지 현재 대상이 변경되지 않은 상태로 유지됩니다.

사전 요구 사항

- 루트 액세스

절차

1. **systemd** 가 시스템을 시작하는 데 사용하는 현재 기본 대상 장치를 결정합니다.

```
# systemctl get-default
graphical.target
```

2. 현재 로드된 대상을 나열합니다.

```
# systemctl list-units --type target
```

3. 기본적으로 다른 대상 단위를 사용하도록 시스템을 구성합니다.

```
# systemctl set-default <name>.target
```

& *lt;name* >을 기본적으로 사용하려는 대상 단위의 이름으로 바꿉니다.

```
Example:
# systemctl set-default multi-user.target
Removed /etc/systemd/system/default.target
Created symlink /etc/systemd/system/default.target -> /usr/lib/systemd/system/multi-user.target
```

4. 기본 대상 단위를 확인합니다.

```
# systemctl get-default
multi-user.target
```

5. 재부팅하여 변경 사항을 적용합니다.

```
# reboot
```

추가 리소스

- 시스템의 **systemctl(1)**, **systemd.special(7)** 및 **bootup(7)** 도움말 페이지

14.3.3. 현재 대상 변경

실행 중인 시스템에서 재부팅하지 않고 현재 부팅의 대상 장치를 변경할 수 있습니다. 다른 대상으로 전환하면 **systemd** 는 이 대상에 필요한 모든 서비스와 해당 종속 항목을 시작하고 새 대상이 활성화하지 않는 모든 서비스를 중지합니다. 다른 대상을 격리하면 현재 부팅에만 영향을 미칩니다.

절차

1. 선택 사항: 현재 대상을 확인합니다.

```
# systemctl get-default
graphical.target
```

2. 선택 사항: 선택할 수 있는 대상 목록을 표시합니다.

```
# systemctl list-units --type target
```

> **참고**
>
> 단위 파일에 **AllowIsolate=yes** 옵션이 설정된 대상만 격리할 수 있습니다.

3. 현재 부팅 시 다른 대상 단위로 변경합니다.

```
# systemctl isolate <name>.target
```

& *lt;name* >을 현재 부팅 시 사용할 대상 단위의 이름으로 바꿉니다.

```
Example:
# systemctl isolate multi-user.target
```

이 명령은 **multi-user** 및 모든 종속 단위라는 대상 장치를 시작하고 다른 모든 장치를 즉시 중지합니다.

추가 리소스

- 시스템의 **systemctl(1)** 도움말 페이지

14.3.4. 복구 모드로 부팅

시스템이 이후 대상에 도달할 수 없는 경우 문제 해결 또는 복구를 위해 단일 사용자 환경을 제공하는 복구 모드로 부팅하고 일반 부팅 프로세스가 실패합니다. 복구 모드에서는 시스템이 모든 로컬 파일 시스템을 마운트하고 특정 중요한 시스템 서비스를 시작하려고 하지만 네트워크 인터페이스를 활성화하지는 않습니다.

사전 요구 사항

- 루트 액세스

절차

- 복구 모드로 들어가려면 현재 세션의 현재 대상을 변경합니다.

```
# systemctl rescue

Broadcast message from root@localhost on pts/0 (Fri 2023-03-24 18:23:15 CEST):

The system is going down to rescue mode NOW!
```

참고

이 명령은 **systemctl isolate rescue.target** 과 유사하지만 현재 시스템에 로그인된 모든 사용자에게 정보 메시지를 전송합니다.

systemd 가 메시지를 보내지 않도록 하려면**--no-wall** 명령줄 옵션을 사용하여 다음 명령을 입력합니다.

```
# systemctl --no-wall rescue
```

문제 해결 단계

시스템이 복구 모드로 전환할 수 없는 경우 가능한 최소한의 환경을 제공하는 *긴급 모드로* 부팅할 수 있습니다. 긴급 모드에서는 읽기 전용으로 루트 파일 시스템을 마운트하고 다른 로컬 파일 시스템을 마운트하지 않고 네트워크 인터페이스를 활성화하지 않으며 몇 가지 필수 서비스만 시작합니다.

14.3.5. 부팅 프로세스 문제 해결

시스템 관리자는 부팅 시 기본값이 아닌 대상을 선택하여 부팅 프로세스의 문제를 해결할 수 있습니다. 부팅 시 대상을 변경하면 단일 부팅에만 영향을 미칩니다. 가능한 가장 최소한의 환경을 제공하는 *긴급 모드로* 부팅할 수 있습니다.

절차

1. 시스템을 재부팅하고 일반 부팅을 시작하는 **Enter** 키를 제외한 임의의 키를 눌러 부트 로더 메뉴 카운트다운을 중단합니다.
2. 시작할 커널 항목으로 커서를 이동합니다.
3. **E** 키를 눌러 현재 항목을 편집합니다.
4. **linux** 로 시작하는 행 끝으로 이동하고 **Ctrl+E**를 눌러 행 끝으로 이동합니다.

```
linux ($root)/vmlinuz-5.14.0-70.22.1.e19_0.x86_64 root=/dev/mapper/rhel-root ro crash\
kernel=auto resume=/dev/mapper/rhel-swap rd.lvm.lv/swap rhgb quiet
```

5. 대체 부팅 대상을 선택하려면 **linux** 로 시작하는 행의 끝에**systemd.unit=** 매개 변수를 추가합니다.

```
linux ($root)/vmlinuz-5.14.0-70.22.1.e19_0.x86_64 root=/dev/mapper/rhel-root ro crash\
kernel=auto resume=/dev/mapper/rhel-swap rd.lvm.lv/swap rhgb quiet
systemd.unit=<name>.target
```

& ***lt;name*** >을 사용하려는 대상 단위의 이름으로 바꿉니다. 예:**systemd.unit=emergency.target**

6. **Ctrl+X**를 눌러 이러한 설정으로 부팅합니다.

14.4. 시스템 종료, 일시 중지 및 최대 절전 모드

시스템 관리자는 다양한 전원 관리 옵션을 사용하여 전력 소비를 관리하고, 적절한 종료를 수행하여 모든 데이터가 저장되도록 하거나, 시스템을 다시 시작하여 변경 및 업데이트를 적용할 수 있습니다.

14.4.1. 시스템 종료

시스템을 종료하려면 **systemctl** 유틸리티를 직접 사용하거나 **shutdown** 명령을 통해 이 유틸리티를 호출할 수 있습니다.

종료 를 사용하면 다음과 같은 이점이 있습니다.

- **time** 인수를 사용하여 종료를 예약할 수 있습니다. 또한 시스템 종료가 예약되었음을 사용자에게 경고합니다.
- 종료를 취소할 수 있습니다.

추가 리소스

- systemctl을 사용한 전원 관리 명령 개요

14.4.2. 시스템 종료 예약

시스템 관리자는 지연된 종료를 예약하여 사용자가 작업을 저장하고 시스템을 로그아웃하는 시간을 제공할 수 있습니다. **shutdown** 명령을 사용하여 다음 작업을 수행합니다.

- 시스템을 종료하고 특정 시간에 시스템 전원을 끕니다.
- 시스템의 전원을 끄지 않고 시스템 종료 및 중지
- 보류 중인 종료 취소

사전 요구 사항

- 루트 액세스

절차

shutdown 명령을 사용하여 다음 작업을 수행합니다.

- 시스템을 종료할 시간을 지정하고 머신의 전원을 끕니다.

  ```
  # shutdown --poweroff hh:mm
  ```

  여기서 ***hh:mm*** 는 24 시간 표기법의 시간입니다. 새 로그인을 방지하기 위해 시스템을 종료하기 전에 **/run/nologin** 파일이 5분 전에 생성됩니다.

  시간 인수를 사용하는 경우 선택적 *월 메시지* (예: **shutdown --poweroff 13:59 "Attention)**를 지정하여 계획된 종료의 시스템에 로그인한 사용자에게 알릴 수 있습니다. 시스템이 **13:59**에서 종료됩니다..

- 머신의 전원을 끄지 않고 지연 후 시스템을 종료하고 중지합니다.

  ```
  # shutdown --halt +m
  ```

  여기서 ***+m*** 은 지연 시간(분)입니다. **now** 키워드를 **+0** 의 별칭으로 사용할 수 있습니다.

- 보류 중인 종료를 취소합니다.

  ```
  # shutdown -c
  ```

추가 리소스

- **shutdown(8)** 매뉴얼 페이지
- systemctl 명령을 사용하여 시스템 종료

14.4.3. systemctl 명령을 사용하여 시스템 종료

시스템 관리자는 시스템을 종료하고 시스템 전원을 끄거나 **systemctl** 명령을 사용하여 시스템의 전원을 끄지 않고 시스템을 중지할 수 있습니다.

사전 요구 사항

- 루트 액세스

절차

systemctl 명령을 사용하여 다음 작업을 수행합니다.

- 시스템을 종료하고 시스템 전원을 끕니다.

```
# systemctl poweroff
```

- 시스템 전원을 끄지 않고 시스템을 종료하고 중지합니다.

```
# systemctl halt
```

참고

기본적으로 이러한 명령 중 하나를 실행하면 **systemd** 에서 현재 시스템에 로그인한 모든 사용자에게 정보 메시지를 보냅니다. **systemd** 가 이 메시지를 보내지 않도록 하려면 **--no-wall** 명령줄 옵션을 사용하여 선택한 명령을 실행합니다.

14.4.4. 시스템을 다시 시작

시스템을 다시 시작하면 **systemd** 는 실행 중인 모든 프로그램 및 서비스를 중지하고 시스템이 종료되고 즉시 다시 시작합니다. 시스템을 다시 시작하면 다음과 같은 상황에서 도움이 될 수 있습니다.

- 새 소프트웨어 또는 업데이트를 설치한 후
- 시스템 설정을 변경한 후
- 시스템 문제 해결 시

사전 요구 사항

- 루트 액세스

절차

- 시스템을 다시 시작하십시오.

```
# systemctl reboot
```

참고

기본적으로 이 명령을 사용하면 **systemd** 는 현재 시스템에 로그인한 모든 사용자에게 정보 메시지를 보냅니다. **systemd** 가 이 메시지를 보내지 않도록 하려면**--no-wall** 옵션을 사용하여 이 명령을 실행합니다.

14.4.5. 시스템을 일시 중단하고 완화하여 전력 소비 최적화

시스템 관리자는 전력 소비를 관리하고, 시스템에 전력을 절약하고, 시스템의 현재 상태를 유지할 수 있습니다. 이렇게 하려면 다음 모드 중 하나를 적용합니다.

일시 중단

일시 중지는 RAM에 시스템 상태를 저장하고 RAM 모듈을 제외하고 시스템의 대부분의 장치의 전원을 끕니다. 시스템을 다시 켜면 시스템은 다시 부팅하지 않고도 RAM에서 상태를 복원합니다. 시스템 상태가 하드 디스크에 저장되지 않고 RAM에 저장되므로 시스템을 일시 중지 모드에서 복원하는 것은 최대 절전 모드보다 훨씬 빠릅니다. 그러나 일시 중지된 시스템 상태도 정전에 취약합니다.

Hibernate

hibernating은 시스템 상태를 하드 디스크 드라이브에 저장하고 시스템의 전원을 켭니다. 시스템을 다시 켜면 시스템에서 다시 부팅하지 않고도 저장된 데이터에서 해당 상태를 복원합니다. 시스템 상태가 RAM이 아닌 하드 디스크에 저장되므로 시스템은 RAM 모듈에 대한 전기 전원을 유지할 필요가 없습니다. 그러나 이로 인해 시스템을 최대 절전 모드로 복원하는 것은 일시 중지 모드에서 복원하는 것보다 훨씬 느립니다.

하이브리드 절전

이는 하이버네이션 및 일시 중단의 요소를 결합합니다. 시스템은 먼저 하드 디스크 드라이브에 현재 상태를 저장하고 일시 중단과 유사한 저전력 상태를 입력하면 시스템이 더 빨리 다시 시작할 수 있습니다. 하이브리드 절전의 이점은 시스템이 절전 상태 중에 전원을 끊는 경우에도 하이버네이션과 유사하게 하드 디스크에 저장된 이미지에서 이전 상태를 복구할 수 있다는 것입니다.

suspend-ECDHE-hibernate

이 모드는 먼저 시스템을 일시 중단하여 현재 시스템 상태를 RAM에 저장하고 시스템을 저전력 모드로 전환합니다. **HibernateDelaySec** 매개변수에 정의할 수 있는 특정 기간 동안 일시 중지된 경우 시스템 hibernates입니다. 하이버네이션은 시스템 상태를 하드 디스크 드라이브에 저장하고 시스템을 완전히 종료합니다. suspend-hibernate 모드는 여전히 작업을 재개할 수 있는 동안 배터리 전원을 예약할 수 있는 이점을 제공합니다. 또한 이 모드를 사용하면 정전 시 데이터가 저장됩니다.

사전 요구 사항

- 루트 액세스

절차

절전을 위한 적절한 방법을 선택합니다.

- 시스템을 일시 중지합니다.

```
# systemctl suspend
```

- Hibernate 시스템:

```
# systemctl hibernate
```

- Hibernate 및 시스템 일시 중단:

```
# systemctl hybrid-sleep
```

–

- 시스템을 일시 중지한 다음 hibernate를 실행합니다.

```
# systemctl suspend-then-hibernate
```

14.4.6. systemctl을 사용하여 전원 관리 명령 개요

다음 **systemctl** 명령 목록을 사용하여 시스템의 전원 관리를 제어할 수 있습니다.

표 14.4. systemctl 전원 관리 명령 개요

systemctl 명령	설명
systemctl halt	시스템을 중지합니다.
systemctl poweroff	시스템의 전원을 끕니다.
systemctl reboot	시스템을 다시 시작합니다.
systemctl suspend	시스템을 일시 중단합니다.
systemctl hibernate	시스템을 최대 절전 모드로 전환합니다.
systemctl hybrid-sleep	시스템을 최대화 및 일시 중단합니다.

14.4.7. 전원 버튼 동작 변경

컴퓨터의 전원 버튼을 누르면 기본적으로 시스템이 일시 중지되거나 종료됩니다. 기본 설정에 따라 이 동작을 사용자 지정할 수 있습니다.

14.4.7.1. systemd에서 전원 버튼 동작 변경

그래픽이 아닌 **systemd** 대상에서 전원 버튼을 누르면 기본적으로 시스템이 종료됩니다. 기본 설정에 따라 이 동작을 사용자 지정할 수 있습니다.

사전 요구 사항

- 관리 액세스.

절차

1. **/etc/systemd/logind.conf** 구성 파일을 엽니다.
2. **HandlePowerKey=poweroff** 라는 행을 찾습니다.
3. 행이 **#** 기호로 시작하는 경우 이를 제거하여 설정을 활성화합니다.
4. **poweroff** 를 다음 옵션 중 하나로 교체합니다.

   poweroff

   컴퓨터를 종료합니다.

reboot

시스템을 재부팅합니다.

halt

시스템 중단을 시작합니다.

kexec

kexec 재부팅을 시작합니다.

일시 중단

시스템을 일시 중지합니다.

hibernate

시스템 hibernation을 시작합니다.

무시

아무것도 하지 마십시오.

예를 들어 전원 버튼을 누를 때 시스템을 재부팅하려면 다음 설정을 사용합니다.

```
HandlePowerKey=reboot
```

5. 변경 사항을 저장하고 편집기를 종료합니다.

다음 단계

- 그래픽 세션을 사용하는 경우 GNOME에 전원 버튼도 구성합니다. 14.4.7.2절. "GNOME에서 전원 버튼 동작 변경"을 참조하십시오.

14.4.7.2. GNOME에서 전원 버튼 동작 변경

그래픽 로그인 화면 또는 그래픽 사용자 세션에서 전원 버튼을 누르면 기본적으로 시스템이 일시 중지됩니다. 이는 사용자가 전원 버튼을 물리적으로 누르거나 원격 콘솔에서 가상 전원 버튼을 누를 때 두 경우 모두 발생합니다. 다른 전원 버튼 동작을 선택할 수 있습니다.

사전 요구 사항

- **systemd** 에서 전원 버튼 동작을 구성했습니다.14.4.7.1절. "systemd에서 전원 버튼 동작 변경"을 참조하십시오.

절차

1. **/etc/dconf/db/local.d/01-power** 파일에서 시스템 전체 설정을 위한 로컬 데이터베이스를 만듭니다. 다음 콘텐츠를 입력합니다.

```
[org/gnome/settings-daemon/plugins/power]
power-button-action='suspend'
```

suspend 를 다음 전원 버튼 작업으로 교체합니다.

없음

아무것도 하지 않습니다.

일시 중단

시스템을 일시 중단합니다.

hibernate

시스템을 최대 절전 모드로 전환합니다.

대화형

사용자에게 수행할 작업을 요청하는 팝업 쿼리를 표시합니다.
대화형 모드를 사용하면 전원 버튼을 누를 때 60초 후에 시스템이 자동으로 꺼집니다. 그러나 팝업 쿼리에서 다른 동작을 선택할 수 있습니다.

2. 선택 사항: 사용자 설정을 재정의하고 사용자가 변경하지 못하도록 합니다.
/etc/dconf/db/local.d/locks/01-power 파일에 다음 구성을 입력합니다.

```
/org/gnome/settings-daemon/plugins/power/power-button-action
```

3. 시스템 데이터베이스를 업데이트합니다.

```
# dconf update
```

4. 시스템 전체 설정이 적용되려면 로그아웃한 후 다시 로그인합니다.

15장. 시간 동기화 구성

IT 환경에서 정확한 시간 유지가 중요합니다. 모든 네트워크 장치의 일관된 시간은 로그 파일 및 특정 프로토콜의 추적 가능성을 향상시킵니다. 예를 들어 Kerberos는 타임스탬프를 사용하여 재생 공격을 방지합니다.

15.1. CHRONY 제품군을 사용하여 NTP 구성

IT의 여러 가지 이유로 정확한 시간 유지는 중요합니다. 예를 들어 네트워킹에서는 패킷 및 로그의 정확한 타임스탬프가 필요합니다. Linux 시스템에서 **NTP** 프로토콜은 사용자 공간에서 실행되는 데몬에 의해 구현됩니다.

사용자 공간 데몬은 커널에서 실행되는 시스템 시계를 업데이트합니다. 시스템 클록은 다양한 시계 소스를 사용하여 시간을 유지할 수 있습니다. 일반적으로TSC(*Time Stamp Counter*)가 사용됩니다. TSC는 마지막으로 재설정된 이후 사이클 수를 계산하는 CPU 레지스터입니다. 매우 빠르며 해상도가 높으며 중단이 없습니다.

Red Hat Enterprise Linux 8부터 **NTP** 프로토콜은 **chronyd** 데몬에서 구현되며 **chrony** 패키지의 리포지토리에서 사용할 수 있습니다.

다음 섹션에서는 chrony 제품군을 사용하여 NTP를 구성하는 방법을 설명합니다.

15.1.1. Chrony 제품군 소개

Chrony 는 NTP(**Network Time Protocol**) 구현 입니다. Chrony :

- **NTP** 서버와 시스템 시계를 동기화하려면
- 참조 클록과 시스템 클록을 동기화하려면 (예: GPS 수신자)
- 수동 시간 입력과 시스템 시계를 동기화하려면
- **NTPv4(RFC 5905)** 서버 또는 피어로서 네트워크의 다른 컴퓨터에 시간 서비스 제공

Chrony 는 간헐적인 네트워크 연결, 과도하게 혼잡한 네트워크, 온도 변경(일반 컴퓨터 클록은 온도에 민감함) 및 지속적으로 실행되지 않거나 가상 시스템에서 실행되지 않는 시스템을 포함하여 다양한 조건에서 원활하게 작동합니다.

인터넷을 통해 동기화된 두 머신 간의 일반적인 정확성은 몇 밀리초 이내이며, 10마이크로초 내에 LAN에 있는 머신의 경우입니다. 하드웨어 타임스탬프 또는 하드웨어 참조 클록은 두 머신 간의 정확도를 1마이크로초 단위로 동기화할 수 있습니다.

Chrony 는 사용자 공간에서 실행되는 데몬인**chronyd**와 **chronyd** 의 성능을 모니터링하고 실행 중인 다양한 운영 매개 변수를 변경하는 데 사용할 수 있는 명령줄 프로그램인 **chronyc** 로 구성됩니다.

chrony 데몬인 **chronyd** 는 명령줄 **chronyc** 에서 모니터링하고 제어할 수 있습니다. 이 유틸리티는 **chronyd** 의 현재 상태를 쿼리하고 구성을 변경할 수 있도록 여러 명령을 입력할 수 있는 명령 프롬프트를 제공합니다. 기본적으로 **chronyd** 는 **chronyc** 의 로컬 인스턴스에서 명령만 수락하지만 원격 호스트에서도 모니터링 명령을 허용하도록 구성할 수 있습니다. 원격 액세스를 제한해야 합니다.

15.1.2. chronyc를 사용하여 chronyd 제어

chronyc 명령줄 유틸리티를 사용하여 chronyd 를 제어할 수 있습니다.

절차

1. 대화형 모드에서 utility chronyc 명령줄을 사용하여 chronyd 의 로컬 인스턴스를 변경하려면 root 로 다음 명령을 입력합니다.

```
# chronyc
```

일부 제한된 명령을 사용하려면 chronyc 를 root 로 실행해야 합니다.

chronyc 명령 프롬프트는 다음과 같이 표시됩니다.

```
chronyc>
```

2. 모든 명령을 나열하려면 help 를 입력합니다.

3. 또는 다음과 같이 명령과 함께 호출되는 경우 비대화형 명령 모드에서 유틸리티를 호출할 수도 있습니다.

```
chronyc command
```

참고

chronyc 를 사용하여 변경한 내용은 영구적이지 않으며 chronyd 재시작 후 손실됩니다. 영구 변경을 위해 /etc/chrony.conf 를 수정합니다.

15.1.3. Chrony로 마이그레이션

Red Hat Enterprise Linux 7에서는 사용자가 ntp 와 chrony 중에서 선택하여 정확한 시간 유지를 보장할 수 있습니다. ntp 와 chrony, ntpd 및 chronyd 의 차이점은 ntpd와 chronyd 간의 차이점을 참조하십시오.

Red Hat Enterprise Linux 8부터 NTP는 더 이상 지원되지 않습니다. Chrony 는 기본적으로 활성화되어 있습니다. 따라서 ntp 에서 chrony 로 마이그레이션해야 할 수 있습니다.

ntp 에서 chrony 로의 마이그레이션은 대부분의 경우 간단합니다. 프로그램, 구성 파일 및 서비스의 해당 이름은 다음과 같습니다.

표 15.1. ntp에서 chrony로 마이그레이션할 때 프로그램, 구성 파일 및 서비스의 해당 이름

NTP 이름	Chrony 이름
/etc/ntp.conf	/etc/chrony.conf
/etc/ntp/keys	/etc/chrony.keys
ntpd	Chronyd
ntpq	Chronyc
ntpd.service	chronyd.service
ntp-wait.service	chrony-wait.service

ntp 배포에 포함된 ntpdate 및 sntp 유틸리티는 -q 옵션 또는 - t 옵션을 사용하여 chronyd 로 바꿀 수 있습니다. /etc/chrony.conf 를 읽지 않도록 명령줄에서 구성을 지정할 수 있습니다. 예를 들어 ntpdate ntp.example.com 을 실행하는 대신chronyd 를 다음과 같이 시작할 수 있습니다.

```
# chronyd -q 'server ntp.example.com iburst'
2018-05-18T12:37:43Z chronyd version 3.3 starting (+CMDMON +NTP +REFCLOCK +RTC
+PRIVDROP +SCFILTER +SIGND +ASYNCDNS +SECHASH +IPV6 +DEBUG)
2018-05-18T12:37:43Z Initial frequency -2.630 ppm
2018-05-18T12:37:48Z System clock wrong by 0.003159 seconds (step)
2018-05-18T12:37:48Z chronyd exiting
```

이전에 ntp 패키지에 포함되었으며 ntpd만 지원되었던 ntp stat 유틸리티는 이제 ntpd 와 chronyd 를 둘 다 지원합니다. ntpstat 패키지에서 사용할 수 있습니다.

15.1.3.1. 마이그레이션 스크립트

ntp2chrony.py 라는 Python 스크립트는 chrony 패키지(/usr/share/doc/chrony) 설명서에 포함되어 있습니다. 스크립트는 기존 ntp 구성을 chrony 로 자동으로 변환합니다. ntp.conf 파일에서 가장 일반적인 지시문과 옵션을 지원합니다. 변환에서 무시되는 모든 행은 검토를 위해 생성된 chrony.conf 파일에 주석으로 포함됩니다. ntp 키 파일에 지정되지만 ntp .conf에서 신뢰할 수 있는 키로 표시되지 않는 키는 생성된 chrony. keys 파일에 주석으로 포함되어 있습니다.

기본적으로 스크립트는 파일을 덮어쓰지 않습니다. /etc/chrony.conf 또는 /etc/chrony.keys 가 이미 있는 경우 -b 옵션을 사용하여 파일 이름을 백업으로 변경할 수 있습니다. 이 스크립트는 다른 옵션을 지

원합니다. --help 옵션은 지원되는 모든 옵션을 인쇄합니다.

ntp 패키지에 제공된 기본 ntp.conf 를 사용하여 스크립트 호출의 예는 다음과 같습니다.

```
# python3 /usr/share/doc/chrony/ntp2chrony.py -b -v
Reading /etc/ntp.conf
Reading /etc/ntp/crypto/pw
Reading /etc/ntp/keys
Writing /etc/chrony.conf
Writing /etc/chrony.keys
```

이 경우 무시된 유일한 지시문은 noclientlog 지시문에 chrony가 동일하지만 증폭 공격을 완화하기 위해서만 기본 ntp.conf 에 포함되어 있는 disable monitor 입니다.

생성된 chrony.conf 파일에는 일반적으로 ntp.conf 의 restrict 행에 해당하는 다수의 allow 지시문이 포함되어 있습니다. chronyd 를 NTP 서버로 실행하지 않으려면 chrony.conf 에서 allow 지시문을 모두 제거합니다.

15.2. CHRONY 사용

다음 섹션에서는 chronyd 를 설치, 시작 및 중지하는 방법과 chrony 가 동기화되었는지 확인하는 방법을 설명합니다. 섹션에서는 시스템 클락을 수동으로 조정하는 방법도 설명합니다.

15.2.1. Chrony 관리

다음 절차에서는 chronyd 의 설치, 시작, 중지 및 상태를 확인하는 방법을 설명합니다.

절차

1. chrony 제품군은 기본적으로 Red Hat Enterprise Linux에 설치됩니다. 이를 확인하려면 root 로 다음 명령을 실행하십시오.

```
# yum install chrony
```

chrony 데몬의 기본 위치는 /usr/sbin/chronyd 입니다. 명령행 유틸리티는 /usr/bin/chronyc 에 설치됩니다.

2. chronyd 의 상태를 확인하려면 다음 명령을 실행합니다.

```
$ systemctl status chronyd
chronyd.service - NTP client/server
  Loaded: loaded (/usr/lib/systemd/system/chronyd.service; enabled)
  Active: active (running) since Wed 2013-06-12 22:23:16 CEST; 11h ago
```

3. chronyd 를 시작하려면 root 로 다음 명령을 실행합니다.

```
# systemctl start chronyd
```

시스템 시작 시 chronyd 가 자동으로 시작되도록 하려면 root 로 다음 명령을 실행하십시오.

```
# systemctl enable chronyd
```

4. chronyd 를 중지하려면 root 로 다음 명령을 실행하십시오.

```
# systemctl stop chronyd
```

시스템 시작 시 chronyd 가 자동으로 시작되지 않도록 하려면 root 로 다음 명령을 실행하십시오.

```
# systemctl disable chronyd
```

15.2.2. chrony가 동기화되었는지 확인

다음 절차에서는 chrony 가 추적, 소스및 sourcestats 명령을 사용하여 동기화되었는지 확인하는 방법을 설명합니다.

절차

1. Chrony 추적 을 확인하려면 다음 명령을 실행합니다.

```
$ chronyc tracking
Reference ID    : CB00710F (ntp-server.example.net)
Stratum         : 3
Ref time (UTC)  : Fri Jan 27 09:49:17 2017
System time     :  0.000006523 seconds slow of NTP time
Last offset     : -0.000006747 seconds
```

```
RMS offset      : 0.000035822 seconds
Frequency       : 3.225 ppm slow
Residual freq   : 0.000 ppm
Skew            : 0.129 ppm
Root delay      : 0.013639022 seconds
Root dispersion : 0.001100737 seconds
Update interval : 64.2 seconds
Leap status     : Normal
```

2. **source** 명령은 **chronyd** 가 액세스 중인 현재 시간 소스에 대한 정보를 표시합니다. **chrony** 소스를 확인하려면 다음 명령을 실행합니다.

```
$ chronyc sources
 210 Number of sources = 3
MS Name/IP address         Stratum Poll Reach LastRx Last sample
===============================================================================
====
#* GPS0                      0   4   377    11   -479ns[ -621ns] /-  134ns
^? a.b.c                     2   6   377    23   -923us[ -924us] +/-   43ms
^ d.e.f                      1   6   377    21  -2629us[-2619us] +/-   86ms
```

선택 사항 **-v** 인수를 지정하여 자세한 정보를 출력할 수 있습니다. 이 경우 열의 의미를 알리는 추가 캡션 행이 표시됩니다.

3. **sourcestats** 명령은 **chronyd** 에서 현재 검사하는 각 소스에 대한 드리프트 속도 및 오프셋 추정 프로세스에 대한 정보를 표시합니다. **chrony** 소스 통계를 확인하려면 다음 명령을 실행합니다.

```
$  chronyc sourcestats
210 Number of sources = 1
Name/IP Address            NP  NR  Span  Frequency  Freq Skew  Offset  Std Dev
===============================================================================
====
abc.def.ghi                11   5   46m     -0.001      0.045      1us    25us
```

자세한 정보를 포함하여 선택적 인수 **-v** 를 지정할 수 있습니다. 이 경우 열의 의미를 알리는 추가 캡션 행이 표시됩니다.

추가 리소스

- 시스템의 **chronyc(1)** 도움말 페이지

15.2.3. 시스템 시계 수동 조정

다음 절차에서는 시스템 클락을 수동으로 조정하는 방법을 설명합니다.

절차

1. 시스템 시계를 즉시 전환하려면 줄 바꿈으로 진행 중인 조정을 무시하고 **root** 로 다음 명령을 실행합니다.

```
# chronyc makestep
```

rtcfile 지시문을 사용하는 경우 실시간 클록을 수동으로 조정해서는 안 됩니다. 임의 조정으로 인해실시간 클록 드리프트의 속도를 측정해야 하는 **Chrony**의 필요성에 방해가 됩니다.

15.2.4. chrony 디스패치 스크립트 비활성화

chrony 디스패치 스크립트는 **NTP** 서버의 온라인 및 오프라인 상태를 관리합니다. 시스템 관리자는 디스패치 스크립트를 비활성화하여 **chronyd** 가 서버를 지속적으로 폴링하는 상태를 유지할 수 있습니다.

시스템에서 **NetworkManager**를 활성화하여 네트워킹 구성을 관리하는 경우 **NetworkManager**는 인터페이스 재구성, 중지 또는 시작 작업 중에 **chrony** 디스패처 스크립트를 실행합니다. 그러나 **NetworkManager** 이외의 특정 인터페이스 또는 경로를 구성하는 경우 다음과 같은 상황이 발생할 수 있습니다.

1. **NTP** 서버에 대한 경로가 없는 경우 디스패치 스크립트가 실행되어 **NTP** 서버가 오프라인 상태로 전환될 수 있습니다.

2. 나중에 경로를 설정하면 스크립트는 기본적으로 다시 실행되지 않으며 **NTP** 서버는 오프라인 상태로 유지됩니다.

chronyd 가 별도의 관리 인터페이스가 있는 **NTP** 서버와 동기화할 수 있도록 하려면 디스패치 스크립트를 비활성화합니다.

사전 요구 사항

- 시스템에 **NetworkManager**를 설치하고 활성화합니다.

- 루트 액세스

절차

1. chrony 디스패치 스크립트를 비활성화하려면 다음과 같이 /etc/NetworkManager/dispatcher.d/20-chrony- online 파일을 편집합니다.

```
#!/bin/sh
exit 0
```

참고

chrony 패키지를 업그레이드하거나 다시 설치할 때 패키지된 디스패치 스크립트가 수정된 디스패치 스크립트가 교체됩니다.

15.2.5. 격리된 네트워크에서 시스템의 chrony 설정

인터넷에 연결되지 않은 네트워크의 경우 하나의 컴퓨터가 기본 타이머로 선택됩니다. 다른 컴퓨터는 서버 또는 클라이언트의 직접 클라이언트입니다. 서버에서 드리프트 파일을 시스템 클럭의 평균 드리프트 비율로 수동으로 설정해야 합니다. 서버가 재부팅되면 주변 시스템에서 시간을 확보하고 평균을 계산하여 시스템 시계를 설정합니다. 그 후 드리프트 파일을 기반으로 조정 적용을 재개합니다. settime 명령을 사용하면 드리프트 파일이 자동으로 업데이트됩니다.

다음 절차에서는 격리된 네트워크에서 시스템의 chrony 를 설정하는 방법을 설명합니다.

절차

1. 시스템에서 root 로 실행되는 텍스트 편집기를 사용하여 /etc/chrony.conf 를 다음과 같이 편집합니다.

```
driftfile /var/lib/chrony/drift
commandkey 1
keyfile /etc/chrony.keys
initstepslew 10 client1 client3 client6
local stratum 8
manual
allow 192.0.2.0/24
```

여기서 192.0.2.0/24 는 클라이언트가 연결할 수 있는 네트워크 또는 서브넷 주소입니다. 자세한 내용은 시스템의 chrony.conf(7) 매뉴얼 페이지를 참조하십시오.

2. root 로 실행되는 텍스트 편집기를 사용하여 서버의 직접 클라이언트가 되도록 선택된 시스템에서 다음과 같이 /etc/chrony.conf 를 편집합니다.

```
server ntp1.example.net
driftfile /var/lib/chrony/drift
logdir /var/log/chrony
log measurements statistics tracking
keyfile /etc/chrony.keys
commandkey 24
local stratum 10
initstepslew 20 ntp1.example.net
allow 192.0.2.123
```

여기서 192.0.2.123 은 서버의 주소이며 ntp1.example.net 은 서버의 호스트 이름입니다. 이 구성을 사용하는 클라이언트는 재시작 시 서버와 다시 동기화됩니다.

서버의 직접 클라이언트가 아니어야 하는 클라이언트 시스템에서 /etc/chrony.conf 파일은 local 및 allow 지시문을 생략해야 한다는 점을 제외하고 동일해야 합니다.

격리된 네트워크에서 로컬 참조 모드를 활성화하는 로컬 지시문을 사용할 수도 있습니다. 이를 통해 chronyd 가 NTP 서버로 작동하면 동기화되지 않았거나 클록의 마지막 업데이트가 오래 전에 발생한 경우에도 chronyd가 실시간으로 동기화될 수 있습니다.

네트워크에 있는 여러 서버가 동일한 로컬 구성을 사용하도록 허용하고 둘 이상의 서버를 폴링하는 클라이언트를 혼동하지 않고 서로 동기화할 수 있도록 하려면 로컬 지시문의 고립 옵션을 사용하여 고립 모드를 활성화합니다. 각 서버는 로컬로 다른 모든 서버를 폴링하도록 구성해야 합니다 . 이렇게 하면 최소 참조 ID가 있는 서버만 로컬 참조가 활성 상태이며 다른 서버가 동기화됩니다. 서버가 실패하면 다른 서버가 인계됩니다.

15.2.6. 원격 모니터링 액세스 구성

Chronyc 는 다음 두 가지 방법으로 chronyd 에 액세스할 수 있습니다.

- 인터넷 프로토콜, IPv4 또는 IPv6.
- Unix 도메인 소켓 - root 또는 chrony 사용자가 로컬로 액세스할 수 있습니다.

기본적으로 chronyc 는 Unix 도메인 소켓에 연결됩니다. 기본 경로는 /var/run/chrony/chronyd.sock 입니다. 이 연결이 실패하고 예를 들어 루트가 아닌 사용자가 chronyc 를 실행 중인 경우 chronyc 는

127.0.0.1에 연결을 시도한 다음 ::1에 연결을 시도합니다.

네트워크에서 chronyd 동작에 영향을 미치지 않는 다음 모니터링 명령만 허용합니다.

- 활동
- 수동 목록
- rtcdata
- 완화 중
- 소스
- Sourcestats
- 추적
- waitsync

chronyd가 이러한 명령을 허용하는 호스트 집합은 chronyd 의 구성 파일에서 cmdallow 지시문 또는 chronyc 의 cmdallow 명령으로 구성할 수 있습니다. 기본적으로 명령은 localhost(127.0.0.1 또는 ::1)에서만 허용됩니다.

기타 모든 명령은 Unix 도메인 소켓을 통해서만 허용됩니다. 네트워크를 통해 보내면 chronyd 는 localhost에서 온 경우에도 Not authorized 오류로 응답합니다.

다음 절차에서는 chronyc 를 사용하여 원격으로 chronyd에 액세스하는 방법을 설명합니다.

절차

1.

다음을 /etc/chrony.conf 파일에 추가하여 IPv4 및 IPv6 주소 모두에서 액세스를 허용합니다.

```
bindcmdaddress 0.0.0.0
```

또는

```
bindcmdaddress ::
```

2. cmdallow 지시문을 사용하여 원격 IP 주소, 네트워크 또는 서브넷의 명령을 허용합니다.

   /etc/chrony.conf 파일에 다음 내용을 추가합니다.

```
cmdallow 192.168.1.0/24
```

3. 방화벽에서 포트 323을 열어 원격 시스템에서 연결합니다.

```
# firewall-cmd --zone=public --add-port=323/udp
```

   선택적으로 --permanent 옵션을 사용하여 포트 323을 영구적으로 열 수 있습니다.

```
# firewall-cmd --permanent --zone=public --add-port=323/udp
```

4. 323 포트를 영구적으로 여는 경우 방화벽 구성을 다시 로드합니다.

```
# firewall-cmd --reload
```

추가 리소스

- 시스템의 chrony.conf(5) 도움말 페이지

15.2.7. RHEL 시스템 역할을 사용하여 시간 동기화 관리

time sync 역할을 사용하여 여러 대상 시스템에서 시간 동기화를 관리할 수 있습니다. timesync 역할은 시스템 클럭을 동기화하도록 NTP 또는 PTP 클라이언트로 작동하도록 NTP 또는 PTP 구현을 설치하고 구성합니다.

또한 timesync 역할을 사용하면 ntp 또는 chrony를 사용하여 NTP 프로토콜을 구현하는지 여부에 관계없이 RHEL 6에서 시작하는 모든 Red Hat Enterprise Linux 버전에서 동일한 플레이북을 사용할 수 있기 때문에 쉽게 chrony 로 마이그레이션 할 수 있습니다.

> 주의
>
> timesync 역할은 관리 호스트에서 제공된 또는 탐지된 공급자 서비스의 구성을 대체합니다. 이전 설정은 역할 변수에 지정되지 않았더라도 손실됩니다. timesync_ntp_provider 변수가 정의되지 않은 경우 유일한 보존 설정은 프로바이더 선택 사항입니다.

다음 예제에서는 하나의 서버 풀만 있는 상황에서 timesync 역할을 적용하는 방법을 보여줍니다.

예 15.1. 단일 서버 풀에 대한 timesync 역할을 적용하는 예제 플레이북

```
---
- hosts: timesync-test
  vars:
    timesync_ntp_servers:
      - hostname: 2.rhel.pool.ntp.org
        pool: yes
        iburst: yes
  roles:
    - rhel-system-roles.timesync
```

timesync 역할 변수에 대한 자세한 참조는 rhel-system-roles 패키지를 설치하고 /usr/share/doc/rhel-system-roles/timesync 디렉터리의 README.md 또는 README.html 파일을 참조하십시오.

추가 리소스

- RHEL 시스템 역할을 사용하도록 컨트롤 노드 및 관리형 노드 준비

15.2.8. 추가 리소스

- 시스템의 chronyc(1) 및 chronyd(8) 도움말 페이지

- 자주하는 질문

15.3. HW 타임스탬프링을 사용하는 CHRONY

하드웨어 타임스탬프링은 수신 및 발신 패킷의 정확한 타임스탬프를 제공하는 일부 NIC(Network Interface Controller)에서 지원되는 기능입니다. NTP 타임스탬프는 일반적으로 시스템 클록을 사용하여 커널 및 chronyd 에 의해 생성됩니다. 그러나 HW 타임스탬프가 활성화되면 NIC는 자체 클록을 사용하여 패킷이 링크 계층 또는 물리적 계층을 입력하거나 종료할 때 타임스탬프를 생성합니다. NTP 와 함께 사용할 경우 하드웨어 타임스탬프를 사용하면 동기화의 정확성을 크게 향상시킬 수 있습니다. 최적의 정확성을 위해 NTP 서버와 NTP 클라이언트 모두 하드웨어 타임스탬프를 사용해야 합니다. 이상적인 상황에서 마이크로초 미만의 정확성이 가능할 수 있습니다.

하드웨어 타임스탬프를 사용하는 시간 동기화를 위한 또 다른 프로토콜은 PTP 입니다.

NTP 와 달리PTP 는 네트워크 스위치 및 라우터의 지원을 사용합니다. 동기화의 정확성을 극대화하려면 PTP 를 지원하는 스위치 및 라우터가 있는 네트워크에서 PTP 를 사용하고 이러한 스위치와 라우터가 없는 네트워크에서 NTP 를 선호합니다.

다음 섹션에서는 다음을 수행하는 방법을 설명합니다.

- 하드웨어 타임스탬프 지원 확인
- 하드웨어 타임스탬프 활성화
- 클라이언트 폴링 간격 구성
- 인터리브드 모드 활성화
- 많은 수의 클라이언트를 위해 서버 구성
- 하드웨어 타임스탬프 확인

- PTP-NTP 브리지 구성

15.3.1. 하드웨어 타임스탬프 지원 확인

인터페이스에서 NTP 를 사용한 하드웨어 타임스탬프를 지원하는지 확인하려면 ethtool -T 명령을 사용합니다. ethtool 이 SOF_TIMESTAMPING_TX_ HARDWARE 및 SOF_TIMESTAMPING_TX_QUAWARE 기능과 HWTSTAMPMP_FILTER_ ALL 필터 모드를 나열하는 경우 인터페이스를 NTP 로 하드웨어 타임스탬프에 사용할 수 있습니다.

예 15.2. 특정 인터페이스에서 하드웨어 타임스탬프 지원 확인

```
# ethtool -T eth0
```

출력:

```
Timestamping parameters for eth0:
Capabilities:
        hardware-transmit     (SOF_TIMESTAMPING_TX_HARDWARE)
        software-transmit     (SOF_TIMESTAMPING_TX_SOFTWARE)
        hardware-receive      (SOF_TIMESTAMPING_RX_HARDWARE)
        software-receive      (SOF_TIMESTAMPING_RX_SOFTWARE)
        software-system-clock (SOF_TIMESTAMPING_SOFTWARE)
        hardware-raw-clock    (SOF_TIMESTAMPING_RAW_HARDWARE)
PTP Hardware Clock: 0
Hardware Transmit Timestamp Modes:
        off                   (HWTSTAMP_TX_OFF)
        on                    (HWTSTAMP_TX_ON)
Hardware Receive Filter Modes:
        none                  (HWTSTAMP_FILTER_NONE)
        all                   (HWTSTAMP_FILTER_ALL)
        ptpv1-l4-sync         (HWTSTAMP_FILTER_PTP_V1_L4_SYNC)
        ptpv1-l4-delay-req    (HWTSTAMP_FILTER_PTP_V1_L4_DELAY_REQ)
        ptpv2-l4-sync         (HWTSTAMP_FILTER_PTP_V2_L4_SYNC)
        ptpv2-l4-delay-req    (HWTSTAMP_FILTER_PTP_V2_L4_DELAY_REQ)
        ptpv2-l2-sync         (HWTSTAMP_FILTER_PTP_V2_L2_SYNC)
        ptpv2-l2-delay-req    (HWTSTAMP_FILTER_PTP_V2_L2_DELAY_REQ)
        ptpv2-event           (HWTSTAMP_FILTER_PTP_V2_EVENT)
        ptpv2-sync            (HWTSTAMP_FILTER_PTP_V2_SYNC)
        ptpv2-delay-req       (HWTSTAMP_FILTER_PTP_V2_DELAY_REQ)
```

15.3.2. 하드웨어 타임스탬프 활성화

하드웨어 타임스탬프를 활성화하려면 /etc/chrony.conf 파일에서 hwtimestamp 지시문을 사용합니다. 지시문은 단일 인터페이스를 지정하거나 와일드카드 문자를 사용하여 이를 지원하는 모든 인터페이스에서 하드웨어 타임스탬프를 활성화할 수 있습니다. linuxptp 패키지의 ptp4l 과 같은 다른 애플리케이션

이 인터페이스에서 하드웨어 타임스탬프를 사용하는 경우 와일드카드 사양을 사용합니다. chrony 구성 파일에는 여러 hwtimestamp 지시문이 허용됩니다.

예 15.3. hwtimestamp 지시문을 사용하여 하드웨어 타임스탬프 활성화

```
hwtimestamp eth0
hwtimestamp eth1
hwtimestamp *
```

15.3.3. 클라이언트 폴링 간격 구성

인터넷의 서버에는 폴링 간격의 기본 범위(64-1024초)가 권장됩니다. 로컬 서버 및 하드웨어 타임스탬프의 경우 시스템 클럭의 오프셋을 최소화하기 위해 더 짧은 폴링 간격을 구성해야 합니다.

/etc/chrony.conf 의 다음 지시문은 1초 폴링 간격을 사용하여 로컬 NTP 서버를 지정합니다.

```
server ntp.local minpoll 0 maxpoll 0
```

15.3.4. 인터리브드 모드 활성화

하드웨어 NTP 어플라이언스가 아니라 소프트웨어 NTP 구현을 실행하는 범용 컴퓨터(예: chrony)는 패킷을 보낸 후에만 하드웨어 전송 타임스탬프를 가져옵니다. 이렇게 하면 서버가 해당 패킷의 타임스탬프를 저장하는 것을 방지할 수 있습니다. 전송 후 생성된 전송 타임스탬프를 수신하는 NTP 클라이언트를 활성화하려면 /etc/chrony.conf의 server 지시문에 xleave 옵션을 추가하여 클라이언트가 NTP 인터리브 모드를 사용하도록 구성합니다.

```
server ntp.local minpoll 0 maxpoll 0 xleave
```

15.3.5. 다수의 클라이언트에 대해 서버 구성

기본 서버 구성을 사용하면 대부분의 수천 개의 클라이언트가 인터리브드 모드를 동시에 사용할 수 있습니다. 다수의 클라이언트에 대해 서버를 구성하려면 /etc/chrony.conf 에서 clientloglimit 지시문을 늘립니다. 이 지시문은 서버에서 클라이언트 액세스 권한을 로깅하기 위해 할당된 최대 메모리 크기를 지정합니다.

```
clientloglimit 100000000
```

15.3.6. 하드웨어 타임스탬프 확인

인터페이스가 하드웨어 타임스탬프를 성공적으로 활성화했는지 확인하려면 시스템 로그를 확인합니

다. 로그에는 하드웨어 타임스탬프가 성공적으로 활성화된 각 인터페이스에 대해 **chronyd** 의 메시지가 포함되어야 합니다.

예 15.4. 하드웨어 타임스탬프가 활성화된 인터페이스에 대한 로그 메시지

```
chronyd[4081]: Enabled HW timestamping on eth0
chronyd[4081]: Enabled HW timestamping on eth1
```

chronyd 가 **NTP** 클라이언트 또는 피어로 구성되면 **chronyc ntpdata** 명령으로 각 **NTP** 소스에 대해 보고된 전송 및 수신 타임스탬프 모드와 인터리브드 모드가 있을 수 있습니다.

예 15.5. 각 **NTP** 소스에 대해 전송 보고, 수신 타임스탬프링 및 인터리브드 모드

```
# chronyc ntpdata
```

출력:

```
Remote address  : 203.0.113.15 (CB00710F)
Remote port     : 123
Local address   : 203.0.113.74 (CB00714A)
Leap status     : Normal
Version         : 4
Mode            : Server
Stratum         : 1
Poll interval   : 0 (1 seconds)
Precision       : -24 (0.000000060 seconds)
Root delay      : 0.000015 seconds
Root dispersion : 0.000015 seconds
Reference ID    : 47505300 (GPS)
Reference time  : Wed May 03 13:47:45 2017
Offset          : -0.000000134 seconds
Peer delay      : 0.000005396 seconds
Peer dispersion : 0.000002329 seconds
Response time   : 0.000152073 seconds
Jitter asymmetry: +0.00
NTP tests       : 111 111 1111
Interleaved     : Yes
Authenticated   : No
TX timestamping : Hardware
RX timestamping : Hardware
Total TX        : 27
Total RX        : 27
Total valid RX  : 27
```

예 15.6. **NTP** 측정의 안정성 보고

```
# chronyc sourcestats
```

하드웨어 타임스탬프를 활성화하면 NTP 측정의 안정성이 정상 부하에서 수십 또는 수백 나노초 내에 있어야 합니다. 이 안정성은 chronyc sourcestats 명령의 출력의 Std Dev 열에 보고됩니다.

출력:

```
210 Number of sources = 1
Name/IP Address          NP  NR  Span  Frequency  Freq Skew  Offset  Std Dev
ntp.local                12   7    11     +0.000      0.019     +0ns    49ns
```

15.3.7. PTP-NTP 브리지 구성

PTP(Precision Time Protocol) 기본 timeserver를 PTP를 지원하는 스위치 또는 라우터가 없는 네트워크에서 사용할 수 있는 경우 컴퓨터는 PTP 클라이언트 및 stratum-1 NTP 서버로만 작동할 수 있습니다. 이러한 컴퓨터에는 두 개 이상의 네트워크 인터페이스가 있어야 하며 기본 시간 서버에 가 있거나 직접 연결할 수 있어야 합니다. 이렇게 하면 네트워크에서 매우 정확한 동기화가 가능합니다.

하나의 인터페이스를 사용하여 PTP 를 사용하여 시스템 시계를 동기화하도록 linuxptp 패키지에서 ptp4l 및 phc2sys 프로그램을 구성합니다.

다른 인터페이스를 사용하여 시스템 시간을 제공하도록 chronyd 를 구성합니다.

예 15.7. 다른 인터페이스를 사용하여 시스템 시간을 제공하도록 chronyd 구성

```
bindaddress 203.0.113.74
hwtimestamp eth1
local stratum 1
```

15.4. CHRONY에서 NTP가 이전에 지원한 일부 설정 수행

ntp 에서 지원하는 이전 Red Hat Enterprise Linux 주 버전에 있는 일부 설정은 chrony 에서 지원되지 않습니다. 다음 섹션은 이러한 설정을 나열하고 chrony 가 있는 시스템에서 이를 수행하는 방법을 설명합니다.

15.4.1. ntpq 및 ntpdc로 모니터링

chrony 는 NTP 모드 6 및 7을 지원하지 않으므로 ntp 배포의 ntpq 및 ntpdc 유틸리티로 Chronyd를 모니터링할 수 없습니다. 다른 프로토콜을 지원하며 chronyc 는 클라이언트 구현입니다. 자세한 내용은 시스템의 chronyc(1) 매뉴얼 페이지를 참조하십시오.

chronyd 에서 대칭화된 시스템 클록의 상태를 모니터링하려면 다음을 수행할 수 있습니다.

- 추적 명령 사용
- ntpstat 유틸리티를 사용하여 chrony 를 지원하고 ntpd와 함께 사용된 것과 유사한 출력을 제공합니다.

예 15.8. 추적 명령 사용

```
$ chronyc -n tracking
Reference ID    : 0A051B0A (10.5.27.10)
Stratum         : 2
Ref time (UTC)  : Thu Mar 08 15:46:20 2018
System time     : 0.000000338 seconds slow of NTP time
Last offset     : +0.000339408 seconds
RMS offset      : 0.000339408 seconds
Frequency       : 2.968 ppm slow
Residual freq   : +0.001 ppm
Skew            : 3.336 ppm
Root delay      : 0.157559142 seconds
Root dispersion : 0.001339232 seconds
Update interval : 64.5 seconds
Leap status     : Normal
```

예 15.9. ntpstat 유틸리티 사용

```
$ ntpstat
synchronised to NTP server (10.5.27.10) at stratum 2
  time correct to within 80 ms
  polling server every 64 s
```

15.4.2. 공개 키 암호화를 기반으로 인증 메커니즘 사용

Red Hat Enterprise Linux 7에서 ntp 는 공개 키 암호화를 기반으로 하는 인증 메커니즘인 Autokey 를 지원합니다.

Red Hat Enterprise Linux 8에서 chronyd 는 Autokey 대신 최신 보안 인증 메커니즘인 NTP(Network Time Security)를 지원합니다. 자세한 내용은 chrony의 NTS(Network Time Security) 개

요를 참조하십시오.

15.4.3. 임시 대칭 연결 사용

Red Hat Enterprise Linux 7에서 ntpd 지원 임시 대칭 연관은 ntp.conf 구성 파일에 지정되지 않은 피어의 패킷에 의해 조작될 수 있습니다. Red Hat Enterprise Linux 8에서 chronyd 는 모든 피어를 chrony.conf 에 지정해야 합니다. 임시 대칭 연결은 지원되지 않습니다.

server 또는 pool 지시문에서 활성화하는 클라이언트/서버 모드를 사용하는 것은 peer 지시문에서 활성화하는 대칭 모드에 비해 더 안전합니다.

15.4.4. 멀티 캐스트/브로드 캐스트 클라이언트

Red Hat Enterprise Linux 7은 클라이언트 구성을 간소화하는 브로드캐스트/멀티캐스트 NTP 모드를 지원했습니다. 이 모드에서는 클라이언트가 개별 서버의 특정 이름이나 주소를 수신 대기하는 대신 멀티캐스트/브로드캐스트 주소로 전송되는 패킷만 수신 대기하도록 구성할 수 있습니다. 이러한 패킷은 시간이 지남에 따라 변경될 수 있습니다.

Red Hat Enterprise Linux 8에서 chronyd 는 브로드캐스트/멀캐스트 모드를 지원하지 않습니다. 주된 이유는 일반 클라이언트/서버 및 대칭 모드보다 덜 정확하고 안전하지 않기 때문입니다.

NTP 브로드캐스트/다중캐스트 설정에서 마이그레이션하는 몇 가지 옵션이 있습니다.

- ntp.example.com과 같은 단일 이름을 다른 서버의 여러 주소로 변환하도록 DNS 구성

  클라이언트는 단일 pool 지시문만 사용하여 여러 서버와 동기화할 정적 구성을 가질 수 있습니다. 풀의 서버를 복원할 수 없거나 동기화에 적합하지 않은 경우 클라이언트는 자동으로 풀의 다른 서버로 교체합니다.

- DHCP를 통해 NTP 서버 목록 배포

  NetworkManager가 DHCP 서버에서 NTP 서버 목록을 가져오면 chronyd 가 자동으로 이를 사용하도록 구성됩니다. 이 기능은 PEERNTP=no 를 /etc/sysconfig/network 파일에 추가하여 비활성화할 수 있습니다.

- PTP(Precision Time Protocol)사용

이 옵션은 주로 서버가 자주 변경되거나, 지정된 서버를 사용하지 않고도 대규모 클라이언트 그룹을 서로 동기화해야 하는 환경에 적합합니다.

PTP 는 멀티캐스트 메시징용으로 설계되었으며 NTP 브로드캐스트 모드와 유사하게 작동합니다. PTP 구현은 linuxptp 패키지에서 사용할 수 있습니다.

PTP 는 일반적으로 네트워크 스위치의 하드웨어 타임스탬프 및 지원이 제대로 작동해야 합니다. 그러나 PTP 는 소프트웨어 타임스탬프를 사용하고 네트워크 스위치에서 지원하지 않는 브로드캐스트 모드에서 NTP 보다 더 잘 작동할 것으로 예상됩니다.

하나의 통신 경로에 매우 많은 PTP 클라이언트가 있는 네트워크에서는 클라이언트에서 생성하는 네트워크 트래픽 양을 줄이기 위해 hybrid_e2e 옵션으로 PTP 클라이언트를 구성하는 것이 좋습니다. chronyd 를 NTP 클라이언트로 실행하는 컴퓨터를 구성하고 NTP 서버를 구성하여 멀티 캐스트 메시징을 사용하여 동기화된 시간을 다수의 컴퓨터에 분산하도록 기본 PTP timeserver로 작동할 수 있습니다.

15.5. CHRONY의 NTP(NETWORK TIME SECURITY) 개요

NTP(Network Time Security)는 주요 클라이언트를 확장하도록 설계된 NTP(Network Time Protocol) 인증 메커니즘입니다. 클라이언트 시스템으로 이동하는 동안 서버 시스템에서 수신한 패킷이 변경되지 않은지 확인합니다. NTP(Network Time Security)에는 서버와 클라이언트 간에 사용되는 암호화 키를 자동으로 생성하는 NTPS-KE(Key Establishment) 프로토콜이 포함되어 있습니다.

주의

NTS는 FIPS 및 OSPP 프로필과 호환되지 않습니다. FIPS 및 OSPP 프로필을 활성화하면 NTS로 구성된 chronyd 가 치명적 메시지와 함께 중단될 수 있습니다. GNUTLS_FORCE_FIPS_MODE=0 을 /etc/sysconfig/chronyd 파일에 추가하여 chronyd 서비스의 OSPP 프로필 및 FIPS 모드를 비활성화할 수 있습니다.

15.5.1. 클라이언트 구성 파일에서 NTP(네트워크 시간 보안) 활성화

기본적으로 NTP(Network Time Security)는 활성화되어 있지 않습니다. /etc/chrony.conf 에서 NTS 를 활성화할 수 있습니다. 이를 위해 다음 단계를 수행하십시오.

사전 요구 사항

- **NTS**를 지원하는 서버

절차

클라이언트 구성 파일에서 다음을 수행합니다.

1. 권장되는 **iburst** 옵션 외에 **the nts** 옵션을 사용하여 **server**를 지정합니다.

```
For example:
server time.example.com iburst nts
server nts.netnod.se iburst nts
server ptbtime1.ptb.de iburst nts
```

2. 시스템 부팅 중에 **NTPS-KE(Network Time Security-Key Establishment)** 세션을 반복하지 않으려면 **chrony.conf** 에 다음 행을 추가합니다.

```
ntsdumpdir /var/lib/chrony
```

3. **/etc/sysconfig/network** 에 다음 행을 추가하여 **DHCP** 에서 제공하는 **NTP(Network Time Protocol)** 서버와의 동기화를 비활성화합니다.

```
PEERNTP=no
```

4. 변경 사항을 저장하십시오.

5. **chronyd** 서비스를 다시 시작하십시오.

```
systemctl restart chronyd
```

검증

- **NTS** 키가 성공적으로 설정되었는지 확인합니다.

```
# chronyc -N authdata

Name/IP address  Mode KeyID Type KLen Last Atmp  NAK Cook CLen
==================================================================
```

```
time.example.com NTS    1  15 256 33m   0   0   8 100
nts.sth1.ntp.se  NTS    1  15 256 33m   0   0   8 100
nts.sth2.ntp.se  NTS    1  15 256 33m   0   0   8 100
```

KeyID,Type 및 KLen 의 값은 0이 아니어야 합니다. 값이 0이면 chronyd 에서 시스템 로그에 오류 메시지가 있는지 확인합니다.

- 클라이언트가 NTP를 측정하고 있는지 확인합니다.

```
# chronyc -N sources

MS Name/IP address Stratum Poll Reach LastRx Last sample
===============================================================
time.example.com  3      6  377   45  +355us[ +375us] +/-  11ms
nts.sth1.ntp.se   1      6  377   44  +237us[ +237us] +/-  23ms
nts.sth2.ntp.se   1      6  377   44  -170us[ -170us] +/-  22ms
```

Reach 열에는 0이 아닌 값이 있어야 합니다. 이상적인 377. 값이 377를 거의 얻지 못하거나 377를 얻지 못하면 네트워크에서 NTP 요청 또는 응답이 손실되고 있음을 나타냅니다.

추가 리소스

- 시스템의 chrony.conf(5) 도움말 페이지

15.5.2. 서버에서 NTP(Network Time Security) 활성화

자체 NTP(Network Time Protocol) 서버를 실행하는 경우 서버 NTP(Network Time Security) 지원을 활성화하여 클라이언트가 안전하게 동기화하도록 할 수 있습니다.

NTP 서버가 다른 서버, 즉 Stratum 1 서버인 경우 동기화에 NTS 또는 대칭 키를 사용해야 합니다.

사전 요구 사항

- PEM 형식의 서버 개인 키
- 필요한 중간 인증서가 있는 서버 인증서 PEM 형식

절차

1. chrony.conf 에서 개인 키와 인증서 파일을 지정합니다. 예를 들면 다음과 같습니다.

```
ntsserverkey /etc/pki/tls/private/<ntp-server.example.net>.key
ntsservercert /etc/pki/tls/certs/<ntp-server.example.net>.crt
```

2. 그룹 소유권을 설정하여 chrony 시스템 사용자가 키와 인증서 파일을 모두 읽을 수 있는지 확인합니다. 예를 들면 다음과 같습니다.

```
# chown :chrony /etc/pki/tls//<ntp-server.example.net>.
```

3. the ntsdumpdir /var/lib/chrony 지시문이 chrony.conf 에 있는지 확인합니다.

4. chronyd 서비스를 다시 시작하십시오.

```
# systemctl restart chronyd
```

중요

서버에 방화벽이 있는 경우 NTP 및 Network Time Security-Key Establishment(NTS-KE)에 대한 UDP 123 및 TCP 4460 포트를 모두 허용해야 합니다.

검증

- 다음 명령을 사용하여 클라이언트 시스템에서 빠른 테스트를 수행합니다.

```
$ chronyd -Q -t 3 'server

ntp-server.example.net iburst nts maxsamples 1'
2021-09-15T13:45:26Z chronyd version 4.1 starting (+CMDMON +NTP +REFCLOCK
+RTC +PRIVDROP +SCFILTER +SIGND +ASYNCDNS +NTS +SECHASH +IPV6
+DEBUG)
2021-09-15T13:45:26Z Disabled control of system clock
2021-09-15T13:45:28Z System clock wrong by 0.002205 seconds (ignored)
2021-09-15T13:45:28Z chronyd exiting
```

시스템 클록 잘못된 메시지는 NTP 서버가 NTS-KE 연결을 수락하고 NTS 보호 NTP 메시지로 응답하고 있음을 나타냅니다.

- 서버에서 관찰된 NTS-KE 연결 및 인증된 NTP 패킷을 확인합니다.

```
# chronyc serverstats

NTP packets received       : 7
NTP packets dropped        : 0
Command packets received   : 22
Command packets dropped    : 0
Client log records dropped : 0
NTS-KE connections accepted: 1
NTS-KE connections dropped : 0
Authenticated NTP packets: 7
```

NTS-KE 연결 값이 0이 아닌 NTP 패킷 필드인 경우 클라이언트가 NTS-KE 포트에 연결하고 인증된 NTP 요청을 보낼 수 있었습니다.

16장. LANGPACK 사용

langpack 은 시스템에 설치된 모든 패키지에 대해 번역, 사전 및 로케일을 포함하는 추가 애드온 패키지를 설치하는 메타 패키지입니다.

Red Hat Enterprise Linux 8 시스템에서 langpacks 설치는 langpacks -<langcode > 언어 메타 패키지 및 RPM 약한 종속성 (Supplements 태그)을 기반으로 합니다.

선택한 언어에 대해 langpack 을 사용할 수 있는 사전 요구 사항은 두 가지가 있습니다. 이러한 사전 요구 사항이 충족되면 언어 메타 패키지에서 트랜잭션 세트에서 선택한 언어에 대한 langpack을 자동으로 가져옵니다.

사전 요구 사항

- 선택한 언어의 langpacks-<langcode> 언어 메타 패키지가 시스템에 설치되었습니다.

  Red Hat Enterprise Linux 8에서 langpacks 메타 패키지는 Anaconda 설치 프로그램을 사용하여 운영 체제의 초기 설치를 통해 자동으로 설치됩니다. 이러한 패키지는 Application Stream 리포지토리에서 사용할 수 있기 때문입니다.

  자세한 내용은 langpacks를 제공하는 언어 검사를 참조하십시오.

- locale 패키지를 검색하려는 기본 패키지가 시스템에 이미 설치되어 있습니다.

16.1. LANGPACK을 제공하는 언어 확인

이 절차에 따라 langpack을 제공하는 언어를 확인합니다.

절차

- 다음 명령을 실행합니다.

```
# yum list langpacks-*
```

16.2. RPM에 약한 종속성 기반 LANGPACK 작업

이 섹션에서는 **RPM** 약한 종속성 기반 **langpack**을 쿼리하여 언어 지원을 설치하거나 제거할 때 수행할 수 있는 여러 작업에 대해 설명합니다.

16.2.1. 이미 설치된 언어 지원 목록 나열

이미 설치된 언어 지원을 나열하려면 다음 절차를 사용하십시오.

절차

- 다음 명령을 실행합니다.

```
# yum list installed langpacks*
```

16.2.2. 언어 지원 가능 여부 확인

모든 언어에 대해 언어 지원을 사용할 수 있는지 확인하려면 다음 절차를 사용하십시오.

절차

- 다음 명령을 실행합니다.

```
# yum list available langpacks*
```

16.2.3. 언어에 설치된 패키지 나열

모든 언어로 설치된 패키지를 나열하려면 다음 절차를 따르십시오.

절차

- 다음 명령을 실행합니다.

```
# yum repoquery --whatsupplements langpacks-<locale_code>
```

16.2.4. 언어 지원 설치

언어 지원을 새로 추가하려면 다음 절차를 사용하십시오.

절차

- 다음 명령을 실행합니다.

```
# yum install langpacks-<locale_code>
```

16.2.5. 언어 지원 삭제

설치된 언어 지원을 모두 제거하려면 다음 절차를 사용하십시오.

절차

- 다음 명령을 실행합니다.

```
# yum remove langpacks-<locale_code>
```

16.3. GLIBC-LANGPACK-<LOCALE_CODE>를 사용하여 디스크 공간 절약

현재 모든 로케일이 **/usr/lib/locale/locale-archive** 파일에 저장되어 많은 디스크 공간이 필요합니다.

디스크 공간이 컨테이너 및 클라우드 이미지와 같은 중요한 문제이거나 몇 개의 로케일만 필요한 시스템에서는 **glibc locale langpack** 패키지(**glibc-langpack-<locale_code>**)를 사용할 수 있습니다.

로케일을 개별적으로 설치하고 패키지 설치 공간이 더 작은 경우에는 다음 절차를 사용하십시오.

절차

- 다음 명령을 실행합니다.

```
# yum install glibc-langpack-<locale_code>
```

Anaconda를 사용하여 운영 체제를 설치할 때 **glibc-langpack-<locale_code>** 는 설치 중 사용한 언어와 추가 언어로 선택한 언어에 대해 설치됩니다. 모든 로케일을 포함하는 **glibc-all-langpack** 은 기본적으로 설치되므로 일부 로케일이 중복됩니다. 하나 이상의 선택된 언어에 **glibc-langpack-<locale_code>** 를 설치한 경우 설치 후 **glibc-all-langpacks** 를 삭제하여 디스크 공간을 절약할 수 있습니다.

glibc- all -langpacks 대신 선택된 **glibc-langpack-<locale_code>** 패키지만 설치하면 런타임 성능에 영향을 미칩니다.

참고

디스크 공간이 문제가 아닌 경우 **glibc-all-langpacks** 패키지를 사용하여 설치된 모든 로케일을 유지합니다.

17장. 나중에 분석을 위해 충돌한 커널 덤프

시스템이 충돌한 이유를 분석하기 위해 **kdump** 서비스를 사용하여 나중에 분석을 위해 시스템 메모리의 내용을 저장할 수 있습니다. 이 섹션에서는 **kdump** 에 대한 간략한 소개와 **RHEL** 웹 콘솔을 사용하여 **kdump** 를 구성하거나 해당 **RHEL** 시스템 역할을 사용하는 방법에 대한 정보를 제공합니다.

17.1. KDUMP란 무엇입니까?

kdump 는 크래시 덤프 메커니즘을 제공하고 크래시 덤프 또는 **vmcore** 파일이라는 덤프 파일을 생성하는 서비스입니다. **vmcore** 파일에는 분석 및 문제 해결에 도움이 되는 시스템 메모리 내용이 있습니다. **kdump** 는 **kexec** 시스템 호출을 사용하여 두 번째 커널로 부팅하고 재부팅 없이 *캡처 커널* 을 캡처한 다음 충돌된 커널의 메모리 내용을 캡처하여 파일에 저장합니다. 두 번째 커널은 시스템 메모리의 예약된 부분에서 사용할 수 있습니다.

중요

커널 크래시 덤프는 시스템 오류가 발생한 경우 사용 가능한 유일한 정보일 수 있습니다. 따라서 운영 **kdump** 는 미션 크리티컬한 환경에서 중요합니다. **Red Hat**은 일반 커널 업데이트 주기에서 **kexec-tools** 를 정기적으로 업데이트하고 테스트하는 것이 좋습니다. 이는 새 커널 기능을 설치할 때 특히 중요합니다.

머신의 설치된 모든 커널 또는 지정된 커널에서만 **kdump** 를 활성화할 수 있습니다. 이 기능은 시스템에 사용된 커널이 여러 개 있을 때 유용하며 그 중 일부는 충돌할 수 있다는 우려가 없을 정도로 안정적입니다. **kdump** 를 설치하면 기본 **/etc/kdump.conf** 파일이 생성됩니다. **/etc/kdump.conf** 파일에는 **kdump** 설정을 사용자 지정하도록 편집할 수 있는 기본 최소 **kdump** 구성이 포함되어 있습니다.

17.2. 웹 콘솔에서 KDUMP 메모리 사용량 및 대상 위치 구성

kdump 커널에 대한 메모리 예약을 구성하고 **RHEL** 웹 콘솔 인터페이스를 사용하여 **vmcore** 덤프 파일을 캡처할 대상 위치도 지정할 수 있습니다.

사전 요구 사항

- 웹 콘솔을 설치하고 액세스할 수 있어야 합니다.

  자세한 내용은 웹 콘솔 설치를 참조하십시오.

절차

1.

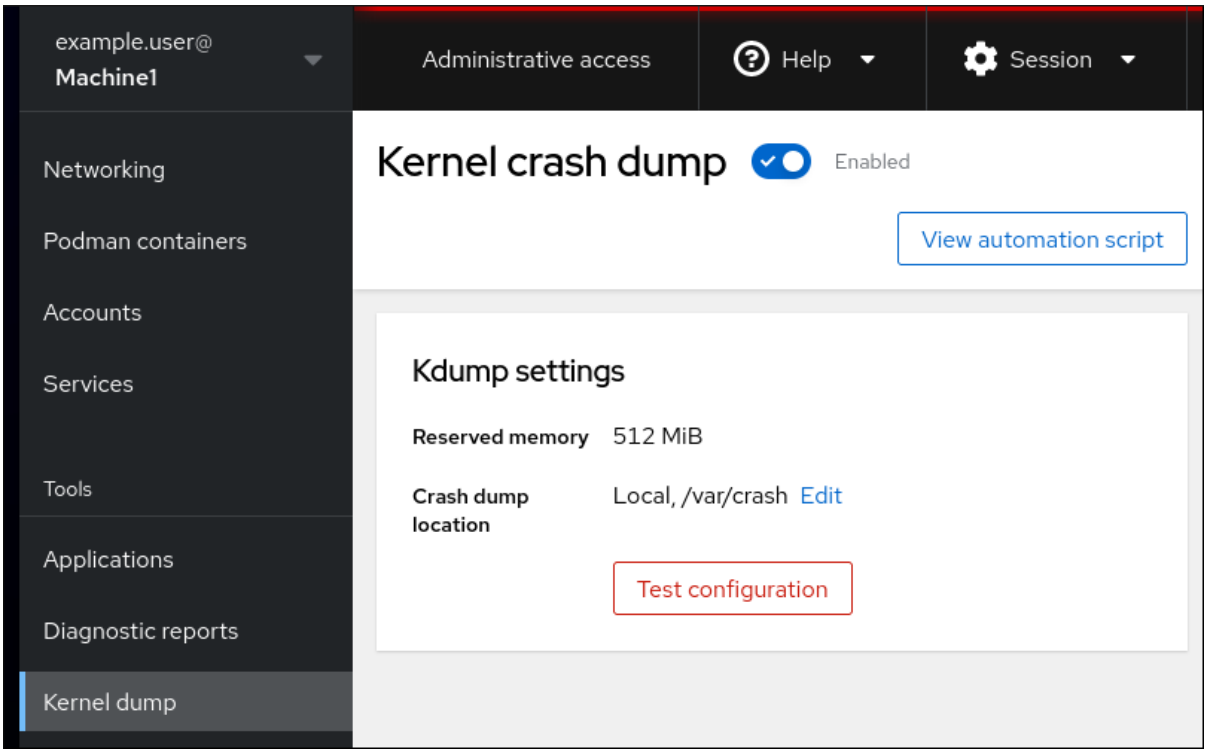
웹 콘솔에서 커널 덤프 탭을 열고 **Kernel crash dump** 스위치를 **on**으로 설정하여 **kdump** 서비스를 시작합니다.

2. 터미널에서 **kdump** 메모리 사용량을 구성합니다. 예를 들면 다음과 같습니다.

```
$ sudo grubby --update-kernel ALL --args crashkernel=512M
```

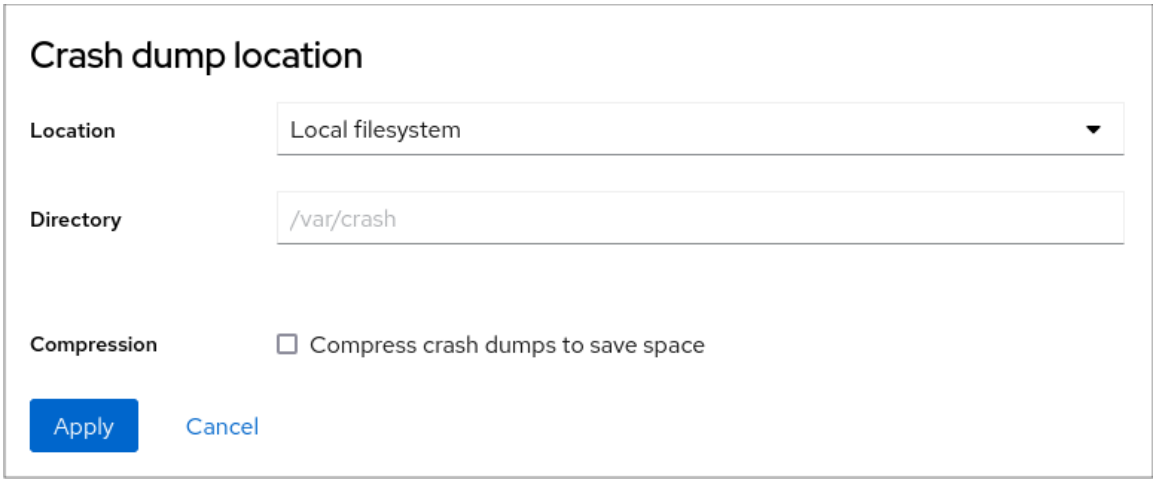
시스템을 다시 시작하여 변경 사항을 적용합니다.

3. 커널 덤프 탭에서 **Crash** 덤프 위치 필드의 끝에 있는 편집 을 클릭합니다.



4. **vmcore** 덤프 파일을 저장할 대상 디렉터리를 지정합니다.

- 로컬 파일 시스템의 드롭다운 메뉴에서 로컬 파일 시스템을 선택합니다.



- **SSH** 프로토콜을 사용하여 원격 시스템의 경우 드롭다운 메뉴에서 **SSH**를 통한 **Remote**를 선택하고 다음 필드를 지정합니다.
  - 서버 필드에 원격 서버 주소를 입력합니다.
  - **SSH** 키 필드에 **SSH** 키 위치를 입력합니다.
  - 디렉터리 필드에 대상 디렉터리를 입력합니다.
- **NFS** 프로토콜을 사용하여 원격 시스템의 경우 드롭다운 메뉴에서 **Remote over NFS**를 선택하고 다음 필드를 지정합니다.
  - 서버 필드에 원격 서버 주소를 입력합니다.
  - 내보내기 필드에 **NFS** 서버의 공유 폴더 위치를 입력합니다.
  - 디렉터리 필드에 대상 디렉터리를 입력합니다.

참고

Compression 확인란을 선택하여 **vmcore** 파일의 크기를 줄일 수 있습니다.

–

5. 선택 사항: 자동화 스크립트 보기를 클릭하여 자동화 스크립트를 표시합니다.

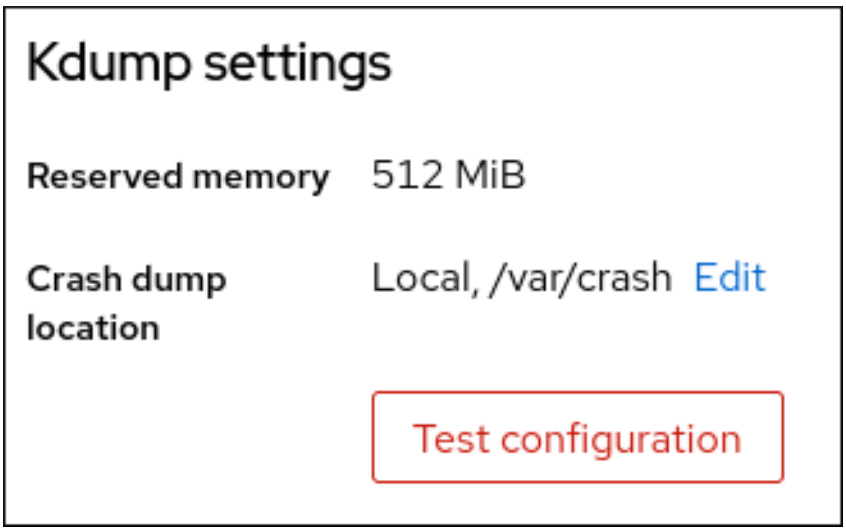
   생성된 스크립트가 있는 창이 열립니다. 쉘 스크립트와 **Ansible** 플레이북 생성 옵션 탭 사이를 탐색할 수 있습니다.

6. 선택 사항: **Copy to** 클립보드를 클릭하여 스크립트를 복사합니다.

   이 스크립트를 사용하여 여러 시스템에 동일한 구성을 적용할 수 있습니다.

검증

1. 설정 테스트를 클릭합니다.



2. **kdump** 설정 테스트에서 **Crash** 시스템을 클릭합니다.

> 주의
>
> 시스템 충돌을 시작하면 커널 작업이 중지되고 데이터 손실로 인해 시스템이 중단됩니다.

추가 리소스

- 지원되는 kdump 대상

17.3. RHEL 시스템 역할을 사용하는 KDUMP

RHEL 시스템 역할은 여러 RHEL 시스템을 원격으로 관리하는 일관된 구성 인터페이스를 제공하는 Ansible 역할 및 모듈의 컬렉션입니다. kdump 역할을 사용하면 여러 시스템에서 기본 커널 덤프 매개 변수를 설정할 수 있습니다.

> **주의**
>
> kdump 역할은 /etc/kdump.conf 파일을 대체하여 관리 호스트의 kdump 구성을 완전히 대체합니다. 또한 kdump 역할이 적용된 경우 /etc/sysconfig/kdump 파일을 대체하여 역할 변수로 지정하지 않아도 이전 kdump 설정도 교체됩니다.

다음 예제 플레이북은 kdump 시스템 역할을 적용하여 크래시 덤프 파일의 위치를 설정하는 방법을 보여줍니다.

```
---
- hosts: kdump-test
  vars:
    kdump_path: /var/crash
  roles:
    - rhel-system-roles.kdump
```

kdump 역할 변수에 대한 자세한 참조는 rhel-system-roles 패키지를 설치하고 /usr/share/doc/rhel-system-roles/kdump 디렉터리의 README.md 또는 README.html 파일을 참조하십시오.

추가 리소스

- RHEL 시스템 역할 소개

17.4. 추가 리소스

- kdump 설치

- 명령줄에서 kdump 구성
- 웹 콘솔에서 kdump 구성

18장. 시스템 복구 및 복원

기존 백업을 사용하여 시스템을 복구하고 복원하기 위해 **Red Hat Enterprise Linux**는 **Relax-and-recovery(ReaR)** 유틸리티를 제공합니다.

유틸리티를 재해 복구 솔루션 및 시스템 마이그레이션에도 사용할 수 있습니다.

유틸리티를 사용하면 다음 작업을 수행할 수 있습니다.

- 이미지를 사용하여 부팅 가능한 이미지를 생성하고 기존 백업에서 시스템을 복원합니다.
- 원래 스토리지 레이아웃을 복제합니다.
- 사용자 및 시스템 파일을 복원합니다.
- 시스템을 다른 하드웨어로 복원합니다.

또한 재해 복구의 경우 특정 백업 소프트웨어를 **ReaR**과 통합할 수도 있습니다.

ReaR 설정에는 다음과 같은 고급 단계가 포함됩니다.

1. **ReaR** 설치.
2. **ReaR** 구성 파일을 수정하여 백업 방법 세부 정보를 추가합니다.
3. 복구 시스템 생성.
4. 백업 파일을 생성합니다.

18.1. REAR 설정

다음 단계에 따라 **Relax-and-Recover(ReaR)** 유틸리티를 사용하는 패키지를 설치하고 복구 시스템을 생성하고 백업을 구성 및 생성합니다.

사전 요구 사항

- 백업 복원 계획에 따라 필요한 구성이 준비됩니다.

  ReaR과 함께 완전히 통합된 기본 제공 방법인 **NETFS** 백업 방법을 사용할 수 있습니다.

절차

1. 다음 명령을 실행하여 **ReaR** 유틸리티를 설치합니다.

```
# yum install rear
```

2. 선택한 편집기에서 **ReaR** 구성 파일을 수정합니다. 예를 들면 다음과 같습니다.

```
# vi /etc/rear/local.conf
```

3. **/etc/rear/local.conf** 에 백업 설정 세부 정보를 추가합니다. 예를 들어 **NETFS** 백업 방법의 경우 다음 행을 추가합니다.

```
BACKUP=NETFS
BACKUP_URL=backup.location
```

   backup.location 을 백업 위치의 **URL**로 바꿉니다.

4. 새 파일이 생성될 때 이전 백업 아카이브를 유지하도록 **ReaR**을 구성하려면 구성 파일에 다음 행도 추가합니다.

```
NETFS_KEEP_OLD_BACKUP_COPY=y
```

5. 백업을 점진적으로 만들려면 각 실행에서 변경된 파일만 백업하려면 다음 행을 추가합니다.

```
BACKUP_TYPE=incremental
```

6. 복구 시스템을 생성합니다.

```
# rear mkrescue
```

7. 복원 계획에 따라 백업을 수행합니다. 예를 들어 NETFS 백업 방법의 경우 다음 명령을 실행합니다.

```
# rear mkbackuponly
```

또는 다음 명령을 실행하여 단일 단계에서 복구 시스템 및 백업을 생성할 수 있습니다.

```
# rear mkbackup
```

이 명령은 rear mkrescue 및 rear mk backuponly 명령의 기능을 결합합니다.

18.2. REAR 예약

rear 패키지의 /etc/cron.d/ rear crontab 파일은 매일 오전 1:30에 rear mkrescue 명령을 자동으로 실행하여 복구 시스템을 정기적으로 생성하기 위한 Relax-and-Recover(ReaR) 유틸리티를 예약합니다. 명령은 데이터 백업이 아닌 복구 시스템만 생성합니다. 데이터의 주기적인 백업을 직접 예약해야 합니다. 예를 들면 다음과 같습니다.

절차

- rear mkbackuponly 명령을 예약하는 다른 crontab을 추가할 수 있습니다.
- 기존 crontab을 변경하여 기본 /usr/sbin/rear checklayout || /usr/sbin/rear mkrescure 명령 대신 rear mkbackup 명령을 실행할 수도 있습니다.
- 외부 백업 방법이 사용 중인 경우 외부 백업을 예약할 수 있습니다. 자세한 내용은 ReaR에서 사용하는 백업 방법에 따라 다릅니다.

참고

rear 패키지에 제공된 /etc/cron.d/ rear crontab 파일은 더 이상 사용되지 않는 기능 쉘 및 명령줄 을 참조하십시오. 기본적으로 백업을 수행하는 데 충분하지 않기 때문입니다.

18.3. 64비트 IBM Z 아키텍처에서 REAR RESCUE 이미지 사용

기본 Relax 및 Recover(ReaR) 기능은 이제 64비트 IBM Z 아키텍처에서 사용할 수 있으며 RHEL 8.8 이후 완전히 지원됩니다. IBM Z에서 z/VM 환경에서만 복구 이미지를 생성할 수 있습니다. LPAR(Logical partitions) 백업 및 복구는 테스트되지 않았습니다.

중요

64비트 IBM Z 아키텍처에서는 리어 패키지 버전 2.6-9.el8 이상에서만 지원됩니다. 이전 버전은 기술 프리뷰 기능으로만 사용할 수 있습니다. Red Hat 기술 프리뷰 기능의 지원 범위에 대한 자세한 내용은 https://access.redhat.com/support/offerings/techpreview 을 참조하십시오.

현재 사용 가능한 출력 방법은 초기 프로그램 로드(IPL)입니다. IPL은 zIPL 부트 로더와 함께 사용할 수 있는 커널 및 초기 RAM 디스크(initrd)를 생성합니다.

사전 요구 사항

- Rear가 설치되어 있습니다.
  - ReaR을 설치하려면 yum install rear 명령을 실행합니다.

절차

다음 변수를 /etc/rear/local.conf 에 추가하여 64비트 IBM Z 아키텍처에서 복구 이미지를 생성하도록 ReaR을 구성합니다.

1. IPL 출력 방법을 구성하려면 OUTPUT=IPL 을 추가합니다.

2. 백업 방법 및 대상을 구성하려면 RuntimeClass 및 octets _URL 변수를 추가합니다. 예를 들면 다음과 같습니다.

```
BACKUP=NETFS

BACKUP_URL=nfs://<nfsserver name>/<share path>
```

중요

로컬 백업 스토리지는 현재 64비트 IBM Z 아키텍처에서 지원되지 않습니다.

3. 선택 사항: 커널 및 initrd 파일을 저장하도록 OUTPUT_URL 변수를 구성할 수도 있습니다. 기본적으로 OUTPUT_URL 은 etcdctl_URL에 맞게 조정됩니다.

4. 백업 및 복구 이미지 생성을 수행하려면 다음을 수행합니다.

```
# rear mkbackup
```

5. 이렇게 하면 EgressIP _URL 또는 OUTPUT_URL (설정된 경우) 변수에서 지정한 위치에 커널 및 initrd 파일이 생성되고 지정된 백업 방법을 사용하여 백업을 생성합니다.

6. 시스템을 복구하려면 3단계에서 만든 ReaR 커널 및 initrd 파일을 사용하고, zipl 부트 로더, 커널, initrd를 사용하여 준비한 zipl boot loader, kernel 및 initrd를 사용하여 준비한 FCP(Direct Attached Storage Device) 또는 FCP(Fibre Channel Protocol)에서 부팅하십시오. 자세한 내용은 준비 DASD 사용을 참조하십시오.

7. rescue 커널 및 initrd 가 부팅되면 ReaR rescue 환경을 시작합니다. 시스템 복구를 진행합니다.

주의

현재 복구 프로세스는 시스템에 연결된 모든 DASD(Direct Attached Storage 장치)를 다시 포맷합니다. 시스템 스토리지 장치에 중요한 데이터가 있는 경우 시스템 복구를 시도하지 마십시오. 여기에는 복구 환경에서 부팅하는 데 사용된 zipl 부트 로더, ReaR 커널 및 initrd로 준비된 장치도 포함됩니다. 복사본을 유지해야 합니다.

추가 리소스

- z/VM에서 설치

- 준비 DASD 사용.

19장. 동적 프로그래밍 언어 설치 및 사용

Red Hat은 Python, PHP, Tcl/TK와 같은 다양한 프로그래밍 언어를 제공합니다. 이를 사용하여 자체 애플리케이션 및 서비스를 개발할 수 있습니다.

19.1. PYTHON 소개

Python은 개체 지향, 명령적, 기능적, 절차적 패러다임과 같은 여러 프로그래밍 패러다임을 지원하는 고급 프로그래밍 언어입니다. Python에는 동적 의미 체계가 있으며 범용 프로그래밍에 사용할 수 있습니다.

Red Hat Enterprise Linux를 사용하면 시스템 툴, 데이터 분석 툴 또는 웹 애플리케이션을 제공하는 패키지와 같이 시스템에 설치된 많은 패키지가 Python으로 작성됩니다. 이러한 패키지를 사용하려면 python* 패키지가 설치되어 있어야 합니다.

19.1.1. Python 버전

호환되지 않는 Python 버전은 널리 사용되는 Python 2.x 및 Python 3.x입니다. RHEL 8에서는 다음 버전의 Python을 제공합니다.

표 19.1. RHEL 8의 Python 버전

버전	설치할 패키지	명령 예	이후 사용 가능	라이프 사이클
Python 3.6	**python3**, **python36**	**python3**, **python3.6**, **pip3**, **pip3.6**	RHEL 8.0	전체 RHEL 8
Python 2.7	**python2**	**python2**, **pip2**	RHEL 8.0	짧음
Python 3.8	**python38**	**python3.8**, **pip3.8**	RHEL 8.2	짧음
Python 3.9	**python39**	**python3.9**, **pip3.9**	RHEL 8.4	짧음
Python 3.11	**python3.11**	**python3.11**, **pip3.11**	RHEL 8.8	짧음
Python 3.12	**python3.12**	**python3.12**, **pip3.12**	RHEL 8.10	짧음

지원 기간에 대한 자세한 내용은 Red Hat Enterprise Linux 라이프 사이클 및 Red Hat Enterprise Linux Application Streams 라이프 사이클을 참조하십시오.

3.9까지의 각 Python 버전은 별도의 모듈에 배포됩니다. Python 3.11 및 Python 3.12는 python3.11 및 python3.12 패키지를 포함하여 수정되지 않은 RPM 패키지 제품군으로 배포됩니다.

동일한 RHEL 8 시스템에 여러 Python 버전을 병렬로 설치할 수 있습니다.

중요

Python 버전을 설치하거나, 호출하거나, 다른 방식으로 상호 작용할 때 항상 Python 버전을 지정합니다. 예를 들어 패키지 및 명령 이름에 python 대신 python 3 을 사용합니다. 모든 Python 관련 명령에도 버전이 포함되어야 합니다(예: pip3,pip2,pip3.8,pip3.9,pip3.11 또는 pip3.12).

버전이 없는 python 명령(/usr/bin/python)은 RHEL 8에서 기본적으로 사용할 수 없습니다. alternatives 명령을 사용하여 구성할 수 있습니다. 자세한 내용은 버전이 없는 Python 구성을참조하십시오.

alternatives 명령을 사용하여 변경한 사항을 제외하고 /usr/bin/python 에 대한 수동 변경 사항은 업데이트 시 덮어쓸 수 있습니다.

시스템 관리자는 다음과 같은 이유로 Python 3을 사용합니다.

- Python 3은 Python 프로젝트의 주요 개발 방향을 나타냅니다.
- 업스트림 커뮤니티의 Python 2 지원은 2020년에 종료되었습니다.
- 인기 있는 Python 라이브러리는 업스트림에서 Python 2 지원을 중단하고 있습니다.
- Red Hat Enterprise Linux 8의 Python 2는 라이프사이클이 짧고 고객을 위해 Python 3 으로 원활하게 전환하는 것을 목표로 합니다.

개발자의 경우 Python 3은 Python 2에 비해 다음과 같은 이점이 있습니다.

- Python 3를 사용하면 표현, 유지 관리 및 올바른 코드를 보다 쉽게 작성할 수 있습니다.

- Python 3으로 작성된 코드는 더 길게 됩니다.

- Python 3에는 asyncio, f-strings, 고급 압축 풀기, 키워드 전용 인수, 연결된 예외 등의 새로운 기능이 있습니다.

그러나 레거시 소프트웨어를 사용하려면 /usr/bin/python 을 Python 2로 구성해야 할 수 있습니다. 이러한 이유로 Red Hat Enterprise Linux 8과 함께 기본 python 패키지가 배포되지 않으며 버전되지 않은 Python 구성에 설명된 대로 Python 2와 3을 /usr/bin/python 으로 사용할 수 있습니다.

중요

Red Hat Enterprise Linux 8의 시스템 툴은 고객이 직접 사용할 수 없는 내부 platform-python 패키지에서 제공하는 Python 버전 3.6을 사용합니다. Python 3.6용 python36 패키지에서 python3 또는 python3.6 명령을 사용하거나 이후 Python 버전을 사용하는 것이 좋습니다.

다른 패키지에 필요하므로 RHEL 8에서 platform-python 패키지를 제거하지 마십시오.

19.1.2. Python 버전 간의 주요 차이점

RHEL 8에 포함된 Python 버전은 다양한 측면에서 다릅니다.

Python 바인딩

python38 및 python 3.11 및 python3.12 패키지 모음에는 python36 모듈에 제공되는 시스템 툴(RPM, DNF, SELinux 등)에 대한 동일한 바인딩이 포함되지 않습니다. 따라서 기본 운영 체제 또는 바이너리 호환성이 가장 필요한 인스턴스에서 python36 을 사용합니다. 시스템 바인딩이 다양한 Python 모듈의 최신 버전과 함께 필요한 고유 인스턴스의 경우 Python의 venv 또는 virtualenv 환경에 pip 를 통해 설치된 타사 업스트림 Python 모듈과 함께 python36 모듈을 사용합니다.

Python 3.11 및 Python 3.12 가상 환경은 virtualenv대신 venv 를 사용하여 생성해야 합니다.

python3- virtualenv 패키지에서 제공하는 RHEL 8의 virtualenv 유틸리티는 Python 3.11 및 Python 3.12와 호환되지 않습니다. virtualenv 를 사용하여 가상 환경을 생성하려고 하면 오류 메시지와 함께 실패합니다. 예를 들면 다음과 같습니다.

```
$ virtualenv -p python3.11 venv3.11
Running virtualenv with interpreter /usr/bin/python3.11
ERROR: Virtual environments created by virtualenv < 20 are not compatible with Python 3.11.
```

```
ERROR: Use python3.11 -m venv instead.
```

Python 3.11 또는 Python 3.12 가상 환경을 생성하려면 표준 라이브러리에서 venv 모듈을 사용하는 python3.11 -m venv 또는 python3.12 -m venv 명령을 사용합니다.

19.2. PYTHON 설치 및 사용

Red Hat Enterprise Linux 8에서 Python 3은 python36,python38 및 python39 모듈에서 제공하며 AppStream 리포지토리의 python3.11 및 python3.12 패키지 제품군에서 제공하는 3.6, 3.8 및 3.9 버전으로 배포됩니다.

> **주의**
>
> 버전이 없는 python 명령을 사용하여 Python을 설치하거나 실행하는 작업은 모호성으로 인해 기본적으로 작동하지 않습니다. 항상 Python 버전을 지정하거나 alternatives 명령을 사용하여 시스템 기본 버전을 구성합니다.

19.2.1. Python 3 설치

python27,python36,python38, python39 모듈, python3.11 및 python3.12 패키지 제품군을 포함하여 RHEL 8 모듈을 병렬로 설치할 수 있습니다.

mod_wsgi 모듈을 제외하고 Python 3.8, Python 3.9, Python 3.11 및 Python 3.12를 각 버전에 대해 빌드하고 Python 3.6과 병렬로 설치할 수 있습니다. Apache HTTP 서버의 제한으로 인해 python3-mod_wsgi,python38-mod_wsgi,python39-mod_wsgi,python3.11-mod_wsgi 또는 python3.12-mod_wsgi 패키지 중 하나만 시스템에 설치할 수 있습니다.

절차

- python36 모듈에서 Python 3.6을 설치하려면 다음을 사용합니다.

```
# yum install python3
```

python36:3.6 모듈 스트림은 자동으로 활성화됩니다.

- python38 모듈에서 Python 3.8을 설치하려면 다음을 사용합니다.

```
# yum install python38
```

python38:3.8 모듈 스트림은 자동으로 활성화됩니다.

- python39 모듈에서 Python 3.9를 설치하려면 다음을 사용합니다.

```
# yum install python39
```

python39:3.9 모듈 스트림은 자동으로 활성화됩니다.

- python3.11 RPM 패키지에서 Python 3.11을 설치하려면 다음을 사용합니다.

```
# yum install python3.11
```

- python3.12 RPM 패키지에서 Python 3.12를 설치하려면 다음을 사용합니다.

```
# yum install python3.12
```

검증

- 시스템에 설치된 Python 버전을 확인하려면 필수 Python 버전에 고유한 python 명령과 함께 --version 옵션을 사용합니다.
  - Python 3.6의 경우:

```
$ python3 --version
```

  - Python 3.8의 경우:

```
$ python3.8 --version
```

  - Python 3.9의 경우:

```
$ python3.9 --version
```

- Python 3.11의 경우:

```
$ python3.11 --version
```

- Python 3.12의 경우

```
$ python3.12 --version
```

추가 리소스

- 사용자 공간 구성 요소 설치, 관리 및 제거

19.2.2. 추가 Python 3 패키지 설치

Python 3.6용 애드온 모듈이 있는 패키지는 일반적으로 python3- 접두사를 사용하고, Python 3.8용 패키지에는 python38- 접두사, python39- 접두사가 포함되어 있으며 Python 3.11용 패키지에는 python3.11- 접두사가 포함되어 있으며 Python 3.12용 패키지에는 python3.12- 접두사가 포함되어 있습니다. 아래 예제와 같이 추가 Python 패키지를 설치할 때 접두사를 항상 포함합니다.

절차

- Python 3.6용 Requests 모듈을 설치하려면 다음을 사용합니다.

```
# yum install python3-requests
```

- Cython 확장을 Python 3.8로 설치하려면 다음을 사용합니다.

```
# yum install python38-Cython
```

- Python 3.9에서 pip 패키지 설치 프로그램을 설치하려면 다음을 사용합니다.

```
# yum install python39-pip
```

- Python 3.11에서 pip 패키지 설치 프로그램을 설치하려면 다음을 사용하십시오.

-

```
# yum install python3.11-pip
```

- Python 3.12에서 pip 패키지 설치 프로그램을 설치하려면 다음을 사용합니다.

```
# yum install python3.12-pip
```

추가 리소스

- Python 애드온 모듈에 대한 업스트림 문서

19.2.3. 개발자를 위한 추가 Python 3 툴 설치

개발자를 위한 추가 Python 툴은 해당 python38-devel 또는 python39-devel 모듈의 CodeReady Linux Builder(CRB) 리포지토리 또는 python3.11-* 또는 python3.12-* 패키지를 통해 배포됩니다.

python3-pytest 패키지(Python 3.6용) 및 해당 종속 항목은 AppStream 리포지토리에서 사용할 수 있습니다.

CRB 리포지터리는 다음을 제공합니다.

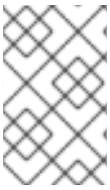
- python38-pytest 패키지 및 해당 종속 항목을 포함하는 python38-devel 모듈입니다.
- python39-pytest 패키지 및 해당 종속성과 python39-debug 및 python39-Cython 패키지가 포함된 python39-devel 모듈입니다.
- 다음을 포함하는 python3.11-* 패키지
  - python3.11-pytest 및 해당 종속 항목
  - python3.11-idle
  - python3.11-debug

- python3.11-Cython

- python3.12-* 패키지는 python3.11-* 와 유사한 패키지 세트를 포함합니다.

중요

CodeReady Linux Builder 리포지토리의 콘텐츠는 Red Hat에서 지원하지 않습니다.



참고

RHEL에서 모든 업스트림 Python 관련 패키지를 사용할 수 있는 것은 아닙니다.

python3*-pytest 패키지를 설치하려면 다음 절차를 사용하십시오.

절차

1. Python 3.8 이상의 경우 CodeReady Linux Builder 리포지터리를 활성화합니다.

```
# subscription-manager repos --enable codeready-builder-for-rhel-8-x86_64-rpms
```

2. Python 3.8 또는 3.9의 경우 해당 python3*-devel 모듈을 활성화합니다. 예를 들면 다음과 같습니다.

```
# yum module enable python39-devel
```

3. python3*-pytest 패키지를 설치합니다.

- Python 3.6의 경우:

```
# yum install python3-pytest
```

- Python 3.8의 경우:

```
# yum install python38-pytest
```

- Python 3.9의 경우:

```
# yum install python39-pytest
```

- Python 3.11의 경우:

```
# yum install python3.11-pytest
```

- Python 3.12의 경우

```
# yum install python3.12-pytest
```

추가 리소스

- CodeReady Linux Builder 내에서 콘텐츠를 활성화하고 사용하는 방법
- 패키지 매니페스트

19.2.4. Python 2 설치

일부 애플리케이션 및 스크립트는 아직 Python 3으로 완전히 포팅되지 않았으며 Python 2를 실행해야 합니다. Red Hat Enterprise Linux 8은 Python 3 및 Python 2의 병렬 설치를 허용합니다. Python 2 기능이 필요한 경우 AppStream 리포지토리에서 사용할 수 있는 python27 모듈을 설치합니다.

> 주의
>
> Python 3은 Python 프로젝트의 주요 개발 방향입니다. Python 2에 대한 지원은 단계적으로 폐지되고 있습니다. python27 모듈은 Red Hat Enterprise Linux 8보다 지원 기간이 짧습니다.

절차

- python27 모듈에서 Python 2.7을 설치하려면 다음을 사용합니다.

```
# yum install python2
```

python27:2.7 모듈 스트림이 자동으로 활성화됩니다.

Python 2용 애드온 모듈이 있는 패키지는 일반적으로 python2- 접두사를 사용합니다. 아래 예제와 같이 추가 Python 패키지를 설치할 때 접두사를 항상 포함합니다.

- Python 2용 Requests 모듈을 설치하려면 다음을 사용합니다.

```
# yum install python2-requests
```

- Cython 확장을 Python 2로 설치하려면 다음을 사용합니다.

```
# yum install python2-Cython
```

검증

- 시스템에 설치된 Python 버전을 확인하려면 다음을 사용합니다.

```
$ python2 --version
```

참고

원래 python27, python36, python38 및 python 39 모듈을 포함하여 RHEL 8 모듈을 병렬로 설치할 수 있습니다.

추가 리소스

- RHEL 8에서 사용자 공간 구성 요소 설치, 관리 및 제거

19.2.5. Python 2에서 Python 3으로 마이그레이션

개발자는 Python 2로 작성된 이전 코드를 Python 3으로 마이그레이션할 수 있습니다.

대규모 코드 기반을 Python 3으로 마이그레이션하는 방법에 대한 자세한 내용은 The Conservative Python 3 Porting Guide 를 참조하십시오.

이 마이그레이션 후 원래 Python 2 코드는 Python 3 인터프리터에서 해석할 수 있게 되며 Python 2 인터프리터에서도 해석할 수 있습니다.

19.2.6. Python 사용

Python 인터프리터 또는 Python 관련 명령을 실행하는 경우 항상 버전을 지정합니다.

사전 요구 사항

- 필요한 Python 버전이 설치되어 있는지 확인합니다.
- Python 3.11 또는 Python 3.12용 타사 애플리케이션을 다운로드하여 설치하려면 python3.11-pip 또는 python3.12-pip 패키지를 설치합니다.

절차

- Python 3.6 인터프리터 또는 관련 명령을 실행하려면 다음을 사용합니다.

```
$ python3
$ python3 -m venv --help
$ python3 -m pip install package
$ pip3 install package
```

- Python 3.8 인터프리터 또는 관련 명령을 실행하려면 다음을 사용합니다.

```
$ python3.8
$ python3.8 -m venv --help
$ python3.8 -m pip install package
$ pip3.8 install package
```

- Python 3.9 인터프리터 또는 관련 명령을 실행하려면 다음을 사용합니다.

```
$ python3.9
$ python3.9 -m venv --help
$ python3.9 -m pip install package
```

```
$ pip3.9 install package
```

- Python 3.11 인터프리터 또는 관련 명령을 실행하려면 다음을 사용합니다. 예를 들면 다음과 같습니다.

```
$ python3.11
$ python3.11 -m venv --help
$ python3.11 -m pip install package
$ pip3.11 install package
```

- Python 3.12 인터프리터 또는 관련 명령을 실행하려면 다음을 사용합니다. 예를 들면 다음과 같습니다.

```
$ python3.12
$ python3.12 -m venv --help
$ python3.12 -m pip install package
$ pip3.12 install package
```

- Python 2 인터프리터 또는 관련 명령을 실행하려면 다음을 사용합니다.

```
$ python2
$ python2 -m pip install package
$ pip2 install package
```

19.3. 버전이 없는 PYTHON 구성

시스템 관리자는 alternatives 명령을 사용하여 /usr/bin/python 에 있는 버전이 없는 python 명령을 구성할 수 있습니다. 필수 패키지 python3,python38,python39,python3.11,python3.12 또는 python2) 은 각 버전에 버전되지 않은 명령을 구성하기 전에 설치해야 합니다.

중요

/usr/bin/python 실행 파일은 alternatives 시스템에 의해 제어됩니다. 업데이트 시 수동 변경 사항을 덮어쓸 수 있습니다.

추가 Python 관련 명령(예: pip3)에는 구성 가능한 버전이 없는 변형이 없습니다.

19.3.1. 버전이 지정되지 않은 python 명령 직접 구성

선택한 버전의 Python으로 직접 버전이 지정되지 않은 python 명령을 구성할 수 있습니다.

사전 요구 사항

- 필요한 Python 버전이 설치되어 있는지 확인합니다.

절차

- 버전이 없는 python 명령을 Python 3.6으로 구성하려면 다음을 사용합니다.

```
# alternatives --set python /usr/bin/python3
```

- 버전이 없는 python 명령을 Python 3.8로 구성하려면 다음을 사용합니다.

```
# alternatives --set python /usr/bin/python3.8
```

- 버전이 없는 python 명령을 Python 3.9로 구성하려면 다음을 사용합니다.

```
# alternatives --set python /usr/bin/python3.9
```

- 버전이 지정되지 않은 python 명령을 Python 3.11로 구성하려면 다음을 사용합니다.

```
# alternatives --set python /usr/bin/python3.11
```

- 버전이 지정되지 않은 python 명령을 Python 3.12로 구성하려면 다음을 사용합니다.

```
# alternatives --set python /usr/bin/python3.12
```

- 버전이 없는 python 명령을 Python 2로 구성하려면 다음을 사용합니다.

```
# alternatives --set python /usr/bin/python2
```

19.3.2. 필수 Python 버전으로 대화형으로 버전이 없는 python 명령 구성

필수 Python 버전에 대화식으로 버전이 없는 python 명령을 구성할 수 있습니다.

사전 요구 사항

- 필요한 Python 버전이 설치되어 있는지 확인합니다.

절차

1. 버전이 없는 python 명령을 대화형으로 구성하려면 다음을 사용합니다.

```
# alternatives --config python
```

2. 제공된 목록에서 필요한 버전을 선택합니다.

3. 이 구성을 재설정하고 버전이 없는 python 명령을 제거하려면 다음을 사용합니다.

```
# alternatives --auto python
```

19.3.3. 추가 리소스

- 시스템의 alternatives(8) 및 unversioned-python(1) 도움말 페이지

19.4. PYTHON 3 RPM 패키지

대부분의 Python 프로젝트에서는 패키징에 Setuptools를 사용하고 setup.py 파일에서 패키지 정보를 정의합니다. Setuptools 패키징에 대한 자세한 내용은 Setuptools 설명서 를 참조하십시오.

Setuptools 패키징과 비교하여 다음과 같은 이점을 제공하는 RPM 패키지로 Python 프로젝트를 패키징할 수도 있습니다.

- 다른 RPM에 패키지의 종속성 사양 (Python이 아닌 경우)

- 암호화 서명

  암호화 서명을 사용하면 RPM 패키지의 콘텐츠를 나머지 운영 체제와 함께 검증, 통합 및 테스트할 수 있습니다.

19.4.1. Python 패키지에 대한 사양 파일 설명

spec 파일에는 rpmbuild 유틸리티에서 RPM을 빌드하는 데 사용하는 지침이 포함되어 있습니다. 지침은 일련의 섹션에 포함되어 있습니다. 사양 파일에는 섹션이 정의된 두 가지 주요 부분이 있습니다.

- preamble (본문에서 사용되는 일련의 메타데이터 항목이 포함되어 있음)
- 본문 (명령의 주요 부분이 포함되어 있음)

Python 프로젝트용 RPM SPEC 파일은 Python이 아닌 RPM SPEC 파일과 비교하여 몇 가지 세부 사항이 있습니다. 특히 Python 라이브러리의 모든 RPM 패키지의 이름에는 항상 버전을 결정하는 접두사를 포함해야 합니다(예: Python 3.6, Python 3.8, python38 용 python38, Python 3.9용 python3.11, Python 3.11용 python3.11 또는 Python 3.12용 python3.12).

다른 세부 사항은 python3-detox 패키지에 대한 다음 사양 파일 예제에 표시됩니다. 이러한 세부 사항에 대한 설명은 아래 예제를 참조하십시오.

```
%global modname detox                                   (1)

Name:         python3-detox                             (2)
Version:      0.12
Release:      4%{?dist}
Summary:      Distributing activities of the tox tool
License:      MIT
URL:          https://pypi.io/project/detox
Source0:      https://pypi.io/packages/source/d/%{modname}/%{modname}-%{version}.tar.gz

BuildArch:    noarch

BuildRequires:  python36-devel                          (3)
BuildRequires:  python3-setuptools
BuildRequires:  python36-rpm-macros
BuildRequires:  python3-six
BuildRequires:  python3-tox
BuildRequires:  python3-py
BuildRequires:  python3-eventlet

%?python_enable_dependency_generator                    (4)

%description

Detox is the distributed version of the tox python testing tool. It makes efficient use of
multiple CPUs by running all possible activities in parallel.
Detox has the same options and configuration that tox has, so after installation you can run it
in the same way and with the same options that you use for tox.
```

```
  $ detox

%prep
%autosetup -n %{modname}-%{version}

%build
%py3_build                                  5

%install
%py3_install

%check
%{__python3} setup.py test                  6

%files -n python3-%{modname}
%doc CHANGELOG
%license LICENSE
%{_bindir}/detox
%{python3_sitelib}/%{modname}/
%{python3_sitelib}/%{modname}-%{version}*

%changelog
...
```

1. modname 매크로에는 Python 프로젝트 이름이 포함됩니다. 이 예에서는 detox 입니다.

2. Python 프로젝트를 RPM으로 패키징하는 경우 python3 접두사는 항상 프로젝트의 원래 이름에 추가해야 합니다. 여기에서 원래 이름은 detox 이고 RPM의 이름은 python3-detox 입니다.

3. BuildRequires 는 이 패키지를 빌드하고 테스트하는 데 필요한 패키지를 지정합니다. BuildRequires에서 Python 패키지를 빌드하는 데 필요한 도구를 제공하는 항목을 항상 포함합니다 (python36-devel 및 python3-setuptools). /usr/bin/ python3 인터프리터 지시문이 있는 파일이 /usr/ bin/python 3.6 으로 자동으로 변경되도록 python36-rpm-macros 패키지가 필요합니다.

4. 모든 Python 패키지가 올바르게 작동하려면 다른 패키지가 필요합니다. 이러한 패키지는 사양 파일에도 지정해야 합니다. 종속 항목을 지정하려면 %python_enable_dependency_generator 매크로를 사용하여 setup.py 파일에 정의된 종속성을 자동으로 사용할 수 있습니다. 패키지에 Setuptools를 사용하여 지정되지 않은 종속성이 있는 경우 추가 Requires 지시문 내에 지정합니다.

5. %py3_build 및 %py3_install 매크로는 설치 위치, 사용할 인터프리터 및 기타 세부 정보를 지정하는 추가 인수와 함께 setup.py build 및 setup.py install 명령을 각각 실행합니다.

6

확인 섹션에서는 올바른 버전의 **Python**을 실행하는 매크로를 제공합니다. **%{__python3}** 매크로에는 **Python 3** 인터프리터의 경로(예: **/usr/bin/python3**)가 포함되어 있습니다. 리터럴 경로가 아닌 매크로를 항상 사용하는 것이 좋습니다.

19.4.2. Python 3 RPM의 공통 매크로

사양 파일에서 항상 값을 하드 코딩하지 않고 다음 **Macros for Python 3 RPMs** 테이블에 설명된 매크로를 사용합니다.

매크로 이름에서 버전이 없는 **python** 대신 항상 **python3** 또는 **python2** 를 사용합니다 . **SPEC** 파일의 **BuildRequires** 섹션에 있는 특정 **Python 3** 버전을 **python36-rpm-macros,python38-rpm-macros,python39-rpm-macros,python3.11-rpm-macros** 또는 **python3.12-rpm-macros.**

표 **19.2. Python 3 RPM**의 매크로

매크로	일반 정의	설명
%{__python3}	/usr/bin/python3	Python 3 인터프리터
%{python3_version}	3.6	Python 3 인터프리터의 전체 버전입니다.
%{python3_sitelib}	/usr/lib/python3.6/site-packages	net-Python 모듈이 설치된 위치.
%{python3_sitearch}	/usr/lib64/python3.6/site-packages	아키텍처별 확장 기능이 포함된 모듈이 설치된 위치.
%py3_build		시스템 패키지에 적합한 인수를 사용하여 **setup.py build** 명령을 실행합니다.
%py3_install		시스템 패키지에 적합한 인수를 사용하여 **setup.py install** 명령을 실행합니다.

19.4.3. Python RPM 자동 제공

Python 프로젝트를 패키징할 때 이러한 디렉터리가 있는 경우 결과 **RPM**에 다음 디렉터리가 포함되어 있는지 확인합니다.

- .dist-info
- .egg-info
- .egg-link

이러한 디렉토리에서 RPM 빌드 프로세스는 자동으로 가상 pythonX.Ydist 를 생성합니다(예: python3.6dist(detox). 이러한 가상 제공은 %python_enable_dependency_generator 매크로로 지정된 패키지에 의해 사용됩니다.

19.5. PYTHON 스크립트에서 인터프리터 지시문 처리

Red Hat Enterprise Linux 8에서 실행 가능한 Python 스크립트는 주요 Python 버전을 명시적으로 지정하는 인터프리터 지시문(Hashbangs 또는 shebangs)을 사용해야 합니다. 예를 들면 다음과 같습니다.

```
#!/usr/bin/python3
#!/usr/bin/python3.6
#!/usr/bin/python3.8
#!/usr/bin/python3.9
#!/usr/bin/python3.11
#!/usr/bin/python3.12
#!/usr/bin/python2
```

RPM 패키지를 빌드할 때 /usr/lib/rpm/redhat/brp-mangle-shebangs BRP(Buildroot Policy) 스크립트가 자동으로 실행되며 모든 실행 파일에서 인터프리터 지시문을 수정하려고 시도합니다.

BRP 스크립트는 다음과 같은 모호한 인터프리터 지시문으로 Python 스크립트를 찾을 때 오류를 생성합니다.

```
#!/usr/bin/python
```

또는

```
#!/usr/bin/env python
```

19.5.1. Python 스크립트에서 인터프리터 지시문 수정

RPM 빌드 시 빌드 오류를 일으키는 Python 스크립트에서 인터프리터 지시문을 수정합니다.

사전 요구 사항

- Python 스크립트의 인터프리터 지시문 중 일부는 빌드 오류가 발생합니다.

절차

인터프리터 지시문을 수정하려면 다음 작업 중 하나를 완료합니다.

- platform-python-devel 패키지에서 pathfix.py 스크립트를 적용합니다.

```
# pathfix.py -pn -i %{__python3} PATH ...
```

  여러 개의 *PATH*s 를 지정할 수 있습니다. *PATH* 가 디렉터리인 경우 pathfix.py 는 모호한 인터프리터 지시문이 있는 ^[a-zA-Z0-9_]+\.py$ 패턴과 일치하는 Python 스크립트를 재귀적으로 스캔합니다. 이 명령을 %prep 섹션 또는 %install 섹션 끝에 추가합니다.

- 패키지된 Python 스크립트를 수정하여 예상되는 형식을 준수하도록 합니다. 이를 위해 pathfix.py 도 RPM 빌드 프로세스 외부에서 사용할 수 있습니다. RPM 빌드 외부에서 pathfix.py 를 실행하는 경우 위의 예제에서 *%{__python3}* 을 /usr/bin/python3 과 같은 인터프리터 지시문 경로로 바꿉니다.

패키지된 Python 스크립트에 Python 3.6 이외의 버전이 필요한 경우 필수 버전을 포함하도록 이전 명령을 조정합니다.

19.5.2. 사용자 정의 패키지의 /usr/bin/python3 인터프리터 지시문 변경

기본적으로, /usr/bin/python3 형식의 인터프리터 지시문은 Red Hat Enterprise Linux의 시스템 도구에 사용되는 platform-python 패키지에서 Python을 가리키는 인터프리터 지시문으로 교체됩니다. 사용자 지정 패키지의 /usr/bin/python3 인터프리터 지시문을 AppStream 리포지토리에서 설치한 특정 Python 버전을 가리키도록 변경할 수 있습니다.

절차

- 특정 버전의 Python에 대한 패키지를 빌드하려면 각 python 패키지의 *python**-rpm-macros 하위 패키지를 spec 파일의 BuildRequires 섹션에 추가합니다. 예를 들어 Python 3.6의 경우 다음 행을 포함합니다.

```
BuildRequires:  python36-rpm-macros
```

결과적으로 사용자 정의 패키지의 **/usr/bin/python3** 인터프리터 지시문은 자동으로 **/usr/bin/python3.6** 으로 변환됩니다.

참고

BRP 스크립트가 인터프리터 지시문을 확인하고 수정하지 못하게 하려면 다음 **RPM** 지시문을 사용합니다.

```
%undefine __brp_mangle_shebangs
```

19.6. PHP 스크립팅 언어 사용

PHP(하이퍼 텍스트 전처리기)는 주로 서버 측 스크립팅에 사용되는 범용 스크립팅 언어이며, 웹 서버를 사용하여 **PHP** 코드를 실행할 수 있습니다.

RHEL 8에서 **PHP** 스크립팅 언어는 **php** 모듈에서 제공하며, 여러 스트림(버전)에서 사용할 수 있습니다.

사용 사례에 따라 선택한 모듈 스트림의 특정 프로필을 설치할 수 있습니다.

- **common -** 웹 서버를 사용하는 서버 측 스크립팅의 기본 프로필입니다. 여기에는 널리 사용되는 몇 가지 확장 기능이 포함되어 있습니다.
- **minimal -** 이 프로필은 웹 서버를 사용하지 않고 **PHP**로 스크립팅하기 위한 명령줄 인터페이스만 설치합니다.
- **devel -** 이 프로필에는 **common** 프로필의 패키지와 개발을 위한 추가 패키지가 포함되어 있습니다.

19.6.1. PHP 스크립팅 언어 설치

선택한 **php** 모듈 버전을 설치할 수 있습니다.

절차

- default 프로필로 php 모듈 스트림을 설치하려면 다음을 사용합니다.

```
# yum module install php:stream
```

스트림 을 설치하려는 PHP 버전으로 교체합니다.

예를 들어 PHP 8.0을 설치하려면 다음을 수행합니다.

```
# yum module install php:8.0
```

기본 common 프로필은 php-fpm 패키지도 설치하고 Apache HTTP 서버 또는 nginx와 함께 사용할 PHP를 사전 구성합니다.

- php 모듈 스트림의 특정 프로필을 설치하려면 다음을 사용합니다.

```
# yum module install php:stream/profile
```

스트림 을 원하는 버전 및 *프로필로* 교체할 프로필 이름으로 교체합니다.

예를 들어 웹 서버 없이 사용할 PHP 8.0을 설치하려면 다음을 수행합니다.

```
# yum module install php:8.0/minimal
```

추가 리소스

- RHEL 8에서 사용 가능한 이전 버전의 PHP에서 업그레이드하려면 이후 스트림으로 전환을 참조하십시오.

- RHEL 8 모듈 및 스트림 관리에 대한 자세한 내용은 사용자 공간 구성 요소 설치, 관리 및 제거를 참조하십시오.

19.6.2. 웹 서버에서 PHP 스크립팅 언어 사용

19.6.2.1. Apache HTTP 서버와 함께 PHP 사용

Red Hat Enterprise Linux 8에서 Apache HTTP 서버를 사용하면 PHP를 FastCGI 프로세스 서버로 실행할 수 있습니다. FPM(FascGI Process Manager)은 웹 사이트가 높은 부하를 관리할 수 있는 대체 PHP FastCGI 데몬입니다. PHP는 RHEL 8에서 기본적으로 FastCGI Process Manager를 사용합니다.

FastCGI 프로세스 서버를 사용하여 PHP 코드를 실행할 수 있습니다.

사전 요구 사항

- PHP 스크립팅 언어가 시스템에 설치되어 있습니다.

  PHP 스크립팅 언어 설치를 참조하십시오.

절차

1. httpd 모듈을 설치합니다.

```
# yum module install httpd:2.4
```

2. Apache HTTP 서버 시작 :

```
# systemctl start httpd
```

   또는 Apache HTTP 서버가 이미 시스템에서 실행 중인 경우 PHP를 설치한 후 httpd 서비스를 다시 시작하십시오.

```
# systemctl restart httpd
```

3. php-fpm 서비스를 시작합니다.

```
# systemctl start php-fpm
```

4. 선택 사항: 부팅 시 두 서비스가 모두 시작되도록 활성화합니다.

```
# systemctl enable php-fpm httpd
```

5. PHP 설정에 대한 정보를 얻으려면 /var/www/html/ 디렉터리에 다음 콘텐츠를 사용하여

index.php 파일을 생성합니다.

```
# echo '<?php phpinfo(); ?>' > /var/www/html/index.php
```

6. index.php 파일을 실행하려면 브라우저에서 다음을 가리킵니다.

```
http://<hostname>/
```

7. 선택 사항: 특정 요구 사항이 있는 경우 구성을 조정합니다.

- /etc/httpd/conf/httpd.conf - generic httpd configuration
- /etc/httpd/conf.d/php.conf - httpd에 대한 PHP별 설정
- /usr/lib/systemd/system/httpd.service.d/php-fpm.conf - 기본적으로 php-fpm 서비스는 httpd로 시작됩니다.
- /etc/php-fpm.conf - FPM 기본 설정
- /etc/php-fpm.d/www.conf - 기본 www 풀 구성

예 19.1. "Hello, World!" 실행 중. Apache HTTP 서버를 사용하는 PHP 스크립트

1. /var/www/html/ 디렉터리에 프로젝트에 대한 hello 디렉터리를 생성합니다.

```
# mkdir hello
```

2. 다음 내용으로 /var/www/html/hello/ 디렉터리에 hello.php 파일을 생성합니다.

```
# <!DOCTYPE html>
<html>
<head>
<title>Hello, World! Page</title>
</head>
<body>
<?php
```

```
    echo 'Hello, World!';
?>
</body>
</html>
```

3. Apache HTTP 서버 시작 :

```
# systemctl start httpd
```

4. hello.php 파일을 실행하려면 브라우저에서 다음을 가리킵니다.

```
http://<hostname>/hello/hello.php
```

그 결과 "Hello, World!" 텍스트가 있는 웹 페이지가 표시됩니다.

추가 리소스

- Apache HTTP 웹 서버 설정

19.6.2.2. nginx 웹 서버와 함께 PHP 사용

nginx 웹 서버를 통해 PHP 코드를 실행할 수 있습니다.

사전 요구 사항

- PHP 스크립팅 언어가 시스템에 설치되어 있습니다.

  PHP 스크립팅 언어 설치를 참조하십시오.

절차

1. nginx 모듈 스트림을 설치합니다.

```
# yum module install nginx:stream
```

스트림 을 설치하려는 nginx 버전으로 교체합니다.

예를 들어 **nginx** 버전 **1.18**을 설치하려면 다음을 수행합니다.

```
# yum module install nginx:1.18
```

2. **nginx** 서버를 시작합니다.

```
# systemctl start nginx
```

또는 **nginx** 서버가 이미 시스템에서 실행 중인 경우 **PHP**를 설치한 후 **nginx** 서비스를 다시 시작하십시오.

```
# systemctl restart nginx
```

3. **php-fpm** 서비스를 시작합니다.

```
# systemctl start php-fpm
```

4. 선택 사항: 부팅 시 두 서비스가 모두 시작되도록 활성화합니다.

```
# systemctl enable php-fpm nginx
```

5. **PHP** 설정에 대한 정보를 얻으려면 **/usr/share/nginx/html/** 디렉터리에 다음 콘텐츠를 사용하여 **index.php** 파일을 생성합니다.

```
# echo '<?php phpinfo(); ?>' > /usr/share/nginx/html/index.php
```

6. **index.php** 파일을 실행하려면 브라우저에서 다음을 가리킵니다.

```
http://<hostname>/
```

7. 선택 사항: 특정 요구 사항이 있는 경우 구성을 조정합니다.

- **/etc/nginx/nginx.conf - nginx** 기본 구성

- /etc/nginx/conf.d/php-fpm.conf - nginx에 대한 FPM 구성
- /etc/php-fpm.conf - FPM 기본 설정
- /etc/php-fpm.d/www.conf - 기본 www 풀 구성

예 19.2. "Hello, World!" 실행 중. nginx 서버를 사용하는 PHP 스크립트

1. /usr/share/nginx/html/ 디렉터리에 프로젝트의 hello 디렉터리를 생성합니다.

```
# mkdir hello
```

2. 다음 콘텐츠를 사용하여 /usr/share/nginx/html/hello/ 디렉터리에 hello.php 파일을 생성합니다.

```
# <!DOCTYPE html>
<html>
<head>
<title>Hello, World! Page</title>
</head>
<body>
<?php
   echo 'Hello, World!';
?>
</body>
</html>
```

3. nginx 서버를 시작합니다.

```
# systemctl start nginx
```

4. hello.php 파일을 실행하려면 브라우저에서 다음을 가리킵니다.

```
http://<hostname>/hello/hello.php
```

그 결과 "Hello, World!" 텍스트가 있는 웹 페이지가 표시됩니다.

추가 리소스

추가 리소스

- NGINX 설정 및 구성

19.6.3. 명령줄 인터페이스를 사용하여 PHP 스크립트 실행

PHP 스크립트는 일반적으로 웹 서버를 사용하여 실행되지만 명령줄 인터페이스를 사용하여 실행할 수도 있습니다.

명령줄만 사용하여 php 스크립트를 실행하려면 php 모듈 스트림의 최소 프로필을 설치합니다.

PHP 스크립팅 언어 설치를 참조하십시오.

사전 요구 사항

- PHP 스크립팅 언어가 시스템에 설치되어 있습니다.

  PHP 스크립팅 언어 설치를 참조하십시오.

절차

1. 텍스트 편집기에서 *filename*.php 파일을 생성합니다.

   파일 이름을 파일 이름으로 바꿉니다.

2. 명령줄에서 생성된 *filename*.php 파일을 실행합니다.

```
# php filename.php
```

예 19.3. "Hello, World!" 실행 중. 명령줄 인터페이스를 사용하는 PHP 스크립트

1. 텍스트 편집기를 사용하여 다음 내용으로 hello.php 파일을 생성합니다.

```
<?php
  echo 'Hello, World!';
?>
```

2. 명령줄에서 **hello.php** 파일을 실행합니다.

```
# php hello.php
```

그 결과 **"Hello, World!"**가 출력됩니다.

19.6.4. 추가 리소스

- **httpd(8) -** 명령줄 옵션의 전체 목록을 포함하는 **httpd** 서비스의 도움말 페이지입니다.
- **httpd.conf(5) - httpd** 구성의 도움말 페이지로, **httpd** 구성 파일의 구조와 위치를 설명합니다.
- **nginx(8) -** 명령줄 옵션 및 신호 목록의 전체 목록을 포함하는 **nginx** 웹 서버의 도움말 페이지입니다.
- **PHP-fpm(8) -** 명령줄 옵션 및 구성 파일의 전체 목록을 설명하는 **PHP FPM**의 도움말 페이지.

19.7. TCL/TK 시작하기

19.7.1. Tcl/Tk 소개

Tcl(도구 명령 언어) 은 동적 프로그래밍 언어입니다. 이 언어의 인터프리터는 **C** 라이브러리와 함께 **tcl** 패키지에서 제공합니다.

Tk (T cl/Tk)와 결합된 **Tcl**을 사용하면 크로스 플랫폼 **GUI** 애플리케이션을 생성할 수 있습니다. **TK** 는 **tk** 패키지에서 제공합니다.

Tk 는 다음 중 하나를 참조할 수 있습니다.

- 여러 언어를 위한 프로그래밍 툴킷

- C, Ruby, Perl, Python과 같은 여러 언어에 사용할 수 있는 Tk C 라이브러리 바인딩
- Tk 콘솔을 인스턴스화하는 wish 인터프리터
- 특정 Tcl 인터프리터에 여러 새 명령을 추가하는 Tk 확장

Tcl/Tk에 대한 자세한 내용은 Tcl/Tk 설명서 또는 Tcl/Tk 설명서 웹 페이지를 참조하십시오.

19.7.2. Tcl/Tk 8.6에서 주요 변경 사항

Red Hat Enterprise Linux 7에서 Tcl/Tk 8.5 를 사용했습니다. Red Hat Enterprise Linux 8에서는 기본 OS 리포지터리에서 Tcl/Tk 버전 8.6 이 제공됩니다.

Tcl/Tk 8.5 에 비해 Tcl/Tk 8.6 의 주요 변경 사항은 다음과 같습니다.

- 객체 지향 프로그래밍 지원
- Stackless 평가 구현
- 향상된 예외 처리
- Tcl로 빌드 및 설치되는 타사 패키지 모음
- 다중 스레드 작업 활성화
- SQL 데이터베이스 기반 스크립트 지원
- IPv6 네트워킹 지원

- 내장된 **Zlib** 압축

- 목록 처리

  Tcl 컨테이너에 대한 변환 표현식을 허용하는 **lmap** 및 **dict** 맵 이라는 두 가지 새 명령을 사용할 수 있습니다.

- 스크립트 별 스택 채널

  두 개의 새 명령인 **chan push** 및 **chan pop** 을 사용할 수 있으므로 **I/O** 채널로의 변환을 추가하거나 제거할 수 있습니다.

Tk 의 주요 변경 사항은 다음과 같습니다.

- 내장 이미지 지원

- 사용 중인 창

  창 또는 위젯에 대한 사용자 상호 작용을 비활성화하고 사용 중인 커서를 표시하는 새 명령인 **tk busy** 를 사용할 수 있습니다.

- 새로운 글꼴 선택 대화 인터페이스

- 눈여겨진 텍스트 지원

- 팔레트 지원에 대한 이동

Tcl 8.5 및 **Tcl 8.6**의 변경 내용은 **Tcl/Tk 8.6** 의 변경 사항을 참조하십시오.

19.7.3. Tcl/Tk 8.6으로 마이그레이션

Red Hat Enterprise Linux 7에서 **Tcl/Tk 8.5** 를 사용했습니다. **Red Hat Enterprise Linux 8**에서는 기

본 OS 리포지터리에서 Tcl/Tk 버전 8.6 이 제공됩니다.

이 섹션에서는 다음을 위해 Tcl/Tk 8.6 로의 마이그레이션 경로에 대해 설명합니다.

- Tcl 확장을 작성하거나 애플리케이션에 Tcl 인터프리터를 포함하는 개발자
- Tcl/Tk를 사용하여 작업을 스크립팅하는 사용자

19.7.3.1. Tcl 확장 개발자를 위한 마이그레이션 경로

코드를 Tcl 8.6 과 호환하려면 다음 절차를 사용하십시오.

절차

1. interp 구조를 사용하도록 코드를 다시 작성합니다. 예를 들어 코드가 interperrorline인 경우 다음 함수를 사용하도록 다시 작성합니다.

```
Tcl_GetErrorLine(interp)
```

   이는 Tcl 8.6 이 인터랩 구조의 멤버에 대한 직접 액세스를 제한하기 때문에 필요합니다.

2. 코드를 Tcl 8.5 및 Tcl 8.6 과 모두 호환하려면 C 또는 C++ 애플리케이션의 헤더 파일에서 다음 코드 스니펫을 사용하거나 Tcl 라이브러리를 포함하는 확장 프로그램의 코드 조각을 사용하십시오.

```
# include <tcl.h>
# if !defined(Tcl_GetErrorLine)
# define Tcl_GetErrorLine(interp) (interp→errorLine)
# endif
```

19.7.3.2. Tcl/Tk를 사용하여 작업을 스크립팅하는 사용자의 마이그레이션 경로

Tcl 8.6 에서는 대부분의 스크립트가 이전 버전의 Tcl 과 동일한 방식으로 작동합니다.

코드를 Tcl 8.6 으로 마이그레이션하려면 다음 절차를 사용하십시오.

절차

- 이식 코드를 작성할 때 **Tk 8.6** 에서 더 이상 지원되지 않는 명령을 사용하지 않도록하십시오 :

```
tkIconList_Arrange
tkIconList_AutoScan
tkIconList_Btn1
tkIconList_Config
tkIconList_Create
tkIconList_CtrlBtn1
tkIconList_Curselection
tkIconList_DeleteAll
tkIconList_Double1
tkIconList_DrawSelection
tkIconList_FocusIn
tkIconList_FocusOut
tkIconList_Get
tkIconList_Goto
tkIconList_Index
tkIconList_Invoke
tkIconList_KeyPress
tkIconList_Leave1
tkIconList_LeftRight
tkIconList_Motion1
tkIconList_Reset
tkIconList_ReturnKey
tkIconList_See
tkIconList_Select
tkIconList_Selection
tkIconList_ShiftBtn1
tkIconList_UpDown
```

  /usr/share/tk8.6/unsupported.tcl 파일에서 지원되지 않는 명령 목록을 확인할 수 있습니다.